

CS3230

@Wxy2003-xy

Contents

1	Asymptotic Analysis	3
1.1	Proof with limit	3
1.2	Reflexivity	5
1.2.1	$f(n) \in O(f(n))$	5
1.2.2	$f(n) \in \Omega(f(n)), f(n) \in \Theta(f(n))$	5
1.3	Transitivity	5
1.3.1	$f(n) \in O(g(n)) \wedge g(n) \in O(h(n)) \implies f(n) \in O(h(n))$	5
1.3.2	$f(n) \in \Omega(g(n)) \wedge g(n) \in \Omega(h(n)) \implies f(n) \in \Omega(h(n))$	5
1.3.3	$f(n) \in \Theta(g(n)) \wedge g(n) \in \Theta(h(n)) \implies f(n) \in \Theta(h(n))$	5
1.3.4	$f(n) \in o(g(n)) \wedge g(n) \in o(h(n)) \implies f(n) \in o(h(n))$	6
1.3.5	$f(n) \in \omega(g(n)) \wedge g(n) \in \omega(h(n)) \implies f(n) \in \omega(h(n))$	6
1.4	Symmetry: $f(n) \in \Theta(g(n)) \iff g(n) \in \Theta(f(n))$	6
1.5	Complementary	6
1.5.1	$f(n) \in O(g(n)) \iff g(n) \in \Omega(f(n))$	6
1.5.2	$f(n) \in o(g(n)) \iff g(n) \in \omega(f(n))$	6
1.6	Problems	6
1.7	Math utils	8
1.8	Techniques	9
2	Recurrence	10
2.1	master theorem	10
2.1.1	Common recurrence table	10
2.2	Telescoping	11
2.2.1	Normalization: $T(n) = T(n^{\frac{1}{c}})$	11
2.2.2	The Iterated Logarithm $\log^* n$	12
2.2.3	Comparison	13
2.3	WA1 Practice on Asymptotic analysis and recurrence	15
2.3.1	$f(n) = 3n + 1, g(n) = 4n$	15
2.3.2	$f(n) = 3^n, g(n) = 2^n \cdot n^{10000}$	15
2.3.3	$f(n) = 2^n, g(n) = 2^{n+1}$	15
2.3.4	$f(n) = 2^{(2^n)}, g(n) = 2^{(2^{n+1})}$	15
2.3.5	$f(n) = \sum_{i=1}^n \frac{1}{2^i}, g(n) = \sum_{i=1}^n \frac{i^{10000}}{2^i}$	16
2.3.6	$f(n) = \log \log(n^n), g(n) = \log \log(2^{\sqrt{n}})$	16
2.3.7	$f(n) = 2^{\sqrt{\log n}}, g(n) = \sqrt{n}$	16
2.3.8	$f(n) = 2^{\sqrt{\log n}}, g(n) = 2^{2^{\sqrt{\log \log n}}}$	16
2.3.9	$f(n) = n^{\frac{1}{\log n}}, g(n) = n^{\frac{1}{(\log n)^2}}$	16
2.3.10	$f(n) = n^{\frac{1}{\log n}}, g(n) = n^{\frac{1}{\sqrt{\log n}}}$	16
2.3.11	$T(n) = 16T(\frac{n}{4}) + 32n \log^{128} n$	17
2.3.12	$T(n) = 16T(\frac{n}{4}) + 64n^2 \log^8 n + 32n \log^{128} n$	18
2.3.13	$T(n) = 16 \cdot T(\frac{n}{4}) + n^4$	18
2.3.14	$T(n) = T(\sqrt{n}) + \log(n)$	19
2.3.15	$T(n) = T(\sqrt{n}) + \log \log(n)$	19
3	Proof of Correctness	20
3.1	Iterative algorithms	20
3.2	Example: Dijkstra	20
3.3	Example: Insertion Sort	20
3.4	Stooge sort	21
3.5	2D Peak Finding via Divide-and-Conquer	22
4	Divide and Conquer	24
4.1	Fast Exponentiation from a Divide-and-Conquer Perspective	24
4.2	Strassen's Matrix Multiplication — Analysis	24
4.3	Karatsuba's Algorithm and Its Divide-and-Conquer Analysis	26
4.4	Divide-and-Conquer Algorithm for the Discrete Fourier Transform	27

5	Lower bound analysis & Linear time sorting & order statistics	28
5.1	Proof of the Lower Bound of Comparison-Based Sorting	28
5.2	Decision Tree Model	28
5.3	Decision-Tree Lower Bounds (Beyond Sorting & Searching)	28
5.3.1	1) Planar Convex Hull: $\Omega(n \log n)$ in algebraic decision trees.	28
5.3.2	2) 2D Maxima / Skyline: $\Omega(n \log n)$ in algebraic decision trees.	29
5.3.3	3) Planar Point Location: $\Omega(\log n)$ queries.	29
5.3.4	4) Minimum Spanning Tree (MST): $\Omega(m)$ edge-weight comparisons.	29
5.3.5	5) Majority Element Decision (is there an element with count $> n/2$?): $\Omega(n)$.	29
5.3.6	6) Segment Intersection Emptiness (no pair intersects?): $\Omega(n \log n)$.	29
5.3.7	7) Voronoi Diagram / Delaunay Triangulation: $\Omega(n \log n)$.	29
5.3.8	8) Linear Programming in Fixed Dimension d (e.g., $d = 2, 3$): $\Omega(n)$.	29
5.3.9	9) Closest Pair (planar): $\Omega(n \log n)$ in algebraic decision trees.	29
5.3.10	10) Transitive Reduction / Cover Graph of a Total Order: $\Omega(n \log n)$.	29
5.4	Linear-Time Sorting: Stable Counting Sort and Radix Sort	30
5.4.1	Stable Counting Sort	30
5.4.2	Radix Sort	31
5.5	Selection/Order statistics	32
5.5.1	Randomized Quickselect for the i -th Smallest Element	32
5.5.2	Median of Medians	33
5.6	Tutorial problems	34
5.7	Tutorial: Order Statistics (Model Answers)	35
6	Randomized Algorithms	38
6.1	Freivalds	38
6.1.1	Probability Analysis	38
6.1.2	Success Probability Amplification	38
6.2	Coupon collection and Hashing	38
6.3	Randomized Quicksort: Analysis	38
6.4	Reservoir sampling	40
6.5	Fisher–Yates (a.k.a. Knuth/Durstenfeld) Shuffle	40
6.6	Relationship Between Expected Runtime and Tail Decay	41
7	Dynamic Programming	44
7.1	The Knapsack Problem and Its Variants	44
7.2	Longest Increasing Subsequence and Its Variants	45
7.2.1	Longest Increasing Subsequence (1D)	45
7.2.2	Longest Contiguous Increasing Subarray with Switching Between Price Sources	45
7.2.3	LIS in 2D: Longest Increasing Sequence in the Plane	46
7.2.4	LIS in a 2D Grid (Monotone Paths)	46
7.2.5	Longest Chain in Higher Dimensions	47
7.3	Twenty Classic Dynamic Programming Problems	49
7.3.1	1. 0/1 Knapsack (maximize value within capacity)	49
7.3.2	2. Coin Change (min coins, unbounded)	49
7.3.3	3. Longest Increasing Subsequence ($O(n^2)$)	49
7.3.4	4. Edit Distance (Levenshtein)	49
7.3.5	5. Matrix Chain Multiplication	50
7.3.6	6. Longest Common Subsequence	50
7.3.7	7. Subset Sum (feasibility)	50
7.3.8	8. Weighted Interval Scheduling	50
7.3.9	9. Rod Cutting	50
7.3.10	10. Palindrome Partitioning II (min cuts)	50
7.3.11	11. Egg Dropping	51
7.3.12	12. Catalan Numbers (count valid parentheses/BSTs)	51
7.3.13	13. Word Break (feasibility with dictionary)	51
7.3.14	14. Distinct Subsequences (count $S \rightarrow T$)	51
7.3.15	15. Longest Palindromic Subsequence	51
7.3.16	16. Tree Minimum Vertex Cover (rooted)	51
7.3.17	17. Traveling Salesman (bitmask DP)	52
7.3.18	18. Grid Min Path Sum (right/down moves)	52
7.3.19	19. Interleaving String (feasibility)	52
7.3.20	20. Bellman–Ford Algorithm from a Dynamic Programming Perspective	52
7.3.21	21. Floyd–Warshall Algorithm (All-Pairs Shortest Paths)	53
7.3.22	22. Maximum Score from Performing Multiplication Operations (LeetCode 1770)	54
7.4	High-Dimensional DP	55
7.4.1	1) LCS of Three Strings (3D)	55
7.4.2	2) 0/1 Knapsack with Exactly K Items (3D with rolling over items)	55
7.4.3	3) 2D Knapsack (Weight & Volume) (3D with rolling on items)	56
7.4.4	4) Count Grid Paths with $\leq K$ Turns (4D split into two 3D arrays)	56

7.4.5	5) Traveling Salesman (Bitmask DP: exponential dimension)	57
7.4.6	6) Minimum Cost to Merge Stones (Dynamic Programming Analysis)	57
7.5	Pseudo-polynomial DP	58
7.5.1	0/1 Knapsack, coin-change etc	58
7.5.2	Counting (r, ℓ) -Spaced Subsets in $O(rn \log n)$ Time	58
8	Probability & Statistics Formulae for Algorithm Analysis	60
8.1	Basic Probability Identities	60
8.2	Indicator Trick & Linearity	60
8.3	Expectation/Variance Identities	60
8.4	Inequalities (Concentration)	60
8.5	Common Inequalities for Mathematical Proofs	60
8.6	Common Taylor Series Expansions	62
8.7	Common Distributions (means/vars)	63
8.8	Asymptotic Approximations	63
8.9	Classic “Balls-into-Bins” Facts	63
8.10	Coupon Collector	63
8.11	Conditioning & Decomposition	63
8.12	Jensen & Convexity	64
8.13	“With High Probability” (w.h.p.)	64
8.14	Divisibility, GCD, and Bézout	64
8.15	Modular Arithmetic & Inverses	64
8.16	Chinese Remainder Theorem (CRT)	64
8.17	Prime Factorization Facts	64
8.18	Primitive Roots & Orders	65
8.19	Combinatorics Mod Prime	65
8.20	Base Representation & Digit Properties	65
8.21	Sieve & Multiplicative Precomputation	65
8.22	Miscellaneous Handy Facts	65
9	Greedy Algorithm	66
9.1	Example: Hoffman Code	66
9.2	Greedy Choice Proofs examples	67
9.2.1	Dijkstra’s Algorithm as a Greedy Algorithm	67
9.2.2	Prim’s Algorithm as a Greedy Algorithm	68
9.2.3	Burning CDs Problem	68
9.2.4	Analysis of Statements for the CD Burning Problem	70
9.3	Maximizing Profit in the Fish Selling Problem	70
9.4	Activity Selection Problem	71
9.5	Dominating Set Variant of the Activity Selection Problem	71
10	Amortization Analysis	73
10.1	Aggregate method	73
10.2	Accounting method	73
10.3	Potential method	73
10.4	Amortized Analysis: Choosing Accounting and Potential Functions	74
10.5	Amortized Analysis of Dynamic Hash Tables with Rehashing	75
10.6	Amortized Analysis of Data Structures via the Potential Method	76
10.6.1	Self-Adjusting Binary Search Tree (Splay Tree)	76
10.6.2	Binary Heap Priority Queue	77
10.6.3	Fibonacci Heap	78
10.6.4	Disjoint Set Union (Union-Find) with Union-by-Rank and Path Compression	79
10.6.5	B-Tree	80
10.6.6	AVL Tree	81
10.6.7	Resizable Hash Table with Linked-List Chaining	81
10.7	Practice: WA2 task 1	82
10.7.1	Claim: <code>increment(i)</code> has $O(1)$ amortized cost.	82
10.7.2	There exists a sequence of n operations with one operation costing $\Omega(n)$	83
10.7.3	Cost of the n -th operation.	83
10.7.4	There exists a finite sequence of operations that the process does not terminate	83

11 Common Graph Algorithms and Their Time Complexities	84
11.1 Shortest Path Algorithms	84
11.1.1 Dijkstra's Algorithm (Binary Heap)	84
11.1.2 Dijkstra's Algorithm (Fibonacci Heap)	84
11.1.3 Bellman-Ford	84
11.1.4 Floyd-Warshall	84
11.1.5 Johnson's Algorithm	84
11.1.6 SPFA (Shortest-Path Faster Algorithm)	84
11.2 Minimum Spanning Tree Algorithms	84
11.2.1 Kruskal's Algorithm (Union-Find)	84
11.2.2 Prim's Algorithm (Binary Heap)	84
11.2.3 Prim's Algorithm (Fibonacci Heap)	84
11.3 Maximum Flow Algorithms	84
11.3.1 Ford-Fulkerson	84
11.3.2 Edmonds-Karp	85
11.3.3 Dinic's Algorithm	85
11.3.4 Push-Relabel (Goldberg-Tarjan)	85
11.4 Connectivity and Components	85
11.4.1 Tarjan's SCC Algorithm	85
11.4.2 Kosaraju's SCC Algorithm	85
11.5 Matching and Assignment	85
11.5.1 Hopcroft-Karp	85
11.5.2 Hungarian Algorithm	85
11.6 Planar Graph Algorithms	85
11.6.1 Planar SSSP	85
11.6.2 Planar Max-Flow	85
11.7 All-Pairs Shortest Paths (APSP)	85
11.7.1 APSP via Repeated Dijkstra	85
11.7.2 APSP via Repeated Bellman-Ford	85
11.7.3 APSP via Fast Matrix Multiplication	85
12 Reduction	86
12.1 $p(n)$ -time reduction	86
12.2 Running time composition	86
12.3 polynomial time reduction	86
12.4 Input size: problem encoding	86
12.5 Example: Karp Reduction from 3SAT to INDEPENDENT SET	86
12.6 Tutorial:	88
12.6.1 Q1. Graph Coloring	88
12.6.2 Q2. Partition versus Ball-Partition	88
12.6.3 Q3. Partition versus Knapsack	89
12.6.4 Q4. Hamiltonian Cycle (HC) versus Travelling Salesperson Problem (TSP)	89
13 NP Complete	90
13.1 $3\text{-SAT} \leq_P \text{IS}$	91
13.2 $3\text{-SAT} \leq_P \text{VC}$	91
13.3 NP-Hardness and Its Relationship to NP	92
13.3.1 Karp Reductions: Propagation of Easiness and Hardness	93
14 Common NPC proofs	97
14.1 Reduction from CIRCUIT-SAT to SAT (NP-Completeness of SAT)	97
14.2 Reduction from 3-SAT to NAE-3-SAT ⁺	98
14.3 Reduction from 3-SAT to 0-1-LINEAR-SAT	101
14.4 Reduction from 3-SAT to EXACT-1 3-SAT	102
14.5 Reduction from 3-SAT to Group Interval Scheduling (Group Size ≥ 3)	104
14.6 Reduction from 3-SAT to PLANAR 3-SAT	105
14.7 Reduction from 3SAT to Bounded-Occurrence 3SAT	106
14.8 Reduction from 3-SAT to 3-DIMENSIONAL MATCHING	107
14.9 Reduction from 3-DIMENSIONAL MATCHING to 4-PARTITION and then to 3-PARTITION	109
14.10 Reduction from 3-DIMENSIONAL MATCHING to PARTITION INTO TRIANGLES	111
14.11 Direct Reduction from X3C to PARTITION INTO TRIANGLES	112
14.12 Reduction from PLANAR 3-SAT to PARTITION INTO TRIANGLES	114
14.13 Reduction from 3-SAT to VERTEX COVER	115
14.14 Reduction from VERTEX COVER to HITTING SET	117
14.15 Reduction from VERTEX COVER to SET COVER	119
14.16 Reduction from SET COVER to DOMINATING SET	120
14.17 WA3 Task 2: Complexity of MIS Variants	121
14.18 More Reductions Overview and Examples	123
14.18.1 Polynomial-Time Reductions	123

14.18.2	Reduction: 3SAT to Vertex 3-Coloring	123
14.18.3	Reduction: 3SAT to Hamiltonian Cycle	123
14.18.4	Reduction: 3SAT to Super Mario	123
14.18.5	Takeaways	123
14.193	Partition $\leq_P$ 2-partition(partition)	124
14.20	Partition $\leq_P$ 0/1 Knapsack	124
14.210/1	Knapsack $\leq_P$ coin change	124

1 Asymptotic Analysis

$$\begin{aligned}
O(g) &: \{f : \exists c > 0 \wedge n_0 > 0, \forall n \geq n_0, 0 \leq f(n) \leq c \cdot g(n)\} \\
\Omega(g) &: \{f : \exists c > 0 \wedge n_0 > 0, \forall n \geq n_0, 0 \leq c \cdot g(n) \leq f(n)\} \\
\Theta(g) &: \{f : \exists c_1, c_2 > 0 \wedge n_0 > 0, \forall n \geq n_0, c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)\} \\
o(g) &: \{f : \forall c > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq f(n) < c \cdot g(n)\} \\
\omega(g) &: \{f : \forall c > 0, \forall n \leq n_0, 0 \leq c \cdot g(n) < f(n)\}
\end{aligned}$$

1.1 Proof with limit

Limit-Based Proofs for Asymptotic Notation

Hint. Work with eventually-positive functions and ratios. Let $f, g : \mathbb{N} \rightarrow \mathbb{R}$ (or $\mathbb{R}_{>0} \rightarrow \mathbb{R}$) with $g(n) \neq 0$ for all large n . Write $R(n) = \frac{|f(n)|}{|g(n)|}$ unless otherwise noted.

Definitions (epsilon/constant form)

$$\begin{aligned}
f(n) \in O(g(n)) &\iff \exists C > 0, N : \forall n \geq N, |f(n)| \leq C |g(n)|. \\
f(n) \in \Omega(g(n)) &\iff \exists c > 0, N : \forall n \geq N, |f(n)| \geq c |g(n)|. \\
f(n) \in \Theta(g(n)) &\iff f \in O(g) \text{ and } f \in \Omega(g). \\
f(n) \in o(g(n)) &\iff \forall \varepsilon > 0 \exists N : \forall n \geq N, |f(n)| \leq \varepsilon |g(n)|. \\
f(n) \in \omega(g(n)) &\iff \forall M > 0 \exists N : \forall n \geq N, |f(n)| \geq M |g(n)|. \\
f(n) \sim g(n) &\iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 1 \quad (\text{asymptotic equivalence}).
\end{aligned}$$

Limit criteria (what to prove with limits)

Assume $g(n) > 0$ eventually.

- **Little-o:** $f \in o(g) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0$.
- **Little- ω :** $f \in \omega(g) \iff \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty$ (equivalently $\liminf R(n) = \infty$).
- **Big- Θ :** $f \in \Theta(g)$ if and only if

$$0 < \liminf_{n \rightarrow \infty} \frac{|f(n)|}{|g(n)|} \leq \limsup_{n \rightarrow \infty} \frac{|f(n)|}{|g(n)|} < \infty.$$

In particular, if $\lim \frac{f}{g} = c \in (0, \infty)$ then $f \in \Theta(g)$.

- **Big-O:** A *sufficient and necessary* condition:

$$f \in O(g) \iff \limsup_{n \rightarrow \infty} \frac{|f(n)|}{|g(n)|} < \infty.$$

In particular, if $\lim \frac{f}{g} = c \in [0, \infty)$ then $f \in O(g)$.

- **Big- Ω :** A *sufficient and necessary* condition (for eventually positive f):

$$f \in \Omega(g) \iff \liminf_{n \rightarrow \infty} \frac{|f(n)|}{|g(n)|} > 0.$$

In particular, if $\lim \frac{f}{g} = c \in (0, \infty]$ then $f \in \Omega(g)$.

- **Asymptotic equivalence ($\sim$):** $f \sim g \Rightarrow f \in \Theta(g)$ (take $c = 1$ above). Converse is false in general.

Proof templates

To show $f \in o(g)$: Compute $L = \lim \frac{f}{g}$. If $L = 0$, write:

$$\text{Given } \varepsilon > 0, \exists N : \forall n \geq N, \frac{|f(n)|}{|g(n)|} < \varepsilon.$$

To show $f \in O(g)$: Show $\limsup R(n) = M < \infty$, then choose $C > M + 1$ and N s.t. $R(n) \leq C$ for all $n \geq N$.

To show $f \in \Omega(g)$: Show $\liminf R(n) = m > 0$, then pick $c = m/2$ and N s.t. $R(n) \geq c$ for all $n \geq N$.

To show $f \in \Theta(g)$: Either (i) show $\lim \frac{f}{g} = c \in (0, \infty)$, or (ii) give both O and Ω bounds via limsup/liminf.

To show $f \in \omega(g)$: Show $\lim \frac{f}{g} = \infty$ (or $\liminf R(n) = \infty$). Then for any $M > 0$, pick N with $R(n) \geq M$ for all $n \geq N$.

examples (limit proofs)

- $n^\alpha \in o(n^\beta)$ for $0 \leq \alpha < \beta$: $\lim \frac{n^\alpha}{n^\beta} = \lim n^{\alpha-\beta} = 0$.
- $\log^k n \in o(n^\varepsilon)$ for any fixed $k, \varepsilon > 0$: use $\lim \frac{\log^k n}{n^\varepsilon} = 0$ (e.g. L'Hôpital or known hierarchy).
- $a^n \in \omega(n^k)$ for $a > 1$: $\lim \frac{a^n}{n^k} = \infty$.
- $(n + \sin n) \in \Theta(n)$: $\lim \frac{n + \sin n}{n} = 1$ by squeeze.
- $n^p(\log n)^q \in \Theta(n^p \log^q n)$: trivial $c = 1$.

Notes

- Using ratio limits when $g(n)$ changes sign or is zero infinitely often. Fix by restricting to n large where $g(n) > 0$.
- Concluding Θ from $\lim \frac{f}{g} = 0$ or ∞ (those give o or ω , not Θ).
- Forgetting absolute values in oscillatory cases (e.g. $(-1)^n n$); prefer limsup/liminf or bound with envelopes.
- Assuming O implies a limit exists—false. $f(n) = \sin n$ is $O(1)$ but the ratio to 1 has no limit.

Handy equivalences

$$f \in o(g) \iff g \in \omega(f), \quad f \in O(g) \iff g \in \Omega(f).$$

If $f \sim g$ then $f - g = o(g)$ (and $o(f)$), and $f \in \Theta(g)$.

$$\Theta(f) = O(f) \cap \Omega(f)$$

Proof. 1. prove: $O(f) \cap \Omega(f) \subseteq \Theta(f)$

From definition:

$$\begin{aligned} \Theta(f(n)) &: \{g : \exists c_1, c_2 > 0, \exists n_0 \geq 0, \forall n \geq n_0, c_1 \cdot f(n) \leq g(n) \leq c_2 \cdot f(n)\} \\ O(f(n)) \cap \Omega(f(n)) &: \{g : \exists c_1 > 0, \exists n_1 \geq 0, \forall n \geq n_1, 0 \leq g(n) \leq c_1 \cdot f(n) \\ &\quad \wedge \exists c_2 > 0, \exists n_2 \geq 0, \forall n \geq n_2, 0 \leq c_2 \cdot f(n) \leq g(n)\} \end{aligned}$$

When $0 \leq c_1 \leq c_2$, $O(f(n)) \cap \Omega(f(n)) : \{g : \exists n_1, n_2 \geq 0, \forall n \geq \{\max(n_1, n_2)\}, 0 \leq c_1 f(n) \leq g(n) \leq c_2 f(n)\}$

$\implies O(f) \cap \Omega(f) \subseteq \Theta(f)$

2. prove: $\Theta(f) \subseteq O(f) \cap \Omega(f)$

$(f(n) > 0 \wedge, c_1 \leq, c_2) \implies (0 \leq c_1 f(n) \leq g(n) \wedge 0 \leq g(n) \leq c_2 f(n))$

$\implies \Theta(f) \subseteq O(f) \cap \Omega(f)$

$\implies \Theta(f) = O(f) \cap \Omega(f)$

□

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = 0 \implies f(n) \in o(g(n))$$

Proof. From limit definition, $\forall \epsilon > 0, \exists n_0 \geq 0, \forall n \geq n_0, \frac{f(n)}{g(n)} < \epsilon$. Let $c = \epsilon, \frac{f(n)}{g(n)} < \epsilon, f(n) < c \cdot g(n)$. $f(n) \geq 0, f(n) \in o(g(n))$ □

$$\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty \implies f(n) \in \omega(g(n))$$

Proof. From $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty \iff \lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = 0$, as
 $\lim_{x \rightarrow \infty} \frac{1}{f(x)} = \frac{1}{\lim_{x \rightarrow \infty} f(x)}$, $\lim_{n \rightarrow \infty} \frac{g(n)}{f(n)} = \frac{1}{\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)}} = \frac{1}{\infty} = 0$.
 Similarly, $\forall \epsilon > 0, \exists n_0 \geq 0, \forall n \geq n_0, \frac{g(n)}{f(n)} < \epsilon$. Let $c = \frac{1}{\epsilon}$, $\frac{g(n)}{f(n)} < \epsilon, c \cdot g(n) < f(n)$. □

$$0 < \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty \implies f(n) \in \Theta(g(n))$$

Proof. Definitions:

$\Theta(g) : \{f : \exists c_1, c_2 > 0 \wedge n_0 > 0, \forall n \geq n_0, c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)\}$

$O(g) : \{f : \exists c > 0 \wedge n_0 > 0, \forall n \geq n_0, 0 \leq f(n) \leq c \cdot g(n)\}$

$o(g) : \{f : \forall c > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq f(n) < c \cdot g(n)\}$

Prove: $\Theta(f) = O(f) \setminus o(f)$

Proof. 1. $\Theta(f) \subseteq O(f) \setminus o(f)$

Suppose $g(n) \in \Theta(f), \Theta(f) = O(f) \cap \Omega(f) \implies g(n) \in O(f)$.

Suppose $g(n) \in o(f), (\exists c_1, c_2 > 0, \exists n_1 > 0, \forall n \geq n_1, 0 \leq c_1 \cdot f(n) \leq g(n) \leq c_2 \cdot f(n)) \wedge (\forall c_3 > 0, \exists n_2 > 0, \forall n \geq n_2, 0 \leq g(n) < c_3 \cdot f(n))$

Let $c_3 < c_1, c_1 \cdot f(n) \leq g(n) < c_3 \cdot f(n)$, contradiction,

$\implies g(n) \in \Theta(f) \rightarrow g(n) \notin o(f)$

$\implies \Theta(f) \subseteq O(f) \setminus o(f)$

2.

$$O(f) \setminus o(f) \subseteq \Theta(f) =$$

$$\begin{aligned} & \{g : (\exists c_1 > 0, \exists n_1 > 0, \forall n \geq n_1, 0 \leq g(n) \leq c_1 \cdot f(n)) \\ & \quad \wedge \neg(\forall c_2 > 0, \exists n_2 > 0, \forall n \geq n_2, 0 \leq g(n) < c_2 \cdot f(n))\} \\ & \{g : (\exists c_1 > 0, \exists n_1 > 0, \forall n \geq n_1, 0 \leq g(n) \leq c_1 \cdot f(n)) \\ & \quad \wedge (\exists c_2 > 0, \forall n_2 > 0, \exists n \geq n_2, (g(n) < 0 \vee g(n) \geq c_2 \cdot f(n)))\} \\ & \{g : \exists c_1 > c_2 > 0, \forall n_2 > 0, \exists n_1 \geq n_2, \forall n \geq n_1, 0 \leq c_2 \cdot f(n) \leq g(n) \leq c_1 \cdot f(n)\} \\ & \{g : \exists c_1 > c_2 > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq c_2 \cdot f(n) \leq g(n) \leq c_1 \cdot f(n)\} \end{aligned}$$

□

□

1.2 Reflexivity

1.2.1 $f(n) \in O(f(n))$

Proof. Suppose $f(n) > 0$

Let $c = 1, n_0 = 1, \forall n \geq n_0, 0 \leq f(n) \leq f(n), \implies f(n) \in O(f(n))$ □

1.2.2 $f(n) \in \Omega(f(n)), f(n) \in \Theta(f(n))$

Similarly, by picking the right c and n_0 it's easy to prove existential statements.

1.3 Transitivity

1.3.1 $f(n) \in O(g(n)) \wedge g(n) \in O(h(n)) \implies f(n) \in O(h(n))$

Proof.

$$\exists c_1 > 0, n_1 > 0, \forall n \geq n_1, 0 \leq f(n) \leq c_1 \cdot g(n)$$

$$\exists c_2 > 0, n_2 > 0, \forall n \geq n_2, 0 \leq g(n) \leq c_2 \cdot h(n)$$

Let $c_1 = 1, n_0 = \max(n_1, n_2), 0 \leq f(n) \leq g(n) \leq c_2 \cdot h(n) \implies \exists c_2 > 0, \forall n \geq n_0, 0 \leq f(n) \leq c_2 \cdot h(n) \implies f(n) \in O(h(n))$ □

1.3.2 $f(n) \in \Omega(g(n)) \wedge g(n) \in \Omega(h(n)) \implies f(n) \in \Omega(h(n))$

Similar to $O(f(n))$ proof

1.3.3 $f(n) \in \Theta(g(n)) \wedge g(n) \in \Theta(h(n)) \implies f(n) \in \Theta(h(n))$

$$\exists c_1^{[1]}, c_2^{[1]} > 0, n_1 > 0, \forall n \geq n_1, 0 \leq c_1^{[1]} \cdot g(n) \leq f(n) \leq c_2^{[1]} \cdot g(n)$$

$$\exists c_1^{[2]}, c_2^{[2]} > 0, n_2 > 0, \forall n \geq n_2, 0 \leq c_1^{[2]} \cdot h(n) \leq g(n) \leq c_2^{[2]} \cdot h(n)$$

Let $c_1^{[1]} = c_2^{[1]} = 1, f(n) = g(n)$,

$0 \leq c_1^{[2]} \cdot h(n) \leq f(n) \leq c_2^{[2]} \cdot h(n) \implies f(n) \in \Theta(h(n))$

1.3.4 $f(n) \in o(g(n)) \wedge g(n) \in o(h(n)) \implies f(n) \in o(h(n))$

Proof.

$$\forall c_1 > 0, \exists n_1 > 0, \forall n \geq n_1, 0 \leq f(n) < c_1 \cdot g(n) \quad (1)$$

$$\forall c_2 > 0, \exists n_2 > 0, \forall n \geq n_2, 0 \leq g(n) < c_2 \cdot h(n) \quad (2)$$

From (2), $\forall c_2 > 0, \exists n_2 > 0, \forall n \geq n_2, 0 \leq c_1 \cdot g(n) < c_1 \cdot c_2 \cdot h(n)$

$$\forall c_1, c_2 > 0, \exists n_1, n_2 > 0, \forall n \geq \max(n_1, n_2), 0 \leq f(n) < c_1 \cdot g(n) < c_1 \cdot c_2 \cdot h(n)$$

$$\forall c_1, c_2 > 0, \exists n_1, n_2 > 0, \forall n \geq \max(n_1, n_2), 0 \leq f(n) < c_1 \cdot c_2 \cdot h(n)$$

□

1.3.5 $f(n) \in \omega(g(n) \wedge g(n) \in \omega(h(n))) \implies f(n) \in \omega(h(n))$

$$\forall c_1 > 0, \exists n_1 > 0, \forall n \geq n_1, 0 \leq c_1 \cdot g(n) < f(n)$$

$$\forall c_2 > 0, \exists n_2 > 0, \forall n \geq n_2, 0 \leq c_2 \cdot h(n) < g(n)$$

$$\forall c_1, c_2 > 0, \exists n_1, n_2 > 0, \forall n \geq \max(n_1, n_2), 0 \leq c_1 \cdot c_2 \cdot h(n) < c_1 \cdot g(n) < f(n)$$

1.4 Symmetry: $f(n) \in \Theta(g(n)) \iff g(n) \in \Theta(f(n))$

Proof.

$$f(n) \in \Theta(g(n)) \implies$$

$$\exists c_1, c_2 > 0, n_1 > 0, \forall n \geq n_1, 0 \leq c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n) \implies$$

$$\left(0 \leq g(n) \leq \frac{1}{c_1} \cdot f(n) \leq \frac{c_2}{c_1} \cdot g(n)\right) \wedge \left(0 \leq \frac{c_1}{c_2} \cdot g(n) \leq \frac{1}{c_2} \cdot f(n) \leq g(n)\right) \implies$$

$$0 \leq \frac{1}{c_2} \cdot f(n) \leq g(n) \leq \frac{1}{c_1} \cdot f(n)$$

Identical proof for the other direction

□

1.5 Complementary

1.5.1 $f(n) \in O(g(n)) \iff g(n) \in \Omega(f(n))$

Proof.

$$f(n) \in O(g(n)) \implies$$

$$\exists c > 0 \wedge n_0 > 0, \forall n \geq n_0, 0 \leq f(n) \leq c \cdot g(n) \implies$$

$$0 \leq \frac{1}{c} \cdot f(n) \leq g(n)$$

Proof for the other direction and the next statement are identical.

□

1.5.2 $f(n) \in o(g(n)) \iff g(n) \in \omega(f(n))$

1.6 Problems

$3^{n+1} \in O(3^n)$

Proof.

$$3^{n+1} = 3 \cdot 3^n$$

$$\text{let } c = 4, n_0 = 1, \forall n \geq n_0,$$

$$3^{n+1} < c \cdot 3^n$$

□

$4^n \notin O(2^n)$

Proof. Suppose $f(n) = 4^n \in O(2^n)$,

$$\exists c > 0, n_0 > 0, \forall n \geq n_0, 0 \leq 2^{2 \times n} \leq c \cdot 2^n \implies$$

$$0 \leq 2^n \leq c$$

$$0 \leq n \leq \log_2 c(\text{const})$$

n is upper bounded by a constant, a contradiction

□

$$2^{\lfloor \log_2 n \rfloor} \in \Theta(n)$$

Proof.

$$\text{Let } k = \lfloor \log_2 n \rfloor, 2^k \leq n < 2^{k+1} = 2 \cdot 2^k$$

$$\text{Let } c_1 = \frac{1}{2}, c_2 = 1, n_0 = 2,$$

$$\frac{n}{2} < 2^k \leq n$$

□

For constant i , $a > 0$, $(n + a)^i \in O(n^i)$

Proof.

$$\forall a > 0, \exists n_0 > a, \forall n \geq n_0, f(n) = (n + a)^i \leq (2n)^i = 2^i \cdot n^i,$$

$$\text{Let } c = 2^i, n_0 = a + 1, \forall n \geq n_0, 0 \leq f(n) \leq c \cdot n^i$$

□

Prove: $2^{\log_2 n} \in O(n)$

$$\text{Proof. Let } c = 2, n_0 = 2, \forall n \geq n_0, 2^{\log_2 n} = n \implies 0 \leq n \leq 2n$$

□

Prove: $2^{\log_2 n} \in \Omega(n)$

$$\text{Proof. Let } c = 0.5, n_0 = 2, \forall n \geq n_0, 2^{\log_2 n} = n \implies 0 \leq 0.5n \leq n$$

□

Prove: $2^{\log_2 n} \notin \Theta(\sqrt{n})$

$$\text{Proof. Suppose otherwise, } \exists c_1, c_2 > 0, n_0 > 0, \forall n \geq n_0, 0 \leq c_1 \cdot \sqrt{n} \leq n \leq c_2 \sqrt{n},$$

$$c_1 \leq \sqrt{n} \leq c_2$$

However, $\lim_{n \rightarrow \infty} \sqrt{n} = \infty$, thus cannot be bound by constant c_2 , a contradiction.

□

Prove: $2^{\log_2 n} \notin \omega(n)$

Proof. Suppose otherwise, $\forall c > 0, \exists n_0 > 0, \forall n \geq n_0, 0 \leq c \cdot n < n$. When $c = 2, \forall n_0 > 0, \forall n \geq n_0, 2n \geq n$, a contradiction. □

Rank order of growth

$$f_1(n) = \log n$$

By reflexivity,

$$f_1(n) \in \Theta(\log n)$$

$$f_2(n) = n!$$

By reflexivity,

$$f_2(n) \in \Theta(n!)$$

$$f_3(n) = 2^n + n$$

$$f_3(n) \in \Theta(2^n)$$

Proof.

□

$$f_4(n) = n^{2.3} + 16n$$

$$f_4(n) \in \Theta(n^{2.3})$$

Proof.

□

$$f_5(n) = \log(n^2)$$

$$f_5(n) \in \Theta(\log(n))$$

Proof. Assume base 2, $f_5(n) = \log(n^2) = 2\log(n)$, let $c_1 = 1, c_2 = 3, n_0 = 2$,
 $\forall n \geq n_0, 0 \leq \log(n) \leq \log(n) \leq 3\log(n)$ □

$$\Theta(\log(n)) \subseteq o(n^{2.3}) \subseteq o(2^n) \subseteq o(n!)$$

Proof.

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{\log(n)}{n^{2.3}} &= \lim_{n \rightarrow \infty} \frac{(\log(n))'}{(n^{2.3})'} = \lim_{n \rightarrow \infty} \frac{1}{\ln(2)n^{1.3}} = 0, \\ \implies \forall f(n) \in \Theta(\log(n)), f(n) &\in o(n) \end{aligned}$$

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{n^{2.3}}{2^n} &= \lim_{n \rightarrow \infty} \frac{2.3 \ln(n)}{n \cdot \ln(2)} = \lim_{n \rightarrow \infty} \frac{2.3}{n \cdot \ln(2)} = 0, \\ \implies \forall f(n) \in \Theta(n^{2.3}), f(n) &\in o(2^n) \end{aligned}$$

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{2^n}{n!} &= \lim_{n \rightarrow \infty} \frac{\prod_{r=1}^n 2}{\prod_{r=1}^n r} = 0, \forall n > 2 \\ \implies \forall f(n) \in \Theta(2^n), f(n) &\in o(n!) \end{aligned}$$

□

1.7 Math utils

$$\begin{aligned} a^{\log_b n} &= n^{\log_b a} \\ \log_a b &= \frac{\log_c b}{\log_c a} \end{aligned}$$

Close form formulae for $\sum_{i=1}^n i^k$

$$\begin{aligned} \sum_{i=1}^n i &= \frac{n(n+1)}{2}, \\ \sum_{i=1}^n i^2 &= \frac{n(n+1)(2n+1)}{6}, \\ \sum_{i=1}^n i^3 &= \left(\frac{n(n+1)}{2} \right)^2, \\ \sum_{i=1}^n i^4 &= \frac{n(n+1)(2n+1)(3n^2+3n-1)}{30}, \\ \sum_{i=1}^n i^5 &= \frac{n^2(n+1)^2(2n^2+2n-1)}{12}. \end{aligned}$$

$$\sum_{i=1}^n i^p = \frac{1}{p+1} \sum_{j=0}^p (-1)^j \binom{p+1}{j} B_j n^{p+1-j}, \quad p \in \mathbb{N}$$

where B_j are the Bernoulli numbers. (<https://en.wikipedia.org/wiki/Faulhaber>)

Example

$$\begin{aligned}\sum_{i=1}^k i^8 &= \frac{1}{9} \sum_{j=0}^8 (-1)^j \binom{9}{j} B_j \\ &= \frac{1}{9} \left(\binom{9}{0} (1)k^9 - \binom{9}{1} \left(\frac{1}{2}\right)k^8 + \dots + \binom{9}{8} \left(-\frac{1}{30}\right)k \right) \\ h(k) &= \sum_{i=1}^k i^8 \in O(k^9)\end{aligned}$$

n	B_n	n	B_n	n	B_n	n	B_n
0	1	5	0	10	$\frac{5}{66}$	15	0
1	$-\frac{1}{2}$	6	$\frac{1}{42}$	11	0	16	$-\frac{3617}{510}$
2	$\frac{1}{6}$	7	0	12	$-\frac{691}{2730}$	17	0
3	0	8	$-\frac{1}{30}$	13	0	18	$\frac{43867}{798}$
4	$-\frac{1}{30}$	9	0	14	$\frac{7}{6}$	19	0

Table 1: First 20 Bernoulli numbers B_n

1.8 Techniques

Log trick:

$$\log\left(\prod_{i=1}^n f(x)\right) = \sum_{i=1}^n \log(f(x))$$

Harmonic series

$$\begin{aligned}H_n &= \sum_{i=1}^n \frac{1}{i} = \ln(n) + \gamma + \Theta\left(\frac{1}{n}\right) \\ H_n^{(p)} &= \sum_{i=1}^n \frac{1}{i^p} \quad \text{converges}\end{aligned}$$

n	γ_n	n	γ_n	n	γ_n	n	γ_n
1	1.000000	6	0.658251	11	0.257651	16	0.254040
2	0.806853	7	0.647998	12	0.256744	17	0.253542
3	0.734722	8	0.640544	13	0.255939	18	0.253095
4	0.697123	9	0.634774	14	0.255228	19	0.252691
5	0.673343	10	0.630331	15	0.254599	20	0.252322

Table 2: First 20 approximations $\gamma_n = H_n - \ln n$ to the Euler–Mascheroni constant

For example, for recurrence $T(n) = T(n-1) + \frac{1}{n}$, $T(n) \in \Theta(\log(n))$. If the recurrence have terms like $\frac{n}{(\log(n))^p}$, it usually connects to $H_n^{(p)}$ where $T \rightarrow \dots \rightarrow S(m) = S(m-1) + \frac{1}{m}$
 Sterling formula:
 $n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$

2 Recurrence

2.1 master theorem

Theorem 2.1 (Master Method). *Consider the recurrence*

$$T(n) = aT\left(\frac{n}{b}\right) + f(n),$$

where a, b are constants. Then:

(A) If $f(n) = O(n^{\log_b a - \varepsilon})$ for some constant $\varepsilon > 0$, then

$$T(n) = O(n^{\log_b a}).$$

(B) If $f(n) = \Theta(n^{\log_b a})$, then

$$T(n) = \Theta(n^{\log_b a} \log n).$$

(C) If $f(n) = \Omega(n^{\log_b a + \varepsilon})$ for some constant $\varepsilon > 0$, and if f satisfies the smoothness condition

$$af(n/b) \leq cf(n) \quad \text{for some constant } c < 1,$$

then

$$T(n) = \Theta(f(n)).$$

2.1.1 Common recurrence table

a	b	$p = \log_b a$	n^{-1}	$\log n$	n	$n \log n$	n^2	n^3
1	2	0	$\Theta(1)$	$\Theta(\log^2 n)$	$\Theta(n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(n^3)$
2	2	1	$\Theta(n)$	$\Theta(n)$	$\Theta(n \log n)$	$\Theta(n \log^2 n)$	$\Theta(n^2)$	$\Theta(n^3)$
3	2	$\log_2 3$	$\Theta(n^{\log_2 3})$	$\Theta(n^{\log_2 3})$	$\Theta(n^{\log_2 3})$	$\Theta(n^{\log_2 3})$	$\Theta(n^2)$	$\Theta(n^3)$
4	2	2	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2 \log n)$	$\Theta(n^3)$
1	3	0	$\Theta(1)$	$\Theta(\log^3 n)$	$\Theta(n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(n^3)$
2	3	$\log_3 2$	$\Theta(n^{\log_3 2})$	$\Theta(n^{\log_3 2})$	$\Theta(n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(n^3)$
3	3	1	$\Theta(n)$	$\Theta(n)$	$\Theta(n \log n)$	$\Theta(n \log^2 n)$	$\Theta(n^2)$	$\Theta(n^3)$
4	3	$\log_3 4$	$\Theta(n^{\log_3 4})$	$\Theta(n^{\log_3 4})$	$\Theta(n^{\log_3 4})$	$\Theta(n^{\log_3 4})$	$\Theta(n^2)$	$\Theta(n^3)$
1	4	0	$\Theta(1)$	$\Theta(\log^4 n)$	$\Theta(n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(n^3)$
2	4	$\frac{1}{2}$	$\Theta(n^{\log_4 2})$	$\Theta(n^{\log_4 2})$	$\Theta(n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(n^3)$
3	4	$\log_4 3$	$\Theta(n^{\log_4 3})$	$\Theta(n^{\log_4 3})$	$\Theta(n)$	$\Theta(n \log n)$	$\Theta(n^2)$	$\Theta(n^3)$
4	4	1	$\Theta(n)$	$\Theta(n)$	$\Theta(n \log n)$	$\Theta(n \log^2 n)$	$\Theta(n^2)$	$\Theta(n^3)$

Master Theorem: Worked Examples (3 Cases)

Hint. Let $p = \log_b a$. Compare $f(n)$ to n^p :

Case 1: $f(n) = O(n^{p-\varepsilon}) \Rightarrow T(n) = \Theta(n^p)$.

Case 2: $f(n) = \Theta(n^p \log^k n) \Rightarrow T(n) = \Theta(n^p \log^{k+1} n)$.

Case 3: $f(n) = \Omega(n^{p+\varepsilon})$ and $af(n/b) \leq cf(n)$ for some $c < 1 \Rightarrow T(n) = \Theta(f(n))$.

Case 1: subpolynomially smaller than n^p .

- **1A.** $T(n) = 4T(n/2) + n$. Here $a = 4, b = 2 \Rightarrow p = \log_2 4 = 2$. Since $f(n) = n = O(n^{2-\varepsilon})$ (take $\varepsilon = 1$),

$$T(n) = \Theta(n^2).$$

- **1B.** $T(n) = 2T(n/2) + \sqrt{n}$. $p = \log_2 2 = 1$. $f(n) = n^{1/2} = O(n^{1-\varepsilon})$ with $\varepsilon = 1/2$. $T(n) = \Theta(n)$.

- **1C.** $T(n) = 8T(n/4) + n \log n$. $p = \log_4 8 = \frac{3}{2}$. Since $\log n = O(n^{0.25})$, $n \log n = O(n^{1.5-0.25})$. $T(n) = \Theta(n^{3/2})$.

Case 2: same polynomial order (polylog factor).

- **2A.** $T(n) = 2T(n/2) + n$ (mergesort). $p = 1$, $f(n) = n = \Theta(n^p)$ ($k = 0$). $T(n) = \Theta(n \log n)$.

- **2B.** $T(n) = 9T(n/3) + n^2$. $p = \log_3 9 = 2$, $f(n) = n^2 = \Theta(n^p)$ ($k = 0$). $T(n) = \Theta(n^2 \log n)$.

- **2C.** $T(n) = 4T(n/2) + n^2 \log^3 n$. $p = 2$, $f(n) = n^2 \log^3 n = \Theta(n^p \log^3 n)$. $T(n) = \Theta(n^2 \log^4 n)$.

Case 3: polynomially larger than n^p (with regularity).

- **3A.** $T(n) = 2T(n/2) + n^2$. $p = 1$. $f(n) = n^2 = \Omega(n^{1+\varepsilon})$ with $\varepsilon = 1$. Regularity: $2 \cdot (n/2)^2 = n^2/2 \leq cn^2$ for $c = \frac{1}{2} < 1$. $\boxed{T(n) = \Theta(n^2)}$.
- **3B.** $T(n) = 3T(n/2) + n^2$. $p = \log_2 3 \approx 1.585$. $f(n) = n^2 = \Omega(n^{p+\varepsilon})$ with $\varepsilon \approx 0.415$. Regularity: $3 \cdot (n/2)^2 = \frac{3}{4}n^2$ ($c = 0.75 < 1$). $\boxed{T(n) = \Theta(n^2)}$.
- **3C.** $T(n) = 4T(n/2) + n^3$. $p = 2$. $f(n) = n^3 = \Omega(n^{2+\varepsilon})$. Regularity: $4 \cdot (n/2)^3 = \frac{1}{2}n^3$ ($c = 0.5 < 1$). $\boxed{T(n) = \Theta(n^3)}$.

Borderline note. When $f(n) = n^p / \log n$ or similar, the vanilla Master Theorem does not apply directly; use extended forms (e.g., Akra–Bazzi).

2.2 Telescoping

For recurrence relation

$$T : \mathbb{N} \rightarrow \mathbb{R}, T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

1. Define operator $N_k : (N_k T)(n) = \frac{T(n)}{n^k}$, where $k = \log_b a$
2. Transform $T(n) \rightarrow R(n) := \frac{T(n)}{n^k}$
3. Obtain the recurrence relation of $R(n) = R\left(\frac{n}{b}\right) + \frac{f(n)}{n^k}$
4. Define map $E_b : \mathbb{Z} \rightarrow \mathbb{N}, E_b(m) = b^m$
5. Define operator $P_b : (P_b H)(m) = H(E_b(m)) = H(b^m)$
6. Transform $R(n) \rightarrow S(m) := P_b(R) = R \circ E_b, S(m) = R(b^m)$

$$\begin{array}{ccccc} T(n) & \xrightarrow{N_k} & R(n) & \xrightarrow{P_b} & S(m) \\ \downarrow n \mapsto n/b & & \downarrow n \mapsto n/b & & \downarrow m \mapsto m-1 \\ a \cdot T\left(\frac{n}{b}\right) + f(n) & \longleftrightarrow & R\left(\frac{n}{b}\right) + \frac{f(n)}{n^k} & \longleftrightarrow & S(m-1) + \frac{f(b^{m-1})}{b^{(m-1)k}} \end{array}$$

$$\begin{aligned} T(n) &= n^k R(n) \in \Theta(n^k H(n)) \leftarrow R(n) \in \Theta(G(\log_b n)) = H(n) \\ \leftarrow S(m) &\in \Theta(S(0) + \sum_{i=1}^m \frac{f(b^i)}{b^{i \cdot k}}) = G(m) \end{aligned}$$

$$T(n) \sim n^{\log_b a} \sum_{i=1}^{\lfloor \log_b n \rfloor} \frac{f(b^i)}{b^{i \log_b a}}$$

2.2.1 Normalization: $T(n) = T(n^{\frac{1}{c}})$

$$T(n) = AT(\sqrt[n]{n}) + B \quad \text{include term } \log \log(n)$$

Consider recurrences of the form

$$T(n) = n^a T(n^{1/c}) + f(n),$$

where $a < 1$ and $c > 1$. A useful technique is to normalize the recurrence by dividing out the dominant power of n .

Step 1: Normalize. Define

$$R(n) = \frac{T(n)}{n},$$

so that

$$R(n) = \frac{T(n)}{n} = \frac{n^a T(n^{1/c})}{n} + \frac{f(n)}{n} = n^{a-1} T(n^{1/c}) + \frac{f(n)}{n} = \frac{T(n^{1/c})}{n^{1/c}} + \frac{f(n)}{n} = R(n^{1/c}) + \frac{f(n)}{n}.$$

For the special case $f(n) = n$, this simplifies to

$$R(n) = R(n^{1/c}) + 1.$$

Step 2: Change variables. Let $n = b^m$ (e.g. $b = c$), and define

$$S(m) = R(b^m),$$

which yields

$$S(m) = S\left(\frac{m}{c}\right) + 1.$$

Step 3: Linearize the recurrence. Let $m = c^k$ and define $U(k) = S(c^k)$, giving the simple recurrence

$$U(k) = U(k-1) + 1,$$

whose solution is

$$U(k) = \Theta(k).$$

Step 4: Substitute back. Since $k = \Theta(\log m)$ and $m = \Theta(\log n)$, we obtain

$$R(n) = \Theta(\log \log n), \quad T(n) = n \cdot R(n) = \Theta(n \log \log n).$$

This method cleanly converts non-standard recurrences of the form $T(n^{1/c})$ into linear recurrences in $\log \log n$.
Examples:

Solving $T(n) = n^{2/3}T(n^{1/3}) + n$ via Normalization

We start with the recurrence

$$T(n) = n^{2/3}T(n^{1/3}) + n.$$

Step 1: Normalize. Define

$$R(n) = \frac{T(n)}{n}.$$

Then

$$R(n) = \frac{T(n)}{n} = \frac{n^{2/3}T(n^{1/3})}{n} + 1 = \frac{T(n^{1/3})}{n^{1/3}} + 1 = R(n^{1/3}) + 1.$$

Thus the recurrence simplifies to

$$R(n) = R(n^{1/3}) + 1.$$

Step 2: Logarithmic substitution. Let

$$n = 3^m, \quad S(m) = R(3^m),$$

so that

$$S(m) = S\left(\frac{m}{3}\right) + 1.$$

Step 3: Linearize. Let

$$m = 3^k, \quad U(k) = S(3^k),$$

giving the simple recurrence

$$U(k) = U(k-1) + 1.$$

Hence $U(k) = \Theta(k)$.

Step 4: Substitute back. Since

$$k = \Theta(\log m), \quad m = \Theta(\log n),$$

we obtain

$$R(n) = S(\log n) = \Theta(\log \log n).$$

Finally,

$$T(n) = n R(n) = \Theta(n \log \log n).$$

Thus the recurrence

$$T(n) = n^{2/3}T(n^{1/3}) + n$$

has solution

$$T(n) = \Theta(n \log \log n).$$

2.2.2 The Iterated Logarithm $\log^* n$

The *iterated logarithm* $\log^* n$ is defined as the number of times the logarithm function must be applied before the value drops to 1 or below. Formally,

$$\log^* n = \begin{cases} 0, & n \leq 1, \\ 1 + \log^*(\log n), & n > 1. \end{cases}$$

The base of the logarithm is not important for asymptotics; we typically use base 2.

For example,

$$\log^*(2^{65536}) = 5,$$

since

$$2^{65536} \xrightarrow{\log} 65536 \xrightarrow{\log} 16 \xrightarrow{\log} 4 \xrightarrow{\log} 2 \xrightarrow{\log} 1.$$

The function $\log^* n$ grows extremely slowly—slower than $\log \log \log n$ —and remains below 5 for all practical values of n .

Recurrences involving $\log^* n$. The iterated logarithm commonly appears in divide-and-reduce recurrences where the input size decreases by repeated logarithms or exponentials. Typical examples include:

- Recurrence of the form

$$T(n) = T(\log n) + 1 \implies T(n) = \Theta(\log^* n).$$

- More generally,

$$T(n) = T(n^{1/c}) + 1,$$

when unrolled repeatedly, produces a sequence $n, n^{1/c}, n^{1/c^2}, \dots$ whose length is $\Theta(\log^* n)$, hence

$$T(n) = \Theta(\log^* n).$$

- Recurrences with decreasing exponentials,

$$T(n) = T(\sqrt{n}) + 1,$$

also yield

$$T(n) = \Theta(\log^* n).$$

The function $\log^* n$ thus arises naturally in algorithms whose input size shrinks by repeated logarithmic or root operations, such as union-find with path compression or multi-level divide-and-conquer schemes.

2.2.3 Comparison

$$\alpha(n) \ll \log^* n \ll \log \log n \ll (\log n)^k \ll n^\varepsilon \ll n \ll n^k \ll 2^n \ll n! \ll n^n \ll 2^{2^n}.$$

more

$$1 \ll \alpha(n) \ll \log^* n \ll \log \log \log n \ll \log \log n \ll \log n \ll (\log n)^k \ll e^{(\log n)^\alpha} \ (0 < \alpha < 1) \ll n^\varepsilon \ll n \ll n^k \ll n^{\log n} \ll 2^{\sqrt{n}} \ll 2^n \ll n! \ll n^n \ll 2^{2^n}.$$

Problems

$$T(n) = 4T\left(\frac{n}{4}\right) + \frac{n}{\log(n)}$$

$$\text{Define } R(n) = \frac{T(n)}{n},$$

$$\begin{aligned} R(n) &= \frac{4 \cdot T\left(\frac{n}{4}\right)}{n} + \frac{1}{\log(n)} \\ &= R\left(\frac{n}{4}\right) + \frac{1}{\log(n)} \end{aligned}$$

$$\text{Define } S(m) = R(4^m),$$

$$\begin{aligned} S(m) &= S(m-1) + \frac{4^m}{4^{m-1} \log_4 4} \\ &= S(m-1) + \frac{1}{m \log(4)} \end{aligned}$$

$$S(m) \sim \frac{1}{\log(4)} \sum_{i=1}^m \frac{1}{i}$$

$$S(m) \in \Theta(\log m)$$

Map back to $R(n)$ and $T(n)$,

$$\begin{aligned} S(m) \in \Theta(\log m) &\implies R(n) \in \Theta(\log(\log_4 n)) \\ &\implies T(n) \in n \log(\log(n)) \end{aligned}$$

$$T(n) = 4T\left(\frac{n}{2}\right) + \sqrt{n}$$

Define $R(n) = \frac{T(n)}{n^2}$,

$$\begin{aligned} R(n) &= \frac{4 \cdot T\left(\frac{n}{2}\right)}{n^2} + \frac{\sqrt{n}}{n^2} \\ &= R\left(\frac{n}{2}\right) + n^{-\frac{3}{2}} \end{aligned}$$

Define $S(m) = R(2^m)$,

$$\begin{aligned} S(m) &= S(m-1) + \frac{(2^m)^{-\frac{1}{2}}}{2^{m \cdot \log_2 4}} \\ &= S(m-1) + \frac{2^{-\frac{m}{2}}}{2^{2m}} \\ &= S(m-1) + \frac{1}{2^{\frac{3m}{2}}} \\ S(m) &\sim \sum_{i=1}^m \frac{1}{2^{\frac{3i}{2}}}, \text{ clearly converges} \\ S(m) &\in \Theta(1) \end{aligned}$$

2 Maps back,

$$\begin{aligned} S(m) \in \Theta(1) &\implies R(n) \in \Theta(1) \\ &\implies T(n) \in \Theta(n^2) \end{aligned}$$

$$T(n) = T\left(\frac{n}{2}\right) + 2\sqrt{\log n}$$

1. Using telescoping

$a = 0 \rightarrow \log_b a = 0$, there is no need for $T(n) \mapsto R(n)$.

Define $S(m) = T(2^m)$, we have:

$$\begin{aligned} S(m) &= S(m-1) + 2\sqrt{\log 2^m} \\ &= S(m-1) + 2\sqrt{m} \\ S(m) &\sim \sum_{i=1}^m 2^{\sqrt{i}} \approx \int_1^m 2^{\sqrt{x}} dx \end{aligned}$$

Solve for $\int_1^m 2^{\sqrt{x}} dx$ by substitution: let $u = x^{\frac{1}{2}}$

$$\begin{aligned} \frac{du}{dx} &= \frac{1}{2} x^{-\frac{1}{2}} \\ dx &= \frac{1}{2u} \\ \int 2^{\sqrt{x}} dx &= \int 2u \cdot 2^u du = 2 \int u 2^u du \\ &= \frac{2^u (2u \ln(2) - 1)}{(\ln(2))^2} + C \\ &= \frac{2^{\sqrt{x}} (2\sqrt{x} \ln(2) - 1)}{(\ln(2))^2} + C \\ \int_1^m 2^{\sqrt{x}} dx &= \frac{1}{(\ln(2))^2} \left(2^{\sqrt{m}} (2\sqrt{m} \ln(2) - 1) \right) \end{aligned}$$

Ignoring the constants, we have

$$\begin{aligned} S(m) &\sim \sum_{i=1}^m 2^{\sqrt{i}} \approx c \cdot 2^{\sqrt{m}} \sqrt{m} \\ T(n) &\in \Theta(\sqrt{\log n} \cdot 2^{\sqrt{\log n}}) \end{aligned}$$

Recurrence relation of merging k arrays of size n

$$T(k, n) = 2T\left(\frac{k}{2}, n\right) + \frac{nk}{2}$$

$$T\left(\frac{k}{2}, n\right) = 2T\left(\frac{k}{4}, n\right) + \frac{nk}{4}$$

Observed that the non-recursive function can be tightly bounded by nk ,

$$T(k, n) = 2T\left(\frac{k}{2}, n\right) + cnk$$

With respect to k , Define $R(k, n) = \frac{T(k, n)}{k}$

$$\begin{aligned} R(k, n) &= \frac{2T\left(\frac{k}{2}, n\right)}{k} + cn \\ &= R\left(\frac{k}{2}, n\right) + cn \end{aligned}$$

Define $S(m, n) = R(2^m, n)$,
 $f(k, n) = cnk \rightarrow f(2^m, n) = cn2^m$,

$$\begin{aligned} S(m, n) &= S(m-1, n) + \frac{cn2^m}{2^m} \\ &\sim cn \sum_{i=1}^m 1 \\ S(m, n) &\in \Theta(nm) \end{aligned}$$

Maps back to $R(n)$,

$$\begin{aligned} S(m, n) \in \Theta(n) &\implies R(k, n) \in \Theta(n \log(k)) \\ &\implies T(k, n) \in \Theta(nk \log(k)) \end{aligned}$$

2.3 WA1 Practice on Asymptotic analysis and recurrence

Asymptotic Analysis

2.3.1 $f(n) = 3n + 1$, $g(n) = 4n$

$f(n) \in \Theta(g(n))$

Proof. $\Theta(g) : \{f : \exists c_1, c_2 > 0 \wedge n_0 > 0, \forall n \geq n_0, c_1 \cdot g(n) \leq f(n) \leq c_2 \cdot g(n)\}$

Let $c_1 = \frac{3}{4}, c_2 = 1, n_0 = 1$, trivial □

2.3.2 $f(n) = 3^n$, $g(n) = 2^n \cdot n^{10000}$

$f(n) \in \omega(g(n))$

Proof. $\lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \lim_{n \rightarrow \infty} \frac{3^n}{2^n \cdot n^{10000}} = \lim_{n \rightarrow \infty} \frac{1.5^n}{n^{10000}} = \lim_{n \rightarrow \infty} \frac{\ln(1.5)1.5^n}{10000n^{9999}}$
 $= \dots = \lim_{n \rightarrow \infty} \frac{(\ln 1.5)^{10000} 1.5^n}{10000!} = \infty \implies f(n) \in \omega(g(n))$ □

2.3.3 $f(n) = 2^n$, $g(n) = 2^{n+1}$

$f(n) \in \Theta(g(n))$

Proof. $g(n) = 2 \cdot 2^n$, let $c_1 = \frac{1}{2}, c_2 = 1, n_0 = 2$,
 $\forall n \geq n_0, 1 \cdot 2^n \leq 2^n \leq 2 \cdot 2^n \implies f(n) \in \Theta(g(n))$ □

2.3.4 $f(n) = 2^{(2^n)}$, $g(n) = 2^{(2^{n+1})}$

$f(n) \in o(g(n))$

Proof. Let $u = 2^n$, $f(n) = 2^u$, $g(n) = 2^{2 \cdot u}$, $g(n) = (f(n))^2, \forall n \in \mathbb{Z}^+, f(n) > 1$,
 $f(n) < (f(n))^2, 0 < f(n) < g(n) \implies f(n) \in o(g(n))$ □

2.3.5 $f(n) = \sum_{i=1}^n \frac{1}{2^i}$, $g(n) = \sum_{i=1}^n \frac{i^{10000}}{2^i}$
 $f(n) \in \Theta(g(n))$

Proof. $f(n) = \frac{\frac{1}{2}(\frac{1}{2^n}-1)}{\frac{1}{2}-1} = 1 - \frac{1}{2^n}$, trivial to show this is tightly upper bounded by constant. $f(n) \in \Theta(1)$
Next we will show that the summation $g(n)$ converges.

$$\begin{aligned} \lim_{n \rightarrow \infty} \frac{\frac{n^{10000}}{2^n}}{\frac{(n-1)^{10000}}{2^{n-1}}} &= \lim_{n \rightarrow \infty} \frac{n^{10000}}{2(n-1)^{10000}} = \lim_{n \rightarrow \infty} \frac{10000n^{9999}}{20000(n-1)^{9999}} = \dots \\ &= \lim_{n \rightarrow \infty} \frac{10000!}{2(10000!)} = \frac{1}{2} < 1 \implies \sum_{i=1}^n \frac{i^{10000}}{2^i} \text{ converges} \end{aligned}$$

$g(n) \in \Theta(1) \implies h(n) \in \Theta(g(n))$ by symmetry $\implies f(n) \in \Theta(g(n))$ by transitivity. □

2.3.6 $f(n) = \log \log(n^n)$, $g(n) = \log \log(2^{\sqrt{n}})$
 $f(n) \in \Theta(g(n))$

Proof. $f(n) = \log(n \log(n)) = \log(n) + \log(\log(n))$
 $g(n) = \log(\sqrt{n} \log 2) = \log(\sqrt{n}) + \log(\log 2) = \frac{1}{2} \log(n) + \log \log(2)$

$$L = \frac{f(n)}{g(n)} = \frac{\log n + \log(\log n)}{0.5 \log n + \log(\log 2)} = \frac{1 + \frac{\log \log(n)}{\log(n)}}{\frac{1}{2} \frac{\log \log(2)}{\log(n)}}$$

$$\lim_{n \rightarrow \infty} L = 2$$

$$0 < \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} < \infty \implies f(n) \in \Theta(g(n))$$
□

2.3.7 $f(n) = 2^{\sqrt{\log n}}$, $g(n) = \sqrt{n}$
 $f(n) \in o(g(n))$

Proof. $L = \frac{f(n)}{g(n)}$, $\log(L) = \log(f(n)) - \log(g(n)) = \sqrt{\log n} \log(2) - \frac{1}{2} \log(n)$

$$= -\frac{1}{2} \log n \left(1 - \frac{\log 2 \sqrt{\log n}}{\log n}\right) = -\frac{1}{2} \log n \left(1 - \frac{\log 2}{\sqrt{\log n}}\right)$$

$$\lim_{n \rightarrow \infty} \log(L) = -\infty \implies \lim_{n \rightarrow \infty} L = e^{-\infty} = 0^+ \implies f(n) \in o(g(n))$$
□

2.3.8 $f(n) = 2^{\sqrt{\log n}}$, $g(n) = 2^{2^{\sqrt{\log \log n}}}$
 $f(n) \in \omega(g(n))$

Proof. Let $u = \log \log n$, $f(u) = 2^{\sqrt{2^u}}$, $g(u) = 2^{\sqrt{u}}$

$$L = \frac{f(u)}{g(u)} = \frac{2^{\sqrt{2^u}}}{2^{\sqrt{u}}}, \log(L) = \log(2)(\sqrt{2^u} - \sqrt{u}), \lim_{u \rightarrow \infty} \log(L) = \infty \implies \lim_{u \rightarrow \infty} L = \infty$$

$$u = \log \log n \text{ is monotonous and unbounded, } \lim_{u \rightarrow \infty} \frac{f(u)}{g(u)} = \infty \implies \lim_{n \rightarrow \infty} \frac{f(n)}{g(n)} = \infty \implies f(n) \in \omega(g(n))$$
□

2.3.9 $f(n) = n^{\frac{1}{\log n}}$, $g(n) = n^{\frac{1}{(\log n)^2}}$
 $f(n) \in \Theta(g(n))$

Proof. Let $L = \frac{f(n)}{g(n)} = n^{\frac{1}{\log n} - \frac{1}{(\log n)^2}} = n^{\frac{\log n - 1}{(\log n)^2}}$

$$\log(L) = \log n \left(\frac{\log n - 1}{(\log n)^2} \right) = \frac{\log n - 1}{\log n} = 1 - \frac{1}{\log n},$$

$$\lim_{n \rightarrow \infty} \log(L) = 1 \implies \lim_{n \rightarrow \infty} L = 2 \implies f(n) \in \Theta(g(n))$$
□

2.3.10 $f(n) = n^{\frac{1}{\log n}}$, $g(n) = n^{\frac{1}{\sqrt{\log n}}}$
 $f(n) \in o(g(n))$

Proof. Let $L = \frac{f(n)}{g(n)} = n^{\frac{1}{\log n} - \frac{1}{(\log n)^{\frac{1}{2}}}} = n^{\frac{1 - (\log n)^{\frac{1}{2}}}{\log n}}$

$$\log(L) = \log n \left(\frac{1 - \sqrt{\log n}}{\log n} \right) = 1 - \sqrt{\log n}$$

$$\lim_{n \rightarrow \infty} \log(L) = -\infty \implies \lim_{n \rightarrow \infty} L = 0^+ \implies f(n) \in o(g(n))$$
□

Recurrence

Alternative method proof:

For recurrence relation

$$T : \mathbb{N} \rightarrow \mathbb{R}, T(n) = a \cdot T\left(\frac{n}{b}\right) + f(n)$$

1. Define operator $N_k : (N_k T)(n) = \frac{T(n)}{n^k}$, where $k = \log_b a$
2. Transform $T(n) \rightarrow R(n) := \frac{T(n)}{n^k}$
3. Obtain the recurrence relation of $R(n) = R\left(\frac{n}{b}\right) + \frac{f(n)}{n^k}$
4. Define map $E_b : \mathbb{Z} \rightarrow \mathbb{N}, E_b(m) = b^m$
5. Define operator $P_b : (P_b H)(m) = H(E_b(m)) = H(b^m)$
6. Transform $R(n) \rightarrow S(m) := P_b(R) = R \circ E_b, S(m) = R(b^m)$

$$\begin{array}{ccccc}
 T(n) & \xrightarrow{N_k} & R(n) & \xrightarrow{P_b} & S(m) \\
 \downarrow n \mapsto n/b & & \downarrow n \mapsto n/b & & \downarrow m \mapsto m-1 \\
 a \cdot T\left(\frac{n}{b}\right) + f(n) & \longleftarrow & R\left(\frac{n}{b}\right) + \frac{f(n)}{n^k} & \longleftarrow & S(m-1) + \frac{f(b^{m-k})}{b^{m-k}} \\
 & & & & \vdots \\
 \boxed{T(n) = n^k R(n) \in \Theta(n^k H(n))} & \longleftarrow & R(n) \in \Theta(G(\log_b n)) = H(n) & \longleftarrow & S(m) \in \Theta(S(0) + \sum_{i=1}^m \frac{f(b^i)}{b^{i-k}}) = G(m)
 \end{array}$$

$$T(n) \sim n^{\log_b a} \sum_{i=1}^{\lfloor \log_b n \rfloor} \frac{f(b^i)}{b^{i \log_b a}}$$

This idea applies can be generalized to other recurrences, by having different and potentially more operators N_k, P_0 to arrive at $S(m) = S(m-1) + f'(m)$, to reduce the recurrence problem to a summation problem.

2.3.11 $T(n) = 16T\left(\frac{n}{4}\right) + 32n \log^{128} n$

Define $R(n) = \frac{T(n)}{n^{128}}$:

$$\begin{aligned}
 R(n) &= \frac{T\left(\frac{n}{4}\right)}{\left(\frac{n}{4}\right)^{128}} + \frac{32 \log^{128} n}{n} \\
 &= R\left(\frac{n}{4}\right) + \frac{32 \log^{128} n}{n}
 \end{aligned}$$

Define $S(m) = R(4^m)$:

$$\begin{aligned}
 S(m) &= S(m-1) + \frac{32 \cdot 4^m (\log(4^m))^{128}}{4^{m \cdot \log_4(16)}} \\
 &= S(m-1) + \frac{32 \cdot 4^m m^{128}}{16^m} \\
 &= S(m-1) + \frac{32(m)^{128}}{4^m} \\
 S(m) &\in \Theta\left(S(0) + 32 \sum_{i=1}^m \frac{i^{128}}{4^i}\right)
 \end{aligned}$$

It's easy to show that $f(n) = \sum_{i=0}^m \frac{i^{128}}{4^i} \in O(1)$ as

$$\lim_{n \rightarrow \infty} \frac{n^{128}}{4^n} = \lim_{n \rightarrow \infty} \frac{128n^{127}}{\ln(4)4^n} = \dots = \lim_{n \rightarrow \infty} \frac{c_1}{c_2 4^n} = 0 \quad \text{for some constant } c_1, c_2$$

$$\lim_{n \rightarrow \infty} \frac{n^{128}}{4^n} = 0 \implies f(n) = \frac{n^{128}}{4^n} \in o(1) \implies f(n) \in O(1)$$

$$S(m) \in O(1)$$

$$R(n) \in O(1)$$

$$T(n) \in O(n^2)$$

$$\mathbf{2.3.12} \quad T(n) = 16T(\frac{n}{4}) + 64n^2 \log^8 n + 32n \log^{128} n$$

Define $R(n) = \frac{T(n)}{n^2}$:

$$\begin{aligned} R(n) &= \frac{16 \cdot T(\frac{n}{4})}{(\frac{n^2}{4})} + \frac{64n^2 (\log(n))^8}{n^2} + \frac{32n \log^{128} n}{n^2} \\ R(n) &= \frac{16 \cdot T(\frac{n}{4})}{(\frac{n^2}{4})} + 64(\log(n))^8 + \frac{32 \log^{128} n}{n} \end{aligned}$$

Define $m = 4^n$, $n = \log(m)$, $S(m) = R(4^m)$

$$\begin{aligned} S(m) &= S(m-1) + \frac{64(4^m)^2 (\log(4^m))^8}{4^m \cdot \log_4 16} + \frac{32(4^m) (\log(4^m))^{128}}{4^m \cdot \log_4 16} \\ &= S(m-1) + 64(m \log 4)^8 + \frac{32(m \log 4)^{128}}{4^m} \\ S(m) &\in O\left(S(0) + 64 \cdot (\log 4)^8 \sum_{i=1}^m i^8 + 32 \cdot (\log 4)^{128} \sum_{i=1}^m \frac{i^{128}}{4^i}\right) \end{aligned}$$

We have shown in 1.1 that $g(m) = \sum_{i=1}^m \frac{i^{128}}{4^i} \in O(1)$.

$$\sum_{i=1}^n i^p = \frac{1}{p+1} \sum_{j=0}^p (-1)^j \binom{p+1}{j} B_j n^{p+1-j}, \quad p \in \mathbb{N}$$

where B_j are the Bernoulli numbers. (<https://en.wikipedia.org/wiki/Faulhaber>)

$$\begin{aligned} \sum_{i=1}^k i^8 &= \frac{1}{9} \sum_{j=0}^8 (-1)^j \binom{9}{j} B_j \\ &= \frac{1}{9} \left(\binom{9}{0} (1)k^9 - \binom{9}{1} \left(\frac{1}{2}\right)k^8 + \dots + \binom{9}{8} \left(\frac{-1}{30}\right)k \right) \end{aligned}$$

$$h(k) = \sum_{i=1}^k i^8 \in O(k^9)$$

$$\begin{aligned} S(m) &\in O(S(0) + m^9) \\ &\in O(m^8) \end{aligned}$$

$$\begin{aligned} R(n) &\in O((\log_4 n)^9) \\ T(n) &\in O(n^2 \log^9 n) \end{aligned}$$

$$\mathbf{2.3.13} \quad T(n) = 16 \cdot T(\frac{n}{4}) + n^4$$

Define $R(n) = \frac{T(n)}{n^2}$:

$$\begin{aligned} R(n) &= \frac{T(\frac{n}{4})}{(\frac{n}{4})^2} + n^2 \\ &= R(\frac{n}{4}) + n^2 \end{aligned}$$

Define $S(m) = R(4^m)$:

$$\begin{aligned} S(m) &= S(m-1) + \frac{(4^m)^4}{4^{2m}} \\ &= S(m-1) + 16^m \\ S(m) &\in O(S(0) + \sum_{i=1}^m 16^i) = O(S(0) + \frac{16^{m+1} - 16}{15}) = O(16^m) \end{aligned}$$

Map back to $R(n)$ and $T(n)$:

$$\begin{aligned} S(m) &\in O(16^m) \\ R(n) &\in O(16^{\log_4 n}) = O(n^2) \\ T(n) &\in O(n^4) \end{aligned}$$

2.3.14 $T(n) = T(\sqrt{n}) + \log(n)$
Define $n = 2^m$, $m = \log(n)$

$$\begin{aligned} T(n) &= T(n^{\frac{1}{2}}) + \log(n) \\ T(2^m) &= T(2^{\frac{1}{2}m}) + \log(2^m) \\ &= T(2^{\frac{1}{2}m}) + m \\ R(m) &= R\left(\frac{m}{2}\right) + m \end{aligned}$$

Define $m = 2^k$, $k = \log(m)$

$$\begin{aligned} R(2^k) &= R\left(\frac{2^k}{2}\right) + 2^k \\ S(k) &= S(k-1) + 2^k \\ S(k) &\in O(S(0) + \sum_{i=1}^k 2^i) = O(1 + 2^{k+1} + 2) = O(2^k) \end{aligned}$$

Map back to $R(m)$ and $T(n)$:

$$\begin{aligned} S(k) &\in O(2^k) \\ \implies R(m) &\in O(2^{\log(m)}) = O(m) \\ \implies T(n) &\in O(\log(n)) \end{aligned}$$

2.3.15 $T(n) = T(\sqrt{n}) + \log \log(n)$
Define $n = 2^m$, $m = \log(n)$

$$\begin{aligned} T(n) &= T(n^{\frac{1}{2}}) + \log \log(n) \\ T(2^m) &= T(2^{\frac{1}{2}m}) + \log \log(2^m) \\ &= T(2^{\frac{1}{2}m}) + \log(m) \\ R(m) &= R\left(\frac{m}{2}\right) + \log(m) \end{aligned}$$

Define $m = 2^k$, $k = \log(m)$

$$\begin{aligned} R(2^k) &= R\left(\frac{2^k}{2}\right) + \log(2^k) \\ S(k) &= S(k-1) + k \\ S(k) &\in O(S(0) + \sum_{i=1}^k i) = O\left(\frac{k(k+1)}{2}\right) = O(k^2) \end{aligned}$$

Map back to $R(m)$ and $T(n)$:

$$\begin{aligned} S(k) &\in O(k^2) \\ \implies R(m) &\in O(\log^2(m)) \\ \implies T(n) &\in O((\log \log(n))^2) \end{aligned}$$

3 Proof of Correctness

3.1 Iterative algorithms

Steps

1. Loop invariant: desirable conditions that holds at the start of every iteration
2. Initialization: the loop invariant is true at the start of the first iteration
3. Maintenance: If the loop invariant is true at the start of the current iteration, it is true at the start of the next iteration
4. Termination: The loop invariant is true at the end of the last iteration $\rightarrow$ the algorithm outputs a correct answer

3.2 Example: Dijkstra

Setting: Let $G = (V, E)$ be a directed graph with strictly positive edge weights $w : E \mapsto \mathbb{R}^+$. Fix source vertex $s \in V$, $\forall v \in V$, let $dist(s, v)$ be the length of a shortest path from s to v . If v is unreachable from s , $dist(s, v) = \infty$.
Notation: $dist(s, v)$: the actual shortest distance from s to v ; $d(v)$: the algorithm variable of current shortest possible distance from s to v . $d(v) \geq dist(s, v)$

```
Dijkstra( $G(V, E), s \in V$ ):  
   $d(s) = 0$   
   $R = \{s\}$   
  while( $R \neq V$ ):  
     $v \in V \setminus R, v = \min_{u \in R} (d(u) + w(u, v))$   
     $d(v) = \min_{u \in R} (d(u) + w(u, v))$   
     $R \leftarrow v$ 
```

Loop invariant:

$$\forall v \in R, d(v) = dist(s, v)$$
$$R \subseteq V \text{ is the set of settled vertices}$$

Initialization:

$$d(s) = dist(s, s) = 0$$

Maintenance:

Suppose $\forall u \in R, d(u) = dist(s, u)$, select $v \in V \setminus R$ such that,

$$v = \min_{u \in R} (d(u) + w(u, v))$$
$$d(v) = \min_{u \in R} (d(u) + w(u, v))$$

Prove: $d(v) = dist(s, v)$

1. $d(v) \geq dist(s, v)$

$$dist(s, v) = \min_{u \in V} (d(u) + w(u, v)) \leq \min_{u \in R} (d(u) + w(u, v)) = d(v)$$

2. $d(v) \leq dist(s, v)$

Suppose otherwise, for the shortest path $P = s \vdash^* v$, $\exists x \in V, x \notin R$,

3.3 Example: Insertion Sort

We prove correctness using loop invariants for both the outer and inner loops.

Outer Loop Invariant

Invariant 1 (Outer). *At the start of iteration i ($1 \leq i \leq n$) of the outer loop:*

1. *The prefix $A[1..i-1]$ is sorted in nondecreasing order.*
2. *The multiset of $A[1..i-1]$ equals the multiset of the $i-1$ smallest elements of the original array.*

Initialization. Before the first iteration ($i = 1$), the prefix $A[1..0]$ is empty, which is trivially sorted and contains exactly the 0 smallest elements. Thus the invariant holds.

Maintenance. Assume the invariant holds at the start of iteration i . Let the key be $x = A[i]$. The inner loop shifts elements of $A[1..i-1]$ that are strictly greater than x one position to the right, creating a “hole.” Finally, x is placed into this hole. This preserves the multiset property (no elements are lost or duplicated) and ensures that $A[1..i]$ is sorted. Therefore, the invariant holds at the start of iteration $i+1$.

Termination. The outer loop terminates when $i = n+1$. By the invariant, $A[1..n]$ is sorted and contains the same multiset of elements as the original input. Hence the algorithm outputs a sorted permutation of the input.

Inner Loop Invariant

While inserting x into $A[1..i-1]$, let j be the scanning index. The inner loop maintains:

Invariant 2 (Inner). *At each iteration:*

1. *The segment $A[j+1..i]$ contains exactly those elements of $A[1..i-1]$ greater than x , shifted one position to the right.*
2. *All elements in $A[j+1..i]$ are $\geq x$.*
3. *The hole (unused position) is at $j+1$. All elements left of j remain sorted and $\leq x$.*

Maintenance. If $A[j] > x$, then $A[j]$ is shifted right, maintaining the invariant. If $A[j] \leq x$ (or $j = 0$), the loop terminates, and x is placed into the hole at $j+1$. This produces a sorted prefix $A[1..i]$.

Termination (Total Correctness)

- The inner loop terminates because j decreases each iteration and is bounded below by 0.
- The outer loop terminates because i increases each iteration and is bounded above by n .

Together with partial correctness from the invariants, insertion sort is totally correct.

Stability

If the inner loop condition is **while** $A[j] > x$, then equal keys are never shifted past each other. Thus insertion sort is stable. Using $\geq$ instead would break stability.

Edge Cases

- Empty or single-element array: trivially sorted.
- Already sorted array: no shifts occur.
- Reverse-sorted array: maximal shifts, but still correct.
- All equal elements: no shifts occur; stability preserved.

3.4 Stooge sort

Complexity analysis

We start from the recurrence

$$T(n) = 3T\left(\frac{2n}{3}\right) + 1.$$

Define

$$R(n) = \frac{T(n)}{n}.$$

Then

$$\begin{aligned}
R(n) &= \frac{T(n)}{n} = \frac{3T\left(\frac{2n}{3}\right) + 1}{n} \\
&= 3 \cdot \frac{T\left(\frac{2n}{3}\right)}{n} + \frac{1}{n} \\
&= 3 \cdot \frac{\frac{2n}{3}}{n} \cdot \frac{T\left(\frac{2n}{3}\right)}{\frac{2n}{3}} + \frac{1}{n} \\
&= 2R\left(\frac{2n}{3}\right) + \frac{1}{n}.
\end{aligned}$$

Now set

$$n = \left(\frac{3}{2}\right)^k, \quad S(k) = R\left(\left(\frac{3}{2}\right)^k\right).$$

Then

$$\begin{aligned}
S(k) &= R\left(\left(\frac{3}{2}\right)^k\right) \\
&= 2R\left(\frac{2}{3} \cdot \left(\frac{3}{2}\right)^k\right) + \frac{1}{\left(\frac{3}{2}\right)^k} \\
&= 2R\left(\left(\frac{3}{2}\right)^{k-1}\right) + \left(\frac{2}{3}\right)^k \\
&= 2S(k-1) + \left(\frac{2}{3}\right)^k.
\end{aligned}$$

Solve this linear recurrence. The homogeneous solution is

$$S_h(k) = C \cdot 2^k.$$

For a particular solution, try $S_p(k) = A\left(\frac{2}{3}\right)^k$:

$$\begin{aligned}
A\left(\frac{2}{3}\right)^k &= 2A\left(\frac{2}{3}\right)^{k-1} + \left(\frac{2}{3}\right)^k \\
A &= 3A + 1 \quad \Rightarrow \quad -2A = 1 \quad \Rightarrow \quad A = -\frac{1}{2}.
\end{aligned}$$

Thus

$$S(k) = C \cdot 2^k - \frac{1}{2} \left(\frac{2}{3}\right)^k,$$

so asymptotically $S(k) = \Theta(2^k)$.

Mapping back to n : since $n = \left(\frac{3}{2}\right)^k$, we have

$$2^k = \left(\left(\frac{3}{2}\right)^k\right)^{\log_{3/2} 2} = n^{\log_{3/2} 2},$$

so

$$R(n) = S(k) = \Theta(n^{\log_{3/2} 2}).$$

Therefore

$$T(n) = nR(n) = \Theta(n^{1+\log_{3/2} 2}).$$

Finally, note that

$$1 + \log_{3/2} 2 = \frac{\log(3/2)}{\log(3/2)} + \frac{\log 2}{\log(3/2)} = \frac{\log 3}{\log(3/2)} = \log_{3/2} 3,$$

so we can write the solution as

$$\boxed{T(n) = \Theta(n^{\log_{3/2} 3})}.$$

3.5 2D Peak Finding via Divide-and-Conquer

We are given an $n \times m$ grid A , where each cell $A[i, j]$ contains a numeric value. A cell $A[i, j]$ is a *peak* if

$$A[i, j] \geq A[i-1, j], \quad A[i, j] \geq A[i+1, j], \quad A[i, j] \geq A[i, j-1], \quad A[i, j] \geq A[i, j+1],$$

whenever the neighboring cells exist. The goal is to find *any* peak.

Divide-and-Conquer Strategy

The classical divide-and-conquer algorithm recursively splits the grid and uses structural properties of local maxima across a middle slice. The standard version operates on columns:

1. Select the middle column $c = \lfloor m/2 \rfloor$.
2. Scan column c to find a global maximum entry in that column. Let the row of that entry be r .
3. Compare $A[r, c]$ to its left and right neighbors:
 - If $A[r, c]$ is at least as large as both neighbors, then (r, c) is a 2D peak.
 - If the left neighbor is larger, recursively apply the algorithm on the left half of the grid (columns 1 to $c - 1$).
 - If the right neighbor is larger, recursively apply the algorithm on the right half of the grid (columns $c + 1$ to m).

Thus at each step, we restrict our search to one half of the matrix, similar to binary search, but guided by the maximal element in the middle column.

Correctness Intuition

The correctness follows from a geometric local-search argument. If the entry $A[r, c]$ is not a peak, then one of its horizontal neighbors is strictly larger. Suppose the right neighbor $A[r, c + 1]$ is larger. In this case, any peak must lie in the right half of the matrix:

$$A[r, c + 1] > A[r, c] \implies \text{peak lies in columns } c + 1, \dots, m.$$

This holds because moving from (r, c) to $(r, c + 1)$ moves strictly uphill. A strictly increasing path cannot cross from the larger side back into a smaller side without violating the peak property. This argument ensures the validity of the recursive restriction.

Running Time

Finding the maximum in one column takes $O(n)$ time. At each recursive step, we discard half of the columns, so the recurrence is:

$$T(m) = T(m/2) + O(n).$$

Solving this recurrence yields:

$$T(m) = O(n \log m).$$

For square grids ($n = m$), this is:

$$T(n) = O(n \log n).$$

Remarks

- The algorithm finds a peak, not necessarily the global maximum.
- A more advanced algorithm achieves $O(n + m)$ time using a “greedy ascent” strategy, but the divide-and-conquer version is easier to analyze.
- The approach generalizes to higher dimensions but suffers from exponential-to-the-dimension cost.

This divide-and-conquer method is the standard algorithmic paradigm taught in introductory algorithms courses for 2D peak finding.

4 Divide and Conquer

4.1 Fast Exponentiation from a Divide-and-Conquer Perspective

Problem. Given a real number a and a non-negative integer n , compute a^n efficiently. The naive method multiplies a by itself $n - 1$ times, achieving $O(n)$ time. We seek a divide-and-conquer approach that improves this to $O(\log n)$.

Divide-and-Conquer Idea. Use the identity

$$a^n = \begin{cases} (a^{n/2})^2, & \text{if } n \text{ is even,} \\ a \cdot a^{n-1}, & \text{if } n \text{ is odd.} \end{cases}$$

Thus we reduce the exponent roughly by half at each recursive call. The cost of combining subproblems is $O(1)$, as it involves at most one or two multiplications.

Algorithm.

- If $n = 0$, return 1.
- Recursively compute $x = a^{\lfloor n/2 \rfloor}$.
- If n is even, return $x \cdot x$.
- If n is odd, return $a \cdot x \cdot x$.

Recurrence Relation. Let $T(n)$ denote the running time. Then,

$$T(n) = \begin{cases} T(n/2) + O(1), & \text{if } n \text{ even,} \\ T(n-1) + O(1), & \text{if } n \text{ odd.} \end{cases}$$

Solving the Recurrence. Observe:

- Each *odd* n produces a call to $T(n-1)$, after which the recursion immediately goes to $T(n/2)$.
- There are at most $\lfloor \log_2 n \rfloor$ halvings, because $n \rightarrow \lfloor n/2 \rfloor$ reduces n exponentially.
- Between any two halvings, there can be at most one odd case (since subtracting 1 flips parity).

Thus, the recursion performs at most $O(\log n)$ total steps.

Formally, unwrapping the longest chain through odd numbers:

$$T(n) = T(\lfloor n/2 \rfloor) + O(1) = T(\lfloor n/4 \rfloor) + O(1) = \dots = T(1) + O(\log n) = O(\log n).$$

Conclusion. The divide-and-conquer fast exponentiation algorithm computes a^n in

$$T(n) = O(\log n)$$

time, which is exponentially faster than the naive $O(n)$ repeated multiplication method.

The key insight is that halving the exponent drastically reduces problem size, and handling odd exponents adds only a constant amount of additional work per level of recursion.

4.2 Strassen's Matrix Multiplication — Analysis

Hint. Replace 8 sub-multiplications on quadrants by 7 cleverly combined ones; the overhead is $O(n^2)$ additions, so the recurrence $T(n) = 7T(n/2) + \Theta(n^2)$ solves to $T(n) = \Theta(n^{\log_2 7})$.

Setup and Block Form. Let $A, B \in \mathbb{F}^{n \times n}$ (assume n is a power of 2; otherwise pad with zeros). Partition into quadrants $A = \begin{bmatrix} A_{11} & A_{12} \\ A_{21} & A_{22} \end{bmatrix}$ and $B = \begin{bmatrix} B_{11} & B_{12} \\ B_{21} & B_{22} \end{bmatrix}$, each block of size $(n/2) \times (n/2)$. Classical block multiplication uses 8 block products; Strassen uses 7:

$$\begin{aligned} M_1 &= (A_{11} + A_{22})(B_{11} + B_{22}), & M_2 &= (A_{21} + A_{22})B_{11}, \\ M_3 &= A_{11}(B_{12} - B_{22}), & M_4 &= A_{22}(B_{21} - B_{11}), \\ M_5 &= (A_{11} + A_{12})B_{22}, & M_6 &= (A_{21} - A_{11})(B_{11} + B_{12}), \\ M_7 &= (A_{12} - A_{22})(B_{21} + B_{22}). \end{aligned}$$

Then assemble

$$\begin{aligned} C_{11} &= M_1 + M_4 - M_5 + M_7, & C_{12} &= M_3 + M_5, \\ C_{21} &= M_2 + M_4, & C_{22} &= M_1 - M_2 + M_3 + M_6, \end{aligned} \quad C = \begin{bmatrix} C_{11} & C_{12} \\ C_{21} & C_{22} \end{bmatrix}.$$

Work Recurrence (Arithmetic Count). At problem size n , Strassen performs:

- 7 recursive matrix multiplications of size $n/2$,
- a constant number of additions/subtractions on $(n/2) \times (n/2)$ blocks (classic Strassen: 18 such adds/subs).

Thus (for scalar arithmetic count)

$$T(n) = 7T(n/2) + cn^2, \quad \text{with } c = 18 \cdot \frac{1}{4} = 4.5 \text{ (block adds on size } n/2\text{)}.$$

By the Master Theorem with $a = 7$, $b = 2$, $f(n) = \Theta(n^2)$, and $p = \log_2 7 \approx 2.807355$,

$$T(n) = \Theta(n^{\log_2 7}) = \Theta(n^{2.807355\dots}).$$

Separating operations:

$$\text{Multiplications } M(n) = 7M(n/2), \quad M(1) = 1 \Rightarrow M(n) = n^{\log_2 7}.$$

$$\text{Additions } A(n) = 7A(n/2) + 18\left(\frac{n}{2}\right)^2, \quad A(1) = 0 \Rightarrow A(n) = \Theta(n^{\log_2 7}).$$

A precise summation of the addition tree yields

$$A(n) = \sum_{i=0}^{\log_2 n - 1} 7^i \cdot 18 \left(\frac{n}{2^i} \cdot \frac{1}{2}\right)^2 = 6(n^{\log_2 7} - n^2),$$

so additions are asymptotically the same order as multiplications, but the exponent's constant is smaller; the overall complexity is still $\Theta(n^{\log_2 7})$.

Asymptotic vs Practical.

- **Crossover threshold.** For small n , classical $O(n^3)$ (tiled/BLAS-3) is faster due to low constants, cache locality, and no extra temporaries. Typical cutovers are in the range 64–512 (hardware/library dependent).
- **Numerical stability.** Strassen increases additive cancellations; rounding errors can grow more than classical GEMM. For ill-conditioned problems in floating-point, consider classical multiplication or mixed approaches; over exact rings (integers/modular) this is not an issue.
- **Memory footprint.** Extra temporaries (for M_i and the linear combinations) require $\Theta(n^2)$ additional space. In-place and Winograd variants reduce the number of additions (e.g., from 18 to 15) at the cost of more complicated scheduling.
- **Rectangular/padding.** If n is not a power of two, pad to the next power of two or use hybrid blocking (peel leftover rows/cols). Strassen can be applied hierarchically to rectangular blocks as well.
- **Parallelism.** The 7 subproducts are independent at each level; depth is $\log_2 n$. Good task parallelism is possible, but addition phases and temporaries need careful scheduling to avoid bandwidth bottlenecks.

Pseudocode (Recursive, with Base Cutoff).

```
Strassen(A, B, n):
    if n <= n0: return ClassicalGEMM(A, B)    // cutoff (e.g., 64..256)
    split A,B into quadrants of size n/2
    S1 = A11 + A22;   T1 = B11 + B22;   M1 = Strassen(S1, T1)
    S2 = A21 + A22;   M2 = Strassen(S2, B11)
    T3 = B12 - B22;   M3 = Strassen(A11, T3)
    T4 = B21 - B11;   M4 = Strassen(A22, T4)
    S5 = A11 + A12;   M5 = Strassen(S5, B22)
    S6 = A21 - A11;   T6 = B11 + B12; M6 = Strassen(S6, T6)
    S7 = A12 - A22;   T7 = B21 + B11; M7 = Strassen(S7, T7)
    C11 = M1 + M4 - M5 + M7
    C12 = M3 + M5
    C21 = M2 + M4
    C22 = M1 - M2 + M3 + M6
    return combine(C11,C12,C21,C22)
```

Communication/Cache Considerations (brief). Blocked classical GEMM already achieves high arithmetic intensity. Strassen reduces arithmetic count, but may increase memory traffic due to extra temporaries & combinations. High-performance implementations interleave additions with recursive multiplies and reuse buffers to reduce bandwidth, approaching optimal communication bounds for the algorithmic class.

Takeaways. Strassen improves the exponent from 3 to $\log_2 7 \approx 2.807$, with overhead dominated by $O(n^2)$ linear combinations. It is asymptotically faster, practically competitive beyond a crossover size, less numerically stable than classical GEMM, and serves as a building block for even faster (but mostly impractical) asymptotic algorithms.

4.3 Karatsuba's Algorithm and Its Divide-and-Conquer Analysis

Multiplying two n -digit integers naively takes $\Theta(n^2)$ time, since the grade-school algorithm performs $n \times n$ digit multiplications. Karatsuba's algorithm gives the first nontrivial improvement by applying a divide-and-conquer strategy that reduces the number of recursive multiplications from 4 to 3, yielding a running time of $O(n^{\log_2 3}) \approx O(n^{1.585})$.

High-Level Idea

Let x and y be two n -digit numbers in base 10 (or any base). Split each into two halves of $n/2$ digits:

$$x = a \cdot B^{n/2} + b, \quad y = c \cdot B^{n/2} + d,$$

where B is the base (e.g. 10) and a, b, c, d are the high and low halves.

The product expands to:

$$xy = (aB^{n/2} + b)(cB^{n/2} + d) = acB^n + (ad + bc)B^{n/2} + bd.$$

A direct divide-and-conquer approach would compute the four products $\{ac, ad, bc, bd\}$ recursively, leading to the recurrence

$$T(n) = 4T(n/2) + O(n),$$

which solves to $\Theta(n^2)$, giving no improvement.

Karatsuba observed that the middle term can be computed more cleverly:

$$ad + bc = (a + b)(c + d) - ac - bd.$$

Thus only *three* recursive multiplications are needed:

$$\begin{aligned} P_1 &= ac, \\ P_2 &= bd, \\ P_3 &= (a + b)(c + d). \end{aligned}$$

Then:

$$ad + bc = P_3 - P_1 - P_2,$$

and the final product is

$$xy = P_1B^n + (P_3 - P_1 - P_2)B^{n/2} + P_2.$$

Algorithm Structure

1. Base case: if n is small (e.g. $n = 1$), multiply directly.
2. Split x and y into (a, b) and (c, d) .
3. Recursively compute:

$$P_1 = ac, \quad P_2 = bd, \quad P_3 = (a + b)(c + d).$$

4. Combine:

$$xy = P_1B^n + (P_3 - P_1 - P_2)B^{n/2} + P_2.$$

Each split and recombination takes $O(n)$ time.

Running Time Analysis

Karatsuba reduces the problem size by half and performs three recursive multiplications. Thus the recurrence for an n -digit multiplication is:

$$T(n) = 3T(n/2) + O(n).$$

Using the Master Theorem:

$$T(n) = \Theta(n^{\log_2 3}) = \Theta(n^{1.585\dots}).$$

This is strictly faster than the $\Theta(n^2)$ runtime of the classical multiplication algorithm. The improvement comes directly from decreasing the number of multiplications in the divide-and-conquer expansion.

Discussion

Karatsuba's method was the first subquadratic multiplication algorithm and paved the way for more advanced techniques such as Toom–Cook multiplication, the Schönhage–Strassen algorithm, and FFT-based multiplication. Although modern systems often switch between several multiplication algorithms depending on input size, Karatsuba remains a fundamental example of how algebraic identities paired with divide-and-conquer reasoning can reduce computational complexity.

4.4 Divide-and-Conquer Algorithm for the Discrete Fourier Transform

The Discrete Fourier Transform (DFT) converts an n -point sequence $x_0, x_1, \dots, x_{n-1}$ into frequency components $X_0, X_1, \dots, X_{n-1}$, where

$$X_k = \sum_{j=0}^{n-1} x_j \omega_n^{jk}, \quad \omega_n = e^{-2\pi i/n}.$$

A direct computation requires $\Theta(n^2)$ time. The Fast Fourier Transform (FFT) reduces this to $\Theta(n \log n)$ via a divide-and-conquer decomposition that exploits symmetry in the n th roots of unity.

1. Even–Odd Decomposition

Assume n is a power of 2 (the algorithm can be generalized). Split the sequence into its even- and odd-indexed elements:

$$x_j^{(even)} = x_{2j}, \quad x_j^{(odd)} = x_{2j+1}, \quad j = 0, \dots, n/2 - 1.$$

Rewrite the DFT definition:

$$X_k = \sum_{j=0}^{n-1} x_j \omega_n^{jk} = \sum_{j=0}^{n/2-1} x_{2j} \omega_n^{2jk} + \sum_{j=0}^{n/2-1} x_{2j+1} \omega_n^{(2j+1)k}.$$

Using $\omega_n^2 = \omega_{n/2}$, we obtain:

$$X_k = E_k + \omega_n^k O_k,$$

where

$$E_k = \sum_{j=0}^{n/2-1} x_{2j} \omega_{n/2}^{jk}, \quad O_k = \sum_{j=0}^{n/2-1} x_{2j+1} \omega_{n/2}^{jk}.$$

Thus, computing the n -point DFT reduces to computing two $(n/2)$ -point DFTs.

2. Combining the Subproblems

For $k = 0, \dots, n/2 - 1$:

$$\begin{aligned} X_k &= E_k + \omega_n^k O_k, \\ X_{k+n/2} &= E_k - \omega_n^k O_k. \end{aligned}$$

This gives all n outputs using:

- two DFTs of size $n/2$, and
- n multiplications by powers of ω_n and additions.

3. Recursive Structure

Let $T(n)$ denote the running time of the FFT on n inputs.

Each stage performs:

$$T(n) = 2T(n/2) + \Theta(n).$$

By the Master Theorem:

$$T(n) = \Theta(n \log n).$$

Since the recursion depth is $\log_2 n$, and each level performs $O(n)$ work, the total running time is dominated by combining stages.

4. Key Insights Behind the Speedup

- The roots of unity satisfy strong symmetry:

$$\omega_n^{k+n/2} = -\omega_n^k, \quad \omega_n^{2k} = \omega_{n/2}^k.$$

- These identities allow grouping even and odd input terms, turning one n -point DFT into two $(n/2)$ -point DFTs with only $O(n)$ extra effort.
- Repeating the decomposition recursively gives an $O(n \log n)$ algorithm.

5. Summary

The divide-and-conquer FFT algorithm follows the pattern:

$$\text{DFT}_n \longrightarrow \text{DFT}_{n/2} \text{ on evens} + \text{DFT}_{n/2} \text{ on odds} + \text{combine}.$$

This decomposition leverages the structure of complex roots of unity to reduce the computational cost from quadratic to nearly linear. FFT is now foundational in signal processing, numerical algorithms, polynomial multiplication, and many divide-and-conquer applications.

5 Lower bound analysis & Linear time sorting & order statistics

5.1 Proof of the Lower Bound of Comparison-Based Sorting

In this section, we give a formal proof that any comparison-based sorting algorithm requires $\Omega(n \log n)$ comparisons in the worst case. This lower bound applies to all algorithms that determine the ordering of elements solely by comparing pairs of keys (e.g., mergesort, heapsort, quicksort in its comparison model).

5.2 Decision Tree Model

A comparison-based sorting algorithm on n distinct elements can be viewed as a *binary decision tree*:

- each internal node represents a comparison of the form “is $a_i < a_j$?”;
- each edge corresponds to one of the two possible outcomes of the comparison;
- each leaf corresponds to a permutation of the inputs that is consistent with the sequence of comparison outcomes on the root-to-leaf path.

Any correct sorting algorithm must be able to distinguish among all $n!$ possible orderings of the input. Therefore, the decision tree must contain at least $n!$ leaves.

Leaf–Height Argument

A binary tree of height h can have at most 2^h leaves. Hence, if L is the number of leaves,

$$L \leq 2^h.$$

Since a sorting decision tree must have at least $n!$ leaves, we obtain

$$n! \leq 2^h.$$

Taking logarithms on both sides yields

$$h \geq \log_2(n!).$$

The height h of the decision tree is exactly the worst-case number of comparisons made by the sorting algorithm. Therefore, any comparison-based sorting algorithm must make at least $\log_2(n!)$ comparisons in the worst case.

Applying Stirling’s Approximation

To simplify the bound, we use Stirling’s formula:

$$n! = \Theta\left(\sqrt{n} \left(\frac{n}{e}\right)^n\right).$$

Taking logarithms,

$$\log_2(n!) = \Theta(n \log_2 n - n \log_2 e) = \Theta(n \log n).$$

Thus,

$$\log_2(n!) = \Omega(n \log n).$$

Conclusion

For any comparison-based sorting algorithm,

$$\text{Worst-case comparisons} \geq \log_2(n!) = \Omega(n \log n).$$

This proves that $\Omega(n \log n)$ is a universal lower bound for comparison-based sorting in the worst case. In other words, no algorithm that relies exclusively on pairwise comparisons can have a worst-case running time better than $\Omega(n \log n)$.

5.3 Decision-Tree Lower Bounds (Beyond Sorting & Searching)

Hint. State the *model*: an internal node tests a predicate about the input (comparison, linear, or algebraic test); leaves encode distinct outputs/decisions. Prove that any such tree must have large height via (i) *reduction* from a known hard problem, or (ii) an *adversary* that forces many tests, or (iii) *information* or *combinatorial* counting (many distinguishable instances).

5.3.1 1) Planar Convex Hull: $\Omega(n \log n)$ in algebraic decision trees.

Task. Given n points in $\mathbb{R}^2$, output their convex hull (e.g., the vertices in ccw order).

Model. Algebraic decision tree (tests are signs of constant-degree polynomials in the coordinates).

Claim. Any algorithm requires $\Omega(n \log n)$ tests in the worst case.

Proof idea (reduction from sorting). Given n reals $x_1, \dots, x_n$, map them to points $p_i = (x_i, x_i^2)$ on a parabola. The lower hull lists the p_i in increasing x_i . Thus, from the hull you recover the sorted order of $\{x_i\}$. Since comparison sorting needs $\Omega(n \log n)$ in this model, so does convex hull.

5.3.2 2) 2D Maxima / Skyline: $\Omega(n \log n)$ in algebraic decision trees.

Task. Report all points undominated in $\mathbb{R}^2$ (no other point both $\geq$ in each coord and $>$ in one).

Model. Algebraic decision tree.

Claim. $\Omega(n \log n)$ worst-case tests.

Proof idea (reduction from sorting). Encode numbers x_i as $p_i = (x_i, -x_i^2)$. All points are maximal, but their *left-to-right order* along the *upper* monotone chain matches the sorted order of $\{x_i\}$. Any algorithm that outputs the skyline in order (or even partitions points into consecutive blocks) can be used to sort, implying the lower bound.

5.3.3 3) Planar Point Location: $\Omega(\log n)$ queries.

Task. Given a planar subdivision with $\Theta(n)$ faces, identify the face containing a query point q .

Model. Linear decision tree (tests of the form “is q on which side of line ℓ ?”).

Claim. Any decision tree needs depth $\Omega(\log n)$ in the worst case.

Proof idea (information/packing). There are $\Theta(n)$ possible answers (faces). Each test has at most two outcomes, so a depth- h tree distinguishes at most 2^h outcomes. Thus $2^h \geq n$ and $h \geq \log_2 n - O(1)$.

5.3.4 4) Minimum Spanning Tree (MST): $\Omega(m)$ edge-weight comparisons.

Task. Given a connected graph with m weighted edges, output an MST.

Model. Comparison decision tree over edge weights (tests of the form $w_e \leq w_f$).

Claim. $\Omega(m)$ comparisons are necessary in the worst case.

Proof idea (adversary). Suppose some edge e is never compared/read. The adversary can set its weight to be smaller than all other edges while preserving all previously observed comparisons. If the unique MST must include e under this assignment, any algorithm that did not “inspect” e errs. Therefore every edge must be “touched” in some comparison path $\Rightarrow$ depth $\geq m$ in the worst case.

5.3.5 5) Majority Element Decision (is there an element with count $> n/2$?): $\Omega(n)$.

Task. Decide whether any value occurs more than $n/2$ times.

Model. Equality/inequality comparisons between input keys (decision tree with tests “ $a_i = a_j$?”, “ $a_i < a_j$?”, etc.).

Claim. $\Omega(n)$ queries are necessary.

Proof idea (adversary). If some position i is never inspected, adversary sets all inspected positions to pairwise distinct values and then sets a_i equal to one of them enough times to flip the answer while preserving all observed outcomes. Hence every position must be read/compared along some root-to-leaf path, forcing linear depth.

5.3.6 6) Segment Intersection Emptiness (no pair intersects?): $\Omega(n \log n)$.

Task. Given n non-vertical line segments in the plane, decide if any pair intersects.

Model. Algebraic decision tree.

Claim. $\Omega(n \log n)$ tests in the worst case.

Proof idea (reduction from sorting). Place n points (x_i, x_i^2) on a parabola and connect each to a common point far above; relative left-to-right order of the x_i determines whether specific pairs of these “spokes” cross. An algorithm deciding emptiness across a family of such instances can sort x_i , implying the lower bound. (Standard geometric reductions also map numbers to x -monotone segments whose intersection pattern encodes their order.)

5.3.7 7) Voronoi Diagram / Delaunay Triangulation: $\Omega(n \log n)$.

Task. Build the Voronoi diagram (or its dual Delaunay triangulation) of n points.

Model. Algebraic decision tree.

Claim. $\Omega(n \log n)$ tests are necessary.

Proof idea (reduction from sorting). Map numbers x_i to parabola points (x_i, x_i^2) . The Delaunay triangulation is the sorted path along the convex position; extracting it yields the sorted order. Therefore constructing the structure needs $\Omega(n \log n)$.

5.3.8 8) Linear Programming in Fixed Dimension d (e.g., $d = 2, 3$): $\Omega(n)$.

Task. Maximize a fixed linear objective subject to n linear constraints in $\mathbb{R}^d$ (constant d).

Model. Linear decision tree (sign tests of linear forms).

Claim. $\Omega(n)$ tests are necessary.

Proof idea (adversary). Each constraint can be made binding at the (unique) optimum by small perturbations consistent with previous tests. If some constraint is never tested, the adversary can flip feasibility/optimality while preserving all prior outcomes, so every constraint must be probed $\Rightarrow$ linear depth.

5.3.9 9) Closest Pair (planar): $\Omega(n \log n)$ in algebraic decision trees.

Task. Given n points, find the pair with minimum Euclidean distance.

Model. Algebraic decision tree.

Claim. $\Omega(n \log n)$ tests.

Proof idea (reduction). Encode $x_1 < \dots < x_n$ as points on two shifted x -monotone curves so that the unique closest pair is formed by consecutive x -neighbors in the sorted order. By querying on crafted instances, one can reconstruct the full order from the identity of closest pairs along refined subsets, yielding a sorting reduction (standard in computational geometry), hence the lower bound.

5.3.10 10) Transitive Reduction / Cover Graph of a Total Order: $\Omega(n \log n)$.

Task. Given a comparability graph that encodes an unknown total order over n labeled elements, recover the cover relation (Hasse diagram) of that order.

Model. Comparison decision tree over pairwise order tests.

Claim. $\Omega(n \log n)$ tests.

Proof idea (counting/reduction). There are $n!$ possible total orders; any decision tree distinguishing them has

depth $\geq \log_2(n!) = \Omega(n \log n)$. Since the cover relation uniquely identifies the order, computing it is at least as hard as sorting in the comparison model.

Nitpicks & model notes. (i) $\Omega(n \log n)$ for *geometric* problems is usually shown in the *algebraic* decision-tree model by a parabola (or lifting) *sorting reduction*.

(ii) Linear-time lower bounds ($\Omega(n)$ or $\Omega(m)$) often come from a simple adversary: any unchecked item/constraint/edge can be made decisive while preserving prior tests.

(iii) Point-location's $\Omega(\log n)$ is a pure information bound: $\Theta(n)$ answers under binary branching.

(iv) Beware: randomized/word-RAM models can beat these bounds (e.g., hashing for distinctness), but *not* in pure decision-tree models restricted to comparisons/algebraic tests.

5.4 Linear-Time Sorting: Stable Counting Sort and Radix Sort

Comparison-based sorting has a lower bound of $\Omega(n \log n)$, but when keys come from a restricted domain, we can sort in *linear time* $\Theta(n + k)$ or $\Theta(d(n + k))$ using non-comparison-based techniques. Two fundamental linear-time algorithms are *stable counting sort* and *radix sort*. These algorithms exploit structural properties of the input keys rather than comparing arbitrary elements.

5.4.1 Stable Counting Sort

Counting sort assumes that each of the n input keys is an integer in the range $\{0, 1, \dots, k-1\}$. The algorithm counts occurrences of each key, then performs a prefix-sum scan to determine the final position of each key in the output.

Algorithm outline.

1. Initialize a count array $C[0..k-1]$ to zero.
2. For each input element a_i , increment $C[a_i]$.
3. Compute prefix sums:

$$C[i] \leftarrow \sum_{j=0}^i C[j]$$

so that $C[i]$ stores the number of keys $\leq i$.

4. Traverse the input array in *reverse* order (to ensure stability). For each key a_i , place it in position $C[a_i]$ in the output array and decrement $C[a_i]$.

Complexity. Counting sort runs in $O(n + k)$ time and uses $O(k)$ extra space. It is *stable* because equal elements preserve their relative ordering.

Use cases. Counting sort is effective when $k = O(n)$ or when keys are naturally small integers (e.g., ASCII codes, bucket labels, or digit values).

```

void countingSortStable(vector<int>& a, int maxVal) {
    int n = (int)a.size();
    vector<int> count(maxVal + 1, 0);
    vector<int> output(n);

    // Count occurrences
    for (int x : a) {
        if (x < 0 || x > maxVal) {
            throw runtime_error("countingSortStable: value out of range");
        }
        count[x]++;
    }

    // Prefix sums: count[i] = number of elements <= i
    for (int i = 1; i <= maxVal; i++) {
        count[i] += count[i - 1];
    }

    // Build output in reverse to keep stability
    for (int i = n - 1; i >= 0; i--) {
        int x = a[i];
        int pos = count[x] - 1; // last position for value x
        output[pos] = x;
        count[x]--;
    }

    a.swap(output);
}

```

5.4.2 Radix Sort

Radix sort extends counting sort to handle multi-digit keys such as integers, strings, tuples, or any objects with fixed-length digit representations. Keys are viewed as d -digit numbers in base k :

$$a = \langle a_{d-1}, a_{d-2}, \dots, a_0 \rangle, \quad 0 \leq a_i < k.$$

Least-Significant-Digit (LSD) Radix Sort. LSD radix sort performs d passes:

1. On pass i , stably sort the array using counting sort on digit a_i .
2. Because counting sort is stable, the ordering of more significant digits (already processed in earlier passes) is preserved.

Correctness. Since LSD radix sort processes digits from least to most significant using a stable sorter, the final ordering corresponds to lexicographic ordering on all digits. Each pass uses counting sort, which does not disturb the order established by previous passes.

Running time. Each counting-sort pass costs $O(n + k)$, so the total running time is

$$O(d(n + k)).$$

Radix sort is linear when:

$$d = O(1) \quad \text{and} \quad k = O(n),$$

for example when sorting 32-bit integers with $k = 256$ and $d = 4$ bytes.

Analysis

To optimize, for b -bit word, we break into $\frac{b}{r}$ groups of r -bit words,

$$d = \frac{b}{r} \quad \text{and} \quad k = 2^r$$

each pass takes time $O(n + 2^r)$, total time is

$$O\left(\frac{b}{r}(n + 2^r)\right) \quad \text{is minimized when} \quad r \approx \log_2(n) \quad (n + 2^r \text{ balances})$$

Therefore optimal time complexity is

$$O\left(\frac{b}{\log(n)}(n + 2^{\log(n)})\right) = O\left(\frac{bn}{\log(n)}\right)$$

Consider integers to be in range of $[0, n^d]$,

$$b = \log_2(n^d) = d \log_2(n) \quad \text{bits}$$

$$O\left(\frac{bn}{\log(n)}\right) = O\left(\frac{d \log(n)n}{\log(n)}\right) = O(dn) \quad \text{- linear time}$$

When Linear-Time Sorting Applies

Counting sort and radix sort avoid the $\Omega(n \log n)$ lower bound because the comparison model is bypassed; instead, they exploit direct indexing into digit/character ranges. They are ideal when:

- keys are small integers,
- keys have fixed-length encodings,
- the alphabet size k is not too large,
- a stable linear-time subroutine is available (counting sort).

```
// Helper: get digit 'd' (0 = LSD) in base 'base'
int getDigit(int x, int d, int base) {
    while (d-- > 0) x /= base;
    return x % base;
}

// LSD radix sort using stable counting sort on each digit
// Assumes non-negative integers.
void radixSortLSD(vector<int>& a, int base = 10) {
    if (a.empty()) return;

    // Find max to know how many digits
    int maxVal = *max_element(a.begin(), a.end());
    if (maxVal < 0) throw runtime_error("radixSortLSD: negative values not supported");
```

```

int d = 0; // number of digits
int p = 1;
while (p <= maxVal) {
    p *= base;
    d++;
}

int n = (int)a.size();
vector<int> output(n);

for (int digit = 0; digit < d; digit++) {
    // Counting sort by 'digit' in base 'base'
    vector<int> count(base, 0);

    // Count digit frequencies
    for (int x : a) {
        int dig = getDigit(x, digit, base);
        count[dig]++;
    }

    // Prefix sums
    for (int i = 1; i < base; i++) {
        count[i] += count[i - 1];
    }

    // Build output (reverse for stability)
    for (int i = n - 1; i >= 0; i--) {
        int x = a[i];
        int dig = getDigit(x, digit, base);
        int pos = count[dig] - 1;
        output[pos] = x;
        count[dig]--;
    }

    a.swap(output);
}
}

```

Conclusion

Stable counting sort and radix sort demonstrate that sorting can be performed in linear time when keys have bounded range or fixed digit structure. These methods are fundamental in algorithm design, providing the backbone for linear-time sorting of integers, characters, and encoded data.

5.5 Selction/Order statistics

Given an unsorted array, without duplicates, find the index of the i^{th} smallest/ i^{th} order statistics/rank i element.

Lower bound

Trivially $\geq n$ steps needed.

5.5.1 Randomized Quickselect for the i -th Smallest Element

Beyond sorting all elements, we often only need the i -th order statistic: the element that would appear at position i in the sorted order. The *Randomized Quickselect* algorithm solves this *selection* problem in expected linear time without fully sorting the array.

Problem. Given an unsorted array $A[1..n]$ and an integer i with $1 \leq i \leq n$, find the element of rank i , i.e. the i -th smallest element of A .

Algorithm idea. Randomized Quickselect is essentially the “selection version” of randomized Quicksort:

1. If $n = 1$, return $A[1]$.
2. Choose a pivot p uniformly at random from A .
3. Partition A into three parts:
 - $L = \{x \in A : x < p\}$,
 - $E = \{x \in A : x = p\}$,
 - $R = \{x \in A : x > p\}$.

The partition step is done in linear time, similarly to Quicksort.

4. Let $|L|$ and $|E|$ be the sizes of these sets.
 - If $i \leq |L|$, the i -th smallest element lies in L ; recursively select the i -th smallest in L .

- If $|L| < i \leq |L| + |E|$, then the pivot value p is the i -th smallest (it occupies all ranks in this interval).
- If $i > |L| + |E|$, the i -th smallest element lies in R . Recursively select the $(i - |L| - |E|)$ -th smallest in R .

Crucially, only *one* side of the partition (either L or R) is recursed on; the other side is discarded. This differs from Quicksort, which recurses on both sides.

Expected running time. Let $T(n)$ be the expected time to select the i -th smallest from n elements. The partition step always costs $\Theta(n)$ time. Because the pivot is chosen uniformly at random, with constant probability (e.g. at least $1/2$) the pivot falls into the “middle” region of the array, so that both L and R have size at most a constant fraction (say $3n/4$) of the input.

Conditioning on this event yields the recurrence

$$T(n) \leq T\left(\frac{3n}{4}\right) + cn$$

for some constant $c > 0$, which solves to

$$T(n) = O(n).$$

A more careful expectation argument over all possible pivot positions shows that the expected running time of randomized Quickselect is indeed $\Theta(n)$.

Worst-case behavior. If the random pivot happens to be the smallest or largest element at every step, Quickselect repeatedly recurses on arrays of size $n - 1$, yielding

$$T(n) = T(n - 1) + \Theta(n) = \Theta(n^2).$$

This quadratic bound is the same as the worst-case running time of naive Quicksort. However, due to randomization, this worst case occurs with extremely small probability, and the *expected* running time over all random choices of pivots is linear.

Summary. Randomized Quickselect finds the i -th smallest element in expected $\Theta(n)$ time by:

- choosing a pivot uniformly at random,
- partitioning the array around the pivot in linear time,
- recursing only on the side of the partition that can contain the desired order statistic.

This achieves asymptotically optimal expected time for selection without sorting the entire array.

5.5.2 Median of Medians

Suppose we have n elements, choose r , where we divide n elements into $\lfloor \frac{n}{r} \rfloor$ groups (ignore the remainders). Suppose r is small, we can sort each of the $\lfloor \frac{n}{r} \rfloor$ groups in $O(1)$, total $O(\lfloor \frac{n}{r} \rfloor)$ time.

With $\lfloor \frac{n}{r} \rfloor$ medians, recursively select the median of the medians, and use the median of medians x as the pivot.

We have $\lfloor \lfloor \frac{n}{r} \rfloor / 2 \rfloor$ medians are $\leq x$. In each group corresponding to these medians, at least half are $\leq$ the median, $\implies$ at least $\lceil \frac{r}{2} \rceil \cdot \lfloor \frac{n}{2r} \rfloor$ elements $\leq x$.

Similarly, at least $\lceil \frac{r}{2} \rceil \cdot (\lfloor \frac{n}{2r} \rfloor + 1)$ elements $\geq x$

Now we can use quick select to guaranteed eliminating $\lceil \frac{r}{2} \rceil \cdot (\lfloor \frac{n}{2r} \rfloor + 1)$ elements that are $\geq x$, and execute on at most $n - \lceil \frac{r}{2} \rceil \cdot (\lfloor \frac{n}{2r} \rfloor + 1)$

For certain choice of r we can show that

$$T(n) \leq T\left(\frac{n}{r}\right) + T\left(\frac{1 - \lfloor \frac{r}{2} \rfloor}{\lfloor \frac{n}{2r} \rfloor} n\right) + \Theta(n) \in \Theta(n)$$

For example, let $r = 5$, for large n ,

$$T(n) \leq T\left(\frac{n}{5}\right) + T\left(\frac{7n}{10}\right) + \Theta(n)$$

$$T(n) \leq c \cdot \frac{n}{5} + c \cdot \frac{7n}{10} + \Theta(n)$$

$$T(n) \leq c \cdot \frac{9n}{10} + \Theta(n)$$

$$T(n) \leq c \cdot n - (c \cdot \frac{n}{10} - d \cdot n)$$

For sufficiently large c , that can handle both dn and the initial conditions

$$T(n) \leq cn \implies T(n) \in O(n)$$

This will not work for small r , For example, let $r = 3$, for large n ,

$$\begin{aligned} T(n) &\leq T\left(\frac{n}{3}\right) + T\left(\frac{2n}{3}\right) + \Theta(n) \\ T(n) &\leq c \cdot \frac{n}{3} + c \cdot \frac{3n}{3} + \Theta(n) \\ T(n) &\leq c \cdot n + \Theta(n) \end{aligned}$$

We can only show that $T(n) \in O(n \log(n))$

5.6 Tutorial problems

Q1. Let $x = 10$ to make it easier to visualize this as a normal base 10 multiplication and $n = 2$.

Let

$$A(10) = 352 = 3 \cdot 10^2 + 5 \cdot 10 + 2, \quad (a_2 = 3, a_1 = 5, a_0 = 2)$$

$$B(10) = 221 = 2 \cdot 10^2 + 2 \cdot 10 + 1, \quad (b_2 = 2, b_1 = 2, b_0 = 1)$$

Compute the coefficients of $C(10) = A(10) \times B(10) = 77792$ using the $O(n^2)$ algorithm above.

Q2. Suppose that you are given the following Divide and Conquer (D&C) algorithm:
Rewrite

$$A(x) = x^{n/2} \cdot A_1(x) + A_2(x), \quad B(x) = x^{n/2} \cdot B_1(x) + B_2(x)$$

where $A_1(x), A_2(x), B_1(x), B_2(x)$ are polynomials of degree up to $n/2$.

We now compute four smaller polynomial multiplications:

$$A_1(x) \times B_1(x), \quad A_1(x) \times B_2(x), \quad A_2(x) \times B_1(x), \quad A_2(x) \times B_2(x).$$

Then compute:

$$C(x) = x^n \cdot [A_1(x) \times B_1(x)] + x^{n/2} \cdot [A_1(x) \times B_2(x) + A_2(x) \times B_1(x)] + A_2(x) \times B_2(x).$$

Apply this D&C algorithm to compute the multiplication of the same two polynomials of degree $n = 2$:

$$A(10) = 352 = 10 \cdot (3 \cdot 10 + 5) + 2$$

$$B(10) = 221 = 10 \cdot (2 \cdot 10 + 2) + 1$$

Compute $A_1(10) \times B_1(10)$, $A_1(10) \times B_2(10)$, $A_2(10) \times B_1(10)$, $A_2(10) \times B_2(10)$. Then compute $C(10)$.

Q3. What is the time complexity of that recursive D&C algorithm? recurrence relation

$$\begin{aligned} T(n) &= 4T\left(\frac{n}{2}\right) + O(n) \\ R(n) &= \frac{T(n)}{n^2} = R\left(\frac{n}{2}\right) + \frac{n}{n^2} \\ &= R\left(\frac{n}{2}\right) + \frac{1}{n} \\ S(m) &= S(m-1) + \frac{1}{2^m} \\ &\in O\left(S(0) + \sum_{i=1}^m \frac{1}{2^i}\right) \\ &\in O(1) \\ \rightarrow R(n) &\in O(1) \\ \rightarrow T(n) &\in O(n^2) \end{aligned}$$

Q5. H-index.

Each paper has a certain number of citations. Your H-index is the largest number H such that you have written and published H papers with at least H citations.

Given the number of citations c_i on each of the n papers you have published (sorted in non-increasing order), what is your H-index?

Design the best algorithm to solve this problem as efficiently as possible. Analyze the time complexity in Big-O notation.

```

int hIndex(vector<int>& citations) {
    sort(citations.begin(), citations.end(), greater<int>());
    int n = citations.size(), lo = 0, hi = n - 1, ans = 0;
    while (lo <= hi) {
        int mid = lo + (hi - lo) / 2;
        if (citations[mid] >= mid + 1) {
            ans = mid + 1;
            lo = mid + 1;
        } else {
            hi = mid - 1;
        }
    }
    return ans;
}

```

Q6. Decision Tree.

You are given x balls, all but one of which have the same weight; the remaining one is heavier. Your job is to find which of the balls is heavier.

You have a balance scale, and you want to minimize the worst-case number of weighings needed.

What is the minimum number of weighings needed to find the ball?

Assume we only use the comparison model (comparison returns $<$, $=$, or $>$). What is the lower bound of any algorithm to solve this problem?

Among x balls there exists x possible outcomes (x leaves), and the height of the decision tree is $\log_n(n)$. In each comparison there are 3 possible outcomes, both halves being equal, and either one being heavier or lighter than the other, thus the branching factor of the decision tree is 3. When $x \% 2 \neq 0$, No additional weighing needed for every comparison. the lower bound is $\log_3 n$.

5.7 Tutorial: Order Statistics (Model Answers)

This subsection summarizes the model answers for CS3230 Tutorial 11 (Order Statistics), based on the official document (last modified 7 November 2025). Topics include linear-time sorting, linear-time selection, and the expected-time analysis of randomized Quickselect.

Review: Linear-Time Sorting and Selection. Comparison-based sorting has a lower bound of $\Omega(n \log n)$ via the decision-tree model. However, non-comparison-based algorithms break this bound:

1. **Counting Sort**, stable or non-stable. Applicable only when keys lie in a small integer range $[0..k - 1]$.
2. **Radix Sort** (LSD variant). Requires stable counting sort on each digit; if b -bit integers are split into b/r blocks of r bits, then counting sort must support $k = 2^r$.

The *selection problem* asks for the i -th smallest element of an unsorted array under the comparison model. Methods include:

1. $\mathcal{O}(in)$ “complete search” — slowest when $i \approx n/2$.
2. Reduce to sorting: $\Theta(n \log n)$.
3. Median-of-medians algorithm: worst-case $\Theta(n)$, but impractical.
4. **Randomized Quickselect**: expected $\Theta(n)$ and practical.

Q1. Best running time to find the $\sqrt{n}$ closest points to the origin.

Given n points, we want the $\sqrt{n}$ points closest to $(0, 0)$.

Best running time: $\Theta(n)$.

Using median-of-medians:

1. Compute all n distances: $\Theta(n)$.
2. Select the $\sqrt{n}$ -th smallest distance: $\Theta(n)$.
3. Partition around it: $\Theta(n)$.
4. Output the first $\sqrt{n}$ items: $\Theta(\sqrt{n})$.

Total: $\Theta(n)$, which is optimal since all n points must be inspected. If we do *not* know median-of-medians, then sorting gives $\Theta(n \log n)$ instead.

Q2. Can deterministic Quicksort be modified to worst-case $\Theta(n \log n)$?

- *Answer: Yes.*

Use median-of-medians to choose the true median as pivot each time. Each pivot splits the array into two equal halves, yielding

$$T(n) = 2T(n/2) + \Theta(n) = \Theta(n \log n).$$

However, this version is slower in practice than randomized Quicksort.

Q3. Worst-case vs. lucky-case recurrences for Quickselect.

Part 1 (lucky case). Assume each recursive call reduces n to at most $\frac{9}{10}n$:

$$T(n) = T(9n/10) + \Theta(n)$$

This is Master Theorem Case 3:

$$T(n) = \Theta(n).$$

Part 2 (unlucky case). Worst-case split is $1 : (n - 1)$:

$$T(n) = T(n - 1) + \Theta(n) = \Theta(n^2).$$

Conclusion. Among the choices:

Correct: $O(n)$ for Part 1 and $O(n^2)$ for Part 2.

Q4. Expected-time analysis of Randomized Quickselect.

Define indicator variables for split outcomes. For $k \in \{0, \dots, n - 1\}$,

$$X_k = \begin{cases} 1, & \text{if partition produces a } k : (n - k - 1) \text{ split,} \\ 0, & \text{otherwise.} \end{cases}$$

Q4a. Expected value of X_k . Each of the n pivot positions is equally likely:

$$\mathbb{E}[X_k] = \frac{1}{n}.$$

Q4b. Derivation. If $X_k = 1$,

$$T(n) = T(\max(k, n - k - 1)) + \Theta(n).$$

Thus:

$$T(n) = \sum_{k=0}^{n-1} X_k (T(\max(k, n - k - 1)) + \Theta(n)). \quad (1)$$

Taking expectation:

$$\mathbb{E}[T(n)] = \sum_{k=0}^{n-1} \mathbb{E}[X_k (T(\max(k, n - k - 1)) + \Theta(n))]. \quad (2)$$

Independence allows:

$$\mathbb{E}[T(n)] = \sum_{k=0}^{n-1} \mathbb{E}[X_k] \mathbb{E}[T(\max(k, n - k - 1)) + \Theta(n)]. \quad (3)$$

Since $\mathbb{E}[X_k] = 1/n$,

$$\mathbb{E}[T(n)] = \frac{1}{n} \sum_{k=0}^{n-1} \mathbb{E}[T(\max(k, n - k - 1))] + \Theta(n). \quad (4)$$

By symmetry between k and $n - k - 1$:

$$\mathbb{E}[T(n)] \leq \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} \mathbb{E}[T(k)] + \Theta(n). \quad (5)$$

Q4c. Solving the inequality. Assume inductively $\mathbb{E}[T(k)] \leq ck$ for all $k < n$. Then:

$$\mathbb{E}[T(n)] \leq \frac{2}{n} \sum_{k=\lfloor n/2 \rfloor}^{n-1} ck + dn \leq \frac{2c}{n} \left(\frac{3n^2}{8} \right) + dn = \frac{3c}{4}n + dn.$$

Choose c large enough so that

$$\frac{3c}{4}n + dn \leq cn.$$

Thus:

$$\boxed{\mathbb{E}[T(n)] = O(n).}$$

Summary. Randomized Quickselect runs in expected linear time, despite worst-case $\Theta(n^2)$ when pivots split extremely unbalanced (e.g. sorted arrays with non-random pivots, or many duplicates). Mitigations include:

- randomized tie-breaking,
- three-way partitioning (Dutch national flag),
- perturbing equal keys with tiny random ε .

Q5 (optional). Additional order-statistics practice.

If time permits, discuss the following order-statistics-related LeetCode tasks:

- **query-kth-smallest-trimmed-number** Radix-sort-based trimming, complexity $O(dn)$ if precomputing digit-orderings.
- **kth-smallest-element-in-a-BST** Use inorder traversal; augmented BST for repeated updates.
- **kth-largest-element-in-a-stream** Maintain a binary min-heap of size k , update per stream element.

6 Randomized Algorithms

6.1 Freivalds

To check matrices if $A_{n \times n} \times B_{n \times n} = C$,

```

let  $v = [v_1, v_2, \dots, v_n]^T$ , where  $\forall v_i, P(v_i = 0) = P(v_i = 1) = \frac{1}{2}$ 
if  $ABv = Cv \rightarrow AB = C$ 
else  $\rightarrow AB \neq C$ 

```

6.1.1 Probability Analysis

Case 1: $AB = C \implies ABv = Cv$, the algorithm will always decide correctly

Case 2: $AB \neq C$

let $C^* = AB - C, u = [u_1, u_2, \dots, u_n]^T = C^*v$

The probability of the algorithm giving incorrect answer is when $AB \neq C \wedge ABv = Cv \iff \forall i \in [n], u_i = 0$

There $P(\text{success}) = P(\exists k \in [n] u_k \neq 0)$.

Since $AB \neq C, \exists C^*[i][j] \neq 0$,

$$u_i = \sum_{k=0}^n C^*[i][k] \cdot v_k = X + C^*[i][j] \cdot v_j$$

Regardless the randomness, there exists **at most one**^{*} choice for v_j to make $u_i = 0$ as $v_j = \frac{-X}{C^*[i][j]}$ (Deferred decision).

^{*}(If $C^*[i][j] \cdot v_j = 0$, There cannot be a v_j of different value that makes $u_i = 0$ as the value of $C^*[i][j]$ is fixed; There can be no possible choice of v_j to make $u_i = 0$, as $X, C^*[i][j]$ are both fixed, as v_j can be either 0 or 1 only.)

At most 1 choice for v_j to make $u_i = 0 \rightarrow$ At least 1 choice for v_j to make $u_i \neq 0 \rightarrow P(\text{success}) \geq \frac{1}{2}$

6.1.2 Success Probability Amplification

- One-sided error: (OR for RP, AND for coRP) With probability of success $c > 0$, with t repetition the overall success probability can be amplified to $1 - (1 - c)^t$
- Two-sided error: (BPP) with success probability $\geq \frac{1}{2} + \epsilon$, for some $\epsilon > 0$, t independent runs reduces error to $e^{-\Theta(\epsilon^2 t)}$. To ensure failure probability to be $\leq \delta$,

$$e^{-c\epsilon^2 t} \leq \delta \rightarrow t \geq \frac{c \ln(\frac{1}{\delta})}{\epsilon^2} = \Theta(\frac{\ln(\frac{1}{\delta})}{\epsilon^2})$$

Using success probability amplification the success probability of Freivalds can be increase to $1 - \frac{1}{2^t}$ with t runs

6.2 Coupon collection and Hashing

Both problems can be abstracted into matching m objects into n slots, and seek the probability that every slot contains at least 1 object. We put 1 object into any slot independently with a probability of $\frac{1}{n}$, repeat for m times, the probability that 1 slot contains no object is $(1 - \frac{1}{n})^m$. Using inequality

$$(1 + x) \leq e^x$$

We have

$$(1 - \frac{1}{n})^m = (1 - \frac{1}{n})^{\frac{nm}{n}} \leq e^{-\frac{m}{n}}$$

The probability of any of the n slots to be empty is $n(1 - \frac{1}{n})^m \leq ne^{-\frac{m}{n}}$ (Union),

6.3 Randomized Quicksort: Analysis

Algorithm Choose a pivot uniformly at random from the n elements; partition the array into $<\text{pivot}$ and $>\text{pivot}$ using $n - 1$ comparisons; recurse on the two subarrays. Worst case $O(n^2)$ (if pivots are extreme), but randomization makes this very unlikely and the *expected* running time is $\Theta(n \log n)$.

Expected Comparisons via Pairwise Indicators. Let the sorted keys be $x_1 < \dots < x_n$. For $1 \leq i < j \leq n$, define the indicator

$$X_{i,j} = \mathbf{1}\{x_i \text{ and } x_j \text{ are directly compared}\}.$$

In randomized quicksort, a pair (x_i, x_j) is compared *iff* the first pivot chosen from the subset $\{x_i, x_{i+1}, \dots, x_j\}$ is either x_i or x_j . Since the first pivot is uniform on that subset,

$$\Pr[X_{i,j} = 1] = \frac{2}{j - i + 1}.$$

Let C_n be the (random) number of comparisons. By linearity of expectation,

$$\mathbb{E}[C_n] = \sum_{1 \leq i < j \leq n} \Pr[X_{i,j} = 1] = 2 \sum_{i=1}^{n-1} \sum_{j=i+1}^n \frac{2}{j-i+1} = 2 \sum_{i=1}^{n-1} \left(\frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n-i+1} \right) \in O(n \log(n))$$

A standard harmonic-number simplification yields the exact closed form

$$\boxed{\mathbb{E}[C_n] = 2(n+1)H_n - 4n} \quad \text{where } H_n = \sum_{k=1}^n \frac{1}{k}.$$

Asymptotically $H_n = \ln n + \gamma + o(1)$, so

$$\mathbb{E}[C_n] = 2n \ln n + O(n).$$

Therefore the expected running time is $\Theta(n \log n)$ (partition is linear-time and dominates by a constant factor of comparisons).

Expectation Recurrence (Avg case).

- pivots are selected uniformly at random

$$\forall j \in [n], \Pr[pivot = a_j] = \frac{1}{n}$$

- the permutations for both recursive calls are also uniformly random:

Permutations of $A_s = (a_1, a_2, \dots, a_{j-1})$ appear with equal probability, so does permutations of $A_t = (a_{j+1}, a_{j+2}, \dots, a_n)$

$$A(n) = \frac{1}{n} \sum_{j=1}^n [A(j-1) + A(n-j) + cn] = cn + \frac{2}{n} \sum_{j=0}^{n-1} A(j)$$

Using telescoping, multiply by n

$$n \cdot A(n) = cn^2 + 2 \sum_{j=0}^{n-1} A(j)$$

$$(n-1)A(n-1) = c(n-1)^2 + 2 \sum_{j=0}^{n-2} A(j)$$

$$nA(n) - (n-1)A(n-1) = c(2n-1) + 2A(n-1)$$

$$nA(n) - (n+1)A(n-1) = c(2n-1) + 2A(n-1) - 2A(n-1)$$

$$nA(n) - (n+1)A(n-1) = c(2n-1)$$

$$\frac{nA(n) - (n+1)A(n-1)}{n(n+1)} = \frac{c(2n-1)}{n(n+1)}$$

$$\frac{A(n)}{n+1} - \frac{A(n-1)}{n} = \frac{c(2n-1)}{n(n+1)} < \frac{c(2n+2)}{n(n+1)} = \frac{2c}{n}$$

$$\frac{A(n)}{n+1} - \frac{A(n-1)}{n} < \frac{2c}{n}$$

Telescoping again to solve inequality

$$\frac{A(n)}{n+1} - \frac{A(n-1)}{n} < \frac{2c}{n}$$

$$\frac{A(n-1)}{n} - \frac{A(n-2)}{n-1} < \frac{2c}{n-1}$$

$$\frac{A(n-2)}{n-1} - \frac{A(n-3)}{n-2} < \frac{2c}{n-2}$$

...

$$\frac{A(2)}{3} - \frac{A(1)}{2} < \frac{2c}{2}$$

Sum left and right

:

$$\frac{A(n)}{n+1} - \frac{A(1)}{2} < 2c \left(\frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \right)$$

$$\frac{A(n)}{n+1} < 2c \left(\frac{1}{2} + \frac{1}{3} + \cdots + \frac{1}{n} \right) + \frac{A(1)}{2}$$

$$A(n) \in O(n \log(n))$$

with $T(0) = T(1) = \Theta(1)$. This solves to $T(n) = \Theta(n \log n)$. (One can prove $T(n) \leq an \log n + bn$ by induction using the identity $\sum_{k=1}^{n-1} k \ln k = \frac{n(n-1)}{2} \ln n - \frac{n(n-1)}{4} + O(n)$.)

High-Probability Behavior. For any fixed $\alpha \in (0, 1/2)$, each split is *good* if it yields subproblems of size at most $(1 - \alpha)n$. A random pivot is good with constant probability $p = 1 - 2\alpha$. After $O(\log n)$ good splits, all subproblems become size 1. By Chernoff bounds on the count of good splits among $O(\log n)$ levels, total work is $O(n \log n)$ *with high probability*.

Space and Depth. Recursion depth is $O(\log n)$ in expectation and with high probability; worst case is n . Tail recursion plus choosing the larger side for iteration yields $O(\log n)$ expected stack space.

Worst-Case Probability (tiny). Hitting the worst-case pattern (always picking an extreme pivot) has probability

$$\prod_{k=1}^{n-1} \frac{2}{n+1-k} = \frac{2^{n-1}}{n!},$$

which decays super-exponentially; nevertheless, the algorithm remains *worst-case* $O(n^2)$.

Implementation Notes (brief). Use in-place partition (Lomuto/Hoare), pick a uniform random index per call, and tail-recurse on the smaller side to control stack depth. Deterministic median-of-three improves constants but not the $\Theta(n \log n)$ expectation guarantee of the fully randomized version.

Takeaway. Randomized quicksort runs in expected $\Theta(n \log n)$ time with an exact expected comparison count $2(n+1)H_n - 4n$, uses linear extra work per level, and is efficient in practice due to good cache behavior and small constants.

6.4 Reservoir sampling

How to ensure uniform probability of selection when the size of the sample space is unknown? for example to select a node in a linkedlist at equal probability with only one traverse?

Suppose we have a buffer R of size k , and an infinite incoming stream $\{x_1, x_2, \dots\}$.

```

R ← {x1, x2, ..., xk}
for i ≥ k + 1:
    with probability  $\frac{k}{i}$ , replace a random entry of R with xi

```

We can show that $\forall t \geq i$, $\Pr[x_i \in R_t] = \frac{k}{t}$, i.e. at any time $t \geq k$, R contains a uniformly random subset of $\{x_1, x_2, \dots, x_t\}$.

Proof. Suppose $t = k$, $\Pr[x_i \in R_t] = 1 = \frac{k}{t}$, as per how R is initialized. This is our base case. Suppose for some $t \geq k$, $\Pr[x_i \in R_t] = \frac{k}{t}$,

$$\begin{aligned}
 \Pr[x_i \in R_{t+1}] &= \Pr[x_i \in R_t] \times \Pr[x_i \text{ is not replaced at } t+1] \\
 &= \frac{k}{t} \cdot \left(1 - \frac{k}{t+1} \cdot \frac{1}{k}\right) \\
 &= \frac{k}{t} \cdot \frac{t}{t+1} \\
 &= \frac{k}{t+1}
 \end{aligned}$$

□

6.5 Fisher–Yates (a.k.a. Knuth/Durstenfeld) Shuffle

Hint. The only way to be uniformly random over $n!$ permutations is to make *exactly* $n - 1$ independent choices with the right ranges; anything else (e.g., picking a random index from $[0, n-1]$ each time) introduces bias.

Goal. Given an array $A[0..n-1]$, produce a uniformly random permutation in-place, in $O(n)$ time and $O(1)$ extra space.

Algorithm (Durstenfeld in-place). For i from $n - 1$ down to 1:

pick $j \sim \text{Unif}\{0, \dots, i\}$; swap $A[i] \leftrightarrow A[j]$.

(Allow $j = i$; swapping with itself is fine.)

Correctness (uniformity). At step i there are exactly $i + 1$ choices equally likely. The sequence of choices therefore has probability $\prod_{i=1}^{n-1} \frac{1}{i+1} = \frac{1}{n!}$. Each distinct sequence of choices yields a distinct permutation, hence every permutation occurs with probability $1/n!$. Equivalently, by induction: after processing index i , the suffix $A[i..n-1]$ is a uniformly random set of $n-i$ elements in a uniformly random order, and the step preserves uniformity.

Complexity. Time $O(n)$ (one RNG draw and a swap per i), space $O(1)$, stable distribution over S_n .

Common Pitfalls (nitpicky but fatal).

- Using a fixed range $j \sim \{0, \dots, n-1\}$ for every i (biased).
- Looping i upwards with $j \sim \{0, \dots, i\}$ and swapping $A[i] \leftrightarrow A[j]$ (also biased unless you use the exact “inside-out” scheme).
- Modulo bias: $j = \text{rand}() \% (i+1)$ with poor RNG; if RNG range is not a multiple of $(i+1)$, distribution skews.
- Re-seeding per iteration or using time-based seeds in tight loops (highly correlated).

Inside-Out Variant (build permutation of $[0..n-1]$ from scratch). For $i = 0..n-1$: set $p[i] = i$; pick $j \sim \{0, \dots, i\}$; set $p[i] = p[j]$, $p[j] = i$. This yields a uniform permutation without an extra final shuffle pass.

C++ (correct, modern <random>):

```
// In-place Fisher-Yates (Durstenfeld)
template<class URBG>
void fisher_yates(vector<int>& a, URBG& gen) {
    for (int i = (int)a.size() - 1; i > 0; --i) {
        std::uniform_int_distribution<int> dist(0, i);
        int j = dist(gen);
        std::swap(a[i], a[j]);
    }
}

// Inside-out: generate a random permutation of 0..n-1
template<class URBG>
vector<int> random_permutation(int n, URBG& gen) {
    vector<int> p(n);
    for (int i = 0; i < n; ++i) {
        std::uniform_int_distribution<int> dist(0, i);
        int j = dist(gen);
        p[i] = (j == i ? i : p[j]);
        p[j] = i;
    }
    return p;
}

// Example usage
int main() {
    std::mt19937_64 rng(std::random_device{}());
    vector<int> a = {1,2,3,4,5,6,7,8,9};
    fisher_yates(a, rng);

    auto p = random_permutation(10, rng);
}
```

Notes on RNG Quality. Prefer `std::mt19937` or `mt19937_64` with `uniform_int_distribution`. Avoid `rand()` and `%` (modulo bias). For cryptographic needs, use a CSPRNG.

Testing Uniformity (sanity). Repeat the shuffle many times; each value should appear in each position with frequency $\approx 1/n$ (chi-square test). Correlations between positions should be negligible.

6.6 Relationship Between Expected Runtime and Tail Decay

Let T be the (random) runtime of a randomized procedure $\mathcal{A}$. We consider the following two assertions:

- A1. $\mathbb{E}[T] < \infty$.
- A2. $\lim_{t \rightarrow \infty} \Pr[T > t] = 0$.

A1 implies A2. If $\mathbb{E}[T] < \infty$, then Markov's inequality gives

$$\Pr[T > t] \leq \frac{\mathbb{E}[T]}{t}.$$

Since the right-hand side tends to 0 as $t \rightarrow \infty$, we obtain

$$\lim_{t \rightarrow \infty} \Pr[T > t] = 0,$$

and hence $A1 \Rightarrow A2$.

A2 does not imply A1. A2 merely requires that the tail probability $\Pr[T > t]$ goes to 0, but it imposes no constraint on how *slowly* it goes to 0. It is possible for the tail to decay to 0 so slowly that the expected value diverges.

Consider the following distribution:

$$\Pr[T = 2^i] = 2^{-i} \quad \text{for } i = 1, 2, 3, \dots$$

Then

$$\Pr[T > t] = \sum_{2^i > t} 2^{-i} = \Theta\left(\frac{1}{t}\right) \rightarrow 0,$$

so A2 holds.

However, the expected runtime is

$$\mathbb{E}[T] = \sum_{i=1}^{\infty} 2^i \cdot 2^{-i} = \sum_{i=1}^{\infty} 1 = \infty,$$

so A1 fails. Thus $A2 \not\rightarrow A1$.

Conclusion. The correct statement is

(b) (A1 implies A2) and (A2 does not imply A1).

Q1

In class, we showed that Freivalds' algorithm succeeds with a probability of at least $1/2$. Show that the bound $1/2$ in the analysis is actually the best possible by constructing an input (A, B, C) on which the success probability of Freivalds' algorithm is precisely $1/2$. *Hint: Use small matrices.*

At most 1 choice for v_j to make $u_i = 0 \rightarrow$ At least 1 choice for v_j to make $u_i \neq 0 \rightarrow \Pr(\text{success}) \geq \frac{1}{2}$

So long as we make sure there is exactly one v_j can make $u_i = 0$. For example, only one element in C^* is none zero, or the size of v is only 1. the probability of success is exactly $\frac{1}{2}$.

Q2

Consider the equality testing problem: Alice holds an n -bit string $S_A \in \{0, 1\}^n$, Bob holds an n -bit string $S_B \in \{0, 1\}^n$, and they want to decide whether $S_A = S_B$.

The protocol is as follows:

1. Let S be the set of n^2 smallest prime numbers.
2. Alice samples a number $p \in S$ uniformly at random.
3. Alice sends p and $S_A \bmod p$ to Bob (only $O(\log p) \subseteq O(\log n)$ bits).
4. Bob computes $S_B \bmod p$.
5. If $S_A \bmod p = S_B \bmod p$, Bob decides $S_A = S_B$; otherwise $S_A \neq S_B$.

Show that this randomized communication protocol is correct with probability $\geq 1 - \frac{1}{n}$.

If $S_A = S_B$, $S_A \bmod p = S_B \bmod p$, the algorithm always decides correctly.

Calculate $\Pr(S_A \neq S_B \wedge S_A \bmod p = S_B \bmod p)$;

$$S_A \neq S_B \wedge S_A \bmod p = S_B \bmod p \rightarrow \exists c \in \mathbb{N}, |S_A - S_B| = cp \rightarrow (S_A - S_B | p)$$

Suppose both S_A and S_B have uniformly randomly selected bits, $S_A - S_B$ also has uniformly chosen bits, the probability of $(S_A - S_B | p)$ is at most $\frac{1}{nx}$, the probability of success is at least $1 - \frac{1}{n}$

Remark. By the prime number theorem, $p \in O(n^2 \log n)$, so this protocol requires communicating only $O(\log p) \subseteq O(\log n)$ bits. This demonstrates an exponential separation between randomized and deterministic algorithms, since any deterministic algorithm requires $\Omega(n)$ bits in the worst case.

Q3

Show that any deterministic algorithm solving the equality testing problem requires communicating $\Omega(n)$ bits in the worst case.

To deterministically check the equality of 2 binary strings of n bits, there are 2^n binary strings of the same number of bits, and each comparison can check the equality of 1 bit, thus the height of the decision tree is $\log_2(2^n) = n$

Q4

You are given a graph $G = (V, E)$ without self-loops. Partition its vertex set into $V = V_1 \cup V_2$ randomly as follows:

- Each vertex $v \in V$ flips an unbiased coin independently.
- If heads, $v \in V_1$; if tails, $v \in V_2$.

Show that the expected number of edges crossing (V_1, V_2) is exactly $|E|/2$.

Suppose all vertices are partitioned uniformly randomly into V_1, V_2 , $\forall e(u, v) \in E$, the probability of e crosses between $V_1, V_2 = P((u \in V_1 \wedge v \in V_2) \vee (u \in V_2 \wedge v \in V_1)) = \frac{1}{4} + \frac{1}{4} = \frac{1}{2}$

$$\mathbb{E}(|E|e \text{ cross}) = |E|\mathbb{E}(e \text{ crosses}) = \frac{|E|}{2}$$

Remark. As a corollary, every graph $G = (V, E)$ admits a cut of size at least $|E|/2$.

Q5

Is it possible to improve the bound $|E|/2$ above? If yes, propose some ideas and analyze the new bound.

7 Dynamic Programming

7.1 The Knapsack Problem and Its Variants

The family of knapsack problems is central in algorithm design and dynamic programming. Each variant differs in whether items may be taken once, infinitely many times, or with bounded multiplicities. We discuss the standard recursive formulation, derive each recurrence, and show how the recurrences translate into tabulated DP algorithms.

1. Classical 0/1 Knapsack

Problem. Given n items, each with weight w_i and value v_i , and knapsack capacity W , find the maximum total value using each item at most once.

Recursive formulation. Let $K(i, w)$ be the maximum value achievable using the first i items within capacity w .
Base cases:

$$K(0, w) = 0, \quad K(i, 0) = 0.$$

If item i is too heavy:

$$K(i, w) = K(i - 1, w).$$

Otherwise:

$$K(i, w) = \max(K(i - 1, w), v_i + K(i - 1, w - w_i)).$$

DP table (bottom-up). Construct a table $dp[i][w] = K(i, w)$ for $i = 0 \dots n$, $w = 0 \dots W$:

$$dp[i][w] = \begin{cases} dp[i - 1][w] & w_i > w, \\ \max(dp[i - 1][w], v_i + dp[i - 1][w - w_i]) & w_i \leq w. \end{cases}$$

Time complexity: $\Theta(nW)$.

2. Complete (Unbounded) Knapsack

Problem. Each item i may be taken *infinitely many times*. This changes the structure: the choice to include i does not prevent including it again.

Recursive formulation. Base cases remain 0. If $w_i > w$:

$$K(i, w) = K(i - 1, w).$$

Otherwise:

$$K(i, w) = \max(K(i - 1, w), v_i + K(i, w - w_i)).$$

Key difference: the second term uses $K(i, w - w_i)$ instead of $K(i - 1, w - w_i)$ because we may reuse item i .

DP table formulation.

$$dp[i][w] = \max(dp[i - 1][w], v_i + dp[i][w - w_i]).$$

To optimize, we loop w *increasing*:

$$\text{for } i = 1..n : \quad \text{for } w = w_i..W : \quad dp[w] = \max(dp[w], v_i + dp[w - w_i]).$$

This avoids repetition and ensures correctness.

Interpretation. This variant resembles coin change, rod cutting, or unbounded production tasks.

3. Multiple / Bounded Knapsack

Problem. Each item i may be taken at most c_i times.

Recursive formulation. Naively:

$$K(i, w) = \max_{0 \leq k \leq c_i, kw_i \leq w} (kv_i + K(i - 1, w - kw_i)).$$

This directly follows from enumerating k copies of item i .

DP table formulation. The naive DP:

$$dp[i][w] = \max_{0 \leq k \leq c_i, kw_i \leq w} (dp[i - 1][w - kw_i] + kv_i).$$

This takes $O(nW \cdot \max c_i)$ in general and can be improved by *binary splitting* (a.k.a. binary optimization) of c_i :

$$c_i = (1 + 2 + 4 + \dots + 2^k) + r.$$

This converts bounded knapsack into a series of $O(\log c_i)$ 0/1 knapsack items.

Time complexity. With binary splitting:

$$O\left(W \sum_{i=1}^n \log c_i\right).$$

4. Relationship Between the Variants

Each recurrence contains the same structural choices but differs in the subproblem invoked when an item is chosen:

Variant	Choice if take item i
0/1	$v_i + K(i-1, w-w_i)$
Unbounded	$v_i + K(i, w-w_i)$
Bounded	$\max_{k \leq c_i} (kv_i + K(i-1, w-kw_i))$

Conceptually:

- 0/1 Knapsack disallows reuse $\Rightarrow$ previous row only.
- Complete Knapsack allows reuse $\Rightarrow$ current row reused.
- Bounded Knapsack is between them, requiring enumeration or binary expansion.

5. Summary

Knapsack DP always follows the same overarching pattern:

$$\text{value}(w) = \max\{\text{skip item, take item (once/many/limited)}\}.$$

What differs is the *subproblem* that taking an item leads to:

$$K(i, w-w_i) \text{ (0/1), } K(i, w-w_i) \text{ (unbounded), } K(i-1, w-kw_i) \text{ (bounded).}$$

The transition between the recurrence and the tabulated DP is systematic: rows correspond to items, columns correspond to capacities, and the change of index in the “take” case reflects the allowed multiplicities.

7.2 Longest Increasing Subsequence and Its Variants

The Longest Increasing Subsequence (LIS) problem is a fundamental example of dynamic programming, binary search optimization, and multi-dimensional ordering. This subsection summarizes its DP formulation, the $O(n \log n)$ algorithm, and common extensions such as 2D LIS on point sets and on 2D grids.

7.2.1 Longest Increasing Subsequence (1D)

Problem. Given an array $A[1..n]$, find the longest subsequence

$$i_1 < i_2 < \dots < i_k \quad \text{such that} \quad A[i_1] < A[i_2] < \dots < A[i_k].$$

DP recurrence (classical $O(n^2)$). Let $dp[i]$ be the length of the LIS ending at index i .

$$dp[i] = 1 + \max_{j < i, A[j] < A[i]} dp[j], \quad dp[i] = 1 \text{ if no such } j.$$

The final answer is $\max_i dp[i]$.

Binary-search optimization ($O(n \log n)$). Maintain an array T where $T[\ell]$ stores the minimum possible ending value of an increasing subsequence of length ℓ .

Algorithm:

1. Initialize an empty array T .
2. For each $A[i]$: use binary search to find the smallest ℓ such that $T[\ell] \geq A[i]$.
3. Set $T[\ell] = A[i]$.

The length of T equals the LIS length. This method exploits the monotonic structure of the DP.

7.2.2 Longest Contiguous Increasing Subarray with Switching Between Price Sources

We are given three arrays `price1`, `price2`, and `price3` of equal length n , representing the stock price reported by three different market makers on each day. On every day i , we may freely choose one of the three prices to form a *combined* array `price`. Our goal is to determine the length of the longest contiguous non-empty subarray of `price` whose values are strictly increasing.

DP formulation. Let the three sources be indexed by $k \in \{1, 2, 3\}$ and the days by $i \in \{1, \dots, n\}$. Define

$dp[k][i]$ = length of the longest valid strictly increasing contiguous segment ending at day i if we choose price source k on day i .

Base case. A single day alone forms a strictly increasing subarray of length 1:

$$\text{dp}[k][1] = 1 \quad \forall k \in \{1, 2, 3\}.$$

Recurrence relation. For each day $i > 1$, if we choose price source k on day i , then we may extend a segment ending at day $i - 1$ only if the chosen price is strictly larger than the previous day's value:

$$\text{dp}[k][i] = 1 + \max_{\substack{j \in \{1, 2, 3\} \\ \text{price}_j[i-1] < \text{price}_k[i]}} \text{dp}[j][i-1].$$

If no j satisfies $\text{price}_j[i-1] < \text{price}_k[i]$, then the segment must start anew at day i :

$$\text{dp}[k][i] = 1.$$

Final answer. The length of the longest valid segment is

$$\max_{i=1..n} \max_{k=1..3} \text{dp}[k][i].$$

Complexity. The DP table has only $3n$ states, and each state checks only the three possible previous choices, giving a linear running time:

$$O(n).$$

Space usage can be $O(3n)$ or reduced to $O(1)$ by storing only values for day i and day $i - 1$.

Summary. This problem is solved by a small constant-state dynamic program. At each day, the decision depends only on the strictly increasing condition against the previous day's three possible prices. Contiguosness is enforced naturally because each $\text{dp}[k][i]$ extends only from day $i - 1$, and switching between sources is free.

7.2.3 LIS in 2D: Longest Increasing Sequence in the Plane

A common generalization is LIS over points (x_i, y_i) . We seek the largest subset of points ordered so that

$$x_{i_1} < x_{i_2} < \dots < x_{i_k} \quad \text{and} \quad y_{i_1} < y_{i_2} < \dots < y_{i_k}.$$

Sort + LIS trick. Sort points by:

$$\text{increasing } x, \quad \text{and for equal } x, \text{ decreasing } y.$$

This prevents selecting two points with equal x or forming invalid ties.

After sorting, the problem reduces to LIS on the y -coordinates:

$$\text{LIS on } (y_1, y_2, \dots, y_n).$$

Time:

$$O(n \log n) \quad \text{via the binary search LIS algorithm.}$$

Applications.

- Russian-doll envelopes (LeetCode 354),
- scheduling where both start- and end-times must increase,
- multi-criteria filtering and skyline queries,
- geometry and dominance problems.

7.2.4 LIS in a 2D Grid (Monotone Paths)

Given an $m \times n$ grid A , a “2D LIS path” requires increasing values along a path moving only rightward or downward:

$$A[i_1][j_1] < A[i_2][j_2] < \dots, \quad i_{t+1} \geq i_t, \quad j_{t+1} \geq j_t.$$

DP formulation. Let $\text{dp}[i][j]$ be the length of the longest increasing path ending at (i, j) .

$$\text{dp}[i][j] = 1 + \max \left(\text{dp}[i-1][j] \text{ if } A[i-1][j] < A[i][j], \quad \text{dp}[i][j-1] \text{ if } A[i][j-1] < A[i][j] \right).$$

Boundary:

$$\text{dp}[i][j] = 1 \quad \text{if neither neighbor is smaller.}$$

Time complexity:

$$O(mn).$$

Remarks. This is not the same as LIS on the array formed by flattening the matrix, because 2D movement constraints restrict the transitions.

7.2.5 Longest Chain in Higher Dimensions

For d -dimensional points $p_i = (a_{i,1}, a_{i,2}, \dots, a_{i,d})$, we want the longest chain satisfying

$$p_{i_1} < p_{i_2} < \dots < p_{i_k} \quad \text{in all coordinates.}$$

Typical solution:

- Sort lexicographically by $(a_1, a_2, \dots, a_{d-1})$,
- Apply LIS to the last coordinate, possibly with transformations when ties occur.

This generalizes the 2D LIS sorting trick.

Summary of Patterns

- 1D LIS: $dp[i] = 1 + \max_{j < i, A[j] < A[i]} dp[j]$ or $O(n \log n)$ via patience sorting.
- 2D LIS: sort by one coordinate, apply 1D LIS to the other.
- Grid LIS: DP with directional constraints; no binary-search reduction.
- Higher-dimensional LIS: sort by all but one coordinate, LIS on last.

Across all variants, the structural pattern is:

Sort or order the states so that the remaining dimension becomes 1D LIS.

This unifies many “increasing sequence” problems.

Path DP

Floyd-Warshall: recurrence:

$$T(i, j) = 0, i = j \quad T(i, j) = T()$$

Interval DP

Example: convex polygon triangulation

$$\begin{aligned} T[x, y] &= 0 \quad y \leq x + 1 \\ T[x, y] &= \min_{x < k < y} (T[x, k] + W(x, k, y), T[k, y]) \end{aligned}$$

Bottom-up DP: iterative DP (Ex: coin change)

given the recurrence: $T(n) = \min_{c \in \text{coins}} (T(n - c) + 1)$, we know the state we need to remember is the min number of coins to get $n - c$. Only 1 param $\rightarrow$ 1D DP

```
int coinChangeMin(vector<int>& coins, int amount) {
    const int INF = 1e9;           // sentinel for impossible
    vector<int> dp(amount + 1, INF);
    dp[0] = 0;                     // base: 0 coins to make 0

    for (int a = 1; a <= amount; ++a) {
        for (int c : coins) {
            if (c <= a && dp[a - c] != INF) {
                dp[a] = min(dp[a], dp[a - c] + 1);
            }
        }
    }
    return (dp[amount] == INF ? -1 : dp[amount]);
}
```

Top-down DP: recursion with memoization (Ex: coin change)

```
int helper(int amount, vector<int>& coins, vector<int>& memo) {
    if (amount == 0) return 0;      // base: 0 coins to make 0
    if (amount < 0) return 1e9;     // sentinel: impossible
    if (memo[amount] != -1) return memo[amount];

    int best = 1e9;
    for (int c : coins) {
        best = min(best, 1 + helper(amount - c, coins, memo));
    }
    return memo[amount] = best;
}
```

```

}

int coinChangeMin(vector<int>& coins, int amount) {
    vector<int> memo(amount + 1, -1);
    int res = helper(amount, coins, memo);
    return (res >= 1e9 ? -1 : res); // -1 means impossible
}

```

Notice that in coin change, we only need to maintain a 1D memo table for top-down dp and 1D dp table, as in the naive recursive solution we found out there is only 1 state needs to be remembered, therefore 1 dimensional auxiliary ds is needed. In comparison, for knapsack, where 2 states need to be remembered, remaining weight and current value sum, we need a 2D table to implement dp.

```

int helper (int k, const vector<vector<int>>& c, int cur) {
    if (k < 0) {
        return -1;
    }
    if (k >= 0 && cur == 0) {
        return 0;
    }
    int best = 0;
    return max(helper(k - c[cur][0], c, cur - 1) + c[cur][1],
               helper(k, c, cur - 1));
}

int knapsack(int k, const vector<vector<int>>& c) {
    return helper(k, c, c.size()-1);
}

int knapsack_bu(int k, const vector<vector<int>>& c) {
    vector<vector<int>> dp(c.size() + 1, vector<int>(k+1, -1));
    for (int i = 0; i < k; i++) {
        dp[0][i] = 0;
    }
    for (int i = 0; i < c.size() + 1; i++) {
        dp[i][0] = 0;
    }
    for (int i = 1; i < c.size()+1; i++) {
        for (int j = 1; j < k+1; j++) {
            if (j >= c[i-1][0]) {
                dp[i][j] = max(dp[i-1][j], dp[i-1][j-c[i-1][0]] + c[i-1][1]);
            } else {
                dp[i][j] = dp[i-1][j];
            }
        }
    }
    return dp[c.size()][k-1];
}

```

From another view, to uniquely identify an instance of coin change problem (on the same set of coins available), 1 key is needed (total value); while to uniquely identify an instance of knapsack (on the same set of items), we need 2 keys. This alludes that we can choose the dimension of our dp table based on the number of states need to be maintained.

Higher dimensional DP

In max profit problem,

```

long long maximumProfit(vector<int>& p, int k) {
    int n = p.size();
    if (n < 2 || k == 0) return 0;
    const long long NEG_INF = -(1LL << 60);

    // dp[day][transactions][state]
    // state: 0 = NOT_HOLDING, 1 = HOLDING, 2 = HOLDING_SHORT
    vector<vector<vector<long long>>> dp(
        n, vector<vector<long long>>(k+1, vector<long long>(3, NEG_INF))
    );

    // pad base case: max profit no transaction yet
    dp[0][0][0] = 0; // NOT_HOLDING
    dp[0][0][1] = -(long long)p[0]; // HOLDING
    dp[0][0][2] = (long long)p[0]; // HOLDING_SHORT

    for (int j = 1; j < n; j++) { // day
        for (int i = 0; i <= k; i++) { // transactions
            // if continue to enter NOT_HOLDING
            dp[j][i][0] = max(dp[j][i][0], dp[j-1][i][0]);
            // if choose to enter HOLDING
            dp[j][i][1] = max(dp[j-1][i][1], dp[j-1][i][0] - (long long)p[j]);
            // if choose to enter HOLDING_SHORT
            dp[j][i][2] = max(dp[j-1][i][2], dp[j-1][i][0] + (long long)p[j]);
        }
    }
}

```

```

        if (i >= 1) { // selling only possible after first day
            // if choose to enter NOT_HOLDING
            dp[j][i][0] = max(dp[j][i][0], dp[j-1][i-1][1] + (long long)p[j]);
            dp[j][i][0] = max(dp[j][i][0], dp[j-1][i-1][2] - (long long)p[j]);
        }
    }

    long long ans = 0;
    for (int i = 1; i <= k; i++) {
        ans = max(ans, dp[n-1][i][0]); // need to end with NOT_HOLDING state. (Invariant 5)
    }
    return ans;
}

```

There are 2 obvious states, day and number of transactions. But there are additional constraint in the question, which is the current action state, whether we are holding a stock. This state affects decision making as it is 1. independent from other states, ie any day we may be in any action state; 2. the transition among this action states is one of the constraints posed by the problem.

Here's a quick nudge first: for each problem, name a **state** that captures exactly "what's left to decide," ensure **overlap** (reuse subresults), and pick a **loop order** that enforces the selection rules (0/1 vs unbounded, interval growth, masks, post-order on trees).

7.3 Twenty Classic Dynamic Programming Problems

7.3.1 1. 0/1 Knapsack (maximize value within capacity)

Note: DP solution runs in pseudo-poly time, the problem is NPC

Description. Choose a subset of items (each at most once) to maximize total value without exceeding capacity.

Recurrence/base. Let $dp[i][c]$ be max value using first i items within capacity c . $dp[i][c] = \max(dp[i-1][c], dp[i-1][c-w_i] + v_i)$ if $w_i \leq c$, else $dp[i][c] = dp[i-1][c]$.

Base: $dp[0][c] = 0$, $dp[i][0] = 0$.

```

for i=1..n:
    for c=0..K:
        dp[i][c] = dp[i-1][c]
        if w[i] <= c:
            dp[i][c] = max(dp[i][c], dp[i-1][c-w[i]] + v[i])
answer = dp[n][K]

```

7.3.2 2. Coin Change (min coins, unbounded)

Description. Using unlimited copies of given coin denominations, make amount K with fewest coins.

Recurrence/base. $dp[a] = \min_{d \leq a} (1 + dp[a-d])$.

Base: $dp[0] = 0$; others $+\infty$.

```

dp[0]=0; for a=1..K: dp[a]=INF
for a=1..K:
    for d in coins:
        if d <= a and dp[a-d] < INF:
            dp[a] = min(dp[a], dp[a-d] + 1)
answer = (dp[K]<INF ? dp[K] : -1)

```

7.3.3 3. Longest Increasing Subsequence ($O(n^2)$)

Description. Longest subsequence with strictly increasing values.

Recurrence/base. $dp[i] = 1 + \max_{j < i, a_j < a_i} dp[j]$;

base $dp[i] = 1$.

```

for i=1..n:
    dp[i]=1
    for j=1..i-1:
        if a[j] < a[i]: dp[i]=max(dp[i], dp[j]+1)
answer = max_i dp[i]

```

7.3.4 4. Edit Distance (Levenshtein)

Description. Minimum insert/delete/replace operations to convert string s to t .

Recurrence/base. $dp[i][j] = \min\{dp[i-1][j] + 1, dp[i][j-1] + 1, dp[i-1][j-1] + [s_i \neq t_j]\}$.

Base: $dp[0][j] = j$, $dp[i][0] = i$.

```

for i=0..n: dp[i][0]=i
for j=0..m: dp[0][j]=j
for i=1..n:
    for j=1..m:
        cost = (s[i]==t[j]?0:1)
        dp[i][j]=min(dp[i-1][j]+1, dp[i][j-1]+1, dp[i-1][j-1]+cost)
answer = dp[n][m]

```

7.3.5 5. Matrix Chain Multiplication

Description. Parenthesize a chain of matrices to minimize scalar multiplications.

Recurrence/base. $dp[i][j] = \min_{i \leq k < j} \{dp[i][k] + dp[k+1][j] + p_{i-1}p_kp_j\}$.

Base: $dp[i][i] = 0$.

```

for i=1..n: dp[i][i]=0
for len=2..n:
  for i=1..n-len+1:
    j=i+len-1; dp[i][j]=INF
    for k=i..j-1:
      dp[i][j]=min(dp[i][j], dp[i][k]+dp[k+1][j]+p[i-1]*p[k]*p[j])
answer = dp[1][n]

```

7.3.6 6. Longest Common Subsequence

Description. Longest sequence that appears as a subsequence in both strings.

Recurrence/base. If $s_i = t_j$: $dp[i][j] = dp[i-1][j-1] + 1$, else $dp[i][j] = \max(dp[i-1][j], dp[i][j-1])$.

Base: $dp[0][*] = dp[*][0] = 0$.

```

for i=0..n: dp[i][0]=0
for j=0..m: dp[0][j]=0
for i=1..n:
  for j=1..m:
    if s[i]==t[j]: dp[i][j]=dp[i-1][j-1]+1
    else: dp[i][j]=max(dp[i-1][j], dp[i][j-1])
answer = dp[n][m]

```

7.3.7 7. Subset Sum (feasibility)

Note: DP solution runs in pseudo-poly time, the problem is NPC

Description. Decide if some subset sums exactly to K .

Recurrence/base. $F[i][c] = F[i-1][c] \vee (w_i \leq c \wedge F[i-1][c-w_i])$.

Base: $F[0][0] = \text{true}$, $F[0][c > 0] = \text{false}$.

```

F[0][0]=true; for c=1..K: F[0][c]=false
for i=1..n:
  for c=0..K:
    F[i][c]=F[i-1][c]
    if w[i]<=c and F[i-1][c-w[i]]: F[i][c]=true
answer = F[n][K]

```

7.3.8 8. Weighted Interval Scheduling

Description. Pick non-overlapping weighted intervals to maximize total weight.

Recurrence/base. Sort by finish; let $p(i)$ be last non-overlapping.

$OPT[i] = \max(OPT[i-1], v_i + OPT[p(i)])$. Base $OPT[0] = 0$.

```

OPT[0]=0
for i=1..n:
  OPT[i]=max( OPT[i-1], v[i] + OPT[p(i)] )
answer = OPT[n]

```

7.3.9 9. Rod Cutting

Description. Cut a rod into lengths to maximize sale price.

Recurrence/base. $dp[L] = \max_{1 \leq j \leq L} (price[j] + dp[L-j])$.

Base: $dp[0] = 0$.

```

dp[0]=0
for L=1..N:
  dp[L]=0
  for j=1..L:
    dp[L]=max(dp[L], price[j]+dp[L-j])
answer = dp[N]

```

7.3.10 10. Palindrome Partitioning II (min cuts)

Description. Split a string into palindromic substrings with minimum cuts.

Recurrence/base. Precompute $pal[i][j]$.

$cuts[i] = \min_{0 \leq j \leq i, pal[j][i]} ((j == 0 ? 0 : cuts[j-1] + 1))$.

Base: handled via $j == 0$.

```

build pal[n][n] // O(n^2)
for i=0..n-1:
  cuts[i]=INF
  for j=0..i:
    if pal[j][i]:
      cuts[i]=min(cuts[i], (j==0?0:cuts[j-1]+1))
answer = cuts[n-1]

```

7.3.11 11. Egg Dropping

Description. Minimize worst-case trials to find critical floor with given eggs.

Recurrence/base. $dp[e][f] = 1 + \min_{1 \leq x \leq f} \max(dp[e-1][x-1], dp[e][f-x])$.

Base: $dp[1][f] = f$, $dp[e][0] = 0$.

```

for f=0..F: dp[1][f]=f
for e=2..E: dp[e][0]=0
for e=2..E:
  for f=1..F:
    dp[e][f]=INF
    for x=1..f:
      dp[e][f]=min(dp[e][f], 1+max(dp[e-1][x-1], dp[e][f-x]))
answer = dp[E][F]

```

7.3.12 12. Catalan Numbers (count valid parentheses/BSTs)

Description. Count structures like valid parentheses strings or BST shapes.

Recurrence/base. $C_0 = 1$, $C_n = \sum_{i=0}^{n-1} C_i C_{n-1-i}$.

```

C[0]=1
for n=1..N:
  C[n]=0
  for i=0..n-1:
    C[n]+=C[i]*C[n-1-i]
answer = C[N]

```

7.3.13 13. Word Break (feasibility with dictionary)

Description. Check if a string can be segmented into dictionary words.

Recurrence/base. $dp[i] = \bigvee_{0 \leq j < i} (dp[j] \wedge s[j..i-1] \in D)$.

Base: $dp[0] = \text{true}$.

```

dp[0]=true; for i=1..n: dp[i]=false
for i=1..n:
  for j=0..i-1:
    if dp[j] and (s[j..i-1] in D): dp[i]=true
answer = dp[n]

```

7.3.14 14. Distinct Subsequences (count $S \rightarrow T$)

Description. Number of ways string T appears as a subsequence of S .

Recurrence/base. $dp[i][j] = dp[i-1][j] + [s_i = t_j] \cdot dp[i-1][j-1]$.

Base: $dp[i][0] = 1$, $dp[0][j > 0] = 0$.

```

for i=0..n: dp[i][0]=1
for j=1..m: dp[0][j]=0
for i=1..n:
  for j=1..m:
    dp[i][j]=dp[i-1][j]
    if s[i]==t[j]: dp[i][j]+=dp[i-1][j-1]
answer = dp[n][m]

```

7.3.15 15. Longest Palindromic Subsequence

Description. Longest subsequence that is a palindrome.

Recurrence/base. If $s_i = s_j$: $dp[i][j] = 2 + dp[i+1][j-1]$ else $dp[i][j] = \max(dp[i+1][j], dp[i][j-1])$.

Base: $dp[i][i] = 1$; $dp[i][i-1] = 0$.

```

for i=0..n-1: dp[i][i]=1
for len=2..n:
  for i=0..n-len:
    j=i+len-1
    if s[i]==s[j]:
      dp[i][j] = (len==2?2:2+dp[i+1][j-1])
    else:
      dp[i][j]=max(dp[i+1][j], dp[i][j-1])
answer = dp[0][n-1]

```

7.3.16 16. Tree Minimum Vertex Cover (rooted)

Description. Smallest set of vertices covering all edges in a tree.

Recurrence/base. $dp[u][1] = 1 + \sum_v \min(dp[v][0], dp[v][1])$,

$dp[u][0] = \sum_v dp[v][1]$.

Base: leaf $dp[leaf][1] = 1$, $dp[leaf][0] = 0$.

```

postorder over tree:
  if leaf: dp[u][0]=0; dp[u][1]=1
  else:
    take = 1 + sum_v min(dp[v][0], dp[v][1])
    skip = sum_v dp[v][1]
    dp[u][0]=skip; dp[u][1]=take
answer = min(dp[root][0], dp[root][1])

```

7.3.17 17. Traveling Salesman (bitmask DP)

Description. Shortest Hamiltonian tour visiting all nodes once and returning to start.

Recurrence/base. $dp[mask][i] = \min_{j \in mask \setminus \{i\}} dp[mask \setminus \{i\}][j] + dist[j][i]$.

Base: $dp[1 \ll s][s] = 0$.

```

for mask=0..(1<<n)-1:
  for i in nodes:
    dp[mask][i]=INF
dp[1<<s][s]=0
for mask from 1..(1<<n)-1:
  for i in mask:
    for j in mask, j!=i:
      dp[mask][i]=min(dp[mask][i], dp[mask^(1<<i)][j]+dist[j][i])
answer = min_i dp[(1<<n)-1][i] + dist[i][s]

```

7.3.18 18. Grid Min Path Sum (right/down moves)

Description. Minimum sum path from top-left to bottom-right moving only right or down.

Recurrence/base. $dp[i][j] = w_{i,j} + \min(dp[i-1][j], dp[i][j-1])$.

Base: first row/col cumulative.

```

dp[0][0]=w[0][0]
for i=1..R-1: dp[i][0]=dp[i-1][0]+w[i][0]
for j=1..C-1: dp[0][j]=dp[0][j-1]+w[0][j]
for i=1..R-1:
  for j=1..C-1:
    dp[i][j]=w[i][j]+min(dp[i-1][j], dp[i][j-1])
answer = dp[R-1][C-1]

```

7.3.19 19. Interleaving String (feasibility)

Description. Decide if s_3 can be formed by interleaving s_1 and s_2 in order.

Recurrence/base. $dp[i][j] = (dp[i-1][j] \wedge s1_i = s3_{i+j}) \vee (dp[i][j-1] \wedge s2_j = s3_{i+j})$.

Base: $dp[0][0] = \text{true}$; first row/col by equality chains.

```

dp[0][0]=true
for i=1..n: dp[i][0]= dp[i-1][0] and s1[i]==s3[i]
for j=1..m: dp[0][j]= dp[0][j-1] and s2[j]==s3[j]
for i=1..n:
  for j=1..m:
    dp[i][j]= (dp[i-1][j] and s1[i]==s3[i+j]) or
              (dp[i][j-1] and s2[j]==s3[i+j])
answer = dp[n][m]

```

7.3.20 20. Bellman–Ford Algorithm from a Dynamic Programming Perspective

Problem. Given a directed graph $G = (V, E)$ with edge weights $w : E \rightarrow \mathbb{R}$ and a source $s \in V$, compute the shortest-path distance from s to all vertices, even when some edge weights are negative (but no negative cycles reachable from s).

DP Formulation. Define a DP table $d[k][v]$ as:

$$d[k][v] = \text{shortest-path distance from } s \text{ to } v \text{ using at most } k \text{ edges.}$$

We initialize:

$$d[0][s] = 0, \quad d[0][v] = +\infty \text{ for } v \neq s.$$

Recurrence. For $k \geq 1$:

$$d[k][v] = \min \left(d[k-1][v], \min_{(u,v) \in E} (d[k-1][u] + w(u,v)) \right).$$

This captures the idea:

- Either the shortest k -edge path to v uses $\leq k-1$ edges, or
- It ends with a single edge (u, v) and a $(k-1)$ -edge path to u .

Key Observation. Any simple shortest path in a graph with $|V| = n$ has at most $n - 1$ edges. Thus:

$$d[n - 1][v] = \text{true shortest-path distance from } s \text{ to } v.$$

Bellman–Ford Algorithm. Rather than store a full $n \times n$ DP table, we keep only the previous row:

$$d[v] \leftarrow d[v] \text{ at iteration } k - 1, \quad d'[v] \leftarrow d[v] \text{ at iteration } k.$$

Each iteration performs:

$$\forall (u, v) \in E: \quad d[v] \leftarrow \min(d[v], d[u] + w(u, v)).$$

We repeat this for $n - 1$ iterations. By the DP lemma above, after iteration $n - 1$ the distances are optimal.

Detecting Negative Cycles. If a further iteration (the n th) still changes some $d[v]$, then:

$$d[n][v] < d[n - 1][v],$$

which means that there exists a path with more than $n - 1$ improving relaxations—impossible unless a negative-weight cycle is reachable from s .

DP Interpretation. The classical Bellman–Ford relaxation loop

$$\text{repeat } n - 1 \text{ times: for every edge } (u, v), \text{ relax } d[v]$$

is exactly computing the DP layers $d[1], d[2], \dots, d[n - 1]$: each layer incorporates paths using at most one more edge.

Thus Bellman–Ford is a straightforward instance of dynamic programming on path length: it monotonically improves the DP table until no better solution exists.

Time Complexity. The DP recurrence is evaluated $n - 1$ times over the edge set, giving:

$$T(n, m) = O(nm),$$

which is optimal for this DP formulation.

Conclusion. Bellman–Ford is dynamic programming over path length: it computes shortest paths by repeatedly applying the DP recurrence for paths of increasing edge count, guaranteeing correctness even with negative weights.

7.3.21 21. Floyd–Warshall Algorithm (All-Pairs Shortest Paths)

Description. Computes shortest-path distances between *all* pairs of vertices in a weighted graph (directed or undirected), allowing negative edges but *no* negative cycles reachable between pairs.

DP perspective. Let $dp[k][i][j]$ = shortest distance from i to j using only intermediate vertices from the set $\{1, \dots, k\}$. Transition considers whether we use vertex k as an intermediate node:

$$dp[k][i][j] = \min(dp[k - 1][i][j], dp[k - 1][i][k] + dp[k - 1][k][j]).$$

Final answer is $dp[|V|][i][j]$ for all pairs (i, j) .

Space optimization. We only need the previous layer $k - 1$; thus we collapse into a 2D table:

$$dist[i][j] = \min(dist[i][j], dist[i][k] + dist[k][j]).$$

Base initialization.

$$dist[i][i] = 0, \quad dist[i][j] = w(i, j) \text{ if edge exists, else } \infty.$$

```
for i = 1..n:
  for j = 1..n:
    if i == j: dist[i][j] = 0
    else if edge(i,j): dist[i][j] = w(i,j)
    else: dist[i][j] = INF

for k = 1..n:                # allow intermediate vertices up to k
  for i = 1..n:
    for j = 1..n:
      if dist[i][k] + dist[k][j] < dist[i][j]:
        dist[i][j] = dist[i][k] + dist[k][j]

# dist[i][j] now contains APSP distances
```

Answer. The full APSP matrix $dist[i][j]$ for all vertex pairs.

Time/Space.

$$O(|V|^3) \text{ time,} \quad O(|V|^2) \text{ space.}$$

7.3.22 22. Maximum Score from Performing Multiplication Operations (LeetCode 1770)

Problem. We are given:

- an integer array `nums` of length n ,
- an integer array `multipliers` of length m , with $m \leq n$.

We perform exactly m operations. On the i -th operation ($0 \leq i < m$):

- we choose either the leftmost or rightmost element from the current `nums`,
- multiply it by `multipliers[i]`,
- add it to our total score, and remove that chosen element from `nums`.

We want to maximize the total score after m operations.

State Definition. A key observation is that after i operations, we have removed exactly i elements in total, split between the left end and the right end. The remaining middle segment is uniquely determined by how many we have removed from the left.

Let `nums` be 0-indexed of length n , `multipliers` 0-indexed of length m . Define a DP state:

$\text{dp}[i][\ell]$ = maximum score after performing i operations and having taken ℓ numbers from the left.

Then:

- We have taken $i - \ell$ numbers from the right.
- The next multiplier we will use is `multipliers[i]`.
- The leftmost available index is ℓ .
- The rightmost available index is

$$r = (n - 1) - (i - \ell).$$

Base Case. Before performing any operations:

$$\text{dp}[0][0] = 0,$$

and all other $\text{dp}[0][\ell]$ for $\ell > 0$ are invalid (can be treated as $-\infty$).

Recurrence. At step i ($0 \leq i < m$), suppose we are computing $\text{dp}[i+1][\ell']$. There are two ways to reach it:

1. **Pick from the left.** If at step i we had taken $\ell' - 1$ elements from the left, then we now take the next left element `nums` $[\ell' - 1]$ with multiplier `multipliers` $[i]$:

$$\text{dp}[i+1][\ell'] \supset \text{dp}[i][\ell' - 1] + \text{multipliers}[i] \cdot \text{nums}[\ell' - 1].$$

2. **Pick from the right.** If at step i we had taken ℓ' elements from the left, then we now take a right element. At that moment, we have taken $i - \ell'$ from the right so far, so the rightmost available index is:

$$r = (n - 1) - (i - \ell').$$

We gain:

$$\text{dp}[i+1][\ell'] \supset \text{dp}[i][\ell'] + \text{multipliers}[i] \cdot \text{nums}[r].$$

Combining both choices, for valid indices:

$$\boxed{\text{dp}[i+1][\ell] = \max \left(\text{dp}[i][\ell - 1] + \text{multipliers}[i] \cdot \text{nums}[\ell - 1], \text{dp}[i][\ell] + \text{multipliers}[i] \cdot \text{nums}[(n - 1) - (i - \ell)] \right)}.$$

We fill this DP table for $i = 0, \dots, m - 1$ and $\ell = 0, \dots, i + 1$. The answer is:

$$\boxed{\max_{0 \leq \ell \leq m} \text{dp}[m][\ell]}$$

since after m operations we may have taken any feasible number ℓ from the left.

Why the Recurrence is Correct. After i operations, the configuration of remaining nums is completely determined by two numbers:

- how many picks we have made (i),
- how many of those picks were from the left (ℓ).

Thus, the remaining available segment is from index ℓ to index $r = (n - 1) - (i - \ell)$. The future decisions and score depend only on this segment and which multiplier index i we are at. This justifies that $dp[i][\ell]$ captures all necessary information (optimal substructure).

Each transition corresponds exactly to the only two available choices (pick left or pick right), and we take the maximum over them, so by standard DP cut-and-paste argument, this recurrence computes the optimal value.

Time and Space Complexity. The DP table has dimensions:

$$i = 0, \dots, m \quad \text{and} \quad \ell = 0, \dots, i,$$

so the number of states is:

$$\sum_{i=0}^m (i+1) = O(m^2).$$

Each state is computed in $O(1)$ time from two previous states, giving total time:

$$O(m^2).$$

Space usage is $O(m^2)$ if we store the full 2D table. Since $dp[i+1][\ell]$ depends only on row i , we can optimize to $O(m)$ space by rolling arrays.

In summary, the DP algorithm runs in $O(m^2)$ time and $O(m)$ – $O(m^2)$ space and yields the maximum possible score.

7.4 High-Dimensional DP

Hint. Go up a dimension only when a real constraint requires it (extra budget, exact count, direction/turns, multiple strings, or a set-state like a bitmask). Watch base layers and sentinel values—most bugs are off-by-one or mixing "previous layer" with "current layer."

7.4.1 1) LCS of Three Strings (3D)

Description. Longest common subsequence among S, T, U .

State. $dp[i][j][k]$ = LCS length of $S[1..i], T[1..j], U[1..k]$.

Recurrence.

$$dp[i][j][k] = \begin{cases} dp[i-1][j-1][k-1] + 1 & \text{if } S_i = T_j = U_k \\ \max\{dp[i-1][j][k], dp[i][j-1][k], dp[i][j][k-1]\} & \text{otherwise} \end{cases}$$

Base. Any index 0 gives 0.

```
int lcs3(const string& A, const string& B, const string& C){
    int n=A.size(), m=B.size(), l=C.size();
    vector dp(n+1, vector(m+1, vector<int>(l+1, 0)));
    for(int i=1; i<=n; i++)
        for(int j=1; j<=m; j++)
            for(int k=1; k<=l; k++){
                if(A[i-1]==B[j-1] && B[j-1]==C[k-1])
                    dp[i][j][k]=dp[i-1][j-1][k-1]+1;
                else
                    dp[i][j][k]=max({dp[i-1][j][k],
                                     dp[i][j-1][k],
                                     dp[i][j][k-1]});
            }
    return dp[n][m][l];
}
```

7.4.2 2) 0/1 Knapsack with Exactly K Items (3D with rolling over items)

Description. Choose exactly K_{items} items, total weight $\leq K$, maximize value.

State. $dp[i][c][k]$ = best value using first i items, capacity c , picking exactly k items. **Recurrence.**

$$dp[i][c][k] = \max \left(dp[i-1][c][k], dp[i-1][c-w_i][k-1] + v_i \right)$$

Base. $dp[0][0][0] = 0$; others $-\infty$.

(2-layer roll on i).

```

int knap_exact_k(const vector<int>& W,
                const vector<int>& V, int Kcap, int Kitems){
    const int n=W.size(), NEG=-1e9;
    vector prev(Kcap+1, vector<int>(Kitems+1, NEG)), cur=prev;
    prev[0][0]=0;
    for(int i=1;i<=n;i++){
        cur=prev;
        for(int c=0;c<=Kcap;c++){
            for(int k=0;k<=Kitems;k++){
                int best=prev[c][k];
                if(k>0 && W[i-1]<=c && prev[c-W[i-1]][k-1]>NEG)
                    best=max(best, prev[c-W[i-1]][k-1]+V[i-1]);
                cur[c][k]=best;
            }
        }
        swap(prev,cur);
    }
    int ans=NEG; for(int c=0;c<=Kcap;c++) ans=max(ans, prev[c][Kitems]);
    return ans;
}

```

7.4.3 3) 2D Knapsack (Weight & Volume) (3D with rolling on items)

Description. Each item has weight w_i , volume u_i , value v_i . Maximize value under W and U .

State. $dp[i][w][u]$ = best value using first i items with caps w, u .

Recurrence.

$$dp[i][w][u] = \max \left(dp[i-1][w][u], dp[i-1][w-w_i][u-u_i] + v_i \right)$$

(valid when $w_i \leq w, u_i \leq u$). **Base.** $dp[0][*][*] = 0$.

(2 layers, watch order).

```

int knap_2d(const vector<int>& Wt, const vector<int>& Vol,
            const vector<int>& Val,
            int W, int U){
    const int n=Wt.size(), NEG=-1e9;
    vector prev(W+1, vector<int>(U+1, 0)), cur=prev;
    for(int i=1;i<=n;i++){
        cur=prev;
        for(int w=0; w<=W; w++){
            for(int u=0; u<=U; u++){
                int best=prev[w][u];
                if(Wt[i-1]<=w && Vol[i-1]<=u)
                    best=max(best, prev[w-Wt[i-1]][u-Vol[i-1]] + Val[i-1]);
                cur[w][u]=best;
            }
        }
        swap(prev,cur);
    }
    int ans=0;
    for(int w=0; w<=W; w++)
        for(int u=0; u<=U; u++)
            ans=max(ans, prev[w][u]);
    return ans;
}

```

7.4.4 4) Count Grid Paths with $\leq K$ Turns (4D split into two 3D arrays)

Description. From $(0,0)$ to $(R-1, C-1)$ moving only Right/Down; count paths using at most K turns (direction changes).

State. Let $R[i][j][t]$ be #paths to (i,j) ending with a Right move using t turns, and $D[i][j][t]$ similarly for Down.

Recurrence.

$$\begin{aligned}
 R[i][j][t] &= R[i][j-1][t] + \mathbf{1}_{t>0} \cdot D[i][j-1][t-1], \\
 D[i][j][t] &= D[i-1][j][t] + \mathbf{1}_{t>0} \cdot R[i-1][j][t-1].
 \end{aligned}$$

Base. Start from $(0,0)$: $R[0][1][0] = 1$ (first step right), $D[1][0][0] = 1$ (first step down) if in bounds.

Answer. $\sum_{t=0}^K (R[R-1][C-1][t] + D[R-1][C-1][t])$.

(bounds-checked).

```

long long paths_with_turns(int Rn, int Cn, int K){
    vector R(Rn, vector(Cn, vector<long long>(K+1, 0))),
        D=R;
    if(Cn>1) R[0][1][0]=1;
}

```

```

if(Rn>1) D[1][0][0]=1;
for(int i=0;i<Rn;i++) for(int j=0;j<Cn;j++){
    if(i==0 && j<=1) continue;
    if(j==0 && i<=1) continue;
    for(int t=0;t<=K;t++){
        if(j>0){
            R[i][j][t] += R[i][j-1][t];
            if(t>0) R[i][j][t] += D[i][j-1][t-1];
        }
        if(i>0){
            D[i][j][t] += D[i-1][j][t];
            if(t>0) D[i][j][t] += R[i-1][j][t-1];
        }
    }
}
long long ans=0;
for(int t=0;t<=K;t++) ans += R[Rn-1][Cn-1][t] + D[Rn-1][Cn-1][t];
if(Rn==1 && Cn==1) ans=1; // zero-move path
return ans;
}

```

7.4.5 5) Traveling Salesman (Bitmask DP: exponential dimension)

Description. Shortest Hamiltonian tour visiting all n nodes and returning to start s .

State. $dp[mask][i]$ = min cost to start at s , visit set $mask$, end at i .

Recurrence.

$$dp[mask][i] = \min_{j \in mask \setminus \{i\}} dp[mask \setminus \{i\}][j] + w_{j,i}$$

Base. $dp[1 \ll s][s] = 0$.

Answer. $\min_i dp[(1 \ll n) - 1][i] + w_{i,s}$.

(tight loop).

```

int tsp(const vector<vector<int>>& w, int s=0){
    int n=w.size(), FULL=(1<<n);
    const int INF=1e9;
    vector dp(FULL, vector<int>(n, INF));
    dp[1<<s][s]=0;
    for(int mask=0; mask<FULL; mask++){
        for(int i=0;i<n;i++) if(dp[mask][i]<INF){
            for(int j=0;j<n;j++) if(!(mask&(1<<j)) && w[i][j]<INF){
                dp[mask | (1<<j)][j] = min(dp[mask | (1<<j)][j], dp[mask][i]+w[i][j]);
            }
        }
    }
    int ans=INF;
    for(int i=0;i<n;i++) if(w[i][s]<INF)
        ans=min(ans, dp[FULL-1][i]+w[i][s]);
    return ans;
}

```

7.4.6 6) Minimum Cost to Merge Stones (Dynamic Programming Analysis)

Problem. We are given an array of piles

$$\text{stones}[1..n],$$

and an integer $K \geq 2$. In one operation we may merge exactly K consecutive piles into one new pile whose size is the sum of their stones; this operation costs the total number of stones in those K piles. Our goal is to compute the minimum total cost to merge the entire sequence into a single pile, or determine that it is impossible.

Feasibility Condition. Merging reduces the number of piles by $K - 1$ each time. To end at exactly one pile, we require:

$$(n - 1) \equiv 0 \pmod{K - 1}.$$

If not, the answer is -1. Dynamic programming is only meaningful when this condition holds.

DP Structure. Let $\text{prefix}[i] = \sum_{j=1}^i \text{stones}[j]$ be the prefix sum array.

We define a 3D DP:

$$dp[i][j][m] = \text{minimum cost to merge interval } [i, j] \text{ into exactly } m \text{ piles.}$$

Base Case. For all i :

$$dp[i][i][1] = 0, \quad dp[i][i][m > 1] = +\infty.$$

Recurrence. To merge $[i, j]$ into m piles, we try all valid partitions:

$$dp[i][j][m] = \min_{i \leq t < j} (dp[i][t][1] + dp[t+1][j][m-1]), \quad \text{if } m > 1.$$

To merge into a *single* pile ($m = 1$), we must first form K piles, then merge them:

$$dp[i][j][1] = dp[i][j][K] + \text{sum}(i, j),$$

where

$$\text{sum}(i, j) = \text{prefix}[j] - \text{prefix}[i-1].$$

This follows exactly from the fact that one merging operation consumes K piles at once.

Key Structural Observation. Because every merge reduces pile count by $(K-1)$:

$$dp[i][j][m] < +\infty \text{ only if } ((j-i+1) - m) \equiv 0 \pmod{K-1}.$$

Thus most DP states are invalid and can be skipped.

Complexity Analysis. There are $O(n^2)$ intervals $[i, j]$, and at most K valid m states for each. The recurrence iterates over all possible split points t :

$$T(n) = O(n^3).$$

Optimizing the DP dimension (we only need $m = 1$ and $m = K$ states) yields the standard form:

$$dp[i][j] = \text{minimum cost to merge } [i, j] \text{ into one pile (if possible)}.$$

But the underlying complexity remains $O(n^3)$.

This $O(n^3)$ DP is optimal for the problem under standard assumptions due to the interval / fixed-length merge constraint.

Conclusion. The solution to the minimum-cost merge-stones problem is a classic interval DP with an additional dimension for the number of piles. Its structure is:

$$dp[i][j][1] = dp[i][j][K] + \text{sum}(i, j), \quad dp[i][j][m] = \min_t (dp[i][t][1] + dp[t+1][j][m-1]),$$

yielding:

$$\text{Time: } O(n^3), \quad \text{Space: } O(n^2 K).$$

The feasibility constraint $(n-1) \bmod (K-1) = 0$ is crucial and arises directly from the DP structure and the combinatorics of merging.

Nitpicks. (i) For maximization with impossible states, use a clear $-\infty$ sentinel (e.g., -10^9) and guard additions. (ii) Rolling arrays reduce memory for item-indexed DPs. (iii) For path-counting, prefer `long long/mod`. (iv) In turn-DP, initialize the first step explicitly; most mistakes there are off-by-one on the start cells.

7.5 Pseudo-polynomial DP

7.5.1 0/1 Knapsack, coin-change etc

7.5.2 Counting (r, ℓ) -Spaced Subsets in $O(rn \log n)$ Time

We are given a set S of n distinct non-negative integers, and two integers $r, \ell \geq 0$. A subset $S' \subseteq S$ is called (r, ℓ) -spaced if:

1. the sum of all elements in S' is exactly r , and
2. for any two distinct $a, b \in S'$, we have $|a - b| \geq \ell$.

We aim to count the number of such subsets in time $O(rn \log n)$.

Preprocessing

Sort the elements of S into an array

$$A[0], A[1], \dots, A[n-1],$$

in non-decreasing order. Sorting costs $O(n \log n)$.

Dynamic Programming Definition

For indices $i \in \{0, \dots, n\}$ and sums $j \in \{0, \dots, r\}$, define:

$$x(i, j) = \text{number of } (j, \ell)\text{-spaced subsets using only } A[i], A[i+1], \dots, A[n-1].$$

The desired answer is $x(0, r)$.

Base Cases.

$$x(n, 0) = 1, \quad x(n, j) = 0 \text{ for all } j > 0.$$

Transition: Optimal Substructure

For each state (i, j) , we consider two possibilities:

Case 1: Use element $A[i]$. This is only possible if $A[i] \leq j$. If we include $A[i]$, no further chosen elements may lie in the interval $[A[i], A[i] + \ell)$. Define $f(i)$ as the smallest index $k > i$ such that:

$$A[k] - A[i] \geq \ell.$$

This index can be found in $O(\log n)$ time using binary search over the sorted array.

If we choose $A[i]$, the new target sum becomes $j - A[i]$ and the next allowed index is $f(i)$, giving:

$$x(i, j) \supset x(f(i), j - A[i]).$$

Case 2: Do not use element $A[i]$. We simply skip $A[i]$, giving:

$$x(i, j) \supset x(i + 1, j).$$

DP Recurrence

Thus the recurrence is:

$$x(i, j) = \begin{cases} x(i + 1, j) + x(f(i), j - A[i]) & \text{if } A[i] \leq j, \\ x(i + 1, j) & \text{if } A[i] > j. \end{cases}$$

Time Complexity

There are $(n + 1)(r + 1) = O(rn)$ DP states. Each state requires:

- $O(1)$ time to combine solutions,
- $O(\log n)$ time to compute $f(i)$ using binary search.

Therefore, the total DP time is:

$$O(rn \log n).$$

Including sorting, the overall running time remains:

$$\boxed{O(rn \log n)}.$$

Thus, we obtain an algorithm that counts all (r, ℓ) -spaced subsets in time $O(rn \log n)$, as required.

8 Probability & Statistics Formulae for Algorithm Analysis

Hint. In analyses: (i) turn events into *indicator r.v.'s* and use *linearity of expectation*; (ii) compare random sums to their mean via *Markov/Chebyshev/Chernoff*; (iii) use *conditioning* to simplify dependencies. Mind when independence is required.

8.1 Basic Probability Identities

$$\Pr(A \cup B) \leq \Pr(A) + \Pr(B) \quad (\text{Union bound})$$

$$\Pr(A \cup B) = \Pr(A) + \Pr(B) - \Pr(A \cap B) \quad (\text{Inclusion-Exclusion, 2 terms})$$

$$\Pr(A | B) = \frac{\Pr(A \cap B)}{\Pr(B)}, \quad A \perp B \iff \Pr(A \cap B) = \Pr(A) \Pr(B)$$

$$\text{Law of Total Probability: } \Pr(A) = \sum_i \Pr(A | E_i) \Pr(E_i), \{E_i\} \text{ partition}$$

$$\text{Bayes: } \Pr(H | D) = \frac{\Pr(D | H) \Pr(H)}{\Pr(D)}$$

8.2 Indicator Trick & Linearity

For indicator $I_E = \mathbf{1}\{E\}$: $\mathbb{E}[I_E] = \Pr(E)$. For a sum $X = \sum_{i=1}^n I_i$ (*counting something*):

$$\mathbb{E}[X] = \sum_{i=1}^n \Pr(I_i = 1) \quad (\text{no independence needed})$$

If I_i are pairwise independent:

$$\text{Var}(X) = \sum_{i=1}^n \text{Var}(I_i) = \sum_i p_i(1 - p_i)$$

8.3 Expectation/Variance Identities

$$\mathbb{E}[aX + b] = a\mathbb{E}[X] + b \quad (\text{Linearity})$$

$$\mathbb{E}[X] = \mathbb{E}[\mathbb{E}[X | Y]] \quad (\text{Law of total expectation})$$

$$\text{Var}(X) = \mathbb{E}[X^2] - \mathbb{E}[X]^2$$

$$\text{Var}(X) = \mathbb{E}[\text{Var}(X | Y)] + \text{Var}(\mathbb{E}[X | Y]) \quad (\text{Law of total variance})$$

$$\text{Tail-sum (nonneg. integer } X): \mathbb{E}[X] = \sum_{t \geq 1} \Pr[X \geq t]$$

8.4 Inequalities (Concentration)

$$\Pr(X \geq a) \leq \frac{\mathbb{E}[X]}{a} \quad (\text{Markov})$$

$$\Pr(|X - \mu| \geq t) \leq \frac{\text{Var}(X)}{t^2} \quad (\text{Chebyshev})$$

Chernoff/Hoeffding for sums of bounded/0–1 r.v.'s. Let $X = \sum_{i=1}^n X_i$, $X_i \in [0, 1]$ independent, $\mu = \mathbb{E}[X]$.

$$\Pr[X \geq (1 + \delta)\mu] \leq \exp\left(-\frac{\mu\delta^2}{2 + \delta}\right) \quad (\delta > 0)$$

$$\Pr[X \leq (1 - \delta)\mu] \leq \exp\left(-\frac{\mu\delta^2}{2}\right) \quad (0 < \delta < 1)$$

$$\Pr(|X - \mu| \geq t) \leq 2 \exp\left(-\frac{2t^2}{n}\right) \quad (\text{Hoeffding, if } X_i \in [0, 1])$$

Handy simplification for $0 < \delta \leq 1$: $\Pr(|X - \mu| \geq \delta\mu) \leq 2 \exp(-\mu\delta^2/3)$.

8.5 Common Inequalities for Mathematical Proofs

Many mathematical arguments rely on a small set of fundamental inequalities. Below is a unified chain collecting the most frequently used algebraic, geometric, convexity, and analytic inequalities. Each tool is stated in a form that is easy to substitute directly into proofs.

Master Chain of Useful Inequalities.

(1) Basic Algebraic Inequalities

$$a \leq b \Rightarrow a + c \leq b + c, \quad a \leq b, c \geq 0 \Rightarrow ac \leq bc.$$

$$(a - b)^2 \geq 0 \Rightarrow 2ab \leq a^2 + b^2 \Rightarrow |ab| \leq \frac{a^2 + b^2}{2}.$$

(2) AM–GM–HM Hierarchy

For all $a, b > 0$,

$$\frac{a + b}{2} \geq \sqrt{ab} \geq \frac{2}{\frac{1}{a} + \frac{1}{b}}.$$

(3) Cauchy–Schwarz Inequality

$$\left(\sum_{i=1}^n a_i b_i \right)^2 \leq \left(\sum_{i=1}^n a_i^2 \right) \left(\sum_{i=1}^n b_i^2 \right).$$

(4) Triangle Inequality

$$|x + y| \leq |x| + |y|, \quad \|u + v\| \leq \|u\| + \|v\|.$$

(5) Jensen's Inequality (Convex Functions)

If f is convex and $\lambda_i \geq 0$, $\sum \lambda_i = 1$,

$$f\left(\sum_{i=1}^n \lambda_i x_i\right) \leq \sum_{i=1}^n \lambda_i f(x_i).$$

(6) Bernoulli's Inequality

For $x \geq -1$,

$$(1 + x)^r \geq 1 + rx \quad (r \geq 1 \text{ or } r \leq 0).$$

(7) Weighted AM–GM

If $a_i > 0$, $\alpha_i \geq 0$, $\sum \alpha_i = 1$,

$$a_1^{\alpha_1} a_2^{\alpha_2} \cdots a_n^{\alpha_n} \leq \alpha_1 a_1 + \alpha_2 a_2 + \cdots + \alpha_n a_n.$$

(8) Young's Inequality

For $p, q > 1$, $\frac{1}{p} + \frac{1}{q} = 1$,

$$ab \leq \frac{a^p}{p} + \frac{b^q}{q}.$$

(9) Hölder's Inequality

$$\|fg\|_1 \leq \|f\|_p \|g\|_q, \quad p, q > 1, \quad \frac{1}{p} + \frac{1}{q} = 1.$$

(10) Logarithmic Inequalities

$$\ln(1 + x) \leq x, \quad 1 + x \leq e^x, \quad \ln x \leq x - 1.$$

(11) Exponential Bounds

$$1 + x \leq e^x \leq 1 + x + \frac{x^2}{2} e^{|x|}.$$

(12) Chebyshev's Sum Inequality

For similarly sorted sequences,

$$\frac{1}{n} \sum_{i=1}^n a_i b_i \geq \left(\frac{1}{n} \sum_{i=1}^n a_i \right) \left(\frac{1}{n} \sum_{i=1}^n b_i \right).$$

(13) Rearrangement Inequality

For sorted $a_1 \leq \cdots \leq a_n$,

$$\sum_{i=1}^n a_i b_i \leq \sum_{i=1}^n a_i b_{\sigma(i)} \leq \sum_{i=1}^n a_i b_i^{\downarrow}.$$

(14) Markov and Chebyshev Bounds

$$\Pr[X \geq a] \leq \frac{\mathbb{E}[X]}{a}, \quad \Pr(|X - \mu| \geq t) \leq \frac{\text{Var}(X)}{t^2}.$$

(15) Jensen + Logarithm = AM–GM

$$\ln\left(\frac{a_1 + \cdots + a_n}{n}\right) \geq \frac{1}{n} \sum_{i=1}^n \ln a_i.$$

Notes.

- Inequalities (1)–(4) handle almost all algebraic manipulations in proofs.
- (5)–(8) come from convexity and are widely used in optimization and analysis.
- (9)–(13) generalize inner-product arguments and are essential in analysis.
- (14)–(15) appear frequently in algorithm analysis and probability arguments.

8.6 Common Taylor Series Expansions

In this subsection we list the most frequently used Taylor (and Maclaurin) series expansions. Unless stated otherwise, all expansions are Maclaurin series (expansion at $x = 0$) and hold for sufficiently small $|x|$.

(1) Exponential Function

$$e^x = 1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \cdots = \sum_{n=0}^{\infty} \frac{x^n}{n!}.$$

Useful bounds:

$$1 + x \leq e^x \leq 1 + x + \frac{x^2}{2} e^{|x|}.$$

(2) Natural Logarithm

For $|x| < 1$,

$$\ln(1+x) = x - \frac{x^2}{2} + \frac{x^3}{3} - \frac{x^4}{4} + \cdots = \sum_{n=1}^{\infty} (-1)^{n+1} \frac{x^n}{n}.$$

Special asymptotic form:

$$\ln(1+x) = x - \frac{x^2}{2} + O(x^3).$$

(3) Geometric Series

For $|x| < 1$,

$$\frac{1}{1-x} = 1 + x + x^2 + x^3 + \cdots.$$

(4) Binomial Series

For any real α , and $|x| < 1$,

$$(1+x)^\alpha = \sum_{n=0}^{\infty} \binom{\alpha}{n} x^n = 1 + \alpha x + \frac{\alpha(\alpha-1)}{2} x^2 + \frac{\alpha(\alpha-1)(\alpha-2)}{6} x^3 + \cdots.$$

(5) Sine Function

$$\sin x = x - \frac{x^3}{3!} + \frac{x^5}{5!} - \cdots = \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n+1}}{(2n+1)!}.$$

(6) Cosine Function

$$\cos x = 1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \cdots = \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n}}{(2n)!}.$$

(7) Tangent (Derivative at 0 and Series)

Though more rarely needed, the Maclaurin series is:

$$\tan x = x + \frac{x^3}{3} + \frac{2x^5}{15} + \frac{17x^7}{315} + \cdots.$$

Radius of convergence: $\pi/2$.

(8) Inverse Tangent

For $|x| \leq 1$,

$$\arctan x = x - \frac{x^3}{3} + \frac{x^5}{5} - \cdots = \sum_{n=0}^{\infty} (-1)^n \frac{x^{2n+1}}{2n+1}.$$

(9) Hyperbolic Sine and Cosine

$$\sinh x = x + \frac{x^3}{3!} + \frac{x^5}{5!} + \cdots = \sum_{n=0}^{\infty} \frac{x^{2n+1}}{(2n+1)!}.$$

$$\cosh x = 1 + \frac{x^2}{2!} + \frac{x^4}{4!} + \cdots = \sum_{n=0}^{\infty} \frac{x^{2n}}{(2n)!}.$$

(10) Square Root Expansion

For $|x| < 1$,

$$\sqrt{1+x} = 1 + \frac{x}{2} - \frac{x^2}{8} + \frac{x^3}{16} - \cdots.$$

(11) Reciprocal

For $|x| < 1$,

$$\frac{1}{1+x} = 1 - x + x^2 - x^3 + \cdots.$$

(12) Useful Asymptotic Forms

$$e^x = 1 + x + O(x^2), \quad \ln(1+x) = x - \frac{x^2}{2} + O(x^3),$$

$$\sin x = x + O(x^3), \quad \cos x = 1 - \frac{x^2}{2} + O(x^4).$$

8.7 Common Distributions (means/vars)

$$\text{Bernoulli}(p) : \mathbb{E}[X] = p, \text{Var}(X) = p(1-p)$$

$$\text{Binomial}(n, p) : \mathbb{E}[X] = np, \text{Var}(X) = np(1-p)$$

$$\text{Geometric}(p) \text{ (trials to first success)} : \mathbb{E}[X] = \frac{1}{p}, \text{Var}(X) = \frac{1-p}{p^2}, \text{memoryless}$$

$$\text{Poisson}(\lambda) : \mathbb{E}[X] = \lambda, \text{Var}(X) = \lambda \quad (\text{Binomial} \approx \text{Poisson when } n \rightarrow \infty, p \rightarrow 0, np = \lambda)$$

$$\text{Exponential}(\lambda) : \mathbb{E}[X] = \frac{1}{\lambda}, \text{Var}(X) = \frac{1}{\lambda^2}, \text{memoryless}$$

$$\text{Normal } \mathcal{N}(\mu, \sigma^2) : \Pr\left(\frac{X-\mu}{\sigma} \in [a, b]\right) \approx \Phi(b) - \Phi(a) \quad (\text{CLT for sums})$$

8.8 Asymptotic Approximations

$$H_n = \sum_{k=1}^n \frac{1}{k} = \ln n + \gamma + O\left(\frac{1}{n}\right)$$

$$n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n \quad (\text{Stirling}) \quad \Rightarrow \quad \binom{n}{k} \approx \frac{n^n}{k^k (n-k)^{n-k}} \sqrt{\frac{n}{2\pi k(n-k)}}$$

8.9 Classic “Balls-into-Bins” Facts

Throw m balls into n bins uniformly and independently.

$$\mathbb{E}[\text{occupied bins}] = n \left(1 - \left(1 - \frac{1}{n}\right)^m\right) \approx n(1 - e^{-m/n})$$

$$\Pr(\text{no collisions with } m \text{ balls}) \approx \exp\left(-\frac{m(m-1)}{2n}\right) \quad (\text{birthday paradox})$$

For $m = n$, the maximum load is $\Theta\left(\frac{\log n}{\log \log n}\right)$ w.h.p.

8.10 Coupon Collector

Collect n distinct coupons, each trial draws one uniformly at random.

$$\mathbb{E}[T] = n H_n \approx n \ln n + \gamma n \quad ; \quad \Pr[T > n(\ln n + c)] \leq e^{-c}$$

8.11 Conditioning & Decomposition

$$\mathbb{E}[X] = \sum_i \mathbb{E}[X | E_i] \Pr(E_i), \quad \text{Var}(X) = \mathbb{E}[\text{Var}(X | Y)] + \text{Var}(\mathbb{E}[X | Y])$$

Wald's identity (independent X_i with $\mathbb{E}[X_i] = \mu$, and stopping time N independent of X_i):

$$\mathbb{E}\left[\sum_{i=1}^N X_i\right] = \mathbb{E}[N] \cdot \mu$$

8.12 Jensen & Convexity

For convex g : $\mathbb{E}[g(X)] \geq g(\mathbb{E}[X])$. Frequently used with $g(x) = e^{tx}$ (mgf) in Chernoff-style proofs.

8.13 “With High Probability” (w.h.p.)

A statement holds w.h.p. if $\Pr[\text{event}] \geq 1 - n^{-c}$ for some constant $c > 0$. Typical pattern: apply a union bound over n events after a Chernoff/Hoeffding bound on each.

Nitpicks that save points: (i) **Linearity** never needs independence; **variance additivity** does. (ii) State the range/independence assumptions before using Chernoff/Hoeffding. (iii) For hashing/“pairwise independence,” Chernoff may fail; use Chebyshev or limited-independence bounds.

Discrete Math Theorems Frequently Used in Competitive Programming

Focus: primes, modular arithmetic, and base conversion/manipulation. Short statements you can drop into proofs or code comments.

8.14 Divisibility, GCD, and Bézout

- **Bézout’s identity.** For $a, b \in \mathbb{Z}$, $\exists x, y \in \mathbb{Z}$ with $ax + by = \gcd(a, b)$.
Use: Extended Euclid; modular inverses when $\gcd(a, m) = 1$.
- **LCM–GCD relation.** $\text{lcm}(a, b) \cdot \gcd(a, b) = |ab|$.
- **Euclidean algorithm.** $\gcd(a, b) = \gcd(b, a \bmod b)$, terminates in $O(\log \min\{a, b\})$ steps.

8.15 Modular Arithmetic & Inverses

- **Mod inverse existence.** $a^{-1} \pmod{m}$ exists $\iff \gcd(a, m) = 1$; compute via Extended Euclid.
- **Fermat’s little theorem (FLT).** If p is prime and $p \nmid a$, then $a^{p-1} \equiv 1 \pmod{p}$; hence $a^{-1} \equiv a^{p-2} \pmod{p}$.
- **Euler’s theorem.** If $\gcd(a, m) = 1$, then $a^{\varphi(m)} \equiv 1 \pmod{m}$.
 φ is multiplicative and $\varphi(p^k) = p^k - p^{k-1}$.
- **Exponent reduction.** For $\gcd(a, m) = 1$, $a^b \bmod m$ can be computed with $b \bmod \varphi(m)$ (careful with $b = 0$ and lifting when $\gcd(a, m) \neq 1$).
- **Wilson’s theorem.** p prime $\iff (p-1)! \equiv -1 \pmod{p}$.

8.16 Chinese Remainder Theorem (CRT)

- **Pairwise coprime moduli.** For $x \equiv a_i \pmod{m_i}$, $i = 1..k$, with pairwise coprime m_i and $M = \prod m_i$, the unique solution modulo M is

$$x \equiv \sum_{i=1}^k a_i M_i \overline{M_i} \pmod{M}, \quad M_i = M/m_i, \quad M_i \overline{M_i} \equiv 1 \pmod{m_i}.$$

- **General (two congruences).** $x \equiv a \pmod{m}$, $x \equiv b \pmod{n}$ has a solution $\iff a \equiv b \pmod{g}$ where $g = \gcd(m, n)$.
When solvable, solutions form a class modulo $\text{lcm}(m, n)$; one solution is

$$x \equiv a + m \cdot \left(\frac{b-a}{g} \cdot \overline{m/g} \pmod{n/g} \right) \pmod{\text{lcm}(m, n)}.$$

8.17 Prime Factorization Facts

- **Multiplicativity.** If $\gcd(m, n) = 1$, then $\tau(mn) = \tau(m)\tau(n)$ and $\sigma(mn) = \sigma(m)\sigma(n)$, where

$$\tau(n) = \prod_i (e_i + 1), \quad \sigma(n) = \prod_i \frac{p_i^{e_i+1} - 1}{p_i - 1} \quad \text{for } n = \prod_i p_i^{e_i}.$$

- **Legendre’s formula.** $v_p(n!) = \sum_{k=1}^{\infty} \left\lfloor \frac{n}{p^k} \right\rfloor$ (exponent of prime p in $n!$).

- **Lifting The Exponent (LTE), common case.** For odd prime $p \mid (a-b)$ and $n \in \mathbb{N}$,

$$v_p(a^n - b^n) = v_p(a-b) + v_p(n).$$

Variants handle $+$, $p = 2$, and when $p \nmid ab$; useful in gcd/valuation problems.

8.18 Primitive Roots & Orders

- **Existence.** A primitive root modulo m exists iff $m \in \{1, 2, 4, p^k, 2p^k\}$ for odd prime p .
- **Order divides totient.** If $\gcd(a, m) = 1$, the multiplicative order $\text{ord}_m(a)$ divides $\varphi(m)$; $a^k \equiv 1 \pmod{m} \iff \text{ord}_m(a) \mid k$.

8.19 Combinatorics Mod Prime

- **Lucas's theorem.** For prime p and $n = \sum n_i p^i$, $k = \sum k_i p^i$ in base p ,

$$\binom{n}{k} \equiv \prod_i \binom{n_i}{k_i} \pmod{p}.$$

- **Kummer's theorem.** $v_p\binom{n}{k}$ equals the number of carries when adding k and $n - k$ in base p .

8.20 Base Representation & Digit Properties

- **Horner evaluation.** If $N = (d_\ell d_{\ell-1} \dots d_0)_b$, then

$$N = (((d_\ell b + d_{\ell-1})b + d_{\ell-2}) \dots)b + d_0, \quad N \bmod m \text{ via } r := 0; \quad r = (r \cdot b + d_i) \bmod m.$$

- **Divisibility rules (any base $b \geq 2$).**

$$N \equiv \sum_i d_i \pmod{b-1}, \quad N \equiv \sum_i (-1)^i d_i \pmod{b+1} \quad (\text{LSB } i=0).$$

Use: Quick checks for $(b-1)$ or $(b+1)$ divisors; rolling hash invariants.

- **Digit DP invariant.** Counting numbers by digit constraints in base b typically uses state (pos, tight, carry/sum mod $M, \dots$) with transitions $O(b)$ per digit.
- **Base change.** Value is preserved: $(d)_b = \sum_i d_i b^i = \sum_j e_j c^j = (e)_c$. For large integers in arrays, use chunk base $B = 10^k$ to keep operations in 64-bit.

8.21 Sieve & Multiplicative Precomputation

- **Eratosthenes.** Marks primes $\leq n$ in $O(n \log \log n)$. Linear sieve computes primes and SPF in $O(n)$.
- **Multiplicative DP.** For multiplicative f , precompute $f(p^e)$ on the fly with SPF; $f(nm) = f(n)f(m)$ if $\gcd(n, m) = 1$.

8.22 Miscellaneous Handy Facts

- **CRT-friendly mod split.** For $M = 10^k$, choose $M = M_1 M_2$ with M_1, M_2 coprime (e.g. $2^a 5^b$ separated) and work mod M_1 and M_2 then recombine via CRT.
- **Pigeonhole for cycles.** Sequences defined by $x_{i+1} \equiv f(x_i) \pmod{m}$ enter a cycle by step $\leq m+1$ (useful for detecting periods).
- **Binomial mod composite.** Use prime-power decomposition $m = \prod p_i^{e_i}$, compute $\binom{n}{k} \bmod p_i^{e_i}$ with factorials excluding p_i and v_{p_i} via Legendre, then CRT.

9 Greedy Algorithm

Problems can be solved by combining optimal substructure:

- Multiple/Exponentially growing subproblems $\rightarrow$ recursion:
- Exactly 1 subproblem $\rightarrow$ greedy algorithm

0/1 /Fractional knapsack: for n items $\{(w_1, v_1), (w_2, v_2), \dots, (w_n, v_n)\}$,

$$0/1 \text{ Knapsack } \max\left(\sum_{i=1}^n v_i x_i\right) \wedge \sum_{i=1}^n w_i x_i \leq W$$

where x_i is binary decision variable

$$\text{Fractional Knapsack } \max\left(\sum_{i=1}^n \frac{v_i}{w_i} y_i\right) \wedge \sum_{i=1}^n w_i y_i \leq W$$

where y_i is decision variable $y_i \in [0, w_i]$; w_i is the available weight for item i , $y_i \leq w_i$

Claim: the optimal solution is greedy. Let j^* be the most valuable per weight item: $\max_{i \in N}(\frac{v_i}{w_i})$, $\exists$ an optimal knapsack containing $\min(w_{j^*}, W)$ of j^* (All j^* or all of j^*)

Proof. Suppose otherwise, $\exists$ an optimal knapsack containing $y_{j^*} < \min(w_{j^*}, W)$, for the rest of the knapsack,

$$W' = w_1 + y_2 + \dots + y_n = \min(w_{j^*}, W) - y_{j^*}$$

Where W' is the weight excluding any j^* . We can replace these weights with $\min(w_{j^*}, W) - y_{j^*}$ units of j^* , the weight doesn't change and total value does not decrease, as $\max_{i \in [1, n]}(\frac{v_i}{w_i}) = \frac{v_{j^*}}{w_{j^*}}$. Yes this proof is trivial conceptually. $\square$

Solve a greedy problem:

1. Cast the problem where we have to make a greedy choice and left with exactly one subproblem to solve
2. In general we prove with such exchange proof to show that there is always an optimal solution to the original problem that makes the greedy choice, therefore the greedy choice is safe.
3. Use optimal substructure (proof by contradiction, cut and past argument) to show that we can combine an optimal solution to the subproblem with a greedy choice to get the original solution.

9.1 Example: Hoffman Code

Huffman encoding is a classic example of a greedy algorithm for constructing an optimal prefix-free binary code given a set of characters and their frequencies. The goal is to minimize the total number of bits used to encode a message.

Problem Setting. We are given characters $c_1, \dots, c_k$ with frequencies $f_1, \dots, f_k$ (or probabilities $p_1, \dots, p_k$). We want to assign each character a binary codeword such that:

- the code is *prefix-free* (no codeword is a prefix of another), and
- the expected codeword length

$$L = \sum_{i=1}^k f_i \cdot \text{length}(\text{code}(c_i))$$

is minimized.

Huffman's algorithm is based on the following greedy rule:

Repeatedly combine the two least frequent symbols into a new "combined" symbol.

More concretely:

1. Start with a forest of k single-node trees, one for each character c_i with weight f_i .
2. Repeat until only one tree remains:
 - (a) Select two trees with the smallest weights, say with roots of weight w_1 and w_2 .
 - (b) Create a new internal node with weight $w_1 + w_2$, and make the two trees its left and right children.
3. The final tree is the *Huffman tree*. Assign bit 0 to each left edge and bit 1 to each right edge. The codeword of a character is the sequence of bits along the path from the root to its leaf.

```

Algorithm ChooseCodePattern(p[1..5]):
# Input: p[1..5] = probabilities of the 5 symbols
# Output: which length pattern to use: A = (1,2,3,4,4) or B = (1,3,3,3,3)

1. Sort the symbols so that  $p[1] \geq p[2] \geq p[3] \geq p[4] \geq p[5]$ .
   (Keep track of which symbol is which if you care about actual codewords.)

2. If  $p[2] > p[4] + p[5]$  then
   return "Pattern A: lengths = (1, 2, 3, 4, 4)"
else if  $p[2] < p[4] + p[5]$  then
   return "Pattern B: lengths = (1, 3, 3, 3, 3)"
else
   #  $p[2] = p[4] + p[5]$ ; both patterns give same expected length
   # you can choose either, e.g. default to A or B
   return "Either pattern A or B (same expected length)"

```

The greedy choice is thus: *at each step, merge the two least frequent trees*. The correctness proof shows that there exists an optimal prefix code that makes exactly this choice in its top-level structure (greedy-choice property), and that after merging, the subproblem on the reduced set of symbols is again optimal (optimal substructure).

Example and Tree Illustration. Suppose we have four characters with frequencies:

$$a : 0.4, \quad b : 0.3, \quad c : 0.2, \quad d : 0.1.$$

A greedy Huffman construction works as follows:

1. Merge c and d (smallest two): new node of weight 0.3.
2. Now the weights are 0.3 (from c, d), 0.3 (b), and 0.4 (a). Merge the two 0.3 nodes: new node of weight 0.6.
3. Finally, merge 0.4 and 0.6 into the root of weight 1.0.

Reading off the codes from the tree, we might get for example:

$$a \mapsto 0, \quad b \mapsto 10, \quad d \mapsto 110, \quad c \mapsto 111.$$

Shorter codewords are assigned to more frequent characters, which is exactly what we want to minimize the expected codeword length.

9.2 Greedy Choice Proofs examples

This subsection summarizes the key correctness arguments behind the greedy algorithms presented in Tutorial 07. The emphasis is on the *greedy choice property* (proved via exchange arguments) and the *optimal substructure* (usually shown by contradiction).

9.2.1 Dijkstra's Algorithm as a Greedy Algorithm

Goal. Given a weighted directed graph with non-negative edge weights, Dijkstra's algorithm computes the shortest-path distances from a given source vertex s to all other vertices.

Greedy Perspective. Dijkstra's algorithm maintains a set S of vertices whose shortest-path distances from s have been finalized. At every step, it chooses the next vertex $u \notin S$ with the smallest currently known tentative distance $d[u]$.

$$u = \arg \min_{v \notin S} d[v].$$

This is the *greedy choice*: **pick the closest not-yet-finalized vertex**. The key claim is that once chosen, u 's tentative distance is its true shortest-path distance.

Why This Greedy Choice is Safe. Because all edge weights are non-negative, any alternative path to a vertex outside of S must be at least as long as the already-finalized distances. If a shorter path to u existed, it would have to go through a vertex not yet in S , which would give a larger or equal distance. Thus no future relaxation can produce a shorter path to u , so finalizing u is safe.

This is a classic *cut property*: the closest vertex across the cut $(S, V \setminus S)$ has its shortest path determined.

Algorithm Sketch.

1. Initialize $d[s] = 0$ and $d[v] = \infty$ for $v \neq s$.
2. Maintain a min-priority queue of vertices by their tentative distances.
3. Repeatedly extract the vertex u with minimum $d[u]$, finalize it, and relax all outgoing edges $u \rightarrow v$.
4. Continue until all vertices are finalized.

Complexity. Using a binary heap: $O((V + E) \log V)$. Using a Fibonacci heap: $O(E + V \log V)$.

Greedy Takeaway. Dijkstra is greedy because at each step it commits to the shortest reachable vertex, and the non-negativity property ensures this greedy choice never violates optimality.

9.2.2 Prim's Algorithm as a Greedy Algorithm

Goal. Given a connected weighted undirected graph, Prim's algorithm constructs a Minimum Spanning Tree (MST).

Greedy Perspective. Prim's algorithm maintains a set S of vertices already included in the growing MST. At each step, it chooses the cheapest edge (u, v) such that:

$$u \in S, \quad v \notin S,$$

and adds both the edge and the vertex v to the MST.

This is the *greedy choice*: **pick the lightest edge crossing the cut** $(S, V \setminus S)$.

Why This Greedy Choice is Safe. The correctness follows from the *Cut Property* of MSTs: for any cut of the graph, the minimum-weight edge crossing that cut belongs to every MST.

Since Prim always selects exactly such an edge (the minimum among edges crossing the current cut), every greedy choice is guaranteed to be part of some MST, ensuring global optimality.

Algorithm Sketch.

1. Start from any vertex s , set $S = \{s\}$.
2. Maintain a priority queue of all edges crossing the cut $(S, V \setminus S)$.
3. Repeatedly extract the minimum-weight edge (u, v) .
4. Add v to S and add (u, v) to the MST.
5. Update the queue with new crossing edges.

Complexity. Using adjacency lists and a binary heap: $O(E \log V)$. With a Fibonacci heap: $O(E + V \log V)$.

Greedy Takeaway. Prim is greedy because it always adds the cheapest safe edge extending the partially built MST; the MST cut property guarantees the safety of this greedy step.

9.2.3 Burning CDs Problem

We are given file sizes (each < 100 MB), and each CD can store at most two files. The goal is to minimize the number of CDs.

Optimal substructure. If an optimal solution contains a pair $\{f_1, f_2\}$ that fits on a CD, then removing that pair leaves a strictly smaller instance that must also be solved optimally; otherwise reinserting $\{f_1, f_2\}$ yields a strictly better solution, contradicting optimality. Thus:

$$\text{MinCD}(A) = 1 + \text{MinCD}(A \setminus \{f_1, f_2\}) \quad (\text{for any pair used in some optimal solution}).$$

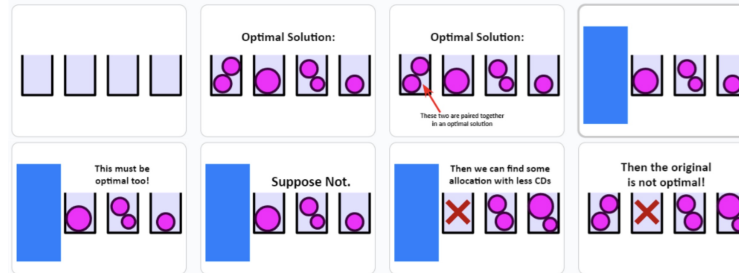


Figure 1: Caption.

Greedy choice: the smallest file must appear in a pair. If the smallest file f is not paired in some optimal solution, choose any pair $\{a, b\}$ in that solution. Since $f \leq a$ and $a + b \leq 100$, we also have $f + b \leq 100$. Swapping a and f preserves feasibility and does not increase the number of CDs. Hence there exists an optimal solution in which f is in a pair.

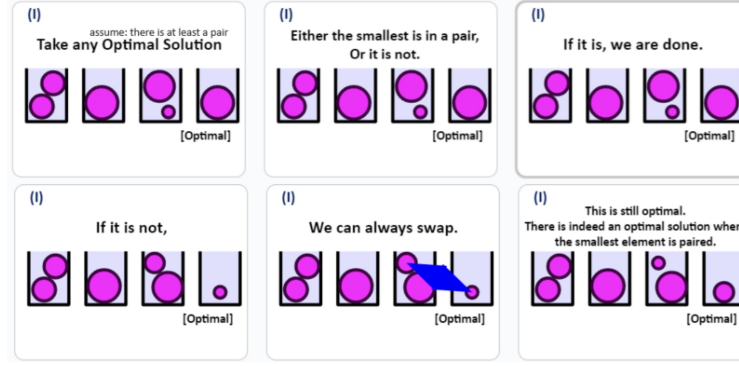
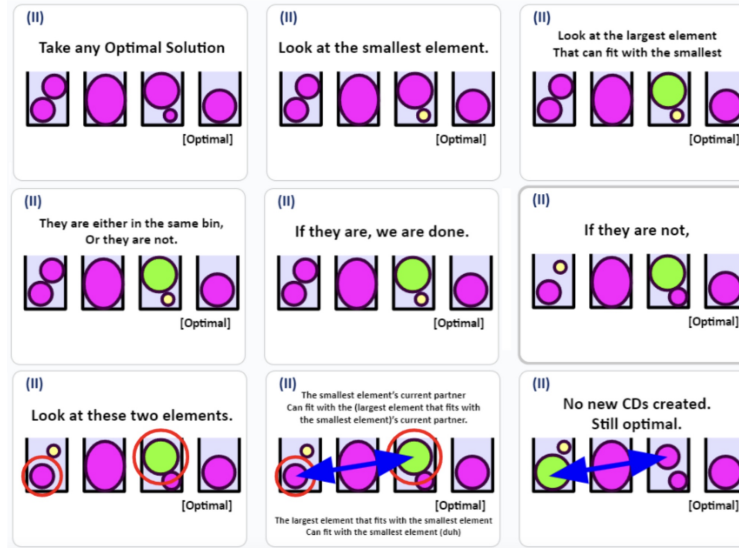


Figure 2: Caption.

Greedy choice: pair the smallest file with the largest file that fits. Let f_1 be the smallest file and let f_2 be the largest file such that $f_1 + f_2 \leq 100$. In any optimal solution where f_1 is paired with some other file $g < f_2$, we may swap g with f_2 :

- if f_2 is unpaired, the swap is trivially feasible;
- if f_2 is paired with another file h , then since $f_2 > g$, the feasibility of $\{g, h\}$ implies $\{f_2, h\}$ also fits.



Thus an optimal solution exists that contains the greedy pair $\{f_1, f_2\}$.

Conclusion. Sorting the files and repeatedly pairing the smallest remaining file with the largest file that fits is a safe greedy choice. The exchange argument guarantees that any optimal solution can be transformed into one that makes the greedy choice.

9.2.4 Analysis of Statements for the CD Burning Problem

We are given a set of file sizes $A = \{a_1, a_2, \dots, a_n\}$ sorted so that $a_1 \geq a_2 \geq \dots \geq a_n > 0$, and a value $k \geq \lceil |A|/2 \rceil$. Each CD holds at most two files, and $\text{MinSizeCD}(A, k)$ denotes the minimum CD capacity needed so that k CDs suffice.

We evaluate the correctness of the following statements:

1. If $\text{MinSizeCD}(A, k) = a_1$, then $k = |A|$.

This is false. Even if the capacity equals the largest file size a_1 , the remaining files might be small enough to be paired efficiently. For example, in $A = \{10, 1, 1, 1, 1\}$, capacity 10 needs fewer than $|A|$ CDs.

2. If $|A|$ is even and $k = |A|/2$, then $\text{MinSizeCD}(A, k) = a_1 + a_n$.

This is false. Counterexample: $A = \{5, 4, 3, 1\}$ and $k = 2$. One might expect capacity $5 + 1 = 6$, but the optimal packing is $(5), (4, 3)$, requiring capacity 7. Thus the stated equality does not hold.

3. If $\text{MinSizeCD}(A, k) = a_1 + a_n$, then $k = |A|/2$.

Also false. Using the same set $A = \{5, 4, 3, 1\}$, capacity $6 = 5 + 1$ would require $k = 3$ CDs, not $2 = |A|/2$. Hence the implication does not hold.

4. If $k > |A|/2$, then

$$\text{MinSizeCD}(A, k) = \max\{a_1, \text{MinSizeCD}(A \setminus \{a_1\}, k - 1)\}.$$

This statement is true. Since $k > |A|/2$, in any feasible solution at least one CD must contain only one file. By a cut-and-paste argument, we can assume without loss of optimality that this single-file CD contains the largest file a_1 . Thus the optimal capacity is the maximum of a_1 and the optimal capacity for the remaining files with $k - 1$ CDs.

5.

Conclusion: The correct choice is (d).

9.3 Maximizing Profit in the Fish Selling Problem

We are given n fishes with weights $w_1, w_2, \dots, w_n$ and m fish sellers, where seller j is described by a pair (x_j, p_j) : he is willing to buy up to x_j fishes at price p_j SGD per kilogram. The goal is to maximize the total profit from selling (some or all) of the fishes.

Optimal Substructure

Suppose we sell fish i (weight w_i) to seller j who currently has quota (x_j, p_j) . We gain $w_i p_j$ SGD, fish i is removed, and the seller state becomes $(x_j - 1, p_j)$; if $x_j - 1 = 0$, seller j is removed.

We claim that the remaining subproblem must be solved optimally. If not, and some other solution yields higher profit, we could append the sale of fish i to seller j , contradicting optimality. Hence the problem has optimal substructure.

Greedy Choice and Exchange Argument

Sort all fishes by non-increasing weight and all sellers by non-increasing price. Let fish i be the heaviest available, and seller j the seller with the highest price p_j . We claim that there exists an optimal solution in which fish i is sold to seller j .

We analyze all possible configurations in an optimal solution:

1. If fish i is already sold to seller j , we are done.
2. If seller j buys some fish $i' \neq i$, and i is sold to some seller j' , then since $w_{i'} \leq w_i$ and $p_{j'} \leq p_j$, exchanging assignments of i and i' does not decrease profit.
3. If seller j buys nothing and fish i is sold to another seller j' , switching fish i to seller j increases or preserves profit since $p_{j'} \leq p_j$.
4. If seller j buys some fish i' and fish i is unsold, we can sell i to seller j instead without losing profit.
5. The case where seller j buys nothing and fish i is unsold is impossible: selling fish i to seller j increases total profit.

In all cases, there exists an optimal solution selling the heaviest fish to the highest-paying seller. This establishes the greedy-choice property.

Greedy Algorithm

1. Sort fishes by non-increasing weight.
2. Sort sellers by non-increasing price.
3. Repeatedly sell the next heaviest fish to the available seller with the highest price per kilogram (reduce seller quota by 1; remove seller if quota becomes zero).

Because we process each fish once and seller quotas only shrink, the greedy procedure performs $O(n)$ assignments.

Time Complexity

Sorting fishes and sellers takes $O(n \log n + m \log m)$. The greedy matching step processes at most n fishes and is $O(n)$. Hence the full algorithm runs in

$$O(n \log n + m \log m),$$

which is polynomial time. Importantly, we never expand seller quotas explicitly (e.g., repeating a seller x_j times), which avoids pseudo-polynomial complexity.

Conclusion

The problem admits optimal substructure and a valid greedy-choice property. Sorting by weight and price and greedily pairing the heaviest fish with the highest-paying seller yields an optimal solution in polynomial time.

9.4 Activity Selection Problem

Each activity a_i occupies an interval $[s_i, f_i)$, and the goal is to select the maximum number of pairwise compatible activities.

Optimal substructure. If an optimal schedule S contains an activity a_j , then the activities that finish before s_j (call this set Before_j) and the activities that start after f_j (call this set After_j) must each form an optimal schedule for their respective subproblems. Otherwise, replacing either part with a strictly better solution yields a contradiction of optimality.

Greedy choice: choose the activity that starts last. Let a^* be the activity with the latest start time among all activities. Let S be an optimal schedule, and let a be the activity in S that starts last (within S). Since a^* starts no earlier than a , replacing a with a^* preserves compatibility with all other activities in S . This exchange preserves the size of the schedule, so the modified schedule remains optimal. Therefore every optimal solution can be made to include a^* .

Conclusion. Processing activities in decreasing order of start times and greedily selecting each compatible activity is a safe greedy strategy. The exchange argument shows that the latest-starting activity is always part of some optimal solution.

9.5 Dominating Set Variant of the Activity Selection Problem

We are given $A = \{a_1, a_2, \dots, a_n\}$, where activity $a_i = [s_i, f_i)$ with $s_i < f_i$. Two activities overlap if their time intervals intersect. A subset $D \subseteq A$ is *dominating* if for every activity $a_i \in A \setminus D$, there exists $a_j \in D$ such that a_i and a_j overlap. Our goal is to compute a smallest dominating subset.

Incorrect Greedy Algorithm

A natural attempt is to repeatedly pick an activity that covers (overlaps) the largest number of uncovered activities:

1. $D \leftarrow \emptyset$; $B \leftarrow A$.
2. While $B \neq \emptyset$:
 - (a) Select $a \in A \setminus D$ maximizing the number of activities in B that overlap with a .
 - (b) Add a to D .
 - (c) Remove from B all activities that overlap with a .
3. Return D .

This algorithm is *incorrect*. A counterexample is:

$$A = \{[1, 2), [1, 3), [2, 3), [2, 4), [3, 4), [3, 5), [4, 5)\}.$$

An optimal dominating set is

$$D^* = \{[1, 3), [3, 5)\}.$$

However, the greedy algorithm first selects $[2, 4)$ (it overlaps the most intervals) and is then forced to take two additional intervals, producing a strictly larger dominating set.

Common mistakes include: (i) choosing from $A \setminus D$ rather than from B , (ii) maximizing instead of minimizing, (iii) treating lengths instead of number of activities, (iv) attempting a proof without a counterexample.

Correct Polynomial-Time Greedy Algorithm

We modify the greedy choice as follows:

1. Select $a_i = [s_i, f_i) \in B$ with the *minimum* finish time.
2. Let S be all activities in $A \setminus D$ that overlap with a_i .
3. From S , select $a_j = [s_j, f_j)$ with the *maximum* finish time.
4. Add a_j to D , remove all activities in B overlapping with a_j .

This greedy rule is correct: in any optimal dominating set, some activity in S must be chosen, otherwise a_i would be undominated. If the optimal solution chooses $a_k \in S$, then replacing a_k by a_j (the one with maximum finish time) still dominates all activities that a_k dominated. If an activity overlapped with a_k but not with a_j , then:

- either it finishes before s_j , contradicting the fact that f_i was the minimum finish time, or
- it starts after $f_j \geq f_k$, meaning it does not overlap with a_k either.

Hence a_j is always safe to choose.

A simpler interpretation:

- Pick the earliest-finishing uncovered interval x .
- Among all activities that overlap x , pick one that finishes as *late* as possible.

Equivalent variations include: (i) replacing “finishes first” with “starts first”, (ii) replacing “maximum finish time” with “maximum number of uncovered overlaps”. However, both modifications cannot be done simultaneously.

Notes and Common Pitfalls

- Minimum dominating set is *not* maximum independent set.
- Tie-breaking rules alone cannot fix the incorrect greedy strategy.
- Selecting the longest interval or earliest/last start/finish time alone is incorrect.
- The algorithm runs in polynomial time: sorting plus linear scanning.

10 Amortization Analysis

10.1 Aggregate method

10.2 Accounting method

Charge the i_{th} operation a fictitious amortized cost $c(i)$, assuming 1 cost is paid for 1 unit of work. This fee is consumed to perform the operation and any amount that is not immediately consumed goes into a bank to be used by subsequent potentially costly operations. Invariant: the bank balance can never go negative.

$$\forall n, \sum_{i=1}^n t(i) \leq \sum_{i=1}^n c(i)$$

This ensure the total amortized cost provides an upper bound on the total true cost.

Example: INSERT in dynamic table

Still we use double upon full. Instead of a cost of 1 for 1 insertion operation, we impose a fictitious cost of 3. We need to prove that

$$\begin{aligned} \forall n \in \mathbb{N}, \sum_{i=1}^n t(i) &\leq \sum_{i=1}^n c(i) \\ \forall n \in \mathbb{N}, n + \sum_{i=1}^{\lfloor \log_2(n) \rfloor} 2^i &\leq \sum_{i=1}^n 3 \end{aligned}$$

Proof. LHS = $n + (2^{\lfloor \log_2(n) \rfloor + 1} - 2)$, $2^{\lfloor \log_2(n) \rfloor + 1} \leq 2n$, LHS $< 3n =$ RHS □

10.3 Potential method

Let $\phi(i)$ be the potential at the end of i^{th} operation, if we maintain invariants

$$\begin{aligned} \phi(0) &= 0 \\ \phi(i) &\geq 0, \forall i \end{aligned}$$

The amortized cost of the i_{th} operation is defined as the actual cost plush the potential difference $\Delta\phi(i) = \phi(i) - \phi(i-1)$.

The amortized cost of n operation:

$$\begin{aligned} \sum_i c_{amortized} &= \sum_i (t(i) + \phi(i) - \phi(i-1)) \\ &= \sum_i t(i) + (\phi(n) - \phi(0)) \\ &= \sum_i t(i) + \phi(n) \end{aligned}$$

$$\sum_i c_{amortized} \in O(g(n)) \implies \sum_i t(i) \in O(g(n))$$

Example: Pop() in dynamic table: half upon resizing

Let $\phi(i) = 2(m_i - n_i)$ where m_i is the table size when carrying out the i^{th} operation, and n_i is the number of element in the table.

1. Not resizing:

- Operation cost $c(i) = 1$
- $\phi(i) = 2(m - n)$, $\phi(i-1) = 2(m - (n-1))$
- $\Delta\phi(i) = 2$

2. Shrinking:

- Operation cost $c(i) = 1 + \frac{m}{2}$
- $\phi(i) = 2(m - n)$, $\phi(i-1) = 2(\frac{m}{2} - (n-1))$
- $\Delta\phi(i) = m$

10.4 Amortized Analysis: Choosing Accounting and Potential Functions

This subsection summarizes the main conceptual ideas behind amortized analysis from Tutorial 08. The focus is on *why* we pick certain amortized charges (accounting method) or certain potential functions (potential method), and how these choices guarantee amortized $O(1)$ or $O(n)$ bounds for sequences of operations.

1. Amortized Analysis: Key Idea

Amortized analysis studies the total cost of a *sequence* of operations. Even if individual operations may be expensive, we show that:

$$\frac{\text{Total true cost}}{\text{Total number of operations}}$$

is small in the worst case. Unlike expected-time analysis, no probability is involved: the guarantee is worst-case per operation over any sequence.

The tutorial emphasizes two methods:

- **Accounting method:** assign a “fictional” charge $C(i)$ to operation i , possibly larger than its true cost $T(i)$.
- **Potential method:** define a potential ϕ that measures “stored work” and use

$$C(i) = T(i) + (\phi(i) - \phi(i-1)).$$

Both methods ensure the bank balance / potential never goes negative.

2. Choosing Accounting Charges

From the tutorial examples, the pattern is:

(a) Dynamic Table Insertion (doubling). We charge \$3 for each insertion:

$$\text{True cost} = 1 \text{ (normal insert)} \quad \text{or} \quad T = \Theta(n) \text{ (when doubling).}$$

The choice “\$3” follows the rule:

Charge normal operations slightly more, so the saved credits pay for rare expensive ones.

Because each element moved during a future doubling was once inserted, assigning an extra \$2 on every insertion exactly accumulates enough credit to pay for all future copies. This makes the amortized cost $O(1)$.

(b) Queue with ENQUEUE/DEQUEUE/DELETE/ADD. Each element that enters the queue is charged \$2:

$$\text{\$1 to enqueue it} \quad + \quad \text{\$1 saved for future removal.}$$

Thus:

$$\text{ENQUEUE}(x) : \$2, \quad \text{DEQUEUE}() : \$0, \quad \text{DELETE}(k) : \$0, \quad \text{ADD}(A) : 2|A|.$$

The bank invariant is: *every element present in the queue holds exactly \$1 for its eventual deletion.* This ensures DEQUEUE and DELETE are free in the amortized sense.

3. Choosing Potential Functions

The potential method requires a ϕ such that:

$$\phi(0) = 0, \quad \phi(i) \geq 0 \quad \forall i.$$

The tutorial provides two guiding heuristics:

Heuristic 1: The potential should decrease sharply during expensive operations. This makes the amortized cost

$$C(i) = T(i) + (\phi(i) - \phi(i-1))$$

small even when $T(i)$ is large.

(a) Queue with ENQUEUE/DEQUEUE/DELETE/ADD. A natural potential is:

$$\phi(i) = \text{current queue size.}$$

The reasoning:

- expensive operations (DELETE, DEQUEUE) shrink the queue, so ϕ decreases sharply, offsetting the true cost;
- cheap operations (ENQUEUE) increase ϕ in a controlled way.

This yields amortized costs:

Operation	$T(i)$	$\phi(i) - \phi(i-1)$	$C(i)$
ENQUEUE	1	+1	2
DEQUEUE	1	-1	0
DELETE(k)	k	$-k$	0
ADD(A)	$ A $	$+ A $	$2 A $

(b) **Dynamic Table with POP() only (halving).** The subtle choice:

$$\phi(i) = \text{number of empty cells} = \text{size}(T) - n.$$

Why this works:

- Before halving, the table has many empty cells, so ϕ is large.
- After halving, ϕ *drops dramatically*, offsetting the heavy cost of copying.
- When POP does not trigger halving, ϕ increases slightly.

The amortized analysis table:

Operation	$T(i)$	$\phi(i) - \phi(i-1)$	$C(i)$
Normal POP	1	+1	2
POP that halves	$n+1$	$-(n-1)$	2

Hence amortized $O(1)$.

4. Key Takeaways for Designing Amortized Proofs

- **Find the invariant:** Which quantity is preserved or slowly built up? (e.g., each inserted element stores enough money for its own copying or deletion.)
- **Account or store potential on the cheap, spend on the expensive:** The expensive operations should cause a large decrease in stored credit/potential.
- **Potential often matches “unused capacity” or “queue length”:** For dynamic tables: unused slots. For queues: number of elements. For stacks/monotonic structures: stack height.
- **Amortization is about sequences, not single operations:** Even if one step is $\Theta(n)$, the average stays $O(1)$.

Summary. The central goal in both accounting and potential methods is to choose an amortized cost or potential that tracks a “stored resource” which accumulates during cheap operations and is spent during expensive ones. This ensures the total amortized cost upper-bounds the total true cost for any operation sequence.

10.5 Amortized Analysis of Dynamic Hash Tables with Rehashing

We maintain a hash table of size M with N stored keys, using Separate Chaining. The load factor is

$$\alpha = \frac{N}{M},$$

which determines the expected chain length. To keep α bounded by a constant, we apply the following resizing rules:

- **Doubling:** If an insertion causes $\alpha > 2$, we double the table size from M to $2M$ and rehash all N keys.
- **Halving:** If a deletion causes $\alpha < \frac{1}{2}$, we halve the table size from M to $M/2$ and rehash all N keys.

Each rehash takes $\Theta(N)$ time. We assume (as permitted) that every individual insertion or deletion takes at most $\alpha + 1 = O(1)$ time when no rehash occurs. The goal is to prove that **amortized** cost of both insertion and deletion remains $O(1)$.

Potential Function

Define the potential function:

$$\phi = 2|N - M| + 2.$$

This potential reflects how far the table size M deviates from the number of keys N .

No-Rehash Case

If no resizing occurs, M stays the same and N changes by exactly 1. Hence

$$\Delta\phi \leq 2.$$

The amortized cost is

$$\text{amortized} = \text{actual} + \Delta\phi \leq 1 + 2 = 3,$$

a constant.

Doubling Rehash

A doubling rehash occurs when an insertion causes $\alpha > 2$, meaning $M = N/2$ just before the rehash. After rehashing, the table size becomes $M' = N$, and the insertion increases N to $N + 1$.

Before rehash:

$$\phi(i-1) = 2|N - N/2| + 2 = 2 \cdot \frac{N}{2} + 2 = N + 2.$$

After rehash:

$$\phi(i) = 2|(N+1) - (N+1)| + 2 = 2.$$

Thus:

$$\Delta\phi = 2 - (N+2) = -N.$$

Actual cost of rehashing all N items plus the insertion is $N + 1$. So amortized cost:

$$(N+1) + (-N) = 1.$$

This is bounded by a constant (in the official solution they simplify it to 3).

Halving Rehash

A halving rehash occurs when a deletion causes $\alpha < \frac{1}{2}$, meaning $M = 2N$ before deletion. After the deletion we have $N' = N - 1$ and the table size becomes $M' = N - 1$.

Before rehash:

$$\phi(i-1) = 2|N - 2N| + 2 = 2N + 2.$$

After rehash:

$$\phi(i) = 2|(N-1) - (N-1)| + 2 = 2.$$

Thus:

$$\Delta\phi = 2 - (2N+2) = -2N.$$

The actual cost of rehashing $N - 1$ keys plus the deletion is N , so amortized cost:

$$N + (-2N) = -N,$$

which is again $O(1)$; following the instructor's simplified bound, this is treated as at most 3.

The amortized analysis table:

Operation	$T(i)$	$\phi(i) - \phi(i-1)$	$C(i) = T(i) + \Delta\phi$
Insert/Delete (no rehash)	1	≤ 2	≤ 3
Insert causing doubling	$N+1$	$2-N$	3
Delete causing halving	$N+1$	$2-N$	3

Hence each operation has amortized cost $O(1)$.

Conclusion

In every case—ordinary insertion/deletion, doubling, or halving—the amortized cost is at most a constant. Therefore:

Amortized cost of insertion and deletion is $O(1)$.

This holds even though resizing (rehashing all keys) is an expensive $\Theta(N)$ -time operation.

10.6 Amortized Analysis of Data Structures via the Potential Method

Potential Method: Reminder

Let D_i be the data structure state after the i -th operation, with real cost c_i . A *potential function* is a mapping

$$\Phi : \{\text{states}\} \rightarrow \mathbb{R}_{\geq 0}$$

such that $\Phi(D_0) = 0$ and $\Phi(D_i) \geq 0$ for all i . The *amortized cost* of operation i is defined as

$$\hat{c}_i = c_i + \Phi(D_i) - \Phi(D_{i-1}).$$

Summing over $i = 1, \dots, m$ gives

$$\sum_{i=1}^m \hat{c}_i = \sum_{i=1}^m c_i + \Phi(D_m) - \Phi(D_0) \geq \sum_{i=1}^m c_i,$$

so the total amortized cost provides an upper bound on the total real cost.

10.6.1 Self-Adjusting Binary Search Tree (Splay Tree)

Model. We consider a *splay tree*, a self-adjusting binary search tree that performs a *splay* (a sequence of rotations) after each access. We analyze operations SEARCH, INSERT, and DELETE in terms of amortized time.

Plain (unbalanced) BSTs have no non-trivial worst-case or amortized guarantees for arbitrary sequences of operations, so we explicitly assume the splay tree variant.

Size and Rank. For each node x , let $s(x)$ be the size (number of nodes) in the subtree rooted at x . Define the rank of x as

$$r(x) = \log s(x),$$

and let the potential of a tree T be

$$\Phi(T) = \sum_{x \in T} r(x) = \sum_{x \in T} \log s(x).$$

Clearly $\Phi(T) \geq 0$ and $\Phi(\emptyset) = 0$.

Single Splay Step. A splay consists of a sequence of rotations, each of which is a *zig*, *zig-zig*, or *zig-zag* step. The real cost of each step is $O(1)$ (constant-time pointer updates).

One proves the *Access Lemma*: splaying node x to the root has amortized cost

$$O(1 + r(T) - r(x)) = O(1 + \log n - \log s(x)),$$

where $n = |T|$ and $r(T)$ is the rank of the root. The proof is a case-by-case analysis over *zig*, *zig-zig*, and *zig-zag*: in each case we bound

$$\hat{c}_{\text{step}} = c_{\text{step}} + \Phi(\text{after}) - \Phi(\text{before})$$

using the ranks of the nodes involved and the fact that logarithms smooth out subtree size changes. The total potential increase over all steps in a splay is then telescoped and bounded.

Amortized Cost of Operations.

- **SEARCH(x):** perform standard BST search, then splay x (or the last accessed node) to the root. Amortized cost: $O(\log n)$ by the Access Lemma.
- **INSERT(x):** insert x as in a standard BST (cost $O(\text{depth}(x))$), then splay x to the root. The splay dominates the cost, so amortized cost: $O(\log n)$.
- **DELETE(x):** splay x to the root, delete it, and re-join the two subtrees (e.g. splay the maximum of the left subtree to the root and attach the right subtree). Each splay is $O(\log n)$ amortized, so overall amortized cost is $O(\log n)$.

Thus, under this potential function, all basic operations on a splay tree have amortized time $O(\log n)$.

10.6.2 Binary Heap Priority Queue

Model. Consider a binary heap implementing a priority queue with operations: INSERT, EXTRACT-MIN, and DECREASE-KEY. We store n elements in an array-based heap.

Trivial Potential. Since each operation is already $O(\log n)$ in the worst case, one can use a very simple potential, e.g. $\Phi(H) = 0$ for all states H , or more critically

$$\Phi(H) = c \cdot n$$

for some constant $c > 0$. This potential does *not* change asymptotic bounds, but lets us formally express amortized costs.

Amortized Costs.

- **INSERT:** place the new element at the end of the array and *bubble up* along at most $\lfloor \log n \rfloor$ levels. Real cost $c_i = O(\log n)$; potential change $\Phi(H_i) - \Phi(H_{i-1}) = O(1)$ (only n changes by 1). Therefore $\hat{c}_i = O(\log n)$.
- **EXTRACT-MIN:** swap the root with the last element, remove the last element, then *bubble down* along at most $\lfloor \log n \rfloor$ levels. Again $c_i = O(\log n)$ and $|\Phi(H_i) - \Phi(H_{i-1})| = O(1)$, so $\hat{c}_i = O(\log n)$.
- **DECREASE-KEY:** bubble up similarly to INSERT, touching at most $O(\log n)$ nodes. Same reasoning yields $\hat{c}_i = O(\log n)$.

Here the potential method is mostly a formal confirmation that nothing pathological happens over a long sequence of heap operations; the amortized bounds match the worst-case bounds.

10.6.3 Fibonacci Heap

Model. A Fibonacci heap is a priority queue supporting:

- INSERT,
- FIND-MIN,
- UNION (meld two heaps),
- EXTRACT-MIN,
- DECREASE-KEY,
- DELETE.

It maintains a collection of heap-ordered trees in a root list, and uses *lazy* consolidation plus *cascading cuts*.
Let:

- $t(H)$ = # of trees in the root list of heap H ,
- $m(H)$ = # of *marked* nodes (nodes that have lost a child since they became a child themselves).

Potential Function. Define

$$\Phi(H) = t(H) + 2m(H).$$

This is always non-negative; a newly created empty heap has $\Phi = 0$.

Operation Analyses. **Insert**(x). We create a singleton tree containing x and add it to the root list.

- Real cost $c_i = O(1)$.
- $t(H)$ increases by 1, $m(H)$ unchanged, so $\Phi(H_i) - \Phi(H_{i-1}) = 1$.
- Amortized cost:

$$\hat{c}_i = c_i + \Delta\Phi = O(1) + 1 = O(1).$$

Find-Min. We maintain a pointer to the minimum root in the root list.

- Real cost $c_i = O(1)$.
- No structural change, so $\Delta\Phi = 0$.
- Amortized cost: $\hat{c}_i = O(1)$.

Union(H_1, H_2). We concatenate the two root lists and update the min pointer.

- Real cost $c_i = O(1)$ (pointer manipulation).
- $t(H)$ becomes $t(H_1) + t(H_2)$ and m is unchanged; this is essentially additive and does not increase asymptotically.
- Amortized cost: $\hat{c}_i = O(1)$.

Extract-Min. Let z be the current minimum root. We:

1. Remove z from the root list.
2. Add all children of z to the root list (unmarking them).
3. *Consolidate*: repeatedly link roots of the same degree, making the larger-key root a child of the smaller-key root.

The expensive part is consolidation. Let $D(n)$ be the maximum degree of any node in a heap of size n (it can be shown that $D(n) = O(\log n)$).

- The real cost c_i is $O(D(n) + t(H))$, which is $O(\log n + t(H))$.
- Potential change:
 - Removing z decreases $t(H)$ by 1.
 - Adding its children to the root list increases $t(H)$ by $\deg(z)$ and sets their marks to 0 (reducing $m(H)$ if they were marked).
 - Each root-link during consolidation decreases $t(H)$ by 1 and may change marks only by a constant.

Overall, one can show

$$\Delta\Phi = \Phi(H_i) - \Phi(H_{i-1}) \leq O(\log n) - \Omega(t_{\text{before}}),$$

so a large amount of structural work (many root links) is paid for by a corresponding decrease in potential.

- Combining these bounds yields an amortized cost $\hat{c}_i = O(\log n)$.

Decrease-Key(x, Δ). We decrease the key of node x by $\Delta > 0$. If the heap order is violated (i.e. x becomes smaller than its parent), we *cut* x from its parent and add x to the root list. If the parent was already marked, we repeat this upwards (cascading cuts); otherwise, we mark the parent.

Let k be the number of nodes cut (including x).

- Real cost $c_i = O(k)$ (each cut and mark/unmark is $O(1)$).
- Potential change:
 - Each cut adds one tree ($t(H)$ increases by 1), but *unmarks* the cut node (decreasing $m(H)$ by 1).
 - Marking the parent increases $m(H)$ by 1.

One can check that each cut reduces $m(H)$ or keeps it bounded, and the potential change over k cuts is at most $O(1) - \Omega(k)$.

- Thus,

$$\hat{c}_i = c_i + \Delta\Phi = O(k) - \Omega(k) + O(1) = O(1).$$

Delete(x). A delete can be implemented as DECREASE-KEY(x, ∞) followed by EXTRACT-MIN. Thus its amortized cost is $O(\log n)$.

Summary. Under the potential $\Phi(H) = t(H) + 2m(H)$, a Fibonacci heap supports:

INSERT, FIND-MIN, UNION, DECREASE-KEY = $O(1)$ amortized, EXTRACT-MIN, DELETE = $O(\log n)$ amortized.

10.6.4 Disjoint Set Union (Union-Find) with Union-by-Rank and Path Compression

Model. We maintain a partition of a set of n elements into disjoint sets, supporting

- MAKE-SET(x): create a singleton set $\{x\}$,
- FIND-SET(x): return the representative of the set containing x ,
- UNION(x, y): merge the sets containing x and y .

We use the usual forest representation with:

- *union-by-rank*: always attach the root of smaller rank to the root of larger rank;
- *path compression*: during FIND-SET, make all nodes on the path point directly to the root.

High-Level Potential Idea. A full potential-based proof of the classic bound is quite involved: Tarjan showed that any sequence of m operations on n elements has total cost

$$O((m+n)\alpha(n)),$$

where α is the inverse Ackermann function (grows extremely slowly).

The potential function Φ assigns a sort of *level* or *rank-based weight* to each node depending on how many times its parent pointer can change before its rank increases or it reaches a higher “phase”. Intuitively:

- Union-by-rank guarantees that a node’s rank increases only $O(\log n)$ times.
- Path compression makes parent pointers jump upwards, and each jump reduces a node’s potential (its level) significantly.
- The complicated mapping from ranks and phases to levels ensures that the potential never becomes negative, and that the amortized cost per operation is $O(\alpha(n))$.

Formally, one defines a hierarchy of functions A_k (Ackermann-like), their inverses, and uses them to classify nodes into levels. The potential function is then something like

$$\Phi = \sum_x f(\text{level}(x)),$$

for a carefully chosen non-decreasing function f . Each pointer change in a FIND-SET significantly decreases the potential of the affected node, paying for the pointer update.

Key Consequence. Without reproducing the full (lengthy) construction, the potential method yields:

- MAKE-SET has $O(1)$ amortized cost,
- UNION has $O(1)$ amortized cost,
- FIND-SET has $O(\alpha(n))$ amortized cost.

Therefore, any intermixed sequence of m operations takes total time $O((m+n)\alpha(n))$.

In all of these examples, the core idea is to identify a potential function that captures *stored work* in the structure (tree heights, number of trees, marked nodes, path lengths), and to show that expensive operations deplete this stored work sufficiently to offset their real cost.

10.6.5 B-Tree

Model. Consider a B-tree of minimum degree $t \geq 2$. Each node (except possibly the root) has between $t-1$ and $2t-1$ keys. Operations: SEARCH, INSERT, DELETE. The expensive work comes from structural changes: *splits*, *merges*, and *borrow*s that can propagate up the tree.

Goal. Show that INSERT and DELETE have amortized cost $O(\log_t n)$, i.e. proportional to the tree height, with only $O(1)$ amortized splits/merges per level.

Potential Function (Slack-Based). For each non-root node v , let $k(v)$ be its number of keys. Define the *slack* of v as

$$s(v) = (2t-1) - k(v),$$

i.e. how many keys it is missing from being completely full. Let

$$\Phi(T) = \sum_{\substack{v \text{ non-root} \\ \text{node in } T}} s(v).$$

Note that $0 \leq s(v) \leq t$ and $\Phi(T) \geq 0$. For the empty tree, we set $\Phi(T_0) = 0$.

Effect of Operations. Insert. An insertion proceeds downwards through $O(\log_t n)$ nodes; we may pre-split full nodes on the way down. Key observations:

- Inserting a key into a non-full node v *decreases* $s(v)$ by 1 (so the potential drops by 1 for that node).
- When a node v with $2t-1$ keys is split into two nodes with $t-1$ and $t-1$ keys (plus one key moved up), the two children each have slack roughly t , whereas previously there was zero slack at v . Thus a split *increases* the local contribution to potential by $O(t)$.
- However, before v can overflow again, at least t more keys must be inserted into that subtree, each of which reduces the potential by at least 1 (as they fill slack). Thus, the extra $O(t)$ potential we pay at a split is “slowly” repaid by many future insertions in that subtree.

By choosing a suitable constant scaling in the definition of $s(v)$ (or equivalently multiplying Φ by a constant), we can ensure that whenever an insertion causes one or more splits along the root-to-leaf path, the amortized cost per split is constant:

$$\hat{c}_i = c_i + \Delta\Phi = O(1) \text{ per split,}$$

and we perform at most $O(\log_t n)$ splits in one insert. Thus, INSERT has amortized cost $O(\log_t n)$.

Delete. Deletion may cause nodes to become *underfull*, triggering merges or borrows. In this case:

- Borrowing from a sibling moves one key between nodes; the net effect on slack (and hence on potential) is $O(1)$.
- A merge of two nodes that are near-minimum size produces a single node with more keys and less total slack; this *decreases* the potential.

Therefore, as with insertion, each structural rebalancing step (merge or borrow) can be paid for by a suitable drop in potential or by the potential that was built up in previous steps. Over any sequence of deletions, the amortized number of rebalancing operations per level remains constant, yielding amortized deletion time $O(\log_t n)$.

Conclusion. Under this slack-based potential, we obtain:

$$\text{SEARCH} = O(\log_t n) \quad (\text{worst-case and amortized}),$$

$$\text{INSERT, DELETE} = O(\log_t n) \quad (\text{amortized}),$$

with only $O(1)$ amortized splits/merges per update.

10.6.6 AVL Tree

Model. An AVL tree is a binary search tree where for every node v ,

$$\text{bf}(v) = \text{height}(\text{left}(v)) - \text{height}(\text{right}(v))$$

satisfies $|\text{bf}(v)| \leq 1$. Rebalancing is done via rotations after insertions and deletions.

Goal. Use the potential method to argue that the number of rotations per update is $O(1)$ amortized, while the search path is $O(\log n)$.

Potential Function (Imbalance). Define, for an AVL tree T ,

$$\Phi(T) = \sum_{v \in T} |\text{bf}(v)|.$$

We clearly have $\Phi(T) \geq 0$, and for a perfectly balanced tree (every node has balance factor 0) we get $\Phi(T) = 0$.

Insertion. Insert a key as in a regular BST; then walk back up the insertion path to restore the AVL property via rotations.

- When we go down the tree to insert, each edge traversal can change the balance factor of a node by at most 1 in absolute value. Thus the downward pass can increase Φ by at most $O(\log n)$ in total.
- Rotations restore balance locally: a single rotation at node(s) x, y reduces $|\text{bf}(x)| + |\text{bf}(y)|$ by at least 1 (often more). Hence each rotation reduces the total potential Φ by $\Omega(1)$.

Let r be the number of rotations triggered by one insertion. Real cost: $c_i = O(\log n + r)$ (search path plus rotations). Potential change: $\Delta\Phi \leq O(\log n) - \Omega(r)$. Hence

$$\hat{c}_i = c_i + \Delta\Phi = O(\log n + r) + O(\log n) - \Omega(r) = O(\log n) + O(\log n) = O(\log n),$$

and, more importantly, the $-\Omega(r)$ term implies that r cannot be large too often; averaged over many operations, the number of rotations per operation is $O(1)$.

Deletion. Deletion is symmetric: removing a key can make some nodes more imbalanced, but each rotation again reduces the potential by at least a constant. A similar argument shows that deletion has amortized cost $O(\log n)$ with $O(1)$ amortized rotations.

Conclusion. With $\Phi(T) = \sum_v |\text{bf}(v)|$, AVL trees satisfy:

$$\text{SEARCH, INSERT, DELETE} = O(\log n) \text{ amortized,}$$

and the expensive balancing operations (rotations) occur only $O(1)$ times on average per update.

10.6.7 Resizable Hash Table with Linked-List Chaining

Model. We store n key-value pairs in a hash table of size m , using separate chaining (linked lists in each bucket). Operations: INSERT, SEARCH, DELETE.

Assume simple uniform hashing: expected chain length is $\alpha = n/m$. We maintain the invariant

$$\frac{1}{4} \leq \alpha \leq 1.$$

Whenever $\alpha > 1$ we *expand* the table (e.g. double m), rehashing all keys. Whenever $\alpha < \frac{1}{4}$ we *shrink* the table (e.g. halve m), again rehashing all keys.

The expensive operations are the expansions/shrinks, which cost $\Theta(n)$.

Potential Function (Load-Factor Deviation). Define

$$\Phi(T) = |2n - m|.$$

We initially have $n = 0$, $m \geq 1$, so $\Phi(T_0) = m$; we can also choose to start with $m = 0$ and $\Phi(T_0) = 0$, but the additive constant does not affect amortized bounds.

Intuition: Φ measures how far we are from the “ideal” balance $m \approx 2n$. Large deviation means we are close to triggering a resize. When we resize, m changes so that $2n$ and m become much closer, hence the potential drops by $\Omega(n)$, paying for the rehash.

Insertion Without Resizing. Suppose $\text{INSERT}(x)$ does not trigger a resize. Then:

- Real cost c_i : expected $O(1)$ to find the right chain and scan it (by uniform hashing), plus constant-time pointer updates.
- Potential change:

$$\Delta\Phi = |2(n+1) - m| - |2n - m|.$$

Since n increases by 1 and m is fixed, we have $|\Delta\Phi| \leq 2$.

Thus the amortized cost is

$$\hat{c}_i = c_i + \Delta\Phi = O(1) + O(1) = O(1).$$

Deletion Without Resizing. Similarly, if deletion does not trigger resizing:

- Real cost $c_i = O(1)$ expected (search in one bucket, pointer removal).
- n decreases by 1, m fixed, so $|\Delta\Phi| \leq 2$.

Hence $\hat{c}_i = O(1)$.

Resize (Expansion). Consider an expansion step where we double the table size: $m' = 2m$, n unchanged. This happens when the load factor exceeds the upper threshold (e.g. $n > m$).

- Real cost $c_i = \Theta(n)$, since we rehash all n keys.
- Before expansion, $n > m$, so $2n - m > n$ and

$$\Phi_{\text{before}} = |2n - m| = 2n - m = \Omega(n).$$

- After expansion, $m' = 2m$ and

$$\Phi_{\text{after}} = |2n - 2m| = 2|n - m|.$$

Right after expansion, $n \leq m$ and typically $n \approx m$, so $|n - m| = O(n)$ but much smaller than the pre-resize deviation. In particular we can choose thresholds so that

$$\Phi_{\text{before}} - \Phi_{\text{after}} = \Omega(n).$$

Thus, the potential drops by $\Omega(n)$ during expansion, which can pay for the $O(n)$ rehash cost. Formally, for a suitable constant factor in the potential,

$$\hat{c}_i = c_i + \Delta\Phi = O(n) - \Omega(n) = O(1).$$

Resize (Shrink). Shrinking is symmetric: when $\alpha < 1/4$, we halve m . Again $c_i = \Theta(n)$, but the deviation $|2n - m|$ is large before shrinking, and becomes much smaller after, so $\Delta\Phi$ is negative and of magnitude $\Omega(n)$, paying for the rehash.

Conclusion. With the potential $\Phi(T) = |2n - m|$ and invariant $\frac{1}{4} \leq \alpha \leq 1$, we get:

$$\text{INSERT, SEARCH, DELETE} = O(1) \text{ amortized,}$$

even though individual operations occasionally perform $\Theta(n)$ work due to resizing. The potential method formalizes that these expensive steps are rare and paid for by many cheap operations.

10.7 Practice: WA2 task 1

```
C
increment(i):
    C[i] = C[i] + 1
    if C[i] % 3 == 0:
        increment(i + 1)
        increment(i + 2)
```

10.7.1 Claim: `increment(i)` has $O(1)$ amortized cost.

Proof. Let $X := \sum_{t=1}^n \text{cost}(t)$ be the total number of invocations (top-level plus all recursive) as each invocation increments exactly one counter once.

Let $T := \sum_i \lfloor C[i]/3 \rfloor$ be the total number of *triggers* (events when an index reaches a multiple of 3). Each trigger creates exactly two recursive invocations, and the only invocations not caused by triggers are the n top-level calls. Hence

$$X = n + 2T \leq n + \frac{2}{3} \sum_i C[i] = n + \frac{2}{3}X,$$

as $\lfloor C[i]/3 \rfloor \leq C[i]/3$. Rearrange, $X \leq 3n$ and the average cost is $\frac{1}{n} \sum_{t=1}^n \text{cost}(t) = \frac{X}{n} \leq 3 \in O(1)$. $\square$

10.7.2 There exists a sequence of n operations with one operation costing $\Omega(n)$

Let $k := \lfloor n/2 \rfloor$. Consider the following sequence of n top-level calls (all of the form `increment(0)` or direct index calls as allowed):

$$\forall t \in \{1, \dots, n-1\} : o_t := \text{increment}(\lceil t/2 \rceil), \quad o_n := \text{increment}(1).$$

Each `increment(j)` with $C[j] \in \{0, 1\}$ just adds 1 and does not trigger recursion unless it lands on $2 \rightarrow 0 \pmod{3}$. Starting from all zeros, exactly two calls to `increment(j)` set $C[j]$ to 2 and cause no recursion. Therefore, after the first $n-1$ operations,

$$\forall i \in \{1, \dots, k\} : C[i] = 2, \quad \text{and} \quad C[i] = 0 \text{ for } i > k.$$

10.7.3 Cost of the n -th operation.

Now execute $o_n = \text{increment}(1)$. At index 1 we have $2 \mapsto 0$, which triggers calls to indices 2 and 3. At any index $j \in \{1, \dots, k\}$ the first time it is invoked during o_n we have $C[j] = 2$, so it goes $2 \mapsto 0$ and triggers two children $(j+1)$ and $(j+2)$. In particular:

1. Every index $i \in \{1, \dots, k\}$ is reachable from 1 by repeatedly adding +1 or +2 (e.g., by induction on i using the edges $j \rightarrow j+1$ and $j \rightarrow j+2$), so each such i is invoked at least once during o_n .
2. On its first invocation, each $i \in \{1, \dots, k\}$ performs at least one activation (itself) and, because $2 \mapsto 0$, it also triggers (recurses further). Thus o_n has at least k activations in total.
3. Therefore the cost of o_n is $\Omega(k)$. With $k = \lfloor n/2 \rfloor$, we obtain an operation of cost $\Omega(n)$ within a sequence of n operations, as for $f(n) = n$

$$\forall n \geq 10, 0 \leq \frac{1}{3}f(n) \leq k$$

10.7.4 There exists a finite sequence of operations that the process does not terminate

Consider this sequence:

1. $o_1 := \text{increment}(1)$
2. $o_2 := \text{increment}(2)$
3. $o_3 := \text{increment}(1)$

Assuming applicative order evaluation in the recursion, after these 3 operations, current counter is $[2, 1, 0, \dots]$, and `increment(2)`, `increment(3)` are on the call stack. after the next `increment(2)` call, counter is $[2, 2, 0, 0]$, call stack contains `increment(3)`, `increment(4)`, `increment(3)`. tracing down we can see the call stack grows

Simple program to empirically prove this sequence does not terminate:

```
static vector<int> counter(1000, 0);
void increment(int idx) {
    if (idx > 1000) {
        cout << "overflow" << endl;
        return;
    }
    counter[idx]++;
    if (counter[idx] % 2 == 0) {
        increment(idx+1);
        increment(idx+2);
    }
}
int main() {
    increment(1);
    increment(2);
    increment(1);
}
```

11 Common Graph Algorithms and Their Time Complexities

Throughout this section, let $|V|$ denote the number of vertices and $|E|$ denote the number of edges. We explicitly use $|V|$ and $|E|$ in all complexity expressions.

11.1 Shortest Path Algorithms

11.1.1 Dijkstra's Algorithm (Binary Heap)

Computes single-source shortest paths for graphs with **non-negative** edge weights.

$$O(|E| \log |V|).$$

Caveat: Fails in the presence of negative-weight edges.

11.1.2 Dijkstra's Algorithm (Fibonacci Heap)

Improves decrease-key operations using a Fibonacci heap.

$$O(|E| + |V| \log |V|).$$

Note: Better theoretical bound, but slower in practice.

11.1.3 Bellman–Ford

Handles negative edges and detects negative cycles reachable from the source.

$$O(|V| |E|).$$

Caveat: Much slower than Dijkstra; used when negative edges exist.

11.1.4 Floyd–Warshall

Dynamic programming algorithm for all-pairs shortest paths (APSP).

$$O(|V|^3) \text{ time, } O(|V|^2) \text{ space.}$$

Note: Works with negative edges but does not detect negative cycles.

11.1.5 Johnson's Algorithm

APSP for sparse graphs using Bellman–Ford + reweighting + repeated Dijkstra.

$$O(|V| |E| \log |V|).$$

Use-case: Best for sparse, weighted graphs with some negative edges.

11.1.6 SPFA (Shortest-Path Faster Algorithm)

A practical improvement of Bellman–Ford.

$$\text{Average: } O(|E|), \quad \text{Worst-case: } O(|V| |E|).$$

Caveat: Worst-case is as bad as Bellman–Ford; adversarial graphs break it.

11.2 Minimum Spanning Tree Algorithms

11.2.1 Kruskal's Algorithm (Union–Find)

Sort edges and merge components greedily.

$$O(|E| \log |V|).$$

Note: Ideal for sparse graphs.

11.2.2 Prim's Algorithm (Binary Heap)

Greedy MST construction using a priority queue.

$$O(|E| \log |V|).$$

11.2.3 Prim's Algorithm (Fibonacci Heap)

Improved version using Fibonacci heaps.

$$O(|E| + |V| \log |V|).$$

11.3 Maximum Flow Algorithms

11.3.1 Ford–Fulkerson

Augmenting-path algorithm.

No polynomial upper bound; runtime depends on capacities.

11.3.2 Edmonds–Karp

Uses BFS to select shortest augmenting paths.

$$O(|V||E|^2).$$

11.3.3 Dinic’s Algorithm

Blocking flows + layered graphs.

$$O(|V|^2|E|) \quad \text{general case,}$$

$$O(\min(|V|^{2/3}, \sqrt{|E|})|E|) \quad \text{unit capacities.}$$

11.3.4 Push–Relabel (Goldberg–Tarjan)

Preflow-based maximum flow.

$$O(|V|^3), \quad O(\sqrt{|V|}|E|) \quad \text{with highest-label heuristics.}$$

11.4 Connectivity and Components

11.4.1 Tarjan’s SCC Algorithm

DFS-based algorithm using low-link values.

$$O(|V| + |E|).$$

11.4.2 Kosaraju’s SCC Algorithm

Two-pass DFS on G and G^{rev} .

$$O(|V| + |E|).$$

11.5 Matching and Assignment

11.5.1 Hopcroft–Karp

Maximum bipartite matching using BFS/DFS layering.

$$O(\sqrt{|V|}|E|).$$

11.5.2 Hungarian Algorithm

Minimum-cost perfect matching in bipartite graphs.

$$O(|V|^3).$$

11.6 Planar Graph Algorithms

11.6.1 Planar SSSP

Single-source shortest paths in planar graphs.

$$O(|V|\log|V|).$$

11.6.2 Planar Max-Flow

Maximum flow specialized for planar graphs.

$$O(|V|^{1.5}).$$

11.7 All-Pairs Shortest Paths (APSP)

11.7.1 APSP via Repeated Dijkstra

Run Dijkstra from every vertex.

$$O(|V||E|\log|V|).$$

Caveat: Requires all edge weights to be non-negative.

11.7.2 APSP via Repeated Bellman–Ford

Run Bellman–Ford from every vertex.

$$O(|V|^2|E|).$$

11.7.3 APSP via Fast Matrix Multiplication

Uses min-plus matrix multiplication.

$$O(|V|^\omega), \quad \omega \approx 2.373.$$

Caveat: Theoretical; constants are extremely large.

12 Reduction

Problem A reduces to problem B:

1. Convert an instance of A α to an instance of B , β
2. Solve the instance of B to obtain a solution of $B(\beta)$
3. Convert $B(\beta)$ back to a solution for α

Example: MAT-MULT $\leftrightarrow$ MAT-SQT

- MAT-SQT reduces to MAT-MULT: given an input C (problem instance) of MAT-SQT, let $A = C$, $B = C$ be the inputs (problem instance) of MAT-MULT, clearly $C^2 = AB$
- MAT-MULT reduces to MAT-SQT: Given A and B (problem instance of MAT-MULT), let $C = \begin{bmatrix} 0 & A \\ B & 0 \end{bmatrix}$, $C^2 = \begin{bmatrix} AB & 0 \\ 0 & BA \end{bmatrix}$, we can learn AB

12.1 p(n)-time reduction

1. Convert an instance of A α to an instance of B , β at most $p(n)$ size, at most $p(n)$ time
2. Solve the instance of B to obtain a solution of $B(\beta)$, at most $T(p(n))$ time
3. Convert $B(\beta)$ back to a solution for α , at most $p(n)$ time

12.2 Running time composition

Suppose:

1. $\exists p(n)$ -time reduction from A to B
2. $\exists T(n)$ -time algorithm to solve B of size n

$\exists (T(p(n)) + O(p(n)))$ – time algorithm to solve A of size n

12.3 polynomial time reduction

$\exists O(n^c)$ -time reduction from A to B for some constant c , $\implies A \leq_P B$

If B can be solved in polynomial time, A can be solved in polynomial time

$$T(n) \in n^{O(1)} \implies (T(p(n)) + O(p(n))) \in n^{O(1)}$$

If A cannot be solved in polynomial time, so does B . (contrapositive)

$$(T(p(n)) + O(p(n))) \notin n^{O(1)} \implies T(n) \notin n^{O(1)}$$

Polynomial time is considered "easy", because polynomial functions are closed under composition. $T(n)$ and $p(n)$ are polynomial, $T(p(n))$ is polynomial

12.4 Input size: problem encoding

12.5 Example: Karp Reduction from 3SAT to INDEPENDENT SET

We show a non-trivial example of a polynomial-time (Karp) reduction:

$$3SAT \leq_p \text{INDEPENDENT SET}.$$

Source Problem. Given a 3-CNF Boolean formula

$$\varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_m,$$

where each clause $C_i = (\ell_{i1} \vee \ell_{i2} \vee \ell_{i3})$ consists of three literals.

Target Problem. The INDEPENDENT SET problem: Given a graph $G = (V, E)$ and integer k , determine whether there exists a subset $S \subseteq V$ of size at least k such that no two vertices in S are adjacent.

Reduction Construction. From the given 3SAT instance, construct a graph $G = (V, E)$ and integer k as follows:

1. For each clause C_i , create three vertices v_{i1}, v_{i2}, v_{i3} , each representing a literal in the clause.
2. Connect these three vertices to form a triangle (K_3). This ensures that at most one vertex (literal) from each clause can be chosen in an independent set.
3. For any two vertices v_{ip} and v_{jq} whose literals are complementary (e.g., x and $\neg x$), add an edge between them.
4. Set $k = m$, the number of clauses.

Intuition. Selecting one vertex from each triangle represents choosing one literal per clause. Edges between complementary literals ensure that no assignment sets both x and $\neg x$ to true.

Correctness Proof.

- ($\Rightarrow$) If the 3SAT formula φ is satisfiable, then there exists an assignment that makes at least one literal in each clause true. Pick for each clause the vertex corresponding to one true literal. Since no complementary literals are both true, these vertices form an independent set of size $m = k$.
- ($\Leftarrow$) If G has an independent set S of size $k = m$, then it contains exactly one vertex from each clause's triangle (since vertices inside the same clause are connected). Because S is independent, no two vertices correspond to complementary literals. Assign each chosen literal to true; extend arbitrarily to remaining variables. Each clause thus has at least one true literal, so φ is satisfiable.

Complexity. The construction creates $3m$ vertices and at most $O(m^2)$ edges, so it runs in polynomial time.

Conclusion.

$$\varphi \in \text{3SAT} \iff (G, k) \in \text{INDEPENDENT SET}.$$

Hence, $\text{3SAT} \leq_p \text{INDEPENDENT SET}$.

Note. Since $(G, k) \in \text{INDEPENDENT SET} \iff (G, |V| - k) \in \text{VERTEX COVER}$, this also gives the well-known reduction $\text{3SAT} \leq_p \text{VERTEX COVER}$.

Tutorial: Reductions and Computational Complexity

1. Lecture Review: Reductions and Computational Complexity

1.1 Revisiting Time Complexity

Time complexity is actually computed based on the input size.

- **Example 1: Sorting**

Input: N (32-bit) integers.

Input size: $O(32 \cdot N) = O(N)$.

Merge sort algorithm runs in $O(N \log N)$ – polynomial with respect to input size.

- **Example 2: Fibonacci**

Input: One single integer, which has value N .

Input size: $O(\log N)$ for just that one integer.

The DP algorithm (that sums the last two Fibonacci values) runs in $O(N)$ – this is *not* polynomial with respect to input size, as there is an exponential gap from $\log N$ to N , i.e. $2^{\log_2 N} = N$.

However, it is *pseudopolynomial* if we consider the input as N and the DP runtime is $O(N)$.

1.2 Reductions

Key Idea:

- We want to solve a problem A
- We know how to solve a problem B
- To solve A , we can translate/reduce problem A to B

Pseudocode of a typical reduction algorithm:

```
Solve_A(instance_of_A):
    instance_of_B = translate_A_to_B(instance_of_A)
    solution_of_B = Solve_B(instance_of_B)
    solution_of_A = translate_B_to_A(solution_of_B)
    return solution_of_A
```

We call this a *polynomial-time reduction* if both translation steps above run in polynomial time. We denote this as $A \leq_p B$.

1.3 Decision vs Optimization Problems

- **Decision Problem:** Any problem where the output is Boolean (YES/NO).
- **Optimization Problem:** Any problem where we want to optimize the output (maximize, minimize, longest, shortest, etc.).

12.6 Tutorial:

12.6.1 Q1. Graph Coloring

Problem. Graph Coloring is the problem of assigning colors to vertices of a graph such that no two adjacent vertices share the same color.

Which statement(s) is/are True?

- (a) If we can solve the optimization problem for Graph-Coloring in polynomial time, we would be able to solve the decision problem for Graph-Coloring in polynomial time. [true]
 - trivial. We can prove that for any optimization problem P_{opt} that is polynomial time solvable, its decision version P_{dec} is polynomial time solvable. Let $Opt(I) = v$, $Dec(I, k) = v > k$
- (b) If we can solve the decision problem for Graph-Coloring in polynomial time, we would be able to solve the optimization problem for Graph-Coloring in polynomial time. [true]
 - Since we know the bound of the solution space of the optimization problem $k \in [1, V]$, we can perform a binary search to find the lowest k such that $Dec(I, k) = true$.
 - $Dec(I, k) = true \wedge \forall k' \in [1, k), Dec(I, k') = false, Opt(I) = k$
- (c) If the decision problem for Graph-Coloring cannot be solved in polynomial time, the optimization problem for Graph-Coloring cannot be solved in polynomial time. [true]
- (d) If the optimization problem for Graph-Coloring cannot be solved in polynomial time, the decision problem for Graph-Coloring cannot be solved in polynomial time. [true]

Define:

- I : problems instance
- $|I|$: problem size
- $S(I)$: set of possible solutions

$$Dec(I, k) \in \mathbb{P} \wedge |S(I)| \in O(p(|I|)) \implies Opt(I) \in \mathbb{P}$$

Proof.

$$\forall s \in S(I), Opt(I) = \min_{s \in S \wedge Dec(I, s)} (s)$$

It takes $O(\log(p(|I|)))$ calls to find the optimal solution.

□

12.6.2 Q2. Partition versus Ball-Partition

Partition: Given a set of positive integers S , can the set be partitioned into two subsets of equal total sum? Example: If $S = \{18, 2, 8, 5, 7, 24\}$, we can partition it as $S_1 = \{18, 2, 5, 7\}$ and $S_2 = \{8, 24\}$, both summing to 32.

Ball-Partition: Given k balls, can we divide them into two boxes with an equal number of balls? Example: If $k = 4$, we can divide them into $\{2, 2\}$.

We try to show that $Partition \leq_p Ball-Partition$ using the following transformation A :

1. From the problem Partition, we are given a set of positive integers S .
2. Define k as the total sum of all integers in S .
3. Use this number k for the Ball-Partition problem.

What is wrong with this transformation?

- (a) The transformation does not run in polynomial time.
- (b) This transformation is correct.
- (c) A YES solution to $A(S)$ does not imply a YES solution to S .
- (d) A YES solution to S does not imply a YES solution to $A(S)$.

12.6.3 Q3. Partition versus Knapsack

Given a Partition instance $\{w_1, w_2, \dots, w_n\}$ with total sum $S = \sum_{i=1}^n w_i$, construct a Knapsack instance $\{(w_1, w_1), (w_2, w_2), \dots, (w_n, w_n)\}$ with capacity $W = S/2$ and threshold $V = S/2$.

Which statement(s) is/are True?

- (a) The transformation runs in polynomial time.
- (b) A YES answer to the Partition instance implies a YES answer to the Knapsack instance.
- (c) A YES answer to the Knapsack instance implies a YES answer to the Partition instance.

12.6.4 Q4. Hamiltonian Cycle (HC) versus Travelling Salesperson Problem (TSP)

Show that $HC \leq_p TSP$.

Define TSP_{dec} as if there exists HC of length at most C in a complete graph G

Steps:

1. Show the transformation algorithm.
2. Show the transformation algorithm runs in polynomial time.
3. Show a YES answer to the HC instance implies a YES answer to the TSP instance.
4. Show a YES answer to the TSP instance implies a YES answer to the HC instance.

Intuition: to ensure a YES instance of HC implies a YES instance of TSP, we force the constructed TSP instance take the Hamilton cycle tour in the HC instance.

1. Given an instance of HC $G = (V, E)$, construct complete graph of size $|V|$, $G' = (V, E')$, $|E'| = \frac{|V| \cdot (|V|-1)}{2}$.
2. $\forall e \in E' \cap E, w(e) = \frac{C}{|V|}; \forall e' \in E' \setminus E, w(e') = C + 1$

This transformation clearly runs in $p(n)$ time.

- $HC(G) = YES \implies TSP_{dec}(G', C) = YES$: if there exists a valid hamilton cycle in HC , this tour is transformed to a tour that visits all vertices in TSP and have a total length of exactly C
- $TSP_{dec}(G', C) = YES \implies HC(G) = YES$: in the constructed graph to ensure all vertices are visited and return to the origin, the only edges can be taken are the one already exist in HC, to satisfy the path length $\leq C$ constraint. If such a tour can be find, meaning it is possible to visit all vertices in HC and return to the origin using only original edges in HC, meaning the original HC instance has a YES answer.

13 NP Complete

There are 3 main types of NPC problem falling into different domains:

- Logical Expression:
 - Circuit-SAT (The OG NPC problem)
 - 3-SAT (Circuit-SAT $\leq_P$ 3-SAT)
 - $\{0, 1, 2\}$ -SAT (3-SAT $\leq_P$ $\{0, 1, 2\}$ -SAT)
- Integer Optimization:
 - Subset sum (from 3-SAT)
 - Partition (from subset sum)
 - 0,1 Knapsack (from partition)
- Graph:
 - Independent set (from 3-SAT / VC)
 - Vertex cover (from 3-SAT / IS)
 - Hamiltonian Cycle (from)
 - Hamiltonian Path (from HC)
 - TSP (from Hamiltonian Cycle)
 - Graph Coloring (from 3-SAT)

Any CNF can be converted to an equivalent 3-SAT expression.

Proof. It suffices to show how to replace each clause by an equivalent conjunction of 3-literal clauses, possibly using fresh variables.

- **Case 1: 1-literal clause.**

For a clause consisting of a single literal x , introduce fresh variables a, b and replace

$$x \equiv (x \vee a \vee b) \wedge (x \vee \neg a \vee b) \wedge (x \vee a \vee \neg b) \wedge (x \vee \neg a \vee \neg b).$$

If x is true, we can satisfy all four clauses by suitable choices of a, b . Conversely, if the conjunction on the right is satisfiable, then in every satisfying assignment at least one literal in each clause is true; the only way this can happen for all four clauses simultaneously is that x is true. Thus the two formulas are equisatisfiable.

- **Case 2: 2-literal clause.**

For a clause $(x \vee y)$, introduce a fresh variable a and replace

$$(x \vee y) \equiv (x \vee y \vee a) \wedge (x \vee y \vee \neg a).$$

If $(x \vee y)$ is true, then both 3-clauses are clearly true regardless of the value of a . Conversely, if both 3-clauses are true, then at least one of x, y must be true (otherwise the two clauses would reduce to a and $\neg a$, which cannot both be satisfied). Hence the formulas are equisatisfiable.

- **Case 3: clause with more than 3 literals.**

Let a clause have the form

$$(\ell_1 \vee \ell_2 \vee \dots \vee \ell_m), \quad m > 3,$$

where each ℓ_i is a literal. Introduce fresh variables $y_1, y_2, \dots, y_{m-3}$ and replace this clause by the conjunction of 3-clauses:

$$\begin{aligned} & (\ell_1 \vee \ell_2 \vee y_1) \\ & \wedge (\neg y_1 \vee \ell_3 \vee y_2) \\ & \wedge (\neg y_2 \vee \ell_4 \vee y_3) \\ & \vdots \\ & \wedge (\neg y_{m-4} \vee \ell_{m-2} \vee y_{m-3}) \\ & \wedge (\neg y_{m-3} \vee \ell_{m-1} \vee \ell_m). \end{aligned}$$

Equisatisfiability:

- If at least one of the literals $\ell_1, \dots, \ell_m$ is true, we can choose the y_i so that all the 3-clauses are satisfied. For example, if some ℓ_j is true, we can propagate suitable truth values along the chain so that each clause has at least one true literal.

- Conversely, if the conjunction of 3-clauses is satisfiable, then along the chain it is impossible for all $\ell_1, \dots, \ell_m$ to be false: if all ℓ_i were false, the clauses force alternating constraints on the y_i that eventually become contradictory. Therefore at least one ℓ_i must be true, so the original long clause is satisfied.

Thus the long clause is equisatisfiable with the conjunction of 3-clauses.

Applying these transformations to every clause of an arbitrary CNF formula yields an equisatisfiable 3-CNF (3-SAT) formula. Hence any CNF can be converted to an equivalent 3-SAT expression. $\square$

13.1 3-SAT $\leq_P$ IS

13.2 3-SAT $\leq_P$ VC

Tutorial: Reductions and Computational Complexity (cont.)

Lecture Review: Reductions and Computational Complexity (cont.) So far, we have been told the following classes of problems:

- **P:** problems that can be solved in polynomial time.
- **NP:** problems that can be verified in polynomial time.

To prove something is in NP:

1. You can verify the “YES” instance in polynomial time via a certificate.
2. State the certificate, then show it verifies the “YES” instance in polynomial time.

NP-hard:

- Problems that can be polynomial-time reduced from all problems in NP.

To prove something is NP-hard:

1. Show that it is harder than (or equal to) any pre-existing NP-hard problem.
2. Show that $\text{A-PROVEN-NP-HARD-PROBLEM} \leq_P \text{SOMETHING}$.

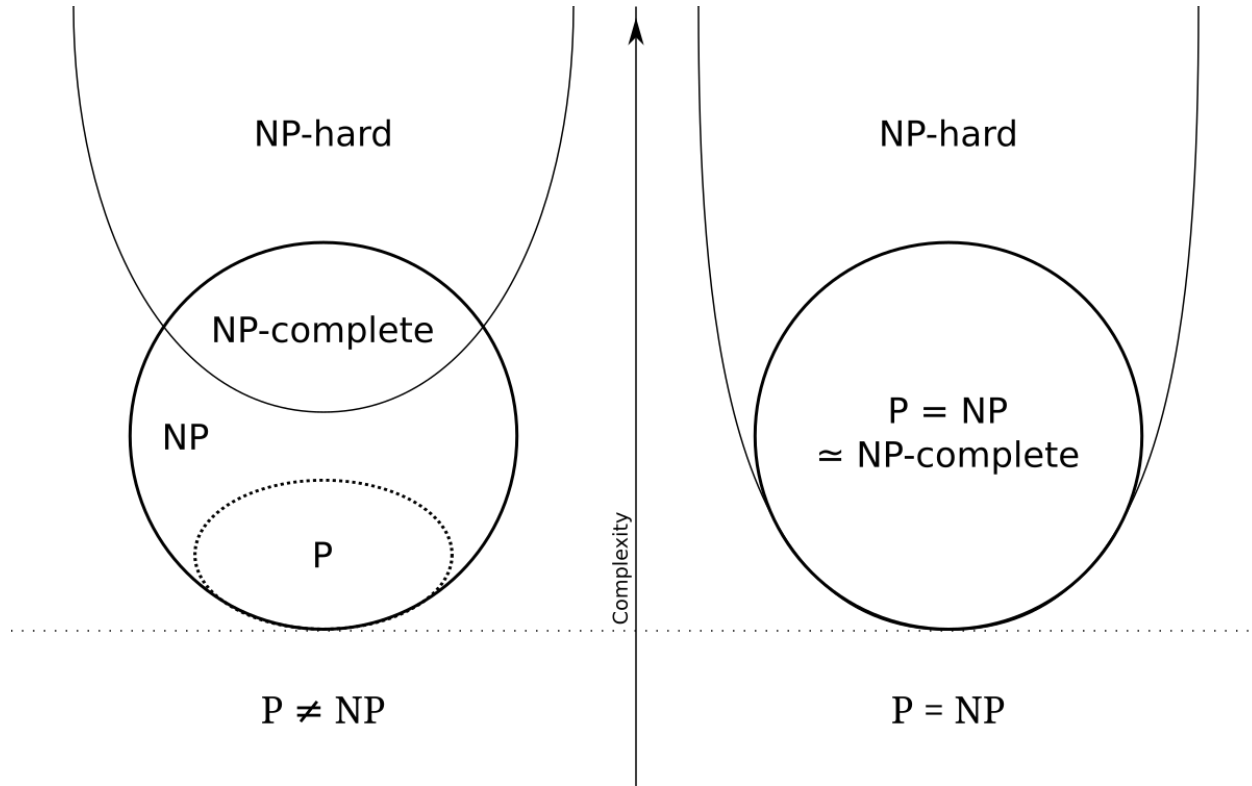
NP-complete:

- Problems that are both in NP and NP-hard.

To prove something is NP-complete:

1. Prove it is in NP.
2. Prove it is NP-hard.

13.3 NP-Hardness and Its Relationship to NP



In computational complexity theory, a decision problem H is called *NP-hard* if every problem $L \in \text{NP}$ can be reduced to H via a polynomial-time many-one reduction. Formally,

$$\forall L \in \text{NP}, \quad L \leq_P H.$$

This means that any polynomial-time algorithm capable of solving H can be used (through the reduction) to solve every problem in NP. Hence, NP-hardness is a *hardness classification*: NP-hard problems are at least as difficult as all NP problems.

NP-Hard vs. NP. A common point of confusion is that NP-hardness does not imply that a problem is in NP. In fact, many NP-hard problems lie strictly outside NP, including:

- PSPACE-complete problems (e.g. TQBF),
- EXPTIME-complete problems,
- recursively enumerable but undecidable problems (e.g. the Halting Problem).

These problems are NP-hard because every NP problem reduces to them, but they are not in NP because they do not necessarily admit polynomial-time verifiers, or may not even be decidable.

NP-Complete Problems. A problem is *NP-complete* if it is both NP-hard and in NP:

$$\text{NPC} = \text{NP} \cap \text{NP-hard}.$$

NP-complete problems are the “hardest” problems within NP: if any NP-complete problem can be solved in polynomial time, then all NP problems can be solved in polynomial time, implying $P = \text{NP}$.

- NP-complete problems lie in the intersection: $\text{NP} \cap \text{NP-hard}$,
- NP-hard problems extend far beyond NP,
- NP-hardness is a hardness notion, not a membership class.

Summary. NP-hardness expresses that a problem is at least as hard as every problem in NP. Such problems need not be in NP; they may lie outside polynomial-time verifiable problems, or even outside decidable problems. NP-complete problems are exactly the NP-hard problems that also lie inside NP.

13.3.1 Karp Reductions: Propagation of Easiness and Hardness

Given decision problems A and B with a Karp reduction $A \leq_P B$, we analyze which complexity-class implications are valid. Recall that a Karp reduction allows us to transform instances of A into instances of B in polynomial time while preserving membership:

$$x \in A \iff f(x) \in B.$$

This enables problem A to be solved using a solver for B , and similarly enables verification for A using a verifier for B .

Incorrect Implications. Several answer choices incorrectly claim:

- $A \in P \Rightarrow B \in P$, or
- $A \in NP \Rightarrow B \in NP$.

These are false because a reduction $A \leq_P B$ only shows how to solve or verify A using an algorithm for B ; it does *not* provide any method to solve or verify B using an algorithm for A . Thus, easiness does *not* propagate from A to B under a Karp reduction.

Correct Implications. The correct propagation rules under $A \leq_P B$ are:

$$B \in P \Rightarrow A \in P, \quad B \in NP \Rightarrow A \in NP.$$

The first implication holds because a polynomial-time algorithm for B , preceded by the reduction f , yields a polynomial-time algorithm for A . The second implication holds because an NP verifier for B , preceded by f , yields a polynomial-time verifier for A : the certificate for A can be reused as the certificate for the reduced instance of B .

These two implications correspond exactly to option (d), which is therefore the correct answer.

Takeaway. A Karp reduction $A \leq_P B$ tells us:

- **Easiness flows backward:** If B is easy (in P or NP), then A is easy.
- **Hardness flows forward:** If A is hard (e.g., NP-hard), then B is also hard.

Thus, under a Karp reduction, complexity never flows from A to B : it always flows in the opposite direction.

Graph Coloring: Algorithms & Analysis

Hint. Anchor your reasoning with two easy bounds: the clique number $\omega(G)$ is a lower bound on $\chi(G)$, and a greedy pass in a good order gives a usable upper bound; a tight search prunes between them.

Problem. Given an undirected graph $G = (V, E)$, find the minimum number of colors $\chi(G)$ so that adjacent vertices receive different colors. Let $n = |V|$, $m = |E|$.

Basic Bounds (useful in analysis).

$$\omega(G) \leq \chi(G) \leq \Delta(G) + 1, \quad \chi(G) \leq d(G) + 1$$

where Δ is the max degree and $d(G)$ is the degeneracy (max core number). Brooks' theorem sharpens the upper bound to $\chi(G) \leq \Delta(G)$ except for cliques and odd cycles.

Greedy (Welsh–Powell) & Degeneracy Orders. **Idea.** Order vertices (e.g., by nonincreasing degree or degeneracy order) and color each with the smallest available color.

- **Complexity:** $O(n + m)$ with adjacency lists and bitsets per neighborhood.
- **Guarantee:** Uses at most $d(G) + 1$ colors (optimal on chordal graphs with perfect elimination order).

Pseudocode (Greedy).

```
order = DegeneracyOrder(G) // O(n+m)
for v in order:
    forbid = {color[u] : u in N(v) and colored(u)}
    color[v] = smallest nonnegative int not in forbid
```

DSATUR Heuristic (Brélaz). **Idea.** Repeatedly pick the uncolored vertex with the highest *saturation degree* (number of distinct neighbor colors); break ties by highest degree. Color it with the smallest feasible color.

- **Why it works well:** Focuses early on constrained vertices, quickly increases the lower bound and often matches $\chi(G)$ in practice.
- **Complexity:** With efficient updates, $O(n^2) - O(n^2 \log n)$; each of n steps inspects/updates neighbor color sets.
- **Bound use:** The number of colors used by DSATUR is an *upper bound* U ; the current max clique found gives a *lower bound* L .

Pseudocode (DSATUR).

```

U = 0; for v: sat[v]=0, color[v]=-1
for step=1..n:
  v = argmax_{uncolored} (sat[v], degree[v])
  c = smallest color not used by N(v)
  color[v]=c; U = max(U, c+1)
  for u in N(v) with color[u]==-1: update sat[u]

```

Exact Coloring: Branch-and-Bound on a Search Tree. **Idea.** Backtracking assigns colors to vertices one by one, pruning nodes where the partial assignment already forces $\geq U$ colors or conflicts. Tighten bounds using $L = \omega(G)$ (via clique search) and U from fast coloring (DSATUR/greedy).

- **State order:** Use DSATUR order dynamically to branch on the most constrained vertex.
- **Pruning rules:** (i) If current colors used $\geq U$, cut; (ii) if partial clique grows to U , no better solution below; (iii) color-class merging tests.
- **Complexity:** Exponential in the worst case (provably so), but very fast on many practical graphs due to strong pruning.

Recurrence (conceptual). Let $T(s)$ be expected nodes from state s ; branching on k feasible colors gives

$$T(s) \leq \sum_{i=1}^k T(s_i) + O(\deg(v)),$$

with k shrinking as bounds tighten; no closed form, but empirical growth is often much smaller than k^n .

Special Classes (Polynomial-Time).

- **Bipartite graphs:** $\chi = 2$ iff no odd cycle. 2-color via BFS in $O(n + m)$.
- **Chordal graphs:** $\chi = \omega$; color optimally in $O(n + m)$ via perfect elimination order (MCS).
- **Interval/comparability/perfect graphs:** $\chi = \omega$; polynomial-time coloring using structure-specific orders.
- **Planar graphs:** $\chi \leq 4$ (4CT). Linear-time 5-coloring; degeneracy ≤ 5 implies a trivial 6-coloring by greedy.

Pseudocode (2-coloring BFS).

```

for all v: color[v]=-1
for each component root r:
  color[r]=0; queue={r}
  while queue not empty:
    v=pop(); for u in N(v):
      if color[u]==-1: color[u]=1-color[v]; push(u)
      else if color[u]==color[v]: return "not bipartite"
return "2-coloring found"

```

Approximation & Hardness (high level).

- General graph coloring is NP-complete for $k \geq 3$; moreover, hard to approximate within $n^{1-\epsilon}$ for any $\epsilon > 0$ in the worst case.
- Greedy gives a $(\Delta + 1)$ -coloring; degeneracy order yields $(d(G) + 1)$ colors. On sparse graphs, $d(G) \ll \Delta$, giving better guarantees.

ILP / Set Cover Formulation (for exact solvers). **Set cover view:** Color classes are independent sets. Let $\mathcal{I}$ be the family of independent sets; choose a minimum subfamily covering V .

$$\min \sum_{S \in \mathcal{I}} y_S \quad \text{s.t.} \quad \sum_{S \ni v} y_S \geq 1 \quad \forall v, \quad y_S \in \{0, 1\}.$$

Column generation adds violated independent sets via a maximum-weight independent set oracle.

Memory & Engineering Notes. Use bitsets to represent color availability and adjacency (word-level parallelism), maintain tight upper bounds by reusing DSATUR in each node, and keep a running clique lower bound. For large sparse graphs, k-core ordering (degeneracy) yields small color counts and fast runtimes.

Takeaways.

- **Heuristics** (degeneracy, DSATUR) are fast and usually near-optimal.
- **Exact** B&B becomes practical with strong L/U bounds and informed branching.
- **Polynomial** algorithms exist for many structured classes; recognize them early.
- **Theory** is harsh on approximations in general, but structure-aware methods win in practice.

Tutorial 10 Questions

Q1. Which of the following imply $P = NP$?

- (a) There is a problem in P that is also NP-complete. [true]
 (b) There is a problem in P that is also in NP. [$P \subseteq NP$, tautology]
 (c) There is a problem in NP that is also NP-hard. [false. this only shows this problem is npc]

Q2 & Q3. Involving SUBSET-SUM

Given a multiset S of n (usually non-negative) integers $\{S_1, S_2, \dots, S_n\}$ and an integer W , the **SUBSET-SUM** problem asks:

Does there exist a subset $I \subseteq \{1, 2, \dots, n\}$ such that $\sum_{i \in I} S_i = W$?

Example: Given $n = 5$, $S = \{5, 1, 5, 1, 4\}$, and $W = 7$, then this is a YES-instance with certificates indices $\{0, 1, 3\}$ or values $\{5, 1, 1\}$ that sum to 7.

We want to prove that SUBSET-SUM is NP-complete, using the usual two-step proof.

Q2. Prove that SUBSET-SUM is in NP.

Sum and equality check runs in $O(|S|)$

Q3. Prove that SUBSET-SUM is NP-hard.

Proof. Suppose a instance of 3-SAT, where there are n variables, and m clauses:

$\forall x_i \in V, c_j \in C$, define the following numbers:

- $a_i = 10^{i+m} + \sum_{j: x_i \in c_j} 10^j$
- $b_i = 10^{i+m} + \sum_{j: \neg x_i \in c_j} 10^j$
- $p_j = 10^j$
- $q_j = 10^j$

$$W = \sum_{i=m}^{n+m-1} 10^i + 3 \sum_{j=0}^{m-1} 10^j$$

If in 3-SAT $\forall x_i \in V$ evaluates to true, select a_i , else select b_i ;

$\forall c_i \in C$ containing 1 true literal, select q_i ;

$\forall c_i \in C$ containing at most 2 true literal, select p_i ;

For an instance of 3-SAT to evaluate to YES, ie there exist an assignment of variables V such that the CNF evaluates to true, the constructed SUBSET($\{a_i, b_i | i \in [1, n]\} \cup \{p_j, q_j | j \in [0, m-1]\}, W$) evaluates to YES. $\square$

3-SAT(YES) $\implies$ Subsum(YES):

for 3-SAT instance to be YES, following constraints need to be satisfied:

- $\forall c_i \in C, \exists r \in \{1, 2, 3\}$ literals evaluates to true;
- $\forall x_i \in V, x_i \vee \neg x_i$

Suppose $S_s = (S_{s1} \cup S_{s2}) \subseteq S$ to be the subset that has sum W :

$\forall x_i \in V$, if $x_i = \text{true}$, $a_i \in S_{s1}$ else $b_i \in S_{s1} \implies \sum_{S_{s1}} = \sum_{i=m}^{n+m-1} 10^i + (k_i) \sum_{j=0}^{m-1} 10^j$, where $k_i \in \{1, 2, 3\}$

$\forall c_i = (l_{i,1} \vee l_{i,2} \vee l_{i,3}) \in C$, if $l_{i,1} + l_{i,2} + l_{i,3} = 1$, $p_i \in S_{s2}$; if $l_{i,1} + l_{i,2} + l_{i,3} = 2$, $q_i \in S_{s2} \implies \sum_{S_{s2}} = (3-k) \sum_{j=0}^{m-1} 10^j$.

$\sum_{S_{s1} \cup S_{s2}} = \sum_{i=m}^{n+m-1} 10^i + 3 \sum_{j=0}^{m-1} 10^j$

Subsum(YES) $\implies$ 3-SAT(YES)

Suppose a YES instance of constructed subset sum, $S = \{a_i, b_i | i \in [1, n]\} \cup \{p_j, q_j | j \in [0, m-1]\}$, digit $0 \sim m-1 = 3$, meaning:

$\forall i \in [1, n], a_i \oplus b_i \in S_s$, which accounts for digit $0 \sim m-1$ to be 1;

For a clause column $j \in \{0, \dots, m-1\}$. Let $k_j \in \{0, 1, 2, 3\}$ be the number of true literals of C_j under the assignment just defined; equivalently, k_j is the number of chosen assignment items among $\{a_i, b_i\}$ that contribute a 1 in column j . The padding items $\{p_j, q_j\}$ can contribute at most 2 in column j . Since the target digit in column j is 3 and the total equals the target, we must have $k_j \geq 1$. Therefore C_j has at least one true literal.

$$\begin{aligned}
 & \forall i \in [1, n], j \in [0, m-1], \\
 & (x_i \vee \neg x_i) \wedge (\hspace{15em} \text{MSB } 0 \sim n \\
 & ((\sum_{r=1}^3 l_{j,i,r} = 2) \wedge (\sum_{r=1}^3 ((l_{j,i,r} = x_i) \wedge x_i) \vee ((l_{j,i,r} = \neg x_i) \wedge \neg x_i)) = 1) \vee \hspace{1em} \text{LSB } 0 \sim m \text{ Case: 2 true literals} \\
 & ((\sum_{r=1}^3 l_{j,i,r} = 1) \wedge (\sum_{r=1}^3 ((l_{j,i,r} = x_i) \wedge x_i) \vee ((l_{j,i,r} = \neg x_i) \wedge \neg x_i)) = 2) \vee \hspace{1em} \text{LSB } 0 \sim m \text{ Case: 1 true literals} \\
 & ((\sum_{r=1}^3 l_{j,i,r} = 3) \wedge (\sum_{r=1}^3 ((l_{j,i,r} = x_i) \wedge x_i) \vee ((l_{j,i,r} = \neg x_i) \wedge \neg x_i)) = 3) \vee \hspace{1em} \text{LSB } 0 \sim m \text{ Case: 3 true literals} \\
 &)
 \end{aligned}$$

Q4 & Q5. Involving FIND-FAMILY

Definitions:

- An undirected bipartite graph $G = (L \cup R, E)$ has two disjoint vertex sets L and R , where each edge connects one vertex in L and one in R .
- Two vertices $u, v \in L$ are called **siblings** if there exists a vertex $r \in R$ such that both edges (u, r) and (r, v) are present.
- A subset $F \subseteq L$ is said to be a **family** if for all distinct $u, v \in F$, u and v are siblings.

Decision problem: FIND-FAMILY

Given $G = (L \cup R, E)$, is there a family of size $\geq k$?

Figure 3: Example bipartite graph with $L = \{0, 2, 4\}$ and $R = \{1, 3\}$. Here, 0 and 2 are siblings, 2 and 4 are siblings, but $\{0, 2, 4\}$ is not a family since 0 and 4 are not siblings.

We want to prove that FIND-FAMILY is NP-complete, using the usual two-step proof.

Q4. Prove that FIND-FAMILY is in NP.

Q5. Prove that FIND-FAMILY is NP-hard.

CLIQUE $\leq_P$ FIND-FAMILY

Proof. Construction

From an instance of CLIQUE($G = (V, E), k$), construct an instance of FIND-FAMILY($G' = (V', E'), k$), where $V'' = V' \cup U$, $V' = V$, $U = \{u_e | e \in E\}$, $E' = \{(v, u_e), (u_e, v') | v, v' \in V, (v, v') \in E\}$.

CLIQUE(YES) $\implies$ FIND-FAMILY(YES)

Suppose a YES instance of CLIQUE, $\exists S \subseteq V, |S| \geq k, \forall v, v' \in S, v \neq v' \exists v \times v' \in E$. In the construction, $\forall v \in S$ in CLIQUE(YES), $\exists! v' \in F \subseteq V', \exists! u \in U, (v', u) \in E', F$ is a family with size k .

FIND-FAMILY(YES) $\implies$ CLIQUE(YES)

Suppose a YES instance of constructed FIND-FAMILY, $\exists S \subseteq V', |S| = k, \exists! u \in U, (v', u) \in E'$, consider the construction as a function $f : (v, v') \mapsto \{(v, u) \cup (u, v')\}$ $\square$

14 Common NPC proofs

14.1 Reduction from Circuit-SAT to SAT (NP-Completeness of SAT)

In this subsection we present a detailed polynomial-time reduction from the CIRCUIT-SAT problem to the SAT problem. Together with the observation that SAT is in NP, this establishes that SAT is NP-complete. The key idea behind the reduction is to encode the behaviour of each gate in a Boolean circuit using a small collection of CNF clauses, thus constructing a CNF formula whose satisfying assignments correspond exactly to the consistent truth assignments of the circuit.

Conceptual Note. A Boolean circuit is simply a directed acyclic graph with gates (AND, OR, NOT) as internal nodes and Boolean inputs as sources. A circuit evaluates deterministically: once the inputs are assigned, the value of every internal wire is forced by the gate semantics. **The reduction relies on the following idea:**

Introduce one propositional variable for every wire in the circuit, and enforce the correct behaviour of each gate using a constant-size CNF formula.

If all gate constraints are satisfied and the output wire is forced to be 1, we have precisely captured the notion of a “valid evaluation” of the circuit.

Encoding Gates in CNF

Let u, v, w denote propositional variables corresponding to the input and output wires of a gate. We now express each gate relationship using CNF constraints. The preservation of the gate semantics is achieved by encoding the logical equivalence

$$w \leftrightarrow \text{Gate}(u, v)$$

as a conjunction of implications.

AND Gate. For a gate $w = u \wedge v$, we enforce:

$$\begin{aligned} w \Rightarrow u &\Rightarrow (\neg w \vee u), \\ w \Rightarrow v &\Rightarrow (\neg w \vee v), \\ u \wedge v \Rightarrow w &\Rightarrow (\neg u \vee \neg v \vee w). \end{aligned}$$

Thus the CNF block for an AND gate is:

$$(\neg w \vee u) \wedge (\neg w \vee v) \wedge (\neg u \vee \neg v \vee w).$$

OR Gate. For a gate $w = u \vee v$:

$$\begin{aligned} w \Rightarrow (u \vee v) &\Rightarrow (\neg w \vee u \vee v), \\ u \Rightarrow w &\Rightarrow (\neg u \vee w), \\ v \Rightarrow w &\Rightarrow (\neg v \vee w). \end{aligned}$$

Thus the CNF block is:

$$(\neg w \vee u \vee v) \wedge (\neg u \vee w) \wedge (\neg v \vee w).$$

NOT Gate. For a gate $w = \neg u$:

$$\begin{aligned} w \Rightarrow \neg u &\Rightarrow (\neg w \vee \neg u), \\ \neg u \Rightarrow w &\Rightarrow (u \vee w). \end{aligned}$$

So the CNF block is:

$$(\neg w \vee \neg u) \wedge (u \vee w).$$

Conceptual Note. Each of the above CNF blocks admits exactly the satisfying assignments consistent with the intended gate semantics. No illegal combination of wire values can satisfy all clauses of the block.

Constructing the Formula

Let C be a Boolean circuit with input wires $x_1, \dots, x_n$, internal wires $y_1, \dots, y_m$, and a single output wire z . We define a CNF formula φ_C as follows:

1. **Variables.** The variables of φ_C are all wire variables:

$$\{x_1, \dots, x_n, y_1, \dots, y_m, z\}.$$

2. **Gate Constraints.** For every gate in C with input wires (u, v) and output w , append the CNF clauses corresponding to its type (AND/OR/NOT) as described above. Each gate contributes at most 3 clauses of size at most 3.

3. **Output Constraint.** Enforce that the final output wire must evaluate to 1:

$$(z).$$

The resulting formula φ_C is clearly in CNF, and its size is linear in the number of gates of C .

Correctness of the Reduction

We prove that for every Boolean circuit C ,

$$C \text{ is satisfiable} \iff \varphi_C \text{ is satisfiable.}$$

($\Rightarrow$) **If the circuit is satisfiable.** Suppose some assignment to the input wires makes the circuit output 1. Evaluating the circuit gate-by-gate yields unique values for all internal and output wires. Let β denote this full assignment to all wire variables.

Each gate constraint added to φ_C is satisfied by β because β assigns the correct output to each gate given its inputs. Furthermore, $\beta(z) = 1$, so the clause (z) is satisfied. Hence φ_C is satisfiable.

($\Leftarrow$) **If the formula is satisfiable.** Let β be any satisfying assignment of φ_C . In particular, β satisfies all constraints encoding the behaviour of each gate. Since each CNF block admits only the assignments consistent with the truth table of the corresponding gate, the wire assignments under β force each gate to operate correctly.

Thus, if we restrict β to the input wires $x_1, \dots, x_n$ and evaluate the circuit in the usual manner, the induced gate evaluations must agree with β on every internal wire and on the output wire z . Because the final clause (z) is satisfied, $\beta(z) = 1$, and hence the circuit outputs 1 under that input assignment. Therefore, C is satisfiable.

Polynomial-Time Complexity

The construction of φ_C involves a single pass over all gates of the circuit, adding a constant number of constant-size clauses for each. Hence $f(C) = \varphi_C$ is produced in time $\mathcal{O}(|C|)$, i.e., polynomial in the size of the circuit.

Conclusion

Since CIRCUIT-SAT is NP-complete and

$$\text{CIRCUIT-SAT} \leq_p \text{SAT},$$

it follows by definition that SAT is NP-hard. Together with the fact that SAT is in NP, we conclude that SAT is NP-complete.

14.2 Reduction from 3-SAT to NAE-3-SAT⁺

In this subsection we present a polynomial-time reduction from 3-SAT to the NOT-ALL-EQUAL 3-SAT problem with only positive literals, denoted NAE-3-SAT⁺. In NAE-3-SAT⁺ we are given clauses of the form (a, b, c) where a, b, c are positive Boolean variables, and a clause is satisfied under an assignment if *not all three literals have the same truth value*. That is, both $(0, 0, 0)$ and $(1, 1, 1)$ are forbidden; all other assignments satisfy the clause.

Conceptual Note. A clause in ordinary 3-SAT is satisfied whenever *at least one literal is true*. In contrast, a clause in NAE-3-SAT is satisfied whenever the three literals are *not* all equal. This is a symmetry-based notion:

$$(0, 0, 1), (0, 1, 0), (1, 0, 0), (1, 1, 0), (1, 0, 1), (0, 1, 1) \text{ are all acceptable.}$$

Since NAE-3-SAT⁺ forbids negations, achieving the reduction requires introducing a reference variable representing a “truth baseline” and encodings that simulate negation by separating variables from this baseline.

Construction of the Reduction

Let an instance of 3-SAT be a CNF formula

$$\varphi = \bigwedge_{i=1}^m (l_{i1} \vee l_{i2} \vee l_{i3}),$$

where each l_{ij} is either a positive literal x or a negative literal $\neg x$. We construct an instance ψ of NAE-3-SAT⁺ as follows.

Step 1: Introduce a global reference variable. Add a fresh variable t (the “truth baseline”). Intuitively, t will serve as the point against which all literal truth values are compared. A positive literal x will be interpreted as “ x differs from t in the NAE sense” (i.e., the clause is satisfied when x is not equal to t), while a negated literal $\neg x$ will be simulated by the requirement that x *agrees* with t .

Step 2: Simulating literals. For each literal l appearing in the 3-SAT instance, we associate the following pair of NAE conditions:

- If $l = x$ (a positive literal), we will use x directly in clauses together with t .
- If $l = \neg x$, we simulate negation by introducing a fresh variable x' and adding the NAE constraint

$$\text{NAE}(x, x', t),$$

enforcing the parity condition: exactly one of x and x' equals t . In effect, x' becomes an NAE-encoding of “ $\neg x$ ”.

Conceptual Note. The constraint $\text{NAE}(x, x', t)$ enforces:

$$x = t \iff x' \neq t, \quad x \neq t \iff x' = t.$$

Thus x' functions as a negated version of x relative to the baseline t .

Step 3: Converting each clause. Given a 3-SAT clause $(l_1 \vee l_2 \vee l_3)$, we create the NAE clause

$$\text{NAE}(l_1^*, l_2^*, l_3^*),$$

where each l_i^* is the corresponding NAE variable:

- If $l_i = x$, then $l_i^* = x$.
- If $l_i = \neg x$, then $l_i^* = x'$ (defined via the relation $\text{NAE}(x, x', t)$).

Since all NAE clauses use only positive variables x, x', t , the instance is indeed a valid NAE-3-SAT^+ formula.

Correctness of the Reduction

We now argue that

$$\varphi \text{ is satisfiable} \iff \psi \text{ is NAE-satisfiable.}$$

($\Rightarrow$) **If φ is satisfiable.** Let α be a satisfying assignment for φ . Construct an assignment β for the variables of ψ as follows:

1. Choose $t = 0$ (arbitrary; the construction is symmetric).
2. Set $\beta(x) = \alpha(x)$ for all original variables.
3. For each x' representing $\neg x$, set

$$\beta(x') = 1 - \alpha(x),$$

ensuring $\text{NAE}(x, x', t)$ is satisfied.

We claim that each NAE clause $\text{NAE}(l_1^*, l_2^*, l_3^*)$ originating from the 3-SAT clause $(l_1 \vee l_2 \vee l_3)$ is satisfied under β . Indeed, since $(l_1 \vee l_2 \vee l_3)$ is true under α , exactly one of the following holds:

- Some positive x among the three is 1, hence differs from t .
- Some negative $\neg x$ is true, so $x = 0$ and therefore $\beta(x') = 1$, differing from t .

Thus among the triple (l_1^*, l_2^*, l_3^*) at least one variable differs from t , meaning the triple is not monochromatic. Therefore the NAE clause is satisfied. Hence ψ is NAE-satisfiable.

($\Leftarrow$) **If ψ is NAE-satisfiable.** Let β satisfy ψ . We first normalize the assignment by possibly flipping all variables:

If necessary, replace every value b by $1 - b$ so that $\beta(t) = 0$.

This preserves satisfaction of all NAE clauses (NAE is invariant under global complementing).

For each original variable x we set

$$\alpha(x) = \beta(x).$$

For each negated literal $\neg x$ we examine the constraint $\text{NAE}(x, x', t)$. Since $\beta(t) = 0$, exactly one of x and x' equals 1:

$$\beta(x') = 1 \iff \beta(x) = 0.$$

Thus a clause $(l_1 \vee l_2 \vee l_3)$ from φ is satisfied by α : in its corresponding NAE clause, at least one variable differs from $t = 0$, hence is equal to 1, so its source literal in the 3-SAT clause is true. Therefore α satisfies φ .

Polynomial-Time Complexity

The reduction creates one fresh variable x' for each negated literal and a constant number of NAE clauses per original clause. The total size of ψ is thus linear in the size of φ , and the construction is computable in $\mathcal{O}(|\varphi|)$ time.

Conclusion

We have established a polynomial-time reduction

$$3\text{-SAT} \leq_p \text{NAE-3-SAT}^+.$$

Since 3-SAT is NP-complete, it follows that NAE-3-SAT^+ is NP-hard. As NAE-satisfiability can be checked using a polynomial-time verifier, NAE-3-SAT^+ is NP-complete.

Extension: From 3-SAT to General NAE-3-SAT

In the previous subsection we reduced 3-SAT to the positive-literal variant NAE-3-SAT^+ , where clauses must contain only unnegated variables. This required the introduction of a global baseline variable t and auxiliary variables x' to simulate negation in an NAE context.

When the target problem is the standard version of NAE-3-SAT, *which permits negated literals*, the reduction becomes much simpler. Indeed, the Not-All-Equal relation already handles negation naturally, because:

$$\text{NAE}(u, v, w) \text{ is symmetric under complementing any subset of its input variables.}$$

Thus, the key complexity of the $+$ version disappears.

Key Insight. For ordinary NAE-3-SAT, a clause such as

$$(l_1 \vee l_2 \vee l_3)$$

can be translated directly to an NAE clause

$$\text{NAE}(l_1, l_2, l_3, t),$$

where t is a single new variable included in every clause. The role of t is to “break” the monochromatic all-0 assignment, yet because negations are allowed, we do not need x' variables: negated literals may be used directly in the NAE clause.

Construction. Given a 3-SAT instance

$$\varphi = \bigwedge_{i=1}^m (l_{i1} \vee l_{i2} \vee l_{i3}),$$

introduce one new variable t . For each clause (l_{i1}, l_{i2}, l_{i3}) produce the NAE clause

$$\text{NAE}(l_{i1}, l_{i2}, l_{i3}, t).$$

This is legal in NAE-3-SAT because negated literals are allowed.

Why this works. Observe the following facts:

1. The NAE clause

$$\text{NAE}(l_{i1}, l_{i2}, l_{i3}, t)$$

forbids both all-0 and all-1 assignments.

2. If at least one of l_{i1}, l_{i2}, l_{i3} is true, then by choosing $t = 0$ the clause is satisfied.
3. If all three l_{ij} are false, then the clause becomes $\text{NAE}(0, 0, 0, t)$, which requires $t = 1$. But because t appears in *every* clause, some other clause that has a true literal would enforce $t = 0$, creating a contradiction. Hence no satisfying NAE assignment can allow all three literals of a clause to be false.

Consequently, each clause behaves exactly like an OR-clause in the original formula.

Correctness. $(\Rightarrow)$ If φ is satisfiable, fix $t = 0$. Each clause has at least one true literal, so no clause becomes monochromatic. Therefore all NAE clauses are satisfied.

$(\Leftarrow)$ If the NAE formula is satisfiable, t must have a fixed value across all clauses. If a clause had all three literals false, that clause would force $t = 1$, but any clause with a true literal forces $t = 0$, contradicting satisfiability. Therefore no clause has all literals false, meaning every clause has at least one true literal — exactly the 3-SAT condition.

Comparison to the NAE-3-SAT^+ Reduction. The positive-literal version required:

- a baseline variable t ,
- an auxiliary variable x' for each negated literal,
- an NAE constraint $\text{NAE}(x, x', t)$ per negation,
- and NAE clauses using only positive literals.

In general NAE-3-SAT:

- negated literals are allowed directly,
- the parity gadget $\text{NAE}(x, x', t)$ disappears,
- each 3-SAT clause becomes a *single* 4-variable NAE clause,
- and the entire reduction requires only one new variable t .

Thus the standard NAE-3-SAT reduction is significantly simpler and more compact than the positive-literal version.

14.3 Reduction from 3-SAT to 0-1-Linear-SAT

We now present a polynomial-time reduction from 3-SAT to a linear satisfiability problem over $\{0, 1\}$ -valued variables, which we call 0-1-LINEAR-SAT. An instance of 0-1-LINEAR-SAT consists of:

- variables $x_1, \dots, x_n$ with domain $\{0, 1\}$, and
- a finite set of linear inequalities of the form

$$a_1x_1 + \dots + a_nx_n \geq b,$$

where a_i, b are integers.

The question is whether there exists an assignment $x \in \{0, 1\}^n$ satisfying all inequalities simultaneously.

Conceptual Note. A 3-SAT clause $(\ell_1 \vee \ell_2 \vee \ell_3)$ is satisfied iff *at least one* of the three literals is true. For $\{0, 1\}$ -valued variables, “at least one is true” is naturally expressed as a *sum inequality*:

$$\ell_1 + \ell_2 + \ell_3 \geq 1,$$

provided we interpret each literal as a linear expression:

$$\ell = x_j \text{ if it is positive,} \quad \ell = 1 - x_j \text{ if it is negative.}$$

This is the key idea behind the reduction.

Construction of the Reduction

Let a 3-SAT instance be given as a CNF formula

$$\varphi = \bigwedge_{i=1}^m (l_{i1} \vee l_{i2} \vee l_{i3}),$$

over variables $X = \{x_1, \dots, x_n\}$, where each l_{ij} is either x_k or $\neg x_k$ for some k .

We construct an instance of 0-1-LINEAR-SAT as follows.

Variables. For each Boolean variable x_j in φ , we create a 0–1 variable (with the same name) $x_j \in \{0, 1\}$. We interpret $x_j = 1$ as “true” and $x_j = 0$ as “false”.

Literal Encoding. For each literal l we define a corresponding linear expression $E(l)$:

$$E(l) = \begin{cases} x_j, & \text{if } l = x_j, \\ 1 - x_j, & \text{if } l = \neg x_j. \end{cases}$$

Note that if $x_j \in \{0, 1\}$, then $E(l) \in \{0, 1\}$ and

$$E(l) = 1 \iff l \text{ is true under the Boolean assignment.}$$

Clause Encoding. For each clause $C_i = (l_{i1} \vee l_{i2} \vee l_{i3})$ in φ , we add the inequality

$$E(l_{i1}) + E(l_{i2}) + E(l_{i3}) \geq 1.$$

Intuitively, at least one of the three literals must evaluate to 1 (true), so the sum must be at least 1.

Resulting System. The 0-1-LINEAR-SAT instance $\mathcal{L}_\varphi$ consists of:

- variables $x_1, \dots, x_n \in \{0, 1\}$, and
- the m inequalities

$$E(l_{i1}) + E(l_{i2}) + E(l_{i3}) \geq 1, \quad i = 1, \dots, m.$$

Correctness of the Reduction

We prove that φ is satisfiable iff $\mathcal{L}_\varphi$ is satisfiable.

($\Rightarrow$) **If φ is satisfiable.** Assume there exists a Boolean assignment

$$\alpha : \{x_1, \dots, x_n\} \rightarrow \{0, 1\}$$

that satisfies φ . Define a 0–1 vector

$$x_j = \alpha(x_j), \quad j = 1, \dots, n.$$

Consider any clause $C_i = (l_{i1} \vee l_{i2} \vee l_{i3})$. Because α satisfies φ , we must have

$$\alpha(l_{i1}) = 1 \quad \text{or} \quad \alpha(l_{i2}) = 1 \quad \text{or} \quad \alpha(l_{i3}) = 1.$$

By construction of $E(l)$ and the fact that $x_j = \alpha(x_j)$, we have

$$E(l_{ij}) = 1 \iff l_{ij} \text{ is true under } \alpha.$$

Therefore, for each clause,

$$E(l_{i1}) + E(l_{i2}) + E(l_{i3}) \geq 1.$$

So the assignment $(x_1, \dots, x_n)$ satisfies all the inequalities in $\mathcal{L}_\varphi$, and hence $\mathcal{L}_\varphi$ is feasible.

($\Leftarrow$) **If $\mathcal{L}_\varphi$ is satisfiable.** Conversely, suppose there exists $x \in \{0, 1\}^n$ satisfying all inequalities of $\mathcal{L}_\varphi$. Define a Boolean assignment

$$\alpha(x_j) = x_j, \quad j = 1, \dots, n.$$

For any clause $C_i = (l_{i1} \vee l_{i2} \vee l_{i3})$, we know that

$$E(l_{i1}) + E(l_{i2}) + E(l_{i3}) \geq 1,$$

and each $E(l_{ij}) \in \{0, 1\}$. Hence, at least one of the terms $E(l_{i1}), E(l_{i2}), E(l_{i3})$ is equal to 1.

But by construction,

$$E(l_{ij}) = 1 \iff l_{ij} \text{ is true under } \alpha.$$

Thus, for each clause C_i , at least one literal l_{ij} is true under α , so C_i is satisfied. Since this holds for all clauses, α satisfies φ .

Polynomial-Time Complexity

The reduction introduces no new variables beyond the original $x_1, \dots, x_n$, and creates exactly one inequality per clause, with at most three nonzero coefficients and a constant-size right-hand side. Therefore, the size of $\mathcal{L}_\varphi$ is linear in the size of φ , and the construction clearly runs in time $\mathcal{O}(|\varphi|)$.

Conclusion

We have defined a polynomial-time reduction

$$3\text{-SAT} \leq_p 0\text{-1-LINEAR-SAT},$$

by encoding each 3-CNF clause as a linear inequality over $\{0, 1\}$ -valued variables. Since 3-SAT is NP-complete, this implies that 0-1-LINEAR-SAT is NP-hard. Because a candidate 0–1 vector can be verified in polynomial time, 0-1-LINEAR-SAT is NP-complete.

14.4 Reduction from 3-SAT to Exact-1 3-SAT

We present a polynomial-time reduction from 3-SAT to the EXACT-1 3-SAT (also called “1-in-3-SAT”) problem. In EXACT-1 3-SAT each clause consists of exactly three literals, and a clause is satisfied if and only if *exactly one* of the three literals is true. Unlike ordinary 3-SAT, where a clause is satisfied if at least one literal is true, the exact-1 constraint enforces a precise combinatorial structure.

Conceptual Note. A 3-SAT clause

$$(l_1 \vee l_2 \vee l_3)$$

admits many satisfying assignments. In contrast, the EXACT-1 clause (a, b, c) restricts to exactly three allowed assignments:

$$(1, 0, 0), (0, 1, 0), (0, 0, 1).$$

To simulate an OR-clause using exact-1 constraints, we introduce fresh “selector” variables that distribute the truth value among multiple exact-1 clauses. The key idea is to enforce the logical condition:

$$l_1 \vee l_2 \vee l_3 \text{ is true} \iff \text{at least one of } l_1, l_2, l_3 \text{ is true.}$$

This will be encoded through a gadget ensuring exactly one literal among an expanded set is chosen while allowing us to recover the intended OR-semantics.

Construction of the Reduction

Let the 3-SAT instance be

$$\varphi = \bigwedge_{i=1}^m (l_{i1} \vee l_{i2} \vee l_{i3}),$$

where each literal l_{ij} is either x_k or $\neg x_k$.

For each clause $C_i = (l_{i1}, l_{i2}, l_{i3})$ we construct a constant-size gadget using fresh variables. The construction below is the classical Schaefer gadget.

Step 1: Literal Variables. We keep all original variables. Negated literals $\neg x$ are treated directly as Boolean negations inside clauses; no expansion is needed.

Step 2: Introduce Clause Selector Variables. For each clause C_i , create a new auxiliary variable s_i . Intuitively, s_i acts as a “switch” selecting which literal is made true within the exact-1 enforcement.

Step 3: Replace Each Clause with Three Exact-1 Clauses. For the clause $(l_{i1} \vee l_{i2} \vee l_{i3})$, introduce the following three exact-1-clauses:

$$\text{X1}(l_{i1}, s_i, a_i), \quad \text{X1}(l_{i2}, s_i, b_i), \quad \text{X1}(l_{i3}, s_i, c_i),$$

where a_i, b_i, c_i are fresh auxiliary variables.

Here $\text{X1}(u, v, w)$ denotes a clause of the form “*exactly one of u, v, w is true.*”

Intuition. The selector variable s_i appears in all three clauses. If the original clause (l_{i1}, l_{i2}, l_{i3}) is to be satisfied, then at least one literal must be true. The exact-1 constraints ensure that:

- If one of the literals l_{i1}, l_{i2}, l_{i3} is true, that literal will be paired with s_i and one auxiliary variable in such a way that exactly one is true per clause.
- If none of the literals is true, then s_i would be forced to satisfy all three clauses alone, which is impossible because s_i appears identically in all three exact-1 clauses.

Thus the gadget forces at least one literal of the original clause to be true.

Correctness of the Reduction

We now prove:

$$\varphi \text{ is satisfiable} \iff \psi \text{ is Exact-1 satisfiable.}$$

($\Rightarrow$) **If φ is satisfiable.** Assume a Boolean assignment α satisfies each clause (l_{i1}, l_{i2}, l_{i3}) . Fix a clause C_i and choose one literal l_{ij} that is true under α . Set:

- $s_i = 0$, - a_i, b_i, c_i so that exactly the true literal is chosen in each clause.

Then each of:

$$\text{X1}(l_{i1}, s_i, a_i), \text{X1}(l_{i2}, s_i, b_i), \text{X1}(l_{i3}, s_i, c_i)$$

is satisfied: one literal (the true one) becomes the “1” in its clause, and the auxiliary variable handles the remaining exact-1 structure. Doing this independently for each clause produces an assignment satisfying all exact-1 constraints.

($\Leftarrow$) **If ψ is Exact-1 satisfiable.** Suppose β satisfies all exact-1 clauses. Fix a clause gadget corresponding to C_i . The three exact-1 clauses share s_i . If all three original literals l_{i1}, l_{i2}, l_{i3} were false under the assignment to original variables, then s_i would have to be the unique true literal in all three clauses simultaneously. But this is impossible: s_i cannot satisfy three different exact-1 constraints where the other two literals in each clause differ.

Therefore, at least one of the three literals l_{i1}, l_{i2}, l_{i3} must be true under the restriction of β to the original variables. Hence the original 3-SAT clause is satisfied.

Thus the induced assignment on $\{x_1, \dots, x_n\}$ satisfies all clauses of φ .

Polynomial-Time Complexity

The reduction introduces a constant number of new variables and exactly three exact-1 clauses per original clause. Therefore the size of ψ is linear in the size of the input φ , and the construction clearly runs in time $\mathcal{O}(|\varphi|)$.

Conclusion

We have established a polynomial-time reduction:

$$3\text{-SAT} \leq_p \text{EXACT-1 3-SAT}.$$

Since 3-SAT is NP-complete and exact-1-satisfiability is verifiable in polynomial time, EXACT-1 3-SAT is NP-complete.

14.5 Reduction from 3-SAT to Group Interval Scheduling (Group Size ≥ 3)

In the *Group Interval Scheduling* decision problem (GIS), the input is:

- a set of groups $G_1, \dots, G_m$, each group G_i containing several intervals on the real line,
- the requirement to choose *exactly one* interval from each group,
- with the global constraint that all chosen intervals must be pairwise non-overlapping.

The question is whether such a selection exists.

We prove that GIS is NP-complete even when every group contains at least three intervals.

Intuition. Classical interval scheduling is easy because conflicts arise only from overlaps in time. GIS becomes hard because the “choose exactly one per group” requirement acts as a *logical disjunction*. A group of three intervals behaves like a clause $(l_1 \vee l_2 \vee l_3)$: picking interval i means “literal i is true,” and overlapping structure blocks inconsistent choices.

Reduction Overview

Given a 3-CNF formula

$$\varphi = C_1 \wedge C_2 \wedge \dots \wedge C_m, \quad C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3}),$$

we build a GIS instance in which:

- For each clause C_j , a group G_j contains *three* intervals encoding the three literals.
- For each variable x_i , we impose interval conflicts so that the GIS solution is forced to use either all x_i -positive literals or all x_i -negative literals, but never both.

The construction ensures:

$$\text{GIS instance is feasible} \iff \varphi \text{ is satisfiable.}$$

Literal Intervals

For each literal ℓ_{jk} in clause C_j , introduce one interval $L_{jk} \in G_j$.

We place intervals representing *positive* and *negative* occurrences of variable x_i in different horizontal “tracks” so that a selection choosing both would overlap in time and thus be infeasible.

Concretely:

- All positive-literal intervals for x_i are assigned to a common time slot on the “positive track.”
- All negative-literal intervals for x_i are placed on a disjoint, shifted time slot on the “negative track.”
- These two time slots overlap in time, forcing the scheduler to pick either only-positive or only-negative occurrences of x_i .

Thus the GIS instance enforces a consistent truth value for each variable.

Clause Groups

For each clause $C_j = (\ell_{j1}, \ell_{j2}, \ell_{j3})$, create a group:

$$G_j = \{L_{j1}, L_{j2}, L_{j3}\}.$$

Because the GIS requirement demands choosing *exactly one* interval from each G_j , selecting a feasible schedule corresponds to picking exactly one literal from each clause.

If all three literal intervals in G_j overlap with already chosen intervals, the clause becomes unsatisfied — and there is no feasible GIS selection.

Ensuring Non-Overlap Reflects Satisfiability

We order time into *variable blocks*. For each variable x_i :

- Every occurrence of x_i (positive or negative) lies within that variable’s block.
- The positive and negative literal intervals overlap inside the block.

Different variable blocks do not overlap in time, so they do not interfere.

Clause groups are simply unions of literal intervals from different variable blocks. Thus:

- Selecting L_{jk} is allowed iff the truth assignment sets the corresponding literal ℓ_{jk} to true.
- If a clause has no true literal, all three intervals in G_j overlap with previous choices, leaving no feasible selection.

Correctness. ($\Rightarrow$) Suppose the GIS instance has a feasible selection: one interval per clause, with no overlaps. In each variable block, since positive and negative literal intervals overlap, the selection cannot choose literals from both sets. Thus the selected intervals determine a consistent truth assignment. Since a non-overlapping interval was chosen in each clause group, each clause has at least one true literal. Hence the assignment satisfies φ .

($\Leftarrow$) Suppose φ is satisfiable. Assign “positive block picks” to variables that are true and “negative block picks” to variables that are false. For each clause, choose the literal that evaluates to true. By construction, this literal interval does not overlap with any previously chosen intervals. Thus the chosen set gives a feasible GIS solution.

Polynomial-Time Complexity

Each literal becomes exactly one interval. Each clause becomes one group of size three. All time coordinates and overlaps are constructed using simple arithmetic. Thus the reduction runs in linear time $\mathcal{O}(|\varphi|)$.

Conclusion

We have constructed a polynomial-time reduction:

$$3\text{-SAT} \leq_p \text{GROUP INTERVAL SCHEDULING},$$

where each group contains exactly three intervals.

Therefore, GIS with group size ≥ 3 is NP-complete.

14.6 Reduction from 3-SAT to Planar 3-SAT

We show how to reduce an arbitrary instance of 3-SAT to an equivalent instance of PLANAR 3-SAT. In PLANAR 3-SAT, the formula is in 3-CNF and its *variable-clause incidence graph* is planar.

Planar 3-SAT

Given a 3-CNF formula

$$\varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_m,$$

over variables $x_1, \dots, x_n$, the *incidence graph* G_φ is the bipartite graph with:

- a vertex for each variable x_i ,
- a vertex for each clause C_j ,
- an edge (x_i, C_j) whenever x_i or $\neg x_i$ occurs in C_j .

A 3-CNF formula is *planar* if G_φ admits a planar embedding.

In PLANAR 3-SAT, the input is such a planar 3-CNF formula, and the question is whether it is satisfiable.

Conceptual Note. Arbitrary 3-SAT instances may have highly non-planar incidence graphs; edges (wires) can cross arbitrarily. The key idea is to *planarize* the drawing by replacing each geometric crossing of two edges with a small *crossover gadget*—a constant-size planar 3-CNF subformula that behaves like two independent wires crossing: it preserves the truth value along each wire and prevents any logical interaction between them.

Construction Outline

Let φ be an arbitrary 3-CNF formula. We construct a planar 3-CNF formula ψ such that φ is satisfiable iff ψ is satisfiable.

Step 1: Draw the incidence graph. Consider the incidence graph G_φ and draw it in the plane with straight edges or polygonal lines. In general, this drawing may have edge crossings.

We can assume w.l.o.g. that:

- no three edges cross at the same point,
- crossings occur only in edge interiors (not at vertices).

This can be enforced by standard perturbation arguments and by subdividing edges if necessary.

Step 2: Turn edges into wires. Think of each edge (x_i, C_j) as a “wire” transmitting the literal (either x_i or $\neg x_i$) from the variable vertex x_i to the clause vertex C_j . Where a wire must bend, we can subdivide it with degree-2 vertices; these do not affect satisfiability but will give us intermediate literals that must equal the original variable. Equality can be enforced by small 3-CNF gadgets (e.g. clauses encoding $u \leftrightarrow v$).

Step 3: Crossover gadget for one crossing. Consider a single crossing of two wires. Locally, in the drawing, we have four endpoints:

$$a \text{ and } a' \quad (\text{opposite ends of wire 1}), \quad b \text{ and } b' \quad (\text{opposite ends of wire 2}).$$

Without crossings, we would like the wires to connect as:

$$a \longleftrightarrow a', \quad b \longleftrightarrow b',$$

and we want the truth value at a to match a' , and similarly b with b' , with no interaction between the two wires.

We now replace the geometric crossing by a planar 3-CNF *crossover gadget* with internal auxiliary variables $y_1, \dots, y_k$ and clauses such that:

1. For any assignment to (a, a', b, b') with $a = a'$ and $b = b'$, there exists an extension to the y_i ’s satisfying all gadget clauses.
2. If $a \neq a'$ or $b \neq b'$, then no assignment to $y_1, \dots, y_k$ satisfies all gadget clauses.
3. The incidence graph of the gadget is planar and can be embedded in the neighborhood of the crossing without creating new crossings.

Such a gadget is known to exist and can be constructed with a constant number of variables and 3-clauses (e.g. via a chain of equivalence gadgets and a small planar layout). We treat this gadget as a black box with the above properties.

Step 4: Replace all crossings. For each edge crossing in the drawing of G_φ , we:

- delete the crossing point,
- cut the two crossing edges into segments ending at a small neighborhood around the crossing,
- introduce a crossover gadget and connect its four interface literals to the four cut endpoints (a, a', b, b') .

This yields a new bipartite incidence structure with:

- additional auxiliary variables from all gadgets,
- additional clauses (each a 3-clause).

By construction, the new incidence graph can now be drawn without any crossings: each former crossing point is replaced by a gadget drawn inside a small disk, and no new crossings are introduced. Therefore, the resulting formula ψ is a planar 3-CNF formula.

Correctness

We argue that φ is satisfiable if and only if ψ is satisfiable.

($\Rightarrow$) **From φ to ψ .** Let α be a satisfying assignment for φ on the original variables $x_1, \dots, x_n$. We extend α to all auxiliary variables of ψ :

- For each subdivided wire $(x_i, \dots, C_j)$ we set all intermediate literals equal to the truth value of x_i and extend to the equality gadgets accordingly, which is always possible by design of the equality gadgets.
- For each crossover gadget, we already have values at its endpoints (a, a', b, b') consistent with the original wires (since α treats each wire as a single literal). Because $a = a'$ and $b = b'$, the crossover gadget properties guarantee that there exists an assignment to its internal variables satisfying all its clauses.

Hence we obtain a full assignment that satisfies every clause of ψ .

($\Leftarrow$) **From ψ to φ .** Conversely, suppose β is a satisfying assignment of ψ (over both original and auxiliary variables). Consider any original variable x_i and any wire starting at x_i and ending at some clause C_j . Along this wire we have a chain of literals, equality gadgets, and possibly crossover gadgets.

Because all these gadgets enforce equality along each wire, we must have the same truth value at both endpoints of the wire; otherwise at least one gadget would be unsatisfied. In particular, all occurrences of x_i (or $\neg x_i$) that are connected along wires share a consistent truth assignment.

Thus we can define an assignment α on the original variables by restricting β to $x_1, \dots, x_n$. For each original clause C_j in φ , its incident literals in ψ are connected via wires and gadgets in a way that does not create or remove satisfying combinations. Since β satisfies all the clauses of ψ , each original clause C_j must have at least one satisfied literal under α . Therefore α satisfies φ .

Complexity

Let c be the number of crossings in the chosen drawing of G_φ . Each crossing is replaced by a constant-size gadget (constant number of new variables and clauses). The number of crossings can be bounded by a polynomial in the size of G_φ , and constructing the drawing and gadgets can be done in polynomial time.

Hence the size of ψ is polynomial in the size of φ , and the reduction runs in polynomial time.

Conclusion

We have provided a polynomial-time reduction

$$3\text{-SAT} \leq_p \text{PLANAR } 3\text{-SAT},$$

by planarizing the variable–clause incidence graph using crossover gadgets. Since 3-SAT is NP-complete and PLANAR 3-SAT is in NP, it follows that PLANAR 3-SAT is NP-complete.

14.7 Reduction from 3SAT to Bounded-Occurrence 3SAT

In this subsection, we show how to reduce an arbitrary 3SAT instance to an equivalent 3SAT instance in which *each literal appears at most twice*. This restricted version is often called *Constraint 3SAT*, *3SAT-2*, or *bounded-occurrence 3SAT*. This reduction is useful for proving NP-hardness of problems that can only encode a limited number of literal occurrences.

Problem Setting

Given an arbitrary 3CNF formula

$$\varphi = C_1 \wedge C_2 \wedge \dots \wedge C_m \quad \text{over variables } x_1, \dots, x_n,$$

each clause C_j has exactly three literals, but variables may occur many times throughout the formula. Our goal is to transform φ into a logically equivalent formula φ' such that:

- each clause of φ' still contains exactly 3 literals,
- each literal x_i and $\neg x_i$ appears in *at most two* clauses,
- φ is satisfiable iff φ' is satisfiable.

High-Level Idea

If a literal (say x) appears many times, we replace those occurrences by distinct “copies” $x_1, x_2, \dots, x_k$, one for each appearance. To ensure consistency, we enforce that all copies must take the same truth value. This is accomplished using a small chain of *equivalence gadgets* that link the copies together:

$$x_1 \leftrightarrow x_2, \quad x_2 \leftrightarrow x_3, \quad \dots, \quad x_{k-1} \leftrightarrow x_k.$$

Each equivalence ($a \leftrightarrow b$) can be enforced using four 3CNF clauses, each of which mentions a or b only a limited number of times.

Constructing the Bounded-Occurrence Instance

For each variable x that appears $k > 2$ times in φ , do the following:

1. **Introduce k copies:**

$$x^{(1)}, x^{(2)}, \dots, x^{(k)}.$$

2. **Replace the i -th occurrence of x by $x^{(i)}$.** Replace the i -th occurrence of $\neg x$ by $\neg x^{(i)}$.
3. **Add equivalence clauses** ensuring the copied variables agree:

$$x^{(i)} \leftrightarrow x^{(i+1)} \quad \text{for } i = 1, \dots, k-1.$$

Each equivalence is encoded as the conjunction of two implications:

$$(x^{(i)} \rightarrow x^{(i+1)}) \wedge (x^{(i+1)} \rightarrow x^{(i)}),$$

and each implication is expressed in 3CNF using a 3-literal clause by introducing a fresh dummy variable if necessary.

Because each copy $x^{(i)}$ appears:

- once in its original clause,
- once or twice in equivalence constraints,

we ensure that every literal occurs *at most twice*, satisfying the desired degree bound.

Correctness of the Reduction

If φ is satisfiable, then φ' is satisfiable. Take a satisfying assignment for φ . Assign all copies $x^{(1)}, \dots, x^{(k)}$ the same truth value as x . Then all original clauses are satisfied. Equivalence constraints are also satisfied because the values agree.

If φ' is satisfiable, then φ is satisfiable. In any satisfying assignment of φ' , the equivalence clauses enforce:

$$x^{(1)} = x^{(2)} = \dots = x^{(k)}.$$

Thus we can consistently assign x the common truth value of its copies. Substituting this assignment in the original clauses of φ (before copying) satisfies φ .

Conclusion

We have shown a polynomial-time transformation that converts an arbitrary 3SAT instance into an equivalent 3CNF formula in which every literal appears at most twice. This proves that *bounded-occurrence 3SAT* remains NP-hard. Because it is clearly in NP, this variant is NP-complete.

14.8 Reduction from 3-SAT to 3-Dimensional Matching

It is easier to prove $3\text{-SAT} \leq_P \text{constrained } 3\text{-SAT}$ where each literal appears at most twice first.

We show a polynomial-time reduction from 3-SAT to 3-DIMENSIONAL MATCHING (3DM). In 3DM we are given three disjoint sets X, Y, Z and a set of triples $T \subseteq X \times Y \times Z$. The question is whether there exists a subset $M \subseteq T$ such that:

- every element of $X \cup Y \cup Z$ appears in *exactly one* triple of M ,
- i.e. M is a perfect 3-dimensional matching.

Conceptual Note. Think of 3DM as a perfect matching problem in a 3-partite 3-uniform hypergraph. A set of triples is a matching if no two share an X -element, a Y -element, or a Z -element. Our reduction encodes:

- **variables** by *cycles of triples* (a “variable gadget”), where: choosing one set of triples along the cycle corresponds to setting the variable to TRUE, and choosing the complementary set corresponds to FALSE;
- **clauses** by *clause gadgets* that are forced to hook into the variable gadgets in such a way that a clause can be fully matched only if at least one of its literals is satisfied.

The disjointness constraint of the matching enforces both consistency of variable assignments and satisfaction of all clauses.

Input and Output of the Reduction

Let the input be a 3-CNF formula

$$\varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_k,$$

over variables $x_1, \dots, x_n$, where each clause

$$C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3})$$

has exactly three literals. We construct in polynomial time an instance of 3DM with:

$$X, Y, Z \text{ pairwise disjoint}, \quad T \subseteq X \times Y \times Z,$$

such that φ is satisfiable iff there exists a perfect 3-dimensional matching $M \subseteq T$.

Variable Gadgets

Fix a variable x_i . Suppose x_i appears r_i times in φ (as either positive or negative). The variable gadget for x_i will enforce a binary choice: TRUE or FALSE, and then propagate this choice to each occurrence of x_i in the clauses.

Structure. For each x_i , we create:

- a cycle of $2r_i$ “core” elements in X , call them $a_{i,1}, \dots, a_{i,2r_i}$;
- corresponding “tip” elements in Y , $b_{i,1}, \dots, b_{i,2r_i}$;
- for each $j = 1, \dots, 2r_i$, a pair of triples in T of the form

$$(a_{i,j}, a_{i,j+1}, b_{i,j}) \quad (\text{indices mod } 2r_i),$$

one designated as a *T-triple* (representing choosing $x_i = \text{TRUE}$) and the other as an *F-triple* (representing $x_i = \text{FALSE}$).

Enforced Choice. Because we require a perfect matching:

- each core element $a_{i,j}$ must appear in exactly one selected triple;
- the cycle structure forces us to pick either all T-triples around the cycle, or all F-triples around the cycle (a local mixture would leave some element uncovered or used twice).

Thus, for each variable gadget, any perfect matching implicitly chooses:

$$x_i = \text{TRUE} \quad (\text{select all T-triples}) \quad \text{or} \quad x_i = \text{FALSE} \quad (\text{select all F-triples}).$$

Clause Gadgets

For each clause $C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3})$, we build a clause gadget that:

- contains two fresh elements $c_j \in X$ and $d_j \in Y$;
- for each literal ℓ_{jk} in C_j , introduces a triple in T linking (c_j, d_j) to the corresponding variable gadget.

Literal–Clause Link. Each occurrence of a literal ℓ_{jk} is represented by a specific “tip” element $b_{i,t}$ in the gadget of the corresponding variable x_i :

- if $\ell_{jk} = x_i$, then we connect the clause gadget using the part of the variable gadget that is included only when $x_i = \text{TRUE}$ (a T-triple);
- if $\ell_{jk} = \neg x_i$, we connect using the part that is included only when $x_i = \text{FALSE}$ (an F-triple).

Concretely, we add to T a triple of the form

$$(c_j, d_j, z_{jk}),$$

where z_{jk} is an element in Z paired with the appropriate tip from the variable gadget so that this triple can be used in the matching *only if* the variable is set consistently with the literal.

Clause Satisfaction via Matching. Because we require a perfect matching:

- the clause elements c_j and d_j must be covered exactly once;
- the only way to cover both c_j and d_j is to pick exactly one of the three triples corresponding to literals in C_j ;
- that chosen triple can be included in the matching *only if* the corresponding variable gadget choice (T/F cycle) is consistent with the literal being true.

Thus the clause gadget can be fully matched if and only if at least one of its literals is set to true by the variable assignment induced by the variable gadgets.

Cleanup Gadgets

The variable and clause gadgets may leave some elements of X, Y, Z unused. To obtain a perfect matching instance, we add *cleanup gadgets*, which are small, local triplets that:

- cover the remaining unused elements in pairs or triples,
- can always be fully matched regardless of the assignment, and
- do not interfere with the variable/clause structure (their elements are disjoint from those used in the main gadgets).

We choose the number of cleanup triples so that $|X| = |Y| = |Z|$ and a perfect 3D matching exists iff the main gadgets can be simultaneously matched.

Correctness of the Reduction

We sketch the correctness argument.

($\Rightarrow$) **If φ is satisfiable.** Suppose there is a satisfying assignment α for φ . We build a perfect matching $M \subseteq T$ as follows:

- For each variable x_i :
 - if $\alpha(x_i) = \text{TRUE}$, include all T-triples of its cycle into M ;
 - if $\alpha(x_i) = \text{FALSE}$, include all F-triples.

This covers all elements in the variable gadget exactly once.

- For each clause C_j , choose any literal ℓ_{jk} that is true under α . Include the corresponding triple (c_j, d_j, z_{jk}) into M . This covers c_j, d_j and the associated z_{jk} .
- Finally, add cleanup triples to cover all remaining elements.

By construction, no element is used twice and all elements are used at least once, so M is a perfect 3-dimensional matching.

($\Leftarrow$) **If a perfect 3D matching exists.** Conversely, suppose M is a perfect matching for the constructed instance. Then:

- In each variable gadget, the cycle structure forces M to pick either all T-triples or all F-triples. This defines a Boolean assignment α for each variable x_i .
- For each clause gadget, the elements c_j and d_j must be covered once, so M must contain exactly one triple from the three literal-connected triples of that clause. That triple corresponds to a literal that is consistent with the choice made in the variable gadget, hence that literal is true under α .

Thus every clause C_j has at least one true literal under α , so α satisfies φ .

Polynomial-Time Complexity

The construction creates a constant-size gadget per literal occurrence and per clause, plus a linear number of cleanup triples. Therefore, the total size of X, Y, Z, T is polynomial in the size of the input formula φ , and the reduction runs in time $\mathcal{O}(|\varphi|)$.

Conclusion

We have constructed, in polynomial time, an instance of 3DM that has a perfect 3-dimensional matching if and only if the original 3-CNF formula φ is satisfiable. Hence,

$$3\text{-SAT} \leq_p 3\text{-DIMENSIONAL MATCHING}.$$

Since 3-SAT is NP-complete and 3DM is in NP, it follows that 3DM is NP-complete.

14.9 Reduction from 3-Dimensional Matching to 4-Partition and then to 3-Partition

We describe the classic sequence of reductions given in Garey and Johnson [?], which shows that 3-DIMENSIONAL MATCHING (3DM) can be transformed into 4-PARTITION, and then further reduced to 3-PARTITION. Both 4-PARTITION and 3-PARTITION are strongly NP-complete numeric partitioning problems, and this chain of reductions forms the core of many scheduling, packing, and allocation hardness proofs.

3-Dimensional Matching

An instance of 3DM consists of three disjoint sets

$$X, Y, Z, \quad |X| = |Y| = |Z| = q,$$

and a collection of triples

$$T \subseteq X \times Y \times Z.$$

The question is whether there exists a matching $M \subseteq T$ of size q such that every element of $X \cup Y \cup Z$ is contained in exactly one triple of M .

Reduction to 4-Partition

Given a 3DM instance (X, Y, Z, T) , Garey and Johnson construct an instance of 4-PARTITION as follows. 4-PARTITION takes as input a multiset A of integers and a bound B , and asks whether A can be partitioned into $|A|/4$ quadruples, each with sum exactly B .

Encoding elements. Assign each element of $X \cup Y \cup Z$ a distinct digit block: that is, each element corresponds to a place value in a large base- M representation, where M is chosen larger than any possible carry resulting from additions. For each such element u , let

$$w(u) = M^{\text{index}(u)}.$$

Encoding triples. For each triple $t = (x, y, z) \in T$, introduce a number

$$w(t) = B - (w(x) + w(y) + w(z)),$$

where B is chosen so that:

- each valid quadruple must contain exactly one triple-number $w(t)$,
- that triple-number must be grouped with exactly its three element-numbers $w(x), w(y), w(z)$ to sum to B ,
- any illegal combination of digits necessarily either undershoots or overshoots the target sum B .

The multiset. The 4-PARTITION instance contains:

$$A = \{w(x) : x \in X\} \cup \{w(y) : y \in Y\} \cup \{w(z) : z \in Z\} \cup \{w(t) : t \in T\}.$$

The target sum is B .

Correctness. A 3DM matching M of size q exists if and only if A can be partitioned into q quadruples, each consisting of (x, y, z, t) with $t = (x, y, z)$. Because the base- M encoding isolates digit blocks, each quadruple must include exactly one triple-number $w(t)$ and exactly its three associated element-numbers, and no mixing of incompatible triples is possible. Thus:

$$(X, Y, Z, T) \text{ is a YES-instance of 3DM} \iff (A, B) \text{ is a YES-instance of 4-Partition.}$$

Reduction from 4-Partition to 3-Partition

Garey and Johnson next show how to convert any 4-PARTITION instance into an equivalent 3-PARTITION instance. Recall that 3-PARTITION takes a multiset of $3m$ integers and target B with the promise

$$\frac{B}{4} < a_i < \frac{B}{2},$$

and asks whether the items can be partitioned into m triples each summing to B .

Padding construction. Given a 4-PARTITION instance (A, B) , let $A = \{a_1, \dots, a_{4m}\}$. Introduce m new “dummy” integers

$$d_1, \dots, d_m,$$

chosen so that:

- each original quadruple $(a_{i_1}, a_{i_2}, a_{i_3}, a_{i_4})$ of sum B corresponds to a triple

$$(a_{i_1}, a_{i_2}, d_j) \quad \text{and} \quad (a_{i_3}, a_{i_4}, d'_j),$$

where d_j and d'_j are numbers chosen to make the triple-sum exactly B ;

- each (a_i, d_j) pair lies inside the permitted numerical range $(\frac{B}{4}, \frac{B}{2})$;
- no illegal grouping can sum to B because any such attempt violates either the numeric interval constraint or the base- M digit alignment inherited from the 4-Partition encoding.

The resulting multiset has $3m$ elements, all strictly between $B/4$ and $B/2$, and can be partitioned into triples of sum B if and only if the original multiset A can be partitioned into quadruples of sum B .

Correctness. Each quadruple in the 4-Partition corresponds to exactly two triples in the 3-Partition instance, using the dummy numbers to maintain the correct total. The promise bounds guarantee that exactly three numbers must appear in every triple, preventing spurious combinations. Thus:

$$(A, B) \text{ is a YES-instance of 4-Partition} \iff (A', B) \text{ is a YES-instance of 3-Partition.}$$

Conclusion

Combining the above reductions yields:

$$3\text{DM} \leq_p 4\text{-PARTITION} \leq_p 3\text{-PARTITION},$$

establishing 3-PARTITION as strongly NP-complete. The base- M digit-block encoding from 3DM ensures structural constraints, while the padding construction enforces the strict cardinality and interval restrictions of 3-PARTITION.

14.10 Reduction from 3-Dimensional Matching to Partition into Triangles

We describe how to reduce 3-DIMENSIONAL MATCHING (3DM) to PARTITION INTO TRIANGLES (PIT). Technically, the standard NP-hardness proof for PIT in Garey and Johnson proceeds via EXACT COVER BY 3-SETS (X3C). We therefore view the reduction as a composition

$$3\text{DM} \leq_p \text{X3C} \leq_p \text{PIT}.$$

We briefly recall each problem and outline the construction.

Problem Definitions

3-Dimensional Matching (3DM). An instance of 3DM consists of three pairwise disjoint sets

$$X, Y, Z, \quad |X| = |Y| = |Z| = q,$$

and a set of triples

$$T \subseteq X \times Y \times Z.$$

A subset $M \subseteq T$ is a *3-dimensional matching* if no two triples in M share an element (i.e. they are pairwise disjoint in all coordinates). The decision question is whether there exists a matching M of size q (so that every element of $X \cup Y \cup Z$ is used exactly once).

Exact Cover by 3-Sets (X3C). An instance of X3C consists of a finite set U (the “universe”) and a collection $\mathcal{S}$ of 3-element subsets of U . The question is whether there exists a subcollection $\mathcal{C} \subseteq \mathcal{S}$ such that:

- the sets in $\mathcal{C}$ are pairwise disjoint, and
- their union is all of U .

Such a subcollection is called an *exact cover*.

Partition into Triangles (PIT). An instance of PIT is an undirected graph $G = (V, E)$ with $|V|$ a multiple of 3. The question is whether V can be partitioned into $|V|/3$ disjoint triples, each triple inducing a triangle (a copy of K_3) in G , i.e.:

$$\exists \text{ partition } V = V_1 \dot{\cup} V_2 \dot{\cup} \dots \dot{\cup} V_k, \quad |V_i| = 3, \quad G[V_i] \cong K_3 \text{ for all } i.$$

Step 1: Reduction $3\text{DM} \leq_p \text{X3C}$

Given a 3DM instance (X, Y, Z, T) with $|X| = |Y| = |Z| = q$, we build an X3C instance $(U, \mathcal{S})$ as follows:

- Let the universe be

$$U = X \cup Y \cup Z.$$

Because X, Y, Z are disjoint, we have $|U| = 3q$.

- For each triple $(x, y, z) \in T$, create a 3-element subset

$$S_{(x,y,z)} = \{x, y, z\},$$

and put it into $\mathcal{S}$.

Thus, each 3DM triple becomes a 3-element set in X3C, and a collection of q disjoint triples in T corresponds exactly to an exact cover of U .

Correctness of Step 1.

- If there exists a 3-dimensional matching $M \subseteq T$ of size q , then the corresponding family

$$\mathcal{C} = \{S_t : t \in M\} \subseteq \mathcal{S}$$

has q pairwise disjoint 3-sets whose union is U (since every element of $X \cup Y \cup Z$ appears in exactly one triple of M). Hence $(U, \mathcal{S})$ has an exact cover.

- Conversely, if $(U, \mathcal{S})$ has an exact cover $\mathcal{C}$ consisting of q disjoint 3-sets, then each $S \in \mathcal{C}$ corresponds to a unique triple in T , and these triples are pairwise disjoint. Thus we obtain a 3-dimensional matching of size q .

The construction clearly runs in time polynomial in $|T|$, so $3\text{DM} \leq_p \text{X3C}$.

Step 2: Reduction $X3C \leq_p PIT$

The reduction from X3C to PIT is classical and appears in Garey and Johnson (Problem SP4, “Partition into Triangles”). We only sketch the structure.

Given an X3C instance $(U, \mathcal{S})$ with $|U| = 3q$ and $\mathcal{S} = \{S_1, \dots, S_m\}$, we construct a graph $G = (V, E)$ with the following properties:

- For each element $u \in U$ there is an *element gadget* that ensures u is “covered” by exactly one chosen triangle in any valid partition.
- For each 3-set $S_i \in \mathcal{S}$ there is a *set gadget* whose internal structure allows triangles to be selected in a way that either:
 - corresponds to choosing S_i as part of the exact cover, or
 - corresponds to not choosing S_i , in which case other triangles in the gadget are used to “fill up” the vertices without conflicting with element gadgets.
- Edges are added so that:
 1. If a set $S_i = \{u, v, w\}$ is chosen in the cover, this is represented in G by three triangles, each incident to the gadgets for u, v, w respectively.
 2. No vertex can participate in more than one triangle in any partition; hence each element gadget can be “used” by at most one set gadget.

Intuitively, each 3-set S_i is encoded by a small subgraph with nine vertices and seven possible triangles, arranged so that exactly one pattern of triangles corresponds to “selecting” S_i and linking it to the three element gadgets of its elements. In any full partition into triangles, the graph structure forces a consistent choice of such patterns, yielding an exact cover if and only if the partition exists. [:contentReference\[oaicite:1\]index=1](#)

Correctness of Step 2 (Sketch).

- If there is an exact cover $\mathcal{C} \subseteq \mathcal{S}$, we select, for each $S_i \in \mathcal{C}$, the triangle pattern in its gadget that “uses” the appropriate element gadgets. For the sets not in $\mathcal{C}$, we select only internal triangles that do not touch any element gadget. This yields a partition of V into triangles.
- Conversely, suppose G admits a partition of V into triangles. The element gadgets ensure that each element $u \in U$ is linked to exactly one set gadget, so each u is covered by exactly one 3-set. Collecting all such sets gives an exact cover in the X3C instance.

A crucial point is that the gadgets are of constant size and are assembled in a uniform way, so the construction is polynomial-time in $|U| + |\mathcal{S}|$.

Composed Reduction and Complexity

By composing the two reductions, we obtain a polynomial-time mapping

$$f : 3DM \rightarrow PIT,$$

such that for every 3DM instance (X, Y, Z, T) ,

$$(X, Y, Z, T) \text{ is a YES-instance of 3DM} \iff f(X, Y, Z, T) \text{ is a YES-instance of PIT.}$$

Since 3DM is NP-complete, this shows that PARTITION INTO TRIANGLES is NP-hard. As PIT is clearly in NP (a partition into triangles can be verified in polynomial time), we conclude that PIT is NP-complete.

Remark.

In many textbooks and surveys, the hardness of PIT is stated as a reduction from X3C. The above discussion makes explicit that 3DM and X3C are polynomially equivalent, so we are justified in stating that

$$3\text{-DIMENSIONAL MATCHING} \leq_p \text{PARTITION INTO TRIANGLES.}$$

14.11 Direct Reduction from X3C to Partition into Triangles

We now present a more direct reduction from EXACT COVER BY 3-SETS (X3C) to PARTITION INTO TRIANGLES (PIT), without going through 3-dimensional matching explicitly.

Problem Statement (X3C)

An instance of X3C consists of:

- a finite set (universe) $U = \{u_1, \dots, u_{3q}\}$,
- a family $\mathcal{S} = \{S_1, \dots, S_m\}$ of 3-element subsets $S_j \subseteq U$.

Question: does there exist a subfamily $\mathcal{C} \subseteq \mathcal{S}$ such that:

- sets in $\mathcal{C}$ are pairwise disjoint;
- $\bigcup_{S \in \mathcal{C}} S = U$?

Such a subfamily is called an *exact cover*.

Target Problem: Partition into Triangles (PIT)

Given an undirected graph $G = (V, E)$ with $|V|$ a multiple of 3, we ask whether there exists a partition of V into vertex-disjoint triangles (each part induces a K_3).

High-Level Idea. We build a graph G with:

- *element gadgets*, one for each $u \in U$, that must be covered by triangles in a way that mimics “ u is covered exactly once”;
- *set gadgets*, one for each $S_j \in \mathcal{S}$, that can be tiled in two different ways:
 - an *active* tiling using three special edges that “claim” the corresponding elements;
 - an *inactive* tiling that does not touch any element gadgets.

A partition into triangles then chooses some set gadgets in the active mode and others in the inactive mode. Because element gadgets can participate in only one active triangle, the chosen sets form an exact cover.

Construction

Let $(U, \mathcal{S})$ be an X3C instance.

Vertices. We construct $G = (V, E)$ as follows:

1. For each element $u \in U$, create a cycle of $2d(u)$ vertices, where $d(u)$ is the number of sets in $\mathcal{S}$ that contain u :

$$u_1, u'_1, u_2, u'_2, \dots, u_{d(u)}, u'_{d(u)}.$$

2. For each 3-set $S_j = \{u, v, w\}$, create a set gadget with:

- three *interface vertices* s_j^u, s_j^v, s_j^w ,
- some internal vertices forming a small fixed graph that can be partitioned either (i) using triangles incident to s_j^u, s_j^v, s_j^w (active mode), or (ii) using only internal triangles (inactive mode).

Edges in Element Gadgets. For each element u , we connect its cycle vertices so that:

- the cycle $(u_1, u'_1, u_2, u'_2, \dots, u_{d(u)}, u'_{d(u)})$ can be tiled by triangles only if either:
 - exactly one pair (u_i, u'_i) is “taken” by a triangle involving a set gadget (representing that u is covered by S_j), and the remaining vertices are covered by internal triangles, or
 - no set gadget uses u , in which case some vertices remain uncovered (disallowed).

Concretely, we can attach for each pair (u_i, u'_i) a “connector triangle” involving an interface vertex s_j^u from the gadget for the corresponding 3-set S_j (where S_j contains u). Selecting that triangle in the final partition corresponds to choosing S_j as part of the exact cover and using u in that cover.

Edges in Set Gadgets. Each set gadget for $S_j = \{u, v, w\}$ is designed so that:

- If we use the interface vertices s_j^u, s_j^v, s_j^w to form three triangles, each with a corresponding element gadget, then the rest of the gadget can be perfectly partitioned into internal triangles.
- If none of the interface vertices is used externally, then the gadget has an alternative partition covering all its vertices internally.
- It is impossible to partially use some but not all interfaces without leaving a vertex uncovered or creating a conflict.

Thus each 3-set gadget can be in exactly one of two states in a valid triangle partition: *active* (representing that S_j is in the cover) or *inactive*.

Correctness (Sketch)

($\Rightarrow$) **Exact Cover $\Rightarrow$ Triangle Partition.** Suppose $\mathcal{C} \subseteq \mathcal{S}$ is an exact cover of U .

- For each $S_j \in \mathcal{C}$, put its gadget in the active state: use its three interface vertices to connect to the three corresponding element gadgets (one for each $u \in S_j$), filling in the rest of the gadget with internal triangles.
- For each $S_j \notin \mathcal{C}$, put its gadget in the inactive state, covering all its vertices with internal triangles only.
- Every element gadget u is used exactly once, since u belongs to exactly one set in the exact cover. The remaining vertices in the element gadget are covered by its internal triangles.

This yields a partition of V into triangles.

($\Leftarrow$) **Triangle Partition $\Rightarrow$ Exact Cover.** Conversely, suppose G has a partition of its vertices into triangles.

- In each set gadget, the partition must match one of the two allowed patterns, determining whether S_j is active or inactive.
- In each element gadget, the requirement to cover all vertices by triangles ensures that exactly one connector triangle to some set gadget is used; otherwise, some vertex would remain uncovered or used twice.

Thus each element $u \in U$ is claimed by exactly one set gadget S_j that is active. The corresponding family of sets forms an exact cover for U .

Complexity. The gadgets have constant size per element and per set, and are built in a uniform way. Hence the total size of G is polynomial in $|U| + |\mathcal{S}|$, and the reduction runs in polynomial time.

Conclusion

We obtain a polynomial-time reduction

$$\text{X3C} \leq_p \text{PARTITION INTO TRIANGLES},$$

establishing that PIT is NP-hard. Since PIT is clearly in NP, it is NP-complete.

14.12 Reduction from Planar 3-SAT to Partition into Triangles

We now sketch a reduction from PLANAR 3-SAT to PARTITION INTO TRIANGLES (PIT). This offers an alternative NP-hardness proof using planar structure and explicit variable/clause gadgets.

Planar 3-SAT

An instance of Planar 3-SAT is a 3-CNF formula

$$\varphi = C_1 \wedge C_2 \wedge \cdots \wedge C_m$$

over variables $x_1, \dots, x_n$, such that the bipartite incidence graph between variables and clauses is planar. Each clause

$$C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3})$$

has exactly three literals. The problem of deciding satisfiability remains NP-complete even under the planarity restriction.

High-Level Idea. We build a graph G whose vertices can be partitioned into triangles if and only if there is a planar 3-SAT assignment satisfying φ . The graph is composed of:

- *variable gadgets*: each has exactly two distinct triangle partitions, corresponding to setting the variable to true or false;
- *clause gadgets*: each enforces that at least one incident literal is set to true, otherwise the gadget cannot be fully partitioned;
- *wires*: paths of vertices connecting variable gadgets to clause gadgets, transmitting the chosen truth value while preserving the ability (or inability) to tile with triangles.

The planar embedding of the variable-clause incidence graph guides the layout without crossings.

Variable Gadget

For each variable x_i , we construct a cycle of gadgets arranged around its occurrences in clauses. The cycle is constructed so that:

- It has exactly two ways to be partitioned into triangles.
- We interpret one tiling as $x_i = \text{TRUE}$ and the other as $x_i = \text{FALSE}$.

Concretely, we can use a cycle of an even number of vertices with chords added so that:

- In the *true* tiling, all “positive” ports (connection points to clauses where x_i appears positively) are free to connect to clauses, while “negative” ports are blocked by internal triangles.
- In the *false* tiling, the roles of the ports are swapped: negative ports are free, positive ports are blocked.

These ports are degree-2 or degree-3 vertices that will be linked to clause gadgets.

Clause Gadget

For each clause $C_j = (\ell_{j1} \vee \ell_{j2} \vee \ell_{j3})$ we construct a small graph with three designated *literal ports*, one for each literal, connected by paths (wires) to the appropriate variable gadget ports.

The clause gadget is built so that:

- If at least one literal port receives an “open” endpoint consistent with the variable assignment (i.e. the corresponding variable gadget tiling leaves that port available), then there is a way to complete the triangles around the clause gadget so that all its vertices are covered.
- If all three literals are false (their ports blocked by the chosen variable tilings), then the clause gadget cannot be fully partitioned into triangles; some vertex is forced to remain uncovered or some edge is overused.

Wires

A *wire* is a path of small repeatable gadgets connecting a variable port to a clause port. Each wire segment has two possible local tilings that propagate a “signal” while remaining consistent along the path.

Key property:

- If the variable gadget port is open (literal true), the wire can be tiled in a way that delivers an open endpoint to the clause gadget.
- If the port is blocked (literal false), the wire forces a blocking pattern at the clause end.

We route these wires in the plane according to the planar incidence graph of the formula, so no crossings are required.

Correctness (Sketch)

($\Rightarrow$) **Triangle Partition $\Rightarrow$ Satisfying Assignment.** Assume G has a partition of its vertices into triangles.

- In each variable gadget, the structure allows only two global tilings. Thus the partition fixes a truth value for each variable x_i .
- Given this assignment, follow each wire gadget. Because the entire graph is tiled by triangles, the tiling on the variable side propagates consistently to the clause side.
- Inspect each clause gadget. If the gadget is fully covered by triangles, then by design at least one literal port must have been open (true), otherwise the gadget geometry would prevent a complete tiling. Therefore each clause has at least one true literal, and the induced assignment satisfies φ .

($\Leftarrow$) **Satisfying Assignment $\Rightarrow$ Triangle Partition.** Conversely, assume φ has a satisfying assignment α .

- In each variable gadget, choose the tiling corresponding to $\alpha(x_i)$.
- For each wire, use the local tiling that is consistent with the variable gadget and passes the appropriate signal to the clause gadget.
- Because each clause is satisfied, at least one literal port is open; the clause gadget can then be tiled using the “satisfied” pattern that covers all its vertices.

By construction, these choices are consistent and tile the entire graph with vertex-disjoint triangles.

Complexity and Planarity

The size of the graph G is linear in the size of the formula φ : each variable occurrence and each clause introduces a constant-size gadget, and the wires use a linear number of vertices. Therefore, the reduction runs in polynomial time.

Moreover, since the variable–clause incidence graph of φ is planar and we route wires without crossings, the resulting graph inherits a planar embedding (though planarity is not needed for the NP-completeness of PIT on general graphs).

Conclusion

We have sketched a polynomial-time reduction

$$\text{PLANAR 3-SAT} \leq_p \text{PARTITION INTO TRIANGLES}.$$

Since planar 3-SAT is NP-complete and PIT is in NP, this provides another NP-hardness proof for PARTITION INTO TRIANGLES.

14.13 Reduction from 3-SAT to VERTEX COVER

We show that the VERTEX COVER problem is NP-complete. The proof consists of two parts: (i) VERTEX COVER is in NP, and (ii) VERTEX COVER is NP-hard by a polynomial-time reduction from 3SAT.

Vertex Cover is in NP

Given a graph $G = (V, E)$ and an integer k , a nondeterministic algorithm can *guess* a subset $V' \subseteq V$ and verify:

1. $|V'| \leq k$, and
2. for every edge $(v, w) \in E$, at least one of v or w lies in V' .

Both checks can be performed in polynomial time. Thus VERTEX COVER $\in$ NP.

Reduction from 3SAT

Let the input 3SAT instance be (U, C) , where $U = \{x_1, x_2, \dots, x_n\}$ is the set of boolean variables and $C = \{c_1, c_2, \dots, c_m\}$ is the set of clauses, with each clause

$$c_j = (\ell_{j,1} \vee \ell_{j,2} \vee \ell_{j,3}),$$

where each literal $\ell_{j,r}$ is either x_i or $\neg x_i$ for some i .

We construct a graph $G = (V, E)$ and an integer k as follows.

Vertices.

$$V = \{u_i, w_i : 1 \leq i \leq n\} \cup \{z_{j,1}, z_{j,2}, z_{j,3} : 1 \leq j \leq m\}.$$

The vertices u_i and w_i encode the variable x_i and its negation $\neg x_i$, respectively. The vertices $z_{j,1}, z_{j,2}, z_{j,3}$ correspond to the three literals in clause c_j .

Edges. The edge set E contains:

- Variable edges:

$$(u_i, w_i) \quad \text{for all } 1 \leq i \leq n.$$

- Clause edges (forming a triangle for each clause):

$$(z_{j,1}, z_{j,2}), (z_{j,2}, z_{j,3}), (z_{j,1}, z_{j,3}) \quad \text{for all } 1 \leq j \leq m.$$

- Literal–variable consistency edges:

$$(z_{j,r}, u_i) \text{ if } \ell_{j,r} = x_i, \quad (z_{j,r}, w_i) \text{ if } \ell_{j,r} = \neg x_i.$$

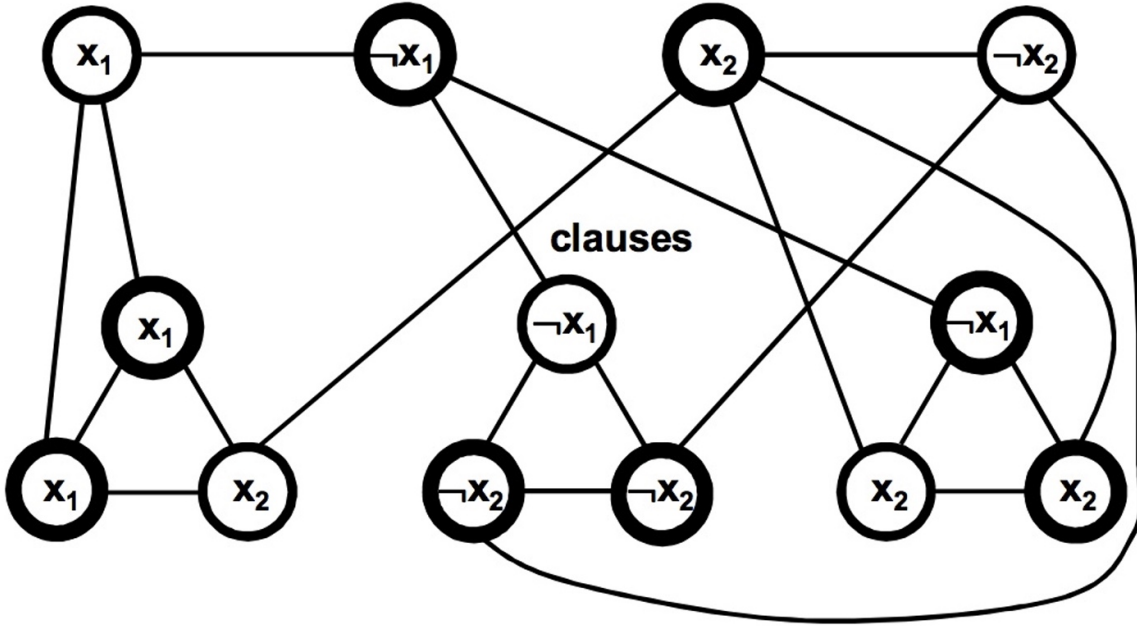
Cover size. We set

$$k = n + 2m.$$

Clearly the reduction can be carried out in polynomial time.

$$(\mathbf{x}_1 \vee \mathbf{x}_1 \vee \mathbf{x}_2) \wedge (\neg \mathbf{x}_1 \vee \neg \mathbf{x}_2 \vee \neg \mathbf{x}_2) \wedge (\neg \mathbf{x}_1 \vee \mathbf{x}_2 \vee \mathbf{x}_2)$$

Variables and negations of variables



$$k = 2(\text{\#clauses}) + (\text{\#variables})$$

Structure of Any Vertex Cover of Size $\leq k$

Observe:

1. For each variable pair (u_i, w_i) , at least one must be taken in any vertex cover.
2. For each clause triangle $(z_{j,1}, z_{j,2}, z_{j,3})$, at least two vertices must be taken to cover all three edges.

Thus any vertex cover of size at most $n + 2m$ must contain:

exactly one of u_i, w_i for each i , and exactly two of $z_{j,1}, z_{j,2}, z_{j,3}$ for each j .

Satisfying Assignment $\Rightarrow$ Vertex Cover

Assume the 3SAT instance is satisfiable. Let the assignment satisfy every clause.

Construct a vertex cover V' as follows:

- For each variable x_i :

$$u_i \in V' \text{ iff } x_i = \text{true}, \quad w_i \in V' \text{ iff } x_i = \text{false}.$$

- For each clause c_j : choose two of $z_{j,1}, z_{j,2}, z_{j,3}$ to include in V' , leaving out $z_{j,r}$ whose corresponding literal $\ell_{j,r}$ is true.

All edges are covered:

- variable edges by construction,
- clause edges because two vertices of each clause triangle are selected,
- literal–variable edges because a literal left out corresponds to a true literal, and its adjacent variable node is in the cover.

Hence V' is a vertex cover of size $n + 2m = k$.

Vertex Cover $\Rightarrow$ Satisfying Assignment

Conversely, suppose G has a vertex cover V' of size at most $n + 2m$. Then:

$$u_i \in V' \text{ or } w_i \in V' \quad (i = 1, \dots, n),$$

and exactly two of $\{z_{j,1}, z_{j,2}, z_{j,3}\}$ lie in V' for each clause.

Define the truth assignment:

$$x_i = \text{true} \iff u_i \in V'.$$

Consider a clause c_j . Let $z_{j,r}$ be the unique clause vertex *not* included in V' . Because the edge $(z_{j,r}, s_i)$ must be covered, where

$$s_i = \begin{cases} u_i & \text{if } \ell_{j,r} = x_i, \\ w_i & \text{if } \ell_{j,r} = \neg x_i, \end{cases}$$

the corresponding s_i must be in V' . Thus $\ell_{j,r}$ is true under the assignment. Hence each clause has at least one true literal, and the formula is satisfied.

Conclusion

The above reduction shows that $3\text{SAT} \leq_P \text{VERTEX COVER}$, and since VERTEX COVER is in NP, the problem is NP-complete.

14.14 Reduction from Vertex Cover to Hitting Set

We show a polynomial-time reduction from VERTEX COVER to HITTING SET . Intuitively, each edge of the graph becomes a 2-element set that must be “hit”, and each vertex is an element which can hit all sets (edges) it is incident to.

Problem Definitions

Vertex Cover. An instance of VERTEX COVER consists of:

- an undirected graph $G = (V, E)$,
- an integer k .

Question: does there exist a subset $C \subseteq V$ with $|C| \leq k$ such that every edge $e = \{u, v\} \in E$ has at least one endpoint in C ? Such a set C is called a *vertex cover*.

Hitting Set. An instance of HITTING SET (set hitting) consists of:

- a finite universe U ,
- a collection $\mathcal{F}$ of subsets of U ,
- an integer k .

Question: does there exist a subset $H \subseteq U$ with $|H| \leq k$ such that H intersects every set $F \in \mathcal{F}$ (i.e. $H \cap F \neq \emptyset$)? Such an H is called a *hitting set* (or transversal).

Conceptual Note. Vertex Cover can be seen as a special case of Hitting Set:

- Vertices $\leftrightarrow$ universe elements.
- Edges $\leftrightarrow$ 2-element subsets that must be hit.
- A vertex cover $\leftrightarrow$ a hitting set.

The reduction just formalizes this observation.

Construction

Let (G, k) be an instance of VERTEX COVER, where $G = (V, E)$.

We construct a HITTING SET instance $(U, \mathcal{F}, k')$ as follows:

- Universe:

$$U := V.$$

That is, each vertex of the graph becomes an element of the universe.

- Family of subsets: for each edge $e = \{u, v\} \in E$, define a subset

$$F_e := \{u, v\} \subseteq U.$$

Then let

$$\mathcal{F} := \{F_e : e \in E\}.$$

- Parameter:

$$k' := k.$$

This completes the description of the hitting set instance.

Correctness of the Reduction

We prove that (G, k) is a YES-instance of VERTEX COVER if and only if $(U, \mathcal{F}, k')$ is a YES-instance of HITTING SET.

($\Rightarrow$) **Vertex Cover $\Rightarrow$ Hitting Set.** Assume there exists a vertex cover $C \subseteq V$ with $|C| \leq k$.

For each edge $e = \{u, v\} \in E$, by definition of vertex cover, at least one of u or v lies in C . Thus:

$$C \cap \{u, v\} \neq \emptyset \quad \text{for all } e = \{u, v\} \in E.$$

Equivalently,

$$C \cap F_e \neq \emptyset \quad \text{for all } F_e \in \mathcal{F}.$$

Therefore C is a hitting set of size at most $k' = k$ for the family $\mathcal{F}$. Hence $(U, \mathcal{F}, k')$ is a YES-instance of HITTING SET.

($\Leftarrow$) **Hitting Set $\Rightarrow$ Vertex Cover.** Conversely, assume there exists a hitting set $H \subseteq U$ with $|H| \leq k'$ such that H intersects every $F_e \in \mathcal{F}$.

Recall that $U = V$ and each $F_e = \{u, v\}$ for an edge $e = \{u, v\} \in E$. Since H is a hitting set:

$$H \cap F_e \neq \emptyset \quad \text{for every } e = \{u, v\} \in E.$$

That is, for every edge $e = \{u, v\}$, we have $u \in H$ or $v \in H$ (possibly both).

Therefore H is a vertex cover of G and

$$|H| \leq k' = k.$$

So (G, k) is a YES-instance of VERTEX COVER.

Complexity

The construction is clearly polynomial-time:

- We set $U = V$, which just copies the vertex set.
- For each edge $e = \{u, v\}$, we create a 2-element subset F_e .

Thus the size of $(U, \mathcal{F}, k')$ is $\mathcal{O}(|V| + |E|)$, and the conversion can be done in time polynomial in the size of G .

Conclusion

We have defined a polynomial-time reduction

$$\text{VERTEX COVER} \leq_p \text{HITTING SET}.$$

Since VERTEX COVER is NP-complete and HITTING SET is in NP, it follows that HITTING SET is NP-complete.

14.15 Reduction from Vertex Cover to Set Cover

We present a polynomial-time reduction from VERTEX COVER to SET COVER. Informally, each *vertex* becomes a *set* of edges it can cover, and each *edge* becomes an *element* that must be covered by at least one chosen set.

Problem Definitions

Vertex Cover. An instance of VERTEX COVER consists of:

- an undirected graph $G = (V, E)$,
- an integer k .

Question: does there exist a set $C \subseteq V$ with $|C| \leq k$ such that every edge $e = \{u, v\} \in E$ has at least one endpoint in C ? Such a set C is called a *vertex cover*.

Set Cover. An instance of SET COVER consists of:

- a universe U of elements,
- a collection of subsets $\mathcal{S} = \{S_1, \dots, S_m\}$ with $S_i \subseteq U$,
- an integer k .

Question: does there exist a subcollection $\mathcal{C} \subseteq \mathcal{S}$ with $|\mathcal{C}| \leq k$ such that

$$\bigcup_{S \in \mathcal{C}} S = U?$$

Such a subcollection $\mathcal{C}$ is a *set cover* of size at most k .

Conceptual Note. There is a natural duality:

- *Edges* correspond to *universe elements* that must be covered.
- Each *vertex* corresponds to the set of edges incident to it.
- A vertex cover is exactly a choice of vertices whose incident-edge sets cover all edges.

The reduction just encodes this duality formally.

Construction

Given a VERTEX COVER instance (G, k) with $G = (V, E)$, we construct a SET COVER instance $(U, \mathcal{S}, k')$ as follows.

- **Universe.** Let

$$U := E,$$

so each edge $e \in E$ becomes a universe element.

- **Subsets.** For each vertex $v \in V$, define a subset

$$S_v := \{e \in E : e \text{ is incident to } v\}.$$

Then let

$$\mathcal{S} := \{S_v : v \in V\}.$$

That is, each vertex gives a set containing exactly the edges it can cover.

- **Parameter.** Set

$$k' := k.$$

This completes the construction of the SET COVER instance.

Correctness of the Reduction

We prove that (G, k) is a YES-instance of VERTEX COVER if and only if $(U, \mathcal{S}, k')$ is a YES-instance of SET COVER.

($\Rightarrow$) **From Vertex Cover to Set Cover.** Assume there is a vertex cover $C \subseteq V$ with $|C| \leq k$.

Consider the subcollection

$$\mathcal{C} := \{S_v : v \in C\} \subseteq \mathcal{S}.$$

We claim that $\mathcal{C}$ covers all of U .

Let $e \in U$ be any universe element. Since $U = E$, e is an edge of G : say $e = \{u, w\} \in E$. Because C is a vertex cover, at least one of u, w is in C . Suppose $u \in C$. By construction,

$$e \in S_u \in \mathcal{C}.$$

Thus e belongs to the union $\bigcup_{S \in \mathcal{C}} S$.

Since this holds for every edge $e \in E = U$, we have

$$\bigcup_{S \in \mathcal{C}} S = U. \quad \text{and} \quad |\mathcal{C}| = |C| \leq k' = k.$$

Hence $(U, \mathcal{S}, k')$ is a YES-instance of SET COVER.

($\Leftarrow$) **From Set Cover to Vertex Cover.** Conversely, suppose there exists a set cover $\mathcal{C} \subseteq \mathcal{S}$ with $|\mathcal{C}| \leq k'$ such that

$$\bigcup_{S \in \mathcal{C}} S = U.$$

Since $\mathcal{S} = \{S_v : v \in V\}$, each set in $\mathcal{C}$ has the form S_v for some vertex $v \in V$. Define

$$C := \{v \in V : S_v \in \mathcal{C}\}.$$

Then $|C| = |\mathcal{C}| \leq k' = k$.

We claim that C is a vertex cover of G .

Let $e \in E$ be any edge. As $U = E$, e is a universe element and must be covered by $\mathcal{C}$, so there exists some $S_v \in \mathcal{C}$ such that $e \in S_v$. By construction of S_v , this means that edge e is incident to vertex v . Since $S_v \in \mathcal{C}$, we have $v \in C$.

Therefore, for every edge $e = \{u, w\}$ in E , at least one endpoint belongs to C . Hence C is a vertex cover of G with size at most k .

Complexity

The size of $(U, \mathcal{S}, k')$ is polynomial in $|V| + |E|$:

- U has one element per edge.
- $\mathcal{S}$ has one set per vertex, and each S_v contains all edges incident to v .

Constructing all sets S_v requires scanning each edge once and adding it to both endpoints' sets. Thus the reduction runs in time $\mathcal{O}(|V| + |E|)$.

Conclusion

We have defined a polynomial-time reduction

$$\text{VERTEX COVER} \leq_p \text{SET COVER}.$$

Since VERTEX COVER is NP-complete and SET COVER is in NP, it follows that SET COVER is NP-complete.

14.16 Reduction from Set Cover to Dominating Set

We present a polynomial-time reduction from SET COVER to DOMINATING SET. The construction is standard: each set becomes a vertex that dominates the elements it contains.

Problem Definitions

Set Cover. An instance of SET COVER consists of:

- a universe $U = \{u_1, \dots, u_n\}$,
- a family of subsets $\mathcal{S} = \{S_1, \dots, S_m\}$ with $S_i \subseteq U$,
- an integer k .

Question: is there a subfamily $\mathcal{C} \subseteq \mathcal{S}$ with $|\mathcal{C}| \leq k$ such that $\bigcup_{S \in \mathcal{C}} S = U$?

Dominating Set. Given a graph $G = (V, E)$ and an integer k , the question is whether there is a subset $D \subseteq V$ with $|D| \leq k$ such that every vertex $v \in V$ is either in D or adjacent to some vertex in D . Such a D is a *dominating set*.

Conceptual Note. Interpretation:

- each set S_i will be a vertex that “dominates” the elements in S_i ,
- picking a vertex in the dominating set corresponds to choosing the set,
- ensuring every element vertex is dominated corresponds to covering all universe elements.

Construction

Given a set cover instance $(U, \mathcal{S}, k)$, we construct a graph $G = (V, E)$ and integer k' as follows.

Vertices. Create two types of vertices:

$$V = V_{\text{set}} \cup V_{\text{elem}},$$

where

- for each set $S_i \in \mathcal{S}$, create a vertex $s_i \in V_{\text{set}}$,
- for each element $u_j \in U$, create a vertex $e_j \in V_{\text{elem}}$.

Edges. Add edges according to set membership:

$$(s_i, e_j) \in E \iff u_j \in S_i.$$

Finally, make all set-vertices adjacent to each other (a clique):

$$\forall i \neq i', (s_i, s_{i'}) \in E.$$

Parameter. Let

$$k' := k.$$

Intuition Behind the Gadget

- Each e_j (element vertex) must be dominated, so at least one s_i with $u_j \in S_i$ must be chosen.
- Making the s_i all adjacent ensures:
 - If we pick any s_i , *all* s -vertices are automatically dominated.
 - There is never a need to include element vertices e_j in the dominating set (this would waste budget).
- Thus a minimum dominating set will consist only of s_i -vertices, which correspond exactly to sets chosen in the cover.

Correctness

($\Rightarrow$) **From Set Cover to Dominating Set.** Suppose $\mathcal{C} \subseteq \mathcal{S}$ is a set cover with $|\mathcal{C}| \leq k$. Let

$$D := \{s_i : S_i \in \mathcal{C}\}.$$

Then $|D| = |\mathcal{C}| \leq k' = k$.

We show D is a dominating set:

- Every set-vertex s_i is adjacent to every vertex in D since V_{set} is a clique.
- Every element-vertex e_j corresponds to an element $u_j \in U$, and since $\mathcal{C}$ is a set cover, there exists $S_i \in \mathcal{C}$ such that $u_j \in S_i$. Hence e_j is adjacent to $s_i \in D$.

Thus all vertices are dominated by D , so (G, k') is a YES-instance of DOMINATING SET.

($\Leftarrow$) **From Dominating Set to Set Cover.** Conversely, suppose $D \subseteq V$ is a dominating set with $|D| \leq k'$.

- If D contains an element-vertex e_j , we can replace it by any adjacent set-vertex (there is always at least one, since e_j must be dominated). This never increases the size of D .
- Repeating this for all such e_j yields a dominating set D' of size $\leq k'$ containing *only* set-vertices.

Let

$$\mathcal{C} := \{S_i : s_i \in D'\}.$$

Every element $u_j \in U$ must be dominated by some $s_i \in D'$, and the edge (s_i, e_j) exists only if $u_j \in S_i$. Therefore every u_j is contained in at least one set $S_i \in \mathcal{C}$, proving that $\mathcal{C}$ is a set cover with $|\mathcal{C}| \leq k$.

Complexity

The construction uses $|V| = m + n$ vertices and $|E| = \sum_{i=1}^m |S_i| + \binom{m}{2}$ edges, which is polynomial in the input size. Therefore the reduction runs in polynomial time.

Conclusion

This construction gives a polynomial-time reduction

$$\text{SET COVER} \leq_p \text{DOMINATING SET}.$$

Hence, since SET COVER is NP-complete and DOMINATING SET is in NP, DOMINATING SET is NP-complete.

14.17 WA3 Task 2: Complexity of MIS Variants

In this task we investigate the computational complexity of several variants of the maximal independent set (MIS) problem. We assume familiarity with the definitions of maximal independent set, Extend-MIS-out (from WA2), and vertex cover (from lectures). We study the following decision problems:

min-MIS.

- **Input:** A simple undirected graph $G = (V, E)$ and an integer $k \geq 0$.
- **Question:** Does G have a maximal independent set $S \subseteq V$ with $|S| \leq k$?

min-VC.

- **Input:** A simple undirected graph $G = (V, E)$ and an integer $k \geq 0$.
- **Question:** Does G have a vertex cover $C \subseteq V$ with $|C| \leq k$?

1. Extend-MIS-out is in NP

A certificate is a maximal independent set $I \subseteq V$ such that $S \cap I = \emptyset$. Verification consists of:

- **Independence:** For each edge $\{u, v\} \in E$, ensure that $\{u, v\} \not\subseteq I$.
- **Maximality:** For each $v \in V \setminus I$, check that v has a neighbor in I .

Both checks take $O(|E|)$ time using adjacency lists. Hence, Extend-MIS-out $\in$ NP.

2. Extend-MIS-out is NP-complete (reduction from 3-SAT)

Given a 3-SAT instance φ , construct (G, S) as follows:

1. For each literal ℓ , create a vertex v_ℓ .
2. For each variable x , create a vertex s_x and make $\{s_x, v_x, v_{\neg x}\}$ a 3-clique. Add s_x to S . This enforces exactly one of v_x or $v_{\neg x}$ to appear in the MIS.
3. For each clause $C = (\ell_1 \vee \ell_2 \vee \ell_3)$, create a vertex s_C adjacent to $v_{\ell_1}, v_{\ell_2}, v_{\ell_3}$, and add s_C to S . This forces at least one literal of each clause to be chosen in any MIS disjoint from S .

Correctness. If φ is satisfiable: Let I contain exactly the vertices v_ℓ corresponding to true literals. This set is independent and maximal, and is disjoint from S .

If (G, S) is a YES-instance: Any maximal independent set I with $I \cap S = \emptyset$ must contain exactly one of $\{v_x, v_{\neg x}\}$ for each variable, defining a truth assignment. Each clause vertex $s_C \in S$ enforces that at least one of its three literals is in I , so the assignment satisfies φ .

Thus Extend-MIS-out is NP-complete.

3. min-MIS is in NP

A certificate is a maximal independent set $S \subseteq V$ with $|S| \leq k$. We verify:

- S is independent (scan edges),
- S is maximal (every $v \notin S$ has a neighbor in S),
- $|S| \leq k$.

All checks take polynomial time, so min-MIS $\in$ NP.

4. The naive reduction from min-VC to min-MIS does not work

Proposed mapping:

- For each edge $e = \{u, v\}$ in G , introduce a new vertex s_e and replace e with the path $u - s_e - v$.
- Set $k' = k$.

Counterexample: Let G be a 3-vertex path. Then $(G, k = 1)$ is a YES-instance of min-VC (choose the middle vertex). But the transformed graph G' becomes a 5-vertex path, in which every maximal independent set has size at least 2. Hence $(G', 1)$ is a NO-instance of min-MIS. Thus this reduction fails.

5. Correct reduction: min-VC $\leq_P$ min-MIS

Let $G = (V, E)$ and k be a min-VC instance. Define $k' = |V| + k$ and construct G' :

1. For each edge $e = \{u, v\}$, replace it with $k' + 1$ internally vertex-disjoint paths of length 2 between u and v .
2. For each vertex $v \in V$, add a new path (v, v', v'') .

Correctness. ($\Rightarrow$) Given a vertex cover C of size $\leq k$, construct a maximal independent set:

$$I := \{v, v'' : v \in C\} \cup \{v' : v \notin C\}.$$

Then $|I| = |V| + |C| \leq k'$, and I is maximal and independent.

($\Leftarrow$) Let I be an MIS of G' with $|I| \leq k' = |V| + k$. For every $v \in V$, I intersects $\{v, v', v''\}$ in exactly either:

$$\{v'\} \quad \text{or} \quad \{v, v''\}.$$

Define $C := \{v : v \in I\}$. Then $|C| = |I| - |V| \leq k$. Maximality of I guarantees C is a vertex cover.

Thus min-VC $\leq_P$ min-MIS, and min-MIS is NP-complete.

14.18 More Reductions Overview and Examples

1. $3SAT \leq_P$ Vertex 3-Coloring,
2. $3SAT \leq_P$ Hamiltonian Cycle,
3. $3SAT \leq_P$ Super Mario (game NP-hardness framework).

Each reduction is based on constructing *gadgets* that encode the logical structure of a 3SAT formula inside the constraints of the target problem.

14.18.1 Polynomial-Time Reductions

A polynomial-time (Karp) reduction from problem A to problem B is an algorithm that maps any instance of A to an instance of B , in polynomial time, such that the two instances have the same answer. Formally,

$$A \leq_P B \iff \text{there exists polytime } f \text{ such that } x \in A \iff f(x) \in B.$$

This implies that B is *at least as hard* as A : a polynomial-time algorithm for B immediately yields one for A . To prove NP-hardness, we typically reduce from a known NP-complete problem such as 3SAT using small, local *gadgets* that simulate variables, clauses, or logical constraints.

14.18.2 Reduction: 3SAT to Vertex 3-Coloring

Given a 3SAT instance, the goal is to construct a graph that is 3-colorable iff the formula is satisfiable. The reduction uses clause gadgets that force the vertices corresponding to literals in a clause to be assigned colors that encode whether the literals can be simultaneously satisfied. The construction guarantees:

- If the formula is satisfiable, a consistent 3-coloring exists.
- If the formula is unsatisfiable, no proper 3-coloring exists.

Thus Vertex 3-Coloring is NP-hard, and because it is also in NP, it is NP-complete.

14.18.3 Reduction: 3SAT to Hamiltonian Cycle

To reduce 3SAT to Hamiltonian Cycle, we construct a directed graph in which a Hamiltonian cycle exists iff the formula is satisfiable. The graph uses:

- **Variable gadgets:** long chains that can be traversed in one of two directions, representing TRUE or FALSE assignments.
- **Clause gadgets:** nodes that can be visited only if at least one literal in the clause is satisfied.

A Hamiltonian cycle must visit all clause gadgets, forcing the traversal to correspond to a satisfying assignment. Conversely, if no satisfying assignment exists, at least one clause gadget is inaccessible, preventing a Hamiltonian cycle from existing.

14.18.4 Reduction: 3SAT to Super Mario

The NP-hardness of Super Mario (and many other platform games) is established via a general framework that reduces 3SAT to in-game progression puzzles. The reduction makes use of several carefully designed *gameplay gadgets*:

- **Start & Finish gadgets:** enforce an initial and final configuration (e.g., Mario must be large).
- **Variable gadgets:** force the player to commit to a TRUE or FALSE choice, and prevent switching afterwards.
- **Clause gadgets:** can be “unlocked” by visiting through at least one literal path.
- **Check path:** ensures that the player can reach the goal only if *all* clauses have been unlocked.
- **Crossover gadgets:** allow literal paths to cross without permitting leakage between them.

The player completes the level iff all clauses are satisfied, establishing NP-hardness of the game.

14.18.5 Takeaways

Across all three reductions, a unifying theme emerges:

- Logical constraints in 3SAT are encoded using small gadgets.
- The constructed instance in the target problem mirrors the structure of the Boolean formula.
- The correctness direction always proceeds by showing:

$$\text{Satisfying assignment} \iff \text{Valid solution in the target problem.}$$

These reductions demonstrate the versatility of gadget constructions and the central role of 3SAT in NP-hardness proofs.

14.19 3-Partition $\leq_P$ 2-partition(partition)**14.20 Partition $\leq_P$ 0/1 Knapsack****Problem definition****Partition**

Given a multiset S , whether there exists a partition of S , S_1 and S_2 , such that $\sum S_1 = \sum S_2 = \frac{1}{2} \sum S$

0/1 Knapsack

Given a multiset of n items $I = \{(w_i, v_i) | i \in [1, n]\}$, and a weight limit W ,

$$\text{maximize } \sum_{i=1}^n x_i v_i, \quad \text{such that } \sum_{i=1}^n x_i w_i \leq W$$

for the decision version, for a target value k , whether there exists a selection such that

$$\sum_{i=1}^n x_i v_i \geq k, \quad \text{while } \sum_{i=1}^n x_i w_i \leq W$$

Construction

Given an instance of partition (S) of size n , let $m = \sum S$, the target partition sum is $m/2$, consider the following construction of a 0/1 knapsack instance:

$$I = \{(\frac{S[i]}{2}, \frac{S[i]}{2}) | i \in [1, n]\}, \quad W = m, \quad k = m$$

Partition(YES) $\leq_P$ Knapsack(YES)

Knapsack(YES) $\leq_P$ Partition(YES)

14.21 0/1 Knapsack $\leq_P$ coin change**Problem definition****0/1 Knapsack**